

JavaScript desde primeros principios

Del bit a la aplicación: tipos, estructuras, control, funciones y entorno de ejecución

2026

Contenido

Parte I — Valores y tipos

- Capítulo 1. JavaScript: lenguaje, motor y entorno
- Capítulo 2. Variables, vinculaciones y alcance
- Capítulo 3. Expresiones, operadores y precedencia
- Capítulo 4. El tipo booleano y la lógica de JavaScript
- Capítulo 5. Números: `number` y `bigint`
- Capítulo 6. Cadenas de texto y Unicode
- Capítulo 7. `undefined` y `null`: dos formas de ausencia
- Capítulo 8. Símbolos y protocolos del lenguaje
- Capítulo 9. Conversiones y coerción de tipos

Parte II — Estructuras de datos

- Capítulo 10. Arrays: secuencias ordenadas
- Capítulo 11. Objetos: entidades con propiedades
- Capítulo 12. Referencias, copias e inmutabilidad
- Capítulo 13. `Map` y `Set`

Parte III — Control del flujo

- Capítulo 14. Decisiones y repeticiones
- Capítulo 15. Excepciones y manejo de errores

Parte IV — Funciones

- Capítulo 16. Funciones: definición, parámetros y resultados
- Capítulo 17. Alcance léxico y clausuras
- Capítulo 18. Funciones de orden superior y procesamiento de colecciones
- Capítulo 19. Recursión y estructuras recursivas

Parte V — Texto y archivos

- Capítulo 20. Expresiones regulares
- Capítulo 21. Gestión de archivos con Node.js

Apéndices

- Apéndice A. Fundamentos de la computación digital
- Apéndice B. Un tokenizador Byte Pair Encoding
- Apéndice C. Referencia rápida de expresiones regulares
- Apéndice D. Glosario

Fuentes y lecturas recomendadas

Parte I — Valores y tipos

Capítulo 1. JavaScript: lenguaje, motor y entorno

JavaScript está presente en casi todas las aplicaciones web que usamos. Responde cuando hacemos clic en un botón, valida un formulario, actualiza una pantalla sin recargarla y permite construir interfaces completas. Pero su alcance ya no termina en el navegador: también recibe solicitudes HTTP, consulta bases de datos, automatiza tareas y ejecuta servicios del lado del servidor.

¿Cómo llegó un lenguaje creado para agregar pequeñas interacciones a una página a ocupar tantos lugares distintos? Para responderlo conviene empezar por el problema que le dio origen y separar tres conceptos que suelen confundirse: **el lenguaje, el motor que lo ejecuta y el entorno que le permite interactuar con el mundo**.

1.1 La Web antes de JavaScript

En los primeros años de la Web, una página se parecía más a un documento que a una aplicación. El navegador pedía un archivo HTML a un servidor, lo recibía y lo mostraba. Si el usuario quería realizar otra acción, normalmente debía solicitar una página nueva.

HTML podía expresar la **estructura** del documento y, con el tiempo, CSS permitió describir su **presentación**. Faltaba una tercera pieza: una forma de expresar **comportamiento** dentro del navegador.

Imaginemos un formulario de registro en aquella Web. Sin un lenguaje ejecutándose en el navegador, incluso una verificación sencilla —comprobar que un campo no estuviera vacío— podía requerir este recorrido:

1. el usuario completaba el formulario;
2. el navegador enviaba los datos al servidor;
3. el servidor detectaba el error;
4. el servidor generaba una nueva respuesta;
5. el navegador cargaba otra página para mostrar el mensaje.

Cinco pasos y dos viajes por la red para decir «falta el nombre».

La necesidad era concreta: poder reaccionar inmediatamente a lo que ocurría en la página. Hacía falta un lenguaje suficientemente accesible para programas pequeños, integrado al navegador y capaz de responder a eventos como un clic, el envío de un formulario o la carga de un documento.

1.2 Nacimiento y estandarización

JavaScript fue creado en 1995 por Brendan Eich mientras trabajaba en Netscape. Su primera aparición pública estuvo ligada a Netscape Navigator 2. Aunque sus primeros nombres fueron Mocha y luego LiveScript, terminó llamándose JavaScript. El nombre aprovechaba la popularidad de Java en ese momento, pero los dos lenguajes no son versiones uno del otro: tienen modelos, objetivos e historias diferentes.

El éxito de JavaScript produjo rápidamente un problema nuevo. Otros navegadores necesitaban implementar un lenguaje compatible, pero una tecnología controlada por una sola empresa no era una base saludable para toda la Web. Netscape llevó entonces la propuesta a Ecma International. El trabajo de estandarización comenzó en 1996 y la primera edición del estándar **ECMA-262** fue aprobada en junio de 1997.

Aquí aparecen dos nombres que conviene relacionar correctamente:

- **ECMAScript** es el lenguaje definido por el estándar ECMA-262.
- **JavaScript** es el nombre con el que usamos habitualmente una implementación de ese lenguaje y su ecosistema.

En la conversación cotidiana ambos nombres se refieren al mismo lenguaje. Sin embargo, «ECMAScript» resulta útil cuando hablamos con precisión de la especificación o de una edición, como ES5 o ES2015.

Una línea de tiempo mínima

Año	Hito	Por qué fue importante
1995	Nace JavaScript en Netscape	Las páginas pueden incorporar comportamiento ejecutado en el navegador
1997	Primera edición de ECMA-262	El lenguaje obtiene una especificación independiente y compartida
1999	ECMAScript 3	Consolida buena parte del JavaScript clásico usado durante años
2005	Se populariza el enfoque AJAX	Las páginas intercambian datos con el servidor sin recargarse por completo
2008	Google publica Chrome y el motor V8	La competencia entre motores impulsa grandes mejoras de rendimiento
2009	ECMAScript 5 y primera versión de Node.js	El lenguaje madura y V8 empieza a usarse ampliamente fuera del navegador
2012	Microsoft presenta TypeScript	Se agrega verificación estática de tipos sobre JavaScript
2015	ECMAScript 2015, también llamado ES6	Incorpora módulos, clases, promesas, let, const, funciones flecha y muchas otras mejoras
Desde 2016	Ediciones anuales de ECMAScript	La evolución pasa a ser gradual y predecible, en lugar de depender de grandes saltos

Esta historia muestra una transformación. JavaScript nació para programas pequeños dentro de páginas, pero la Web empezó a exigir aplicaciones cada vez más complejas. Los motores se hicieron más rápidos, el estándar incorporó mejores abstracciones y el ecosistema llevó el lenguaje a nuevos entornos.

1.3 ¿Qué significa «ejecutar JavaScript»?

Cuando escribimos este código:

```
const mensaje = "Hola, programación";  
console.log(mensaje.toUpperCase());
```

nosotros vemos texto. El procesador, en cambio, solo puede ejecutar instrucciones de máquina, construidas a partir de las operaciones elementales del apéndice A. Entre una cosa y la otra debe existir un traductor.

Tradicionalmente se presentan dos modelos:

- Un **compilador** traduce el programa antes de ejecutarlo y suele producir otro programa o código de máquina.
- Un **intérprete** analiza las instrucciones y las va ejecutando mediante otro programa.

Esta distinción sirve como primera aproximación, pero los motores modernos combinan estrategias. Por eso, afirmar simplemente que «JavaScript es interpretado» deja afuera una parte importante de lo que sucede.

Más precisamente:

ECMAScript define cómo debe comportarse el lenguaje, pero no obliga a los motores a utilizar una única estrategia de ejecución.

Un motor moderno puede interpretar una representación intermedia y, al mismo tiempo, compilar ciertas partes a código de máquina. V8 —usado por Chrome y Node.js— combina un intérprete con compiladores de distintos niveles. Otros motores, como SpiderMonkey de Firefox o JavaScriptCore de Safari, tienen arquitecturas propias con la misma clase de objetivo: comenzar a ejecutar pronto y optimizar lo que realmente lo necesita.

1.4 Compilación justo a tiempo

JIT significa *Just-In-Time*, o compilación justo a tiempo. La idea consiste en compilar durante la ejecución, usando información que solamente puede conocerse mientras el programa está funcionando.

De manera conceptual, el proceso puede verse así:

```
código fuente
↓
análisis sintáctico
↓
representación interna / bytecode
↓
ejecución y recolección de información
↓
optimización de las partes más utilizadas
↓
código de máquina especializado
```

¿Por qué esperar a la ejecución para compilar? Porque antes de ejecutar hay información que no está disponible. En JavaScript una misma función puede recibir valores de distintos tipos:

```
function sumar(a, b) {
  return a + b;
}

sumar(10, 20);           // 30
sumar("Hola, ", "Ana"); // "Hola, Ana"
```

Antes de ejecutar el programa, el motor no siempre puede saber qué recibirá `sumar`. Durante la ejecución, en cambio, puede observar su uso. Si una función se invoca miles de veces con números, el motor puede producir una versión optimizada para ese patrón, mucho más rápida que la versión general.

El razonamiento simplificado es el siguiente:

1. **Leer y analizar.** El motor verifica la sintaxis y construye una representación interna del programa.
2. **Comenzar rápidamente.** Genera una forma ejecutable inicial, como bytecode, sin gastar demasiado tiempo en optimizaciones.
3. **Observar.** Mientras el programa corre, registra qué funciones se usan mucho y qué clases de valores reciben.
4. **Optimizar.** Compila las zonas «calientes» a código de máquina más especializado.
5. **Desoptimizar si es necesario.** Si dejan de cumplirse las suposiciones, vuelve a una versión más general sin cambiar el resultado que exige el lenguaje.

Supongamos que esta función procesa miles de precios:


```
function calcularTotal(precios) {
  let total = 0;

  for (const precio of precios) {
    total += precio;
  }

  return total;
}
```

Si siempre recibe arreglos de números, un motor puede optimizar ese camino. Si más tarde aparece un texto, quizá tenga que abandonar esa optimización. Este comportamiento es una posibilidad interna del motor, no un contrato que nuestro código deba intentar controlar.

La conclusión importante no es memorizar los nombres de cada componente de V8. Es comprender que los motores actuales equilibran dos necesidades opuestas:

- **iniciar rápido**, sin compilar exhaustivamente todo el programa;
- **ejecutar rápido**, invirtiendo más trabajo en las partes que se repiten.

Por eso resulta más correcto decir:

JavaScript es un lenguaje cuya implementación moderna suele combinar interpretación y compilación JIT.

1.5 Tres capas: lenguaje, motor y entorno

Para entender cómo JavaScript puede usarse tanto en el navegador como en un servidor, hay que separar tres capas.

El lenguaje

ECMAScript define elementos como:

- valores: números, textos, booleanos, objetos, funciones;
- sintaxis: `if`, `for`, `class`, `import`, funciones flecha;
- reglas: alcance de variables, comparaciones, conversiones;
- objetos estándar: `Array`, `Object`, `Promise`, `Map`, `Math`.

Este código pertenece al lenguaje y puede ejecutarse en cualquier entorno:

```
const notas = [7, 9, 6];
const promedio = notas.reduce((suma, nota) => suma + nota, 0) / notas.length;

console.log(promedio);
```

El motor

El motor implementa la especificación y ejecuta el código. También administra memoria, recupera automáticamente los objetos que ya no se usan mediante el recolector de basura, y aplica estrategias como JIT.

Algunos motores conocidos:

- **V8**, usado por Chrome y Node.js;
- **SpiderMonkey**, usado por Firefox;
- **JavaScriptCore**, usado por Safari.

El entorno o anfitrión

El lenguaje por sí solo no sabe qué es una página web, un archivo del disco o una solicitud HTTP. El entorno incorpora esas capacidades mediante APIs.

En un navegador podemos escribir:

```
const boton = document.querySelector("#guardar");

boton.addEventListener("click", () => {
  document.body.classList.add("guardado");
});
```

`document`, el DOM y los eventos son APIs del navegador; no forman parte del núcleo de ECMAScript.

En Node.js podemos escribir:

```
import { readFile } from "node:fs/promises";

const contenido = await readFile("alumnos.json", "utf8");
console.log(contenido);
```

El acceso al sistema de archivos lo provee Node.js. Un navegador común no expone `node:fs` porque su entorno y sus reglas de seguridad son diferentes: una página web cualquiera no debería poder leer los documentos de quien la visita.

Podemos resumir la relación así:

```
Aplicación
  ↓ usa
APIs del entorno (DOM, archivos, red, procesos...)
  ↓ se apoya en
Motor de JavaScript (V8, SpiderMonkey, JavaScriptCore...)
  ↓ implementa
ECMAScript
```

Esta separación explica algo que suele desconcertar al principio: un código puede ser JavaScript perfectamente válido y, aun así, fallar en cierto lugar. No porque el lenguaje sea distinto, sino porque depende de una API que ese entorno no ofrece. El capítulo 21 trabaja íntegramente sobre APIs de Node.js por esta razón.

1.6 Frontend y backend: el mismo lenguaje, responsabilidades distintas

Cerca del usuario

En el navegador, JavaScript suele encargarse de:

- responder a eventos del usuario;
- leer y modificar el DOM;
- administrar el estado de la interfaz;
- validar datos para ofrecer una respuesta inmediata;
- pedir o enviar información a un servidor;
- actualizar partes de la pantalla.

```
const formulario = document.querySelector("#inscripcion");

formulario.addEventListener("submit", async (evento) => {
  evento.preventDefault();

  const datos = new FormData(formulario);

  await fetch("/api/inscripciones", {
    method: "POST",
    body: datos,
  });
});
```

Hay algo que conviene decir desde temprano: **el frontend no es una frontera de seguridad**. Un usuario puede modificar el código que corre en su navegador, desactivarlo o construir manualmente una solicitud. La validación del frontend mejora la experiencia; el backend debe volver a validar todo lo que importe.

Cerca de los datos y de las reglas

Con la aparición de Node.js, JavaScript obtuvo un entorno ampliamente adoptado fuera del navegador. Node.js utiliza V8 e incorpora APIs para red, archivos, procesos y otras tareas del sistema.

En el backend, JavaScript puede:

- recibir solicitudes HTTP;

- autenticar y autorizar usuarios;
- aplicar reglas de negocio;
- validar entradas no confiables;
- consultar o modificar una base de datos;
- integrarse con otros servicios;
- construir y devolver una respuesta.

```
import { createServer } from "node:http";

const servidor = createServer((solicitud, respuesta) => {
  respuesta.writeHead(200, { "content-type": "application/json" });
  respuesta.end(JSON.stringify({ mensaje: "Hola desde el backend" }));
});

servidor.listen(3000);
```

Usar el mismo lenguaje en ambos lados facilita compartir conocimientos y, en algunos casos, reglas y tipos. No significa que frontend y backend sean lo mismo: se ejecutan en máquinas diferentes, tienen capacidades diferentes y protegen responsabilidades diferentes.

1.7 Tipado estático y tipado dinámico

Un **tipo** representa una categoría de valores y las operaciones que tienen sentido sobre ellos. Sumar números, convertir un texto a mayúsculas o recorrer una colección son operaciones válidas para ciertos tipos y no para otros.

Al comparar sistemas de tipos conviene separar dos preguntas que suelen mezclarse:

1. **¿Cuándo se comprueba la compatibilidad de tipos?** Estático frente a dinámico.
2. **¿Cuánto convierte automáticamente el lenguaje entre tipos?** Más fuerte frente a más débil o coercitivo.

Estas dimensiones están relacionadas, pero no son equivalentes. Confundirlas produce afirmaciones imprecisas muy difundidas.

Tipado estático

En un sistema estático, una herramienta analiza los tipos antes de ejecutar el programa. Las variables y expresiones tienen tipos que el analizador puede verificar, ya sea porque los escribimos o porque los infiere.

```
let cantidad: number = 3;
cantidad = "tres"; // error de tipos
```

Qué aporta:

- detecta muchas inconsistencias antes de que lleguen al usuario;
- mejora el autocompletado y la navegación del editor;
- vuelve explícitos los contratos entre partes del sistema;
- brinda más seguridad al renombrar, mover código o refactorizar;
- reduce la cantidad de estados que una persona debe considerar a la vez.

Qué cuesta:

- agrega una etapa de análisis y, a veces, anotaciones;
- requiere aprender el sistema de tipos y resolver sus errores;
- puede introducir complejidad si se modela más de lo necesario;
- no demuestra que el programa sea correcto ni reemplaza las pruebas;
- no valida por sí mismo datos externos durante la ejecución.

Tipado dinámico

En un sistema dinámico, los valores tienen tipos durante la ejecución y una variable puede referirse a valores de tipos diferentes a lo largo del tiempo.

```
let resultado = 42;
resultado = "cuarenta y dos";
resultado = { valor: 42 };
```

JavaScript permite las tres asignaciones. Esto **no** significa que «no tenga tipos»: `42`, `"cuarenta y dos"` y `{ valor: 42 }` pertenecen a categorías diferentes, y el lenguaje lo sabe. Significa que la variable `resultado` no queda restringida estáticamente a una de ellas.

Podemos consultar parte de esa información durante la ejecución:

```
console.log(typeof 42);           // "number"
console.log(typeof "hola");       // "string"
console.log(typeof true);         // "boolean"
console.log(typeof { nombre: "Ana" }); // "object"
```

Qué aporta:

- permite empezar con poco código ceremonial;
- resulta flexible para prototipos, scripts y datos cambiantes;
- algunas abstracciones se expresan sin describir previamente todos sus tipos;
- el ciclo inicial de experimentación puede ser muy directo.

Qué cuesta:

- ciertas incompatibilidades aparecen solo al ejecutar el camino afectado;
- una función comunica menos sobre lo que espera si no tiene documentación;
- los cambios en sistemas grandes tienen consecuencias más difíciles de rastrear;

- el editor y las herramientas deben inferir a partir de información incompleta.

No existe un ganador universal. El valor de cada estrategia depende del tamaño del programa, su vida útil, el equipo y el costo de un error.

1.8 Coerción: la segunda dimensión

«Fuertemente tipado» y «débilmente tipado» no tienen una definición tan precisa como estático y dinámico. Distintos autores clasifican un mismo lenguaje de maneras diferentes, y conviene desconfiar de las afirmaciones tajantes. La idea más útil es observar cuánto convierte el lenguaje, de forma automática, valores de un tipo a otro.

JavaScript realiza numerosas **coerciones implícitas**, por lo que suele describirse como débilmente tipado o, de manera más precisa, como un lenguaje **dinámico y coercitivo**.

```
console.log("5" + 1);    // "51"
console.log("5" - 1);    // 4
console.log(false == 0); // true
console.log(false === 0); // false
```

¿Por qué los dos primeros resultados difieren?

- El operador `+` también sirve para concatenar textos. Ante un texto y un número, convierte el número a texto.
- El operador `-` solo representa resta numérica. Por eso intenta convertir `"5"` a número.
- `==` permite conversiones antes de comparar.
- `===` compara sin esa coerción y, como los tipos difieren, devuelve `false`.

La coerción puede ser cómoda, pero también puede esconder un error:

```
const precio = 1200;
const cantidad = "2"; // los campos de texto entregan texto

console.log(precio * cantidad); // 2400
console.log(precio + cantidad); // "12002"
```

El primer resultado puede hacernos creer que los datos eran correctos. El segundo revela que `cantidad` nunca fue convertida de manera intencional.

Una versión más clara explicita la conversión y valida el resultado:

```
const precio = 1200;
const entrada = "2";
const cantidad = Number(entrada);

if (!Number.isFinite(cantidad) || cantidad <= 0) {
  throw new Error("La cantidad debe ser un número positivo");
}

const total = precio * cantidad;
```

La lección no es «JavaScript convierte tipos y eso está mal». La coerción forma parte de las reglas del lenguaje y tiene un algoritmo definido y determinista. El objetivo es reconocer cuándo ocurre y evitar que una conversión implícita tome una decisión de negocio por nosotros. El capítulo 9 desarrolla esas reglas en detalle.

Las dos dimensiones juntas

Lenguaje	Momento de comprobación	Tendencia frente a conversiones
C# / Java	Estático	Generalmente fuerte
Python	Dinámico	Generalmente fuerte
JavaScript	Dinámico	Coercitivo; suele clasificarse como débil
TypeScript	Estático durante el desarrollo	Conserva el comportamiento dinámico de JavaScript al ejecutarse

La tabla es orientativa, sobre todo en la segunda columna. Lo esencial es notar que **dinámico no significa necesariamente débil**: Python es el contraejemplo habitual, porque `"5" + 1` allí produce un error en lugar de una conversión silenciosa.

1.9 TypeScript y la transpilación

A medida que las aplicaciones JavaScript crecieron, también crecieron las relaciones que una persona debía mantener en su cabeza: qué datos recibe una función, qué propiedades tiene un objeto, qué puede devolver una operación y qué partes dependen de ellas.

Consideremos esta función:

```
function aplicarDescuento(precio, porcentaje) {
  return precio - precio * porcentaje / 100;
}
```

El programa no expresa si `precio` debe ser un número, si `porcentaje` puede faltar o qué devuelve la función. Quizá esos contratos estén en la documentación; quizá sean solo un acuerdo implícito del equipo.

TypeScript permite escribirlos en el código:

```
function aplicarDescuento(  
  precio: number,  
  porcentaje: number  
): number {  
  return precio - precio * porcentaje / 100;  
}  
  
aplicarDescuento(10_000, 15); // correcto  
aplicarDescuento("10000", 15); // error antes de ejecutar
```

TypeScript fue presentado por Microsoft en 2012 como un **verificador estático de tipos para JavaScript**. Incluye la sintaxis de JavaScript y agrega, entre otras herramientas, anotaciones de tipos, interfaces y tipos genéricos.

Sin embargo, el navegador y Node.js ejecutan JavaScript. Por eso el código TypeScript debe convertirse antes de llegar al entorno de ejecución:

```
TypeScript (.ts)  
├── verificación de tipos  
└── emisión / transpilación  
    ↓  
    JavaScript (.js)  
    ↓  
    motor de JavaScript
```

Por ejemplo, este código TypeScript:

```
const duplicar = (valor: number): number => valor * 2;
```

produce un JavaScript equivalente a este:

```
const duplicar = (valor) => valor * 2;
```

Las anotaciones `: number` desaparecen. Ayudaron durante el desarrollo, pero no existen durante la ejecución.

¿Compilar o transpilar?

Compilar es traducir un programa a otra representación. **Transpilar** suele usarse para un caso más específico: traducir de un lenguaje fuente a otro lenguaje fuente de un nivel semejante. Como TypeScript produce JavaScript, es habitual decir que se transpila.

Las dos palabras no son contradictorias. `tsc` se llama *TypeScript Compiler*, realiza verificación estática y puede emitir JavaScript. En proyectos modernos también es común separar responsabilidades: TypeScript verifica los tipos sin emitir archivos y otra herramienta transforma el código.

La transpilación también permite adaptar JavaScript moderno a entornos más antiguos: una herramienta puede transformar cierta sintaxis reciente en otra equivalente que comprenda un navegador anterior. Esto cambia la forma del código, pero no incorpora las APIs que falten; para eso puede ser necesario un *polyfill*, un fragmento de código que implementa la funcionalidad ausente.

Una limitación fundamental

Como los tipos de TypeScript desaparecen, no validan automáticamente los datos que llegan desde fuera del programa:

```
type Usuario = {  
  id: number;  
  nombre: string;  
};  
  
const respuesta = await fetch("/api/usuario/1");  
const usuario: Usuario = await respuesta.json();
```

La anotación comunica lo que esperamos, pero no obliga al servidor a enviar esa estructura. Los datos provenientes de una API, un formulario, un archivo o una base de datos cruzan una frontera y deben validarse durante la ejecución.

TypeScript puede detectar inconsistencias en nuestro programa antes de ejecutarlo; no puede convertir una entrada externa no confiable en una entrada correcta.

1.10 Los tipos atraviesan fronteras

Imaginemos una pantalla que envía un producto nuevo al servidor. En el navegador, el valor de un campo llega como texto:

```
const precioIngresado = document.querySelector("#precio").value;  
// Por ejemplo: "35000"
```

La interfaz debe convertirlo y avisar temprano si es inválido. Sin embargo, el servidor no puede confiar en que la solicitud provino de esa interfaz ni en que su validación se ejecutó.

El recorrido correcto es:

```
campo de formulario
  ↓ texto
conversión y validación en el frontend
  ↓ solicitud HTTP
validación nuevamente en el backend
  ↓ dato confiable para esa operación
regla de negocio y persistencia
```

TypeScript puede compartir una descripción como `Producto` entre frontend y backend, y así detectar desacuerdos durante el desarrollo. Aun así, la solicitud HTTP contiene datos de ejecución. El backend debe comprobar que `precio` existe, que es un número finito y que cumple las reglas del negocio.

De aquí surge una regla general:

Los tipos estáticos describen contratos dentro del programa; la validación protege las fronteras por las que ingresan datos reales.

1.11 Síntesis conceptual

JavaScript surgió para incorporar comportamiento a la Web, y su uso terminó excediendo largamente ese propósito. Entender esa expansión requiere distinguir tres capas que a menudo se confunden: la especificación ECMAScript define el lenguaje; un motor lo implementa y decide cómo ejecutarlo; un entorno le agrega capacidades para interactuar con el mundo. Que un programa funcione en un lugar y no en otro casi siempre se explica por la tercera capa, no por las dos primeras.

La forma en que se ejecuta el lenguaje también resiste las simplificaciones. El estándar define resultados, no técnicas internas. Los motores actuales combinan interpretación, bytecode y compilación JIT, y usan información recogida durante la ejecución para optimizar lo que se repite. Decir que JavaScript «es interpretado» describe un modelo que ningún motor moderno usa en estado puro.

Sobre los tipos conviene sostener dos distinciones. La primera separa cuándo se comprueban los tipos —estático frente a dinámico— de cuánto convierte el lenguaje por su cuenta. JavaScript es dinámico y coercitivo, y esa segunda propiedad es la que produce los resultados sorprendentes. La segunda distinción separa la verificación estática, que trabaja sobre nuestro programa antes de ejecutarlo, de la validación en tiempo de ejecución, que protege las fronteras por donde entran datos que no controlamos. Son herramientas distintas para problemas distintos, y ninguna reemplaza a la otra ni a las pruebas.

Con este panorama podemos empezar a escribir código. El capítulo siguiente comienza por lo más básico que necesita cualquier programa: darle un nombre a un valor.

Capítulo 2. Variables, vinculaciones y alcance

Un programa transforma información. Para poder hacerlo necesita representar valores — un precio, un nombre, una cantidad, una respuesta afirmativa— y conservar algunos de ellos para usarlos más adelante.

En JavaScript, cada valor pertenece a un tipo. El número `25`, el texto `"25"` y el booleano `true` son tres valores distintos, con operaciones y significados diferentes. Una variable permite asociar uno de esos valores con un nombre dentro del programa.

Este capítulo construye esa idea desde la base: qué es una variable, cómo se declara, desde dónde puede usarse y durante cuánto tiempo existe. Al final presenta el catálogo completo de tipos primitivos, que los capítulos siguientes desarrollan uno por uno.

2.1 De un valor a una variable

El valor `1200` puede representar el precio de un producto. Podríamos escribirlo directamente cada vez que lo necesitamos:

```
console.log(1200 * 2);  
console.log(1200 * 3);
```

Pero el programa no expresa qué significa ese número. Si le damos un nombre, la intención se vuelve visible:

```
const precioUnitario = 1200;  
  
console.log(precioUnitario * 2);  
console.log(precioUnitario * 3);
```

Podemos imaginar una variable como una etiqueta que permite encontrar un valor. En términos más precisos, una declaración crea una **vinculación** entre un identificador y un valor dentro de cierto entorno del programa.

Esa precisión importa. Una variable de JavaScript no queda asociada para siempre a un tipo. Si fue declarada con `let`, puede referirse primero a un número y después a un texto:

```
let respuesta = 42;  
respuesta = "cuarenta y dos";
```

Los valores tienen tipos. La variable es el nombre mediante el cual accedemos al valor actual. Esta es la manifestación concreta del tipado dinámico que describimos en el capítulo 1.

2.2 Declarar, inicializar y asignar

Son tres acciones relacionadas, pero no idénticas:

```
let cantidad;           // declaración
cantidad = 3;           // asignación

let precio = 1500;      // declaración e inicialización
precio = 1800;          // nueva asignación
```

- **Declarar** introduce el nombre en un alcance.
- **Inicializar** le entrega su primer valor.
- **Asignar** cambia el valor al que está vinculada una variable ya existente.

Una declaración con `let` sin inicializador recibe el valor `undefined` cuando su declaración se ejecuta:

```
let observacion;

console.log(observacion); // undefined
```

Una declaración con `const`, en cambio, debe inicializarse en el mismo momento:

```
const comision = "4K1";
// const turno; // SyntaxError: falta el inicializador
```

Tiene sentido: si el nombre no puede reasignarse, la única oportunidad de darle un valor es la declaración.

2.3 Tres formas de declarar: `const`, `let` y `var`

JavaScript ofrece tres palabras para declarar variables. No son tres maneras intercambiables de escribir lo mismo: difieren en reasignación, alcance, redeclaración e inicialización.

Característica	const	let	var
Permite reasignar	No	Sí	Sí
Exige valor inicial	Sí	No	No
Alcance de bloque	Sí	Sí	No
Permite redeclarar en el mismo alcance	No	No	Sí
Acceso antes de la declaración	Error	Error	undefined
Uso recomendado en código moderno	Por defecto	Cuando habrá reasignación	Evitar salvo código heredado

const : una vinculación que no cambia

const impide que el nombre sea asignado a otro valor:

```
const nombreMateria = "Programación IV";

// nombreMateria = "Laboratorio IV";
// TypeError: no se puede reasignar una constante
```

Esto comunica una decisión: después de inicializar `nombreMateria`, el programa no debería reemplazar su valor. Quien lee el código puede darlo por sentado sin revisar el resto del bloque.

Por esa razón, la regla práctica es:

Usar `const` por defecto. Cambiar a `let` solamente cuando la reasignación forme parte del problema.

Ahora bien, `const` **no vuelve inmutable al valor**. Protege la vinculación, no el interior de un objeto:

```
const alumno = { nombre: "Ana", regular: true };

alumno.regular = false; // permitido: cambia el objeto

// alumno = { nombre: "Luis", regular: true };
// no permitido: intenta reemplazar la vinculación
```

Es una de las confusiones más frecuentes al empezar. El capítulo 12 la desarrolla en profundidad, cuando ya conozcamos objetos y referencias. Por ahora alcanza con retener que, para valores primitivos —que son inmutables—, la distinción no se nota: una constante numérica tampoco podría «modificar el valor por dentro».

let: una vinculación reasignable

`let` se usa cuando el valor asociado al nombre debe cambiar:

```
let intentosRestantes = 3;

intentosRestantes = intentosRestantes - 1;
intentosRestantes -= 1;
```

Los casos típicos son acumuladores, contadores o estados que evolucionan:

```
const notas = [8, 6, 9];
let suma = 0;

for (const nota of notas) {
  suma += nota;
}

const promedio = suma / notas.length;
```

`notas` y `promedio` no necesitan reasignarse, así que se declaran con `const`. `suma` sí cambia durante el recorrido y se declara con `let`.

var: la declaración histórica

Antes de ECMAScript 2015, `var` era la forma habitual de declarar variables. Sigue siendo parte del lenguaje por compatibilidad, pero sus reglas resultan menos locales y favorecen errores que `let` y `const` evitan.

```
if (true) {
  var mensaje = "visible fuera del bloque";
}

console.log(mensaje); // funciona
```

Un bloque delimitado por llaves no crea un alcance nuevo para `var`. Si está dentro de una función, su alcance es toda la función; si se declara en el nivel superior de un script tradicional, tiene alcance global.

Además, `var` permite redeclarar el mismo nombre:

```
var estado = "pendiente";
var estado = "enviado"; // permitido
```

Con `let` o `const`, la redeclaración en el mismo alcance es un error, y eso ayuda a detectar una posible confusión:

```
let estado = "pendiente";
// let estado = "enviado"; // SyntaxError
```

Cómo elegir

Para código moderno:

1. elegir `const` si el nombre no será reasignado;
2. elegir `let` si la reasignación es necesaria;
3. reconocer `var` para poder leer código existente, pero evitarlo en código nuevo.

La elección no depende de si el valor «es importante». Depende de si la **vinculación** necesitará cambiar.

2.4 Alcance: ¿desde dónde puede usarse un nombre?

El **alcance**, o *scope*, es la región del código desde la cual una vinculación resulta accesible.

Alcance global

Una declaración situada fuera de funciones y bloques pertenece al nivel superior del script o módulo:

```
const nombreAplicacion = "Gestión Académica";

function mostrarNombre() {
  console.log(nombreAplicacion);
}
```

El uso excesivo de variables globales dificulta saber qué parte del programa puede modificarlas. Los módulos permiten limitar mejor ese alcance: lo que se declara dentro de un módulo no es visible desde otro salvo que se exporte explícitamente.

Alcance de función

Los parámetros y las variables declaradas dentro de una función no son accesibles desde afuera:

```
function calcularDoble(numero) {
  const resultado = numero * 2;
  return resultado;
}

console.log(calcularDoble(5)); // 10
// console.log(resultado);    // ReferenceError
```

`numero` y `resultado` pertenecen a la ejecución de `calcularDoble`.

Alcance de bloque

Un bloque es una región delimitada por llaves. `let` y `const` respetan ese límite:

```
const usuarioAutenticado = true;

if (usuarioAutenticado) {
  const mensaje = "Bienvenido";
  let segundos = 5;

  console.log(mensaje, segundos); // accesibles
}

// console.log(mensaje); // ReferenceError
// console.log(segundos); // ReferenceError
```

Los bloques de `if`, `for`, `while` y los bloques independientes pueden crear alcances léxicos.

Alcances anidados y sombreado

Un alcance interno puede usar nombres del externo. También puede declarar un nombre igual, que temporalmente **sombrea** al anterior:

```
const mensaje = "mensaje exterior";

{
  const mensaje = "mensaje interior";
  console.log(mensaje); // "mensaje interior"
}

console.log(mensaje); // "mensaje exterior"
```

Son dos vinculaciones distintas que comparten nombre. El sombreado es válido, pero puede reducir la claridad; conviene usarlo solamente cuando los dos significados resulten evidentes.

El capítulo 17 retoma este tema con más profundidad, porque el mecanismo por el cual JavaScript decide qué vinculación corresponde a cada nombre —el **alcance léxico**— es la base de las clausuras.

2.5 Vida de una variable

El alcance responde **dónde** puede usarse un nombre. Su vida responde **durante cuánto tiempo** se conserva la vinculación y, eventualmente, el valor al que hace referencia.

Una vinculación local suele crearse al entrar en su entorno y deja de ser necesaria cuando ese entorno ya no puede alcanzarse. El motor recupera automáticamente la memoria mediante el recolector de basura.

Sin embargo, una función interna puede conservar acceso a variables del entorno donde fue creada. Ese mecanismo se llama **clausura**:

```
function crearContador() {
  let cuenta = 0;

  return function incrementar() {
    cuenta += 1;
    return cuenta;
  };
}

const siguiente = crearContador();

console.log(siguiente()); // 1
console.log(siguiente()); // 2
```

Aunque `crearContador` terminó, la función `incrementar` todavía puede alcanzar `cuenta`. Por eso esa vinculación continúa viva.

No hace falta dominar todavía las clausuras; el capítulo 17 las desarrolla por completo. El ejemplo muestra una idea fundamental: la vida de una variable no depende solamente de la línea donde fue declarada, sino de si algún código todavía puede alcanzarla.

2.6 Declaraciones anticipadas y zona muerta temporal

Antes de ejecutar las instrucciones de un alcance, el motor registra sus declaraciones. A este comportamiento se lo suele llamar **hoisting**. Sin embargo, `var`, `let` y `const` no quedan utilizables de la misma manera.

Con `var`, el nombre existe desde el inicio del alcance y se inicializa con `undefined`:

```
console.log(puntaje); // undefined
var puntaje = 10;
```

Una forma conceptual de imaginarlo:

```
var puntaje;
console.log(puntaje);
puntaje = 10;
```

Con `let` y `const`, la vinculación también pertenece al alcance desde su comienzo, pero no puede usarse antes de la declaración:

```
// console.log(puntaje); // ReferenceError
let puntaje = 10;
```

La región entre el comienzo del alcance y la inicialización se llama **zona muerta temporal**, o *Temporal Dead Zone*. No es un defecto: convierte en error explícito lo que con `var` era un `undefined` silencioso y difícil de rastrear.

La enseñanza práctica es sencilla:

Declarar cada nombre cerca de su primer uso y no depender de accesos anteriores a la declaración.

2.7 Identificadores y convenciones de nombres

El nombre de una variable es un **identificador**. JavaScript establece reglas sintácticas para decidir qué identificadores son válidos; el equipo establece convenciones para decidir cuáles son claros.

Reglas del lenguaje

Un identificador habitual puede contener letras, dígitos, `_` y `$`, pero no puede comenzar con un dígito:

```
const cantidad2 = 10;    // válido
const _resultado = 20;   // válido
const $elemento = null;  // válido

// const 2cantidad = 10; // inválido
```

JavaScript distingue mayúsculas y minúsculas:

```
const nombre = "Ana";
const Nombre = "Luis";

console.log(nombre === Nombre); // false
```

Los identificadores pueden usar muchos caracteres Unicode:

```
const año = 2026;
```

Aunque es válido, muchos equipos prefieren limitar los identificadores a caracteres sin tildes, para facilitar búsquedas, herramientas e intercambio entre distribuciones de teclado. Los ejemplos que siguen adoptan esa convención. Lo importante es acordar una y aplicarla de manera consistente.

Las palabras reservadas del lenguaje no pueden usarse como nombres:

```
// const const = 3;
// const import = "archivo";
// const if = true;
```

Convenciones habituales

En JavaScript se usa normalmente **camelCase** para variables y funciones:

```
const cantidadDeAlumnos = 35;
const fechaDeInscripcion = "2026-08-10";

function calcularPromedio() {
  // ...
}
```

El primer término comienza en minúscula y cada término siguiente comienza en mayúscula.

Otras convenciones frecuentes:

- **PascalCase** para clases: `CuentaBancaria`, `ClientePremium`;
- **MAYUSCULAS_CON_GUIONES_BAJOS** para constantes universales o de configuración: `MAX_INTENTOS`, `IVA_GENERAL`;
- sustantivos para valores: `precio`, `alumno`, `fechaCreacion`;
- verbos para funciones: `calcularTotal`, `buscarCliente`, `enviarMensaje`;
- preguntas para booleanos: `esValido`, `tienePermiso`, `puedeEditar`;
- plurales para colecciones: `alumnos`, `productos`, `mensajes`.

No toda declaración con `const` necesita escribirse en mayúsculas. Una variable local como `const usuario = ...` no representa una constante universal. Las mayúsculas se reservan para valores cuyo significado es fijo en todo el dominio o módulo.

Un nombre debe expresar intención

Compárese:

```
const p = 1200;
const c = 3;
const t = p * c;
```

con:

```
const precioUnitario = 1200;
const cantidad = 3;
const total = precioUnitario * cantidad;
```

La segunda versión no necesita un comentario para explicar la operación.

También conviene incluir la unidad cuando puede haber ambigüedad:

```
const duracionMs = 1500;
const distanciaKm = 12.5;
const precioCentavos = 199_900;
```

Un buen nombre no describe la sintaxis del valor; describe qué representa dentro del problema.

2.8 Los tipos primitivos de JavaScript

Un tipo agrupa valores que comparten una representación conceptual y un conjunto de operaciones válidas. JavaScript tiene siete tipos primitivos y un gran grupo adicional: los objetos.

Tipo	Utilidad principal	Ejemplo	Resultado de typeof
boolean	representar verdad o falsedad	true	"boolean"
number	representar cantidades enteras o decimales	42, 3.14	"number"
bigint	representar enteros de precisión arbitraria	9007199254740993n	"bigint"
string	representar texto	"Hola"	"string"
undefined	señalar que no se asignó o no existe un valor	undefined	"undefined"
null	representar intencionalmente ausencia de valor	null	"object", por una particularidad histórica
symbol	crear identificadores únicos	Symbol("id")	"symbol"

Todo lo que no es un valor primitivo es un **objeto**. Los arreglos, las funciones, las fechas y los objetos literales pertenecen a esa categoría general, que la parte III desarrolla.

Los capítulos 5 a 9 recorren estos tipos uno por uno.

Los primitivos son inmutables

Una operación sobre un valor primitivo no lo modifica: produce otro valor.

```
const nombre = "ana";
const nombreEnMayusculas = nombre.toUpperCase();

console.log(nombre);           // "ana"
console.log(nombreEnMayusculas); // "ANA"
```

Aunque una cadena es primitiva, podemos escribir `nombre.toUpperCase()`. JavaScript permite acceder a métodos mediante objetos envoltorio como `String`, `Number` y `Boolean`. Esa ayuda no significa que debamos construir esos envoltorios manualmente:

```
const correcto = false;
const evitar = new Boolean(false);

console.log(Boolean(correcto)); // false
console.log(Boolean(evitar)); // true: todo objeto es truthy
```

Como regla general, usamos `Boolean()`, `Number()` y `String()` como funciones de conversión, sin `new`.

2.9 Inspeccionar el tipo con `typeof`

El operador `typeof` devuelve una cadena que describe el tipo general de un valor:

```
typeof true;           // "boolean"
typeof 42;             // "number"
typeof 42n;            // "bigint"
typeof "hola";         // "string"
typeof undefined;     // "undefined"
typeof Symbol("id");   // "symbol"
typeof {};             // "object"
typeof [];             // "object"
typeof function () {}; // "function"
typeof null;           // "object", particularidad histórica
```

Dos resultados merecen atención.

`typeof null` devuelve `"object"`. Es un error del diseño original que nunca pudo corregirse sin romper código existente. Para comprobar `null` se usa igualdad estricta:

```
valor === null;
```

`typeof []` devuelve `"object"`, porque un arreglo es un objeto especializado. Para distinguirlo:

```
Array.isArray([]); // true
```

`typeof` sirve bien para clasificar primitivos, pero no alcanza para distinguir entre las distintas clases de objetos.

2.10 Síntesis conceptual

Una variable vincula un nombre con un valor dentro de un alcance. Declarar, inicializar y asignar son operaciones distintas, y confundirlas produce errores sutiles: una declaración sin inicializador deja `undefined`, y una asignación sin declaración previa no crea una vinculación en el alcance actual.

La elección entre `const` y `let` no expresa importancia sino intención: `const` declara que la vinculación no cambiará, y por eso es la elección por defecto. Lo que `const` protege es el nombre, no el contenido del valor; para objetos y arreglos esa distinción tiene consecuencias que el capítulo 12 desarrolla. `var` pertenece al pasado del lenguaje y solo hace falta reconocerlo para leer código antiguo.

El alcance determina desde dónde se ve un nombre, y la vida determina hasta cuándo existe. Los dos conceptos suelen coincidir, pero no siempre: una clausura mantiene viva una vinculación después de que su función terminó. La zona muerta temporal, por su parte, convierte en error lo que con `var` era un valor silenciosamente indefinido.

Finalmente, JavaScript tiene siete tipos primitivos, todos inmutables, y una categoría abierta de objetos. Los capítulos siguientes recorren esos tipos: el próximo empieza por las expresiones y los operadores, porque son la herramienta con la que combinamos valores de cualquier tipo.

Capítulo 3. Expresiones, operadores y precedencia

Los capítulos siguientes recorren los tipos de JavaScript uno por uno, y todos ellos van a mostrar valores combinándose: números que se suman, textos que se concatenan, condiciones que se encadenan. Antes de entrar en cada tipo conviene fijar cómo se leen esas combinaciones.

Este capítulo es breve y sirve de herramienta para los que vienen. Trata una pregunta concreta: cuando una línea tiene varios operadores, ¿en qué orden se aplican?

3.1 Expresiones y operadores

Una **expresión** es una porción de código que produce un valor:

```
2 + 3           // produce 5
precio * cantidad // produce un número
edad >= 18      // produce un booleano
nombre.toUpperCase() // produce una cadena
```

Un **operador** combina, transforma o consulta operandos. En `2 + 3`, `+` es el operador y `2` y `3` son sus operandos.

Conviene distinguir una expresión de una **sentencia**. Una expresión produce un valor; una sentencia ejecuta una acción. `nota >= 6` es una expresión: podemos guardarla en una variable. `if (nota >= 6) { ... }` es una sentencia: dirige la ejecución, pero no produce un valor que podamos asignar. El capítulo 14 vuelve sobre esta distinción al comparar `if` con el operador ternario.

3.2 Precedencia

La precedencia determina cómo se agrupan operadores diferentes cuando no hay paréntesis:

```
const resultado = 2 + 3 * 4;
console.log(resultado); // 14
```

La multiplicación tiene mayor precedencia que la suma, así que la expresión se interpreta como:

```
2 + (3 * 4)
```

Los paréntesis cambian el agrupamiento:

```
const resultado = (2 + 3) * 4;  
console.log(resultado); // 20
```

Más ejemplos, útiles para verificar la intuición:

```
10 % 4 + 2    // 4    → (10 % 4) + 2  
2 ** 3 * 4    // 32   → (2 ** 3) * 4  
-2 ** 2       // SyntaxError: hay que escribir (-2) ** 2 o -(2 ** 2)
```

El último caso es deliberado: el lenguaje prohíbe esa combinación justamente porque su lectura sería ambigua para una persona.

3.3 Asociatividad

Cuando dos operadores tienen la misma precedencia, la asociatividad indica cómo se agrupan:

```
20 - 5 - 2  
// (20 - 5) - 2 = 13: asociatividad de izquierda a derecha  
  
2 ** 3 ** 2  
// 2 ** (3 ** 2) = 512: la exponenciación asocia de derecha a izquierda
```

La exponenciación asocia hacia la derecha porque así funciona en notación matemática: 2^3^2 significa 2 elevado a 3^2 , no $(2^3)^2$.

La asignación también se agrupa desde la derecha:

```
let a;  
let b;  
  
a = b = 5;  
// equivale a: a = (b = 5)
```

Esto funciona porque una asignación es en sí misma una expresión: produce el valor asignado.

3.4 Orden práctico de precedencia

No hace falta memorizar la tabla completa del lenguaje, que tiene más de veinte niveles. Conviene reconocer estos grupos, ordenados aproximadamente de mayor a menor precedencia:

Grupo	Operadores representativos
Agrupamiento y acceso	<code>()</code> , <code>obj.prop</code> , <code>obj[clave]</code> , <code>funcion()</code>
Operadores unarios	<code>!</code> , <code>typeof</code> , <code>+</code> , <code>-</code> , <code>~</code> , <code>++</code> , <code>--</code>
Exponenciación	<code>**</code>
Multiplicativos	<code>*</code> , <code>/</code> , <code>%</code>
Aditivos	<code>+</code> , <code>-</code>
Desplazamiento de bits	<code><<</code> , <code>>></code> , <code>>>></code>
Relacionales	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>in</code> , <code>instanceof</code>
Igualdad	<code>===</code> , <code>!==</code> , <code>==</code> , <code>!=</code>
Bit a bit	<code>&</code> , luego <code>^</code> , luego <code> </code>
AND lógico	<code>&&</code>
OR lógico y coalescencia nula	<code> </code> , <code> </code> , <code>??</code>
Condicional	<code>condicion ? valor1 : valor2</code>
Asignación	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , entre otros

Hay dos consecuencias que vale la pena tener presentes.

La primera: los operadores de comparación tienen mayor precedencia que los lógicos, así que `a > 3 && b < 5` se agrupa como esperamos, sin paréntesis.

La segunda: los operadores bit a bit tienen **menor** precedencia que los de igualdad, lo cual sorprende a quien viene de otros lenguajes. Por eso esto no hace lo que parece:

```
if (permisos & ESCRIBIR !== 0) { // se agrupa como permisos & (ESCRIBIR !== 0)
  // ...
}
```

La versión correcta necesita paréntesis:

```
if ((permisos & ESCRIBIR) !== 0) {
  // ...
}
```

El capítulo 5 vuelve sobre este patrón al tratar las banderas de bits.

3.5 Los paréntesis como documentación

Si la lectura de una expresión exige recordar varios niveles de la tabla, los paréntesis pueden hacer explícita la intención:

```
const puedeIngresar = estaActivo && (esDocente || esAlumno);
```

En este caso los paréntesis no cambian el resultado —&& ya tiene mayor precedencia que ||—, pero comunican la regla de negocio: hay que estar activo, y además ser docente o alumno.

Los paréntesis no son solamente una herramienta matemática. También son documentación. La pregunta útil no es «¿son necesarios?» sino «¿ayudan a quien lea esto dentro de seis meses?».

Hay un caso donde el lenguaje directamente los exige. ?? no puede mezclarse con && o || sin paréntesis:

```
// const valor = entrada || respaldo ?? valorFinal; // SyntaxError
const valor = (entrada || respaldo) ?? valorFinal; // correcto
```

La restricción es deliberada: las dos formas de agrupar producen resultados distintos y el comité prefirió obligarnos a elegir antes que fijar un orden arbitrario.

3.6 Síntesis conceptual

Una expresión produce un valor; una sentencia dirige la ejecución. Los operadores combinan operandos, y cuando aparecen varios en una misma línea, dos reglas deciden el agrupamiento: la precedencia entre operadores distintos y la asociatividad entre operadores iguales.

Conocer los grupos principales de precedencia evita la mayoría de las sorpresas. Las dos excepciones que más cuestan son la exponenciación, que asocia hacia la derecha, y los operadores bit a bit, que tienen menos precedencia que las comparaciones.

Para todo lo demás existe una salida que siempre funciona: escribir los paréntesis. Cuestan dos caracteres y ahorran una consulta a la especificación.

Con esta herramienta a mano, los capítulos siguientes pueden concentrarse en cada tipo. Empezamos por el más simple de todos, el que solo tiene dos valores posibles.

Capítulo 4. El tipo booleano y la lógica de JavaScript

El tipo booleano representa la idea más simple que puede representar un programa: algo es verdadero o es falso. Tiene exactamente dos valores.

```
true;  
false;
```

Podría parecer que un tipo con dos valores no da para un capítulo entero. Pero JavaScript permite usar prácticamente cualquier valor donde espera una condición, y sus operadores lógicos no siempre devuelven booleanos. Esas dos decisiones de diseño explican buena parte de la expresividad del lenguaje y también algunas de sus rarezas.

Para entender el comportamiento completo hay que distinguir cinco cosas: los valores booleanos, los valores *truthy* y *falsy*, la conversión explícita e implícita, los operadores lógicos y la evaluación por cortocircuito. Vamos en ese orden.

Este capítulo enlaza además con el apéndice A: las tablas de verdad de NOT, AND y OR que construimos allí son exactamente las que gobiernan `!`, `&&` y `||`.

4.1 El tipo `boolean`

JavaScript tiene dos valores booleanos: `true` y `false`, siempre en minúsculas. Podemos almacenarlos directamente:

```
let activo = true;  
let terminado = false;  
  
console.log(typeof activo); // "boolean"
```

Pero lo más común es que aparezcan como resultado de una comparación:

```
const edad = 20;  
const esMayor = edad >= 18;  
  
console.log(esMayor); // true
```

Un nombre booleano debería poder leerse como una pregunta:

```
const esMayorDeEdad = edad >= 18;  
const tieneSaldo = saldo > 0;  
const puedeComprar = esMayorDeEdad && tieneSaldo;
```

Esa convención hace que las condiciones se lean casi como texto: «si puede comprar...».

4.2 Los operadores de comparación

Las comparaciones producen booleanos:

Operador	Significado
<	menor que
<=	menor o igual que
>	mayor que
>=	mayor o igual que
===	igualdad estricta
!==	desigualdad estricta
==	igualdad con conversión automática
!=	desigualdad con conversión automática

Para código de aplicación se prefieren `===` y `!==`:

```
console.log(5 === 5);    // true
console.log(5 === "5"); // false

console.log(5 == "5");   // true: convierte antes de comparar
```

La igualdad estricta no intenta convertir tipos. Eso vuelve la expresión más local y predecible: si comparamos contra un número, esperamos realmente un número. La igualdad flexible tiene reglas válidas y deterministas, pero obliga a conocer más información para interpretar una sola línea. El capítulo 9 desarrolla esas reglas.

4.3 Una condición no necesita ser un booleano

En muchos lenguajes, la condición de un `if` debe ser estrictamente un valor booleano. JavaScript es más flexible:

```
const nombre = "Ana";

if (nombre) {
  console.log("Hay un nombre");
}
```

`nombre` no contiene un booleano, sino una cadena. Sin embargo JavaScript lo acepta como condición porque, cuando necesita decidir entre verdadero y falso, convierte implícitamente el valor a booleano.

Conceptualmente podemos imaginar que ocurre esto:

```
if (Boolean(nombre)) {  
  console.log("Hay un nombre");  
}
```

La conversión de una cadena no vacía produce `true`:

```
Boolean("Ana"); // true
```

Por lo tanto, el cuerpo del `if` se ejecuta. Esta conversión automática es una manifestación del sistema de tipos dinámico y de las coerciones implícitas que describimos en el capítulo 1.

Lo mismo ocurre en otras estructuras que necesitan decidir:

```
while (valor) {  
  // se repite mientras valor sea truthy  
}
```

y también con los operadores `!`, `&&` y `||`, y con el operador ternario.

4.4 Valores *truthy* y *falsy*

Cuando JavaScript convierte un valor a booleano, el resultado solo puede ser `true` o `false`. Eso divide todos los valores del lenguaje en dos grupos:

- los que se convierten en `false` se llaman ***falsy***;
- los que se convierten en `true` se llaman ***truthy***.

Es importante notar que un valor *truthy* **no es** el booleano `true`:

```
"Hola" === true; // false  
Boolean("Hola"); // true
```

Por eso decimos que `"Hola"` es *truthy*, no que sea `true`.

La lista completa de valores *falsy*

Son pocos, y conviene conocerlos de memoria porque todo lo demás es *truthy*:

```
false
0
-0
0n      // el cero de bigint
""      // cadena vacía
null
undefined
NaN
```

Podemos comprobarlo:

```
Boolean(false);    // false
Boolean(0);         // false
Boolean("");        // false
Boolean(null);      // false
Boolean(undefined); // false
Boolean(NaN);       // false
```

Esto permite escribir:

```
const nombre = "";

if (nombre) {
  console.log("Hay un nombre");
}
```

Como la cadena vacía es *falsy*, el cuerpo del `if` no se ejecuta.

Todo lo demás es *truthy*

```
Boolean(1);        // true
Boolean(-1);        // true
Boolean(3.14);      // true
Boolean("Hola");    // true
Boolean("false");   // true
Boolean("0");        // true
Boolean([]);         // true
Boolean({});         // true
```

Los últimos cuatro casos suelen sorprender, y vale la pena detenerse.

`Boolean("false")` es `true` porque la conversión observa si la cadena está vacía, no qué palabra contiene. `"false"` es una cadena no vacía, punto. Lo mismo vale para `"0"`: no es el número cero, es un texto de un carácter.

Los arreglos y los objetos son *truthy* incluso cuando están vacíos, porque son objetos y **todo objeto es *truthy***. Esto es una fuente frecuente de errores:

```
const carrito = [];

if (carrito) {
  console.log("Se ejecuta siempre"); // aunque el carrito esté vacío
}

if (carrito.length > 0) {
  console.log("Se ejecuta solo si hay elementos"); // la comprobación correcta
}
```

Los navegadores conservan además una excepción histórica muy particular, `document.all`, que se comporta como *falsey* pese a ser un objeto. Existe por compatibilidad con código antiguo y no es un valor con el que debamos diseñar nada.

4.5 Conversión explícita a booleano

Hay dos formas idiomáticas de convertir un valor a booleano.

La función `Boolean()`

```
const tieneNombre = Boolean(nombre);
```

Es la forma más explícita y la más clara para quien está aprendiendo. Se usa sin `new`: `new Boolean(false)` construye un objeto envoltorio que, por ser objeto, resulta *truthy*, exactamente lo contrario de lo que se pretendía.

La doble negación `!!`

```
const tieneCorreo = !!correo;
```

Para entenderla conviene recordar que `!` es el operador de negación lógica:

```
!true; // false
!false; // true
```

`!` convierte primero su operando a booleano y después niega el resultado. Así:

```
!"Hola";
```

equivale conceptualmente a:

```
!Boolean("Hola");
```

Como `"Hola"` es *truthy*, se convierte en `true` y se niega:

```
!true; // false
```

Si volvemos a negar, obtenemos el booleano original:

```
!!"Hola"; // true
```

Por eso `!!valor` es una abreviatura de `Boolean(valor)`:

```
!!"Hola"; // true
!!"";      // false
!!42;      // true
!!0;       // false
!!null;    // false
```

Las dos formas son correctas. `Boolean()` expresa mejor la intención; `!!` es más breve y muy frecuente en código existente, así que conviene reconocerla.

4.6 Los operadores lógicos

Operador	Nombre	Idea
<code>!a</code>	NOT	niega la condición
<code>a && b</code>	AND	ambas condiciones deben ser verdaderas
<code>a b</code>	OR	al menos una condición debe ser verdadera

Cuando los operandos son booleanos, sus tablas de verdad son las del apéndice A:

a	b	!a	a && b	a b
false	false	true	false	false
false	true	true	false	true
true	false	false	false	true
true	true	false	true	true

Un ejemplo típico:

```
const tieneUsuario = true;
const tieneClave = true;

const puedeIniciarSesion = tieneUsuario && tieneClave;
const necesitaAyuda = !puedeIniciarSesion;
```

En cuanto a precedencia, `!` es mayor que `&&`, y `&&` es mayor que `||`:


```
const permitido = esAdministrador || esDocente && estaActivo;

// se interpreta como:
const permitidoExplicito = esAdministrador || (esDocente && estaActivo);
```

Aunque el resultado coincida, los paréntesis comunican mejor la regla de negocio.

4.7 Evaluación por cortocircuito

Aquí empieza lo interesante. `&&` y `||` evalúan de izquierda a derecha y pueden **evitar evaluar el operando derecho**.

Consideremos:

```
A && B
```

Para que el AND sea verdadero, `A` debe ser *truthy*. Si `A` es *falsy*, ya conocemos el resultado: no hace falta mirar `B`.

```
function prueba() {
  console.log("Se ejecutó");
  return true;
}

false && prueba();
```

No se imprime nada. JavaScript encontró un valor *falsy* y sabe que un AND ya no puede resultar verdadero.

Con `||` ocurre lo simétrico. Para que un OR sea verdadero alcanza con que `A` sea *truthy*:

```
true || prueba();
```

Tampoco se imprime nada.

Esto se llama **evaluación por cortocircuito**, y no es solo una optimización: es un comportamiento observable del que depende una gran cantidad de código idiomático. Este patrón, por ejemplo, funciona precisamente por eso:

```
if (usuario && usuario.activo) {
  // ...
}
```

Si `usuario` es `null`, el segundo operando nunca se evalúa y no se produce el `TypeError` que causaría leer una propiedad de `null`.

4.8 && y || devuelven un operando, no un booleano

La segunda decisión de diseño importante: estos operadores **no convierten necesariamente el resultado a true o false**. Devuelven directamente el valor del operando que determinó el resultado.

&& devuelve el primer valor *falsy*

```
10 && 20;  
// 20
```

10 es *truthy*, así que todavía no se puede determinar el resultado. Se evalúa 20. Como no quedan más operandos, 20 es el resultado.

Con una expresión más larga:

```
10 && 0 && 30;  
// 0
```

La evaluación va de izquierda a derecha:

1. 10 es *truthy*: continuar.
2. 0 es *falsy*: detenerse y devolverlo.
3. 30 nunca se evalúa.

Y si ninguno es *falsy*:

```
"Juan" && 25 && "Hola";  
// "Hola"
```

A && B && C busca el primer valor falsy. Si no encuentra ninguno, devuelve el último operando.

|| devuelve el primer valor *truthy*

```
0 || 20;  
// 20
```

0 es *falsy*, así que JavaScript continúa. 20 es *truthy* y se convierte en el resultado.

```
"" || null || "Juan" || "Pedro";  
// "Juan"
```

La evaluación:

1. "" es *falsy*: continuar.
2. null es *falsy*: continuar.

3. "Juan" es *truthy*: detenerse y devolverlo.
4. "Pedro" nunca se evalúa.

Si ninguno es *truthy*, se devuelve el último:

```
0 || "" || null;  
// null
```

La simetría

Una forma sencilla de recordar ambos operadores:

- `&&` busca algo que haga **falsa** la expresión;
- `||` busca algo que haga **verdadera** la expresión.

Por eso `&&` devuelve el primer *falsy* y `||` el primer *truthy*. Si no encuentran un valor que determine el resultado por anticipado, devuelven el último operando.

4.9 Un modelo mental

Cuando JavaScript encuentra `A && B`, podemos imaginar este comportamiento:

```
if (!A) {  
  return A;  
} else {  
  return B;  
}
```

Y para `A || B`:

```
if (A) {  
  return A;  
} else {  
  return B;  
}
```

No es literalmente la implementación interna, pero es un modelo mental excelente que predice correctamente todos los casos:

```
0 && 100; // 0  
0 || 100; // 100
```

4.10 Por qué esto resulta útil

Que devuelvan el operando permite usar los operadores lógicos para **seleccionar valores**, no solo para construir condiciones.

El patrón tradicional de valor alternativo:

```
const nombre = nombreIngresado || "Anónimo";
```

Si el usuario ingresó algo:

```
"Ana" || "Anónimo"; // "Ana"
```

Si dejó el campo vacío:

```
" " || "Anónimo"; // "Anónimo"
```

Y el patrón de acceso condicional:

```
const nombre = usuario && usuario.nombre;
```

Si `usuario` es *falsy*, se detiene y devuelve ese valor. Si es *truthy*, continúa y devuelve `usuario.nombre`.

Este segundo patrón conviene reemplazarlo hoy por el encadenamiento opcional, que expresa la intención de manera más directa y no devuelve un valor de tipo inesperado:

```
const nombre = usuario?.nombre;
```

El capítulo 11 lo desarrolla.

4.11 El operador de coalescencia nula

El patrón `||` para valores alternativos tiene un problema: descarta **cualquier** valor *falsy*, incluidos algunos que podían ser perfectamente válidos.

```
const cantidad = 0;

console.log(cantidad || 10); // 10 – pero cero era un valor legítimo
```

Si el usuario pidió cero unidades, `||` lo reemplaza silenciosamente por diez.

El operador de coalescencia nula, `??`, resuelve exactamente ese caso: solo usa el valor alternativo cuando el primero es `null` o `undefined`.

```
console.log(cantidad ?? 10); // 0
```

La regla para elegir:

- `||` cuando cualquier valor *falsy* deba ser reemplazado;
- `??` cuando `0`, `false` o `""` sean valores válidos y solo la ausencia deba reemplazarse.

En la práctica, `??` suele ser la elección correcta para valores por defecto de configuración, cantidades y campos de formulario.

Como vimos en el capítulo 3, `??` no puede mezclarse con `&&` o `||` sin paréntesis:

```
const valor = (entrada || respaldo) ?? valorFinal;
```

El capítulo 7 vuelve sobre `??` al tratar `null` y `undefined` como las dos formas de ausencia.

4.12 Síntesis conceptual

Un booleano solo puede contener `true` o `false`. Sin embargo, JavaScript admite cualquier valor en un contexto lógico gracias a la conversión implícita, y eso divide todos los valores del lenguaje en *falsy* —una lista corta y memorizable— y *truthy* —todo lo demás, incluidos los arreglos y los objetos vacíos—.

La conversión explícita se escribe `Boolean(valor)` o `!!valor`. La primera forma comunica mejor la intención; la segunda es idiomática y hay que saber leerla.

Los operadores `&&` y `||` tienen dos propiedades que los distinguen de sus equivalentes en otros lenguajes. Evalúan con cortocircuito, así que el operando derecho puede no ejecutarse nunca, y ese comportamiento es la base de patrones muy usados. Y devuelven directamente uno de sus operandos en lugar de un booleano, lo que los convierte en herramientas de selección de valores además de operadores lógicos:

```
"Hola" && 42;    // 42
0 && 42;         // 0
"Hola" || 42;    // "Hola"
0 || 42;         // 42
```

Finalmente, `??` cubre el caso que `||` resuelve mal: cuando `0`, `false` o la cadena vacía son respuestas legítimas y solo la ausencia real debe reemplazarse.

El capítulo siguiente pasa al tipo con el que más operaciones vamos a hacer, y que tiene sus propias sorpresas.

Capítulo 5. Números: `number` y `bigint`

Casi todos los programas cuentan, miden, suman o comparan. JavaScript ofrece dos tipos numéricos primitivos para hacerlo: `number`, que cubre la enorme mayoría de los casos, y `bigint`, para enteros cuya magnitud excede lo que `number` puede representar con exactitud.

A los dos se suma un tema que a primera vista parece lejano y resulta muy útil: las operaciones que trabajan sobre los bits individuales de un entero.

5.1 Un solo tipo para todos los números

En JavaScript **no hay tipos separados como `int`, `float` o `double`** en el nivel del lenguaje. Tanto los enteros como los números con decimales son valores de tipo `number`:

```
const edad = 25;
const temperatura = 36.5;
const saldo = -1500;

console.log(typeof edad);           // "number"
console.log(typeof temperatura);    // "number"
```

Internamente, los valores `number` se representan mediante el formato de punto flotante **IEEE 754 de doble precisión**, con 64 bits. Ese formato reserva bits para el signo, el exponente y la mantisa, y con ellos representa un rango enorme de valores, incluidos los decimales, los infinitos y un valor especial que indica «esto no es un número».

Esa decisión de diseño simplifica el lenguaje —una sola categoría numérica— pero tiene consecuencias que conviene conocer desde el principio, y a las que llegaremos en la sección 5.6.

5.2 Escribir números en el código

Un **literal** es la forma de escribir directamente un valor dentro del código. JavaScript admite varias notaciones.

Decimal y notación científica

```
const entero = 123;
const conDecimales = 3.1416;
const negativo = -45;

const mil = 1e3;           // 1000
const doscientos = 2.5e2; // 250
const milesima = 3e-3;    // 0.003
```

Otras bases

Los prefijos indican en qué base está escrito el literal:

```
const binario = 0b11111111; // 255
const octal = 0o377;        // 255
const hexadecimal = 0xff;   // 255
```

Un ejemplo que muestra el punto con claridad:

```
const a = 42;
const b = 0b101010;
const c = 0o52;
const d = 0x2A;

console.log(a, b, c, d);
// 42 42 42 42
```

Los cuatro contienen **el mismo valor**. Lo único que cambia es cómo se escribió en el programa. Una vez creado, el valor no «recuerda» su notación de origen: es el número cuarenta y dos, y nada más. Es exactamente la distinción entre valor y representación que vimos en la sección A.12.

Separadores de miles

El guion bajo mejora la lectura y no forma parte del valor:

```
const legible = 1_000_000; // 1000000
const precio = 199_900;
```

5.3 Operadores aritméticos

Operador	Operación	Ejemplo	Resultado
+	suma	7 + 3	10
-	resta	7 - 3	4
*	multiplicación	7 * 3	21
/	división	7 / 2	3.5
%	resto	7 % 3	1
**	potencia	2 ** 3	8
++	incremento	contador++	suma uno
--	decremento	contador--	resta uno

Conviene notar que `/` produce un resultado decimal: no hay división entera implícita. Para obtenerla se usa `Math.trunc(a / b)`.

El resto `%` permite, entre otras cosas, reconocer múltiplos:

```
const numero = 14;
const esPar = numero % 2 === 0;
```

Los operadores de asignación combinada expresan actualización:

```
let total = 100;

total += 20; // total = total + 20
total *= 2;  // total = total * 2
```

Prefijo y sufijo

`++` y `--` tienen dos variantes que difieren en el valor que produce la expresión:

```
let a = 5;
const antes = a++; // antes vale 5; luego a vale 6

let b = 5;
const despues = ++b; // primero b vale 6; despues vale 6
```

`a++` devuelve el valor anterior y después incrementa. `++b` incrementa y después devuelve el valor nuevo.

Conviene usarlos cuando su propósito de contador sea evidente y evitar mezclarlos dentro de expresiones más grandes, donde la diferencia entre las dos variantes se vuelve difícil de leer.

5.4 Comparar números

```
const edad = 20;

console.log(edad >= 18); // true
console.log(edad < 65); // true
console.log(edad === 20); // true
```

Si los operandos tienen tipos distintos, algunos operadores comparativos realizan conversiones. La práctica recomendada es convertir las entradas en la frontera del sistema y comparar siempre valores del mismo tipo. El capítulo 9 desarrolla las reglas de conversión.

5.5 Valores numéricos especiales

```
console.log(1 / 0); // Infinity
console.log(-1 / 0); // -Infinity
console.log(Number("x")); // NaN
```

Que `1 / 0` produzca `Infinity` en lugar de lanzar un error forma parte del estándar IEEE 754. El capítulo 15 muestra cómo imponer una regla propia cuando dividir por cero debe considerarse inválido en nuestro dominio.

`NaN` significa *Not a Number*, pero su tipo sigue siendo `number`:

```
console.log(typeof NaN); // "number"
```

Es el resultado de una operación aritmética que no puede producir un número válido. Y tiene una propiedad que sorprende: **no es igual a sí mismo**.

```
console.log(NaN === NaN); // false
console.log(Number.isNaN(NaN)); // true
```

La razón está en el estándar: `NaN` representa «un resultado indeterminado», y dos resultados indeterminados no tienen por qué ser el mismo. La consecuencia práctica es que nunca hay que comparar contra `NaN` con `===`. Se usa `Number.isNaN()`.

Existe también una función global `isNaN()`, más antigua, que convierte su argumento antes de comprobar y por eso da resultados poco intuitivos — `isNaN("hola")` devuelve `true` —. `Number.isNaN()` no convierte y es la que conviene usar.

5.6 Precisión y enteros seguros

Los decimales se representan en binario, y algunos no pueden almacenarse exactamente:

```
console.log(0.1 + 0.2);           // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

Este resultado es probablemente el más famoso de JavaScript, pero **no es un problema de JavaScript**: es una consecuencia de representar números reales en binario con una cantidad finita de bits. Ocurre igual en C, Java, Python y cualquier lenguaje que use IEEE 754.

La razón es la misma por la que un tercio no tiene representación decimal finita: `0.1` no tiene representación binaria finita. El valor almacenado es el binario más cercano, y esa diferencia mínima se hace visible al sumar.

Para cantidades monetarias, la solución habitual es trabajar con la unidad mínima como entero:

```
const precioCentavos = 199_900; // $1999,00
```

Los cálculos se hacen en centavos y la división por cien se aplica solo al mostrar.

El rango de enteros exactos

`number` representa exactamente los enteros entre `Number.MIN_SAFE_INTEGER` y `Number.MAX_SAFE_INTEGER`:

```
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

Ese número es $2^{53} - 1$, porque la mantisa de un `double` tiene 53 bits de precisión efectiva. Fuera de ese rango, dos enteros matemáticamente distintos pueden compartir la misma representación:

```
console.log(9007199254740993 === 9007199254740992); // true
```

Ese es el problema que resuelve `bigint`, en la sección 5.11.

5.7 Funciones y métodos numéricos

El objeto estándar `Number` ofrece comprobaciones y conversiones:

```
Number.isFinite(25);           // true
Number.isInteger(25.0);        // true
Number.isSafeInteger(25);      // true
Number.isNaN(NaN);             // true

Number.parseInt("42", 10);      // 42
Number.parseFloat("3.14");      // 3.14
```

Conviene indicar siempre la base al usar `parseInt`:

```
Number.parseInt("101", 2); // 5
Number.parseInt("101", 10); // 101
```

Sin base explícita, el comportamiento depende del formato del texto, y esa dependencia produce errores difíciles de rastrear cuando la cadena viene de una entrada del usuario.

Hay una diferencia conceptual importante entre `Number()` y `parseInt()`:

```
Number("123px"); // NaN
Number.parseInt("123px", 10); // 123
```

`Number()` exige que **toda** la cadena represente un número. `parseInt()` lee un prefijo numérico y descarta el resto. Ninguna es mejor: se elige según lo que la regla del problema deba aceptar. Si `"123px"` es una entrada inválida, `Number()` lo detecta; si es un formato conocido del que queremos extraer la parte numérica, `parseInt()` sirve.

Formato para mostrar

Los valores numéricos exponen métodos de formato:

```
const precio = 1234.567;

precio.toFixed(2); // "1234.57"
precio.toPrecision(4); // "1235"
precio.toString(16); // "4d2.9126e978d5"
```

Todos **devuelven cadenas**. Formatear no modifica ni reemplaza el valor numérico:

```
const formateado = precio.toFixed(2);
typeof formateado; // "string"
```

Es un error frecuente encadenar operaciones aritméticas después de `toFixed()` y descubrir que el resultado se concatenó en lugar de sumarse.

El objeto Math

Reúne las operaciones matemáticas:

```
Math.abs(-8); // 8
Math.round(3.6); // 4
Math.floor(3.9); // 3
Math.ceil(3.1); // 4
Math.trunc(3.9); // 3
Math.min(4, 2, 9); // 2
Math.max(4, 2, 9); // 9
Math.sqrt(81); // 9
Math.pow(2, 3); // 8, equivalente a 2 ** 3
Math.random(); // valor desde 0 inclusive hasta 1 exclusivo
```

Conviene distinguir `Math.floor()` de `Math.trunc()` para números negativos: `floor` redondea hacia abajo —hacia menos infinito— y `trunc` descarta la parte decimal:

```
Math.floor(-12.9); // -13
Math.trunc(-12.9); // -12
```

Una advertencia sobre `Math.random()`: sirve para simulaciones, juegos y datos de prueba, pero **no genera aleatoriedad apta para seguridad**. Para contraseñas, tokens o identificadores impredecibles hay que usar las APIs criptográficas del entorno, como `crypto.getRandomValues()`.

5.8 Operaciones bit a bit

Los operadores **bit a bit** permiten trabajar con los bits individuales de un entero. Son la puerta de entrada al nivel que estudiamos en el apéndice A.

Operador	Operación
<code>a & b</code>	AND bit a bit
<code>a b</code>	OR bit a bit
<code>a ^ b</code>	XOR bit a bit
<code>~a</code>	NOT bit a bit
<code>a << n</code>	desplazamiento a la izquierda
<code>a >> n</code>	desplazamiento a la derecha conservando el signo
<code>a >>> n</code>	desplazamiento a la derecha rellenando con ceros

La operación se aplica bit a bit, en paralelo, sobre las posiciones correspondientes de ambos operandos.

Con 5 y 3:

```
5 = 0101
3 = 0011

5 & 3 = 0001 = 1
5 | 3 = 0111 = 7
5 ^ 3 = 0110 = 6
```

```
console.log(5 & 3); // 1
console.log(5 | 3); // 7
console.log(5 ^ 3); // 6
```

Vale la pena verificar el AND columna por columna:

```
  0101
& 0011
-----
  0001
```

Solo la última posición tiene un `1` en ambos operandos.

Los operandos se convierten a enteros de 32 bits

Aunque `number` usa punto flotante de 64 bits, **la mayoría de las operaciones bit a bit convierten sus operandos a enteros con signo de 32 bits** antes de operar:

```
5.8 | 0;
// 5
```

Conceptualmente:

```
5.8
↓ conversión a entero de 32 bits
00000000 00000000 00000000 00000101
```

De ahí viene el viejo truco de escribir `x | 0` para forzar un valor al dominio de los enteros. Conviene conocerlo para leer código existente, pero no usarlo:

```
Math.floor(-12.9); // -13
-12.9 | 0;         // -12
Math.trunc(-12.9); // -12
```

`| 0` trunca hacia cero, no redondea hacia abajo, y además falla silenciosamente fuera del rango de 32 bits. Para truncar hay una función que dice lo que hace:

```
Math.trunc(12.9); // 12
```

La limitación a 32 bits es la razón por la que los operadores bit a bit no sirven para procesar enteros grandes. Para eso está `bigint`, que sí los soporta sin ese recorte.

5.9 Desplazamientos

`<<` mueve los bits hacia la izquierda:

```
const x = 0b00000101; // 5
x << 1;
```

```
00000101    5
  ↓
00001010    10
```

Desplazar n posiciones hacia la izquierda equivale conceptualmente a multiplicar por 2^n , siempre dentro del rango de 32 bits:

```
5 << 1; // 10
5 << 2; // 20
```

Hacia la derecha:

```
20 >> 1; // 10
20 >> 2; // 5
```

10100	20
01010	10
00101	5

`>>` conserva el signo, replicando el bit más significativo. `>>>` rellena con ceros, lo que produce resultados muy distintos para números negativos:

```
-8 >> 1; // -4
-8 >>> 1; // 2147483644
```

5.10 Los bits como conjunto de opciones

Una aplicación clásica y todavía muy usada consiste en representar **varias opciones simultáneas dentro de un único número**. Cada bit es una opción independiente.

Supongamos permisos de usuario:

```
const LEER      = 1 << 0; // 0001
const ESCRIBIR  = 1 << 1; // 0010
const BORRAR    = 1 << 2; // 0100
const ADMIN     = 1 << 3; // 1000
```

`1 << n` coloca un 1 en la posición n . Cada constante ocupa una posición distinta:

LEER	0001
ESCRIBIR	0010
BORRAR	0100
ADMIN	1000

Activar opciones con OR

```
let permisos = LEER | ESCRIBIR;
```

```
0001    LEER
| 0010    ESCRIBIR
-----
0011
```

Podemos agregar otra opción sin perder las anteriores:

```
permisos |= BORRAR;
```

Ahora `permisos` vale `0111`: tiene tres opciones activas simultáneamente en un solo número.

Consultar una opción con AND

```
if ((permisos & ESCRIBIR) !== 0) {
  console.log("Puede escribir");
}
```

AND funciona como una **máscara**: deja pasar únicamente el bit que queremos consultar y pone en cero todos los demás. Si el resultado no es cero, ese bit estaba encendido.

Los paréntesis alrededor de `permisos & ESCRIBIR` son obligatorios por la precedencia que vimos en la sección 3.4: `&` tiene menos precedencia que `!==`.

Apagar una opción con AND y NOT

```
permisos &= ~BORRAR;
```

`~BORRAR` invierte todos los bits: donde `BORRAR` tenía su `1`, ahora hay un `0`, y en todas las demás posiciones hay `1`. Al hacer AND con esa máscara, el bit elegido se apaga y el resto queda intacto:

```
0111
& 1011
-----
0011
```

Quedan activos `LEER` y `ESCRIBIR`.

Alternar una opción con XOR

```
permisos ^= ADMIN;
```

XOR invierte el bit correspondiente: si `ADMIN` estaba apagado, se enciende; si estaba encendido, se apaga.

El patrón completo

```
const LEER      = 1 << 0;
const ESCRIBIR  = 1 << 1;
const BORRAR    = 1 << 2;
const ADMIN     = 1 << 3;

let permisos = 0;

// activar
permisos |= LEER;
permisos |= ESCRIBIR;

// consultar
if ((permisos & LEER) !== 0) {
  console.log("Puede leer");
}

// apagar
permisos &= ~ESCRIBIR;

// alternar
permisos ^= ADMIN;
```

Un solo entero representa muchas combinaciones:

```
0000  ninguna
0001  LEER
0011  LEER + ESCRIBIR
0101  LEER + BORRAR
1011  LEER + ESCRIBIR + ADMIN
1111  todas
```

Una forma práctica de recordar los operadores en este contexto:

```
|      prender bits
&      consultar o filtrar bits
& ~    apagar bits
^      alternar bits
<<     colocar un bit en una posición
>>     desplazar bits
```

La idea de fondo es que dejamos de pensar el número solamente como una cantidad y empezamos a usar **cada uno de sus bits como información independiente**. Es exactamente la codificación de la sección A.2, aplicada dentro de un programa.

Este patrón aparece en APIs de sistemas operativos, formatos de archivo, protocolos de red y bibliotecas gráficas. En código de aplicación moderno suele ser más claro usar un Set de cadenas o un objeto de banderas booleanas —el capítulo 13 muestra esa alternativa—, pero hay que saber leerlo.

5.11 El tipo `bigint`

`bigint` representa enteros de precisión arbitraria: no tiene el límite de `Number.MAX_SAFE_INTEGER`. Su literal termina con `n`:

```
const grande = 9_007_199_254_740_993n;  
const otro = BigInt("9007199254740993");
```

Compárese con el problema de la sección 5.6:

```
9007199254740993 === 9007199254740992; // true - con number  
9007199254740993n === 9007199254740992n; // false - con bigint
```

`bigint` **no representa decimales**. La división descarta la parte fraccionaria:

```
const cociente = 5n / 2n;  
console.log(cociente); // 2n
```

Y no se mezcla directamente con `number` en operaciones aritméticas:

```
// 10n + 2; // TypeError  
10n + 2n; // 12n
```

La restricción es deliberada. Mezclar los dos tipos exigiría decidir si el resultado pierde precisión o cambia de tipo, y ninguna de las dos opciones es segura por defecto. La conversión debe ser explícita:

```
BigInt(10); // 10n  
Number(10n); // 10
```

Al convertir de `bigint` a `number` hay que considerar si el valor cabe en el rango de enteros exactos; si no cabe, se pierde precisión silenciosamente.

`bigint` soporta la mayoría de los operadores aritméticos y de bits, con una excepción: no admite el desplazamiento sin signo `>>>`, porque ese operador supone un ancho fijo de 32 bits que `bigint` no tiene.

En la práctica, `bigint` es útil para identificadores de sistemas externos, cálculos criptográficos, marcas de tiempo de alta resolución y aritmética exacta sobre enteros grandes. Para todo lo demás, `number` alcanza.

5.12 Síntesis conceptual

JavaScript tiene un único tipo numérico de uso general, `number`, representado en punto flotante IEEE 754 de doble precisión. Esa uniformidad simplifica el lenguaje y trae dos consecuencias que hay que conocer: algunos decimales no tienen representación exacta,

y los enteros solo son exactos hasta `Number.MAX_SAFE_INTEGER`. Ninguna de las dos es un defecto de JavaScript; son propiedades del formato.

Las notaciones —decimal, binaria, octal, hexadecimal, científica— son formas de escribir el mismo valor. Una vez creado, el número no conserva memoria de cómo fue escrito.

`NaN` merece atención especial: es de tipo `number`, aparece cuando una operación aritmética no puede producir un resultado válido, y no es igual a sí mismo. Se detecta con `Number.isNaN()`.

Los métodos de formato como `toFixed()` producen cadenas, no números. Formatear es una operación de presentación, y el resultado ya no sirve para seguir calculando.

Los operadores bit a bit convierten sus operandos a enteros de 32 bits y permiten tratar un número como un conjunto de banderas independientes: `|` para activar, `&` para consultar, `& ~` para apagar y `^` para alternar. Es el mismo mecanismo de codificación del apéndice A, ahora al alcance de un programa.

Finalmente, `bigint` cubre el caso que `number` no puede: enteros exactos de cualquier magnitud, a cambio de no admitir decimales ni mezclarse automáticamente con `number`.

El capítulo siguiente pasa al otro gran tipo primitivo, el que representa texto, y descubre que allí la relación entre lo que vemos y lo que el programa almacena es todavía menos directa.

Capítulo 6. Cadenas de texto y Unicode

El tipo `string` representa texto. Es probablemente el tipo con el que más vamos a trabajar: nombres, mensajes, identificadores, contenido de archivos, respuestas de servidores y entradas de formularios llegan todos como texto.

Detrás de esa aparente simpleza hay una pregunta que parece trivial y no lo es: **qué es exactamente un carácter**. La respuesta explica por qué `"😊".length` da 2, por qué un acento puede ocupar dos posiciones y por qué ordenar palabras en español requiere más que comparar con `<`.

6.1 Escribir una cadena

JavaScript ofrece tres formas de escribir literales de cadena.

Comillas simples y dobles

```
const simple = 'Hola';
const doble = "Hola";
```

No hay diferencia de comportamiento entre ambas. Se elige una convención para el proyecto y se usa la otra cuando evita escapes innecesarios:

```
const frase1 = "La materia se llama 'Programación IV'";
const frase2 = 'El alumno respondió "presente"';
```

Secuencias de escape

La barra invertida permite incluir caracteres que de otro modo serían difíciles de escribir:

```
const mensaje = "Ella dijo: \"Hola\"";
const ruta = "C:\\documentos\\clase";
const dosLineas = "Primera línea\nSegunda línea";
const tabulado = "Nombre\tNota";
```

Las secuencias más frecuentes:

Secuencia	Significado
\n	nueva línea
\t	tabulación
\\	barra invertida
\"	comilla doble
\'	comilla simple
\u{1F600}	carácter Unicode por su punto de código

Plantillas literales

Las comillas invertidas —*backticks*— crean plantillas literales, que agregan dos capacidades importantes.

La primera es la **interpolación**:

```
const nombre = "Ana";
const nota = 9;

const resumen = `${nombre} obtuvo ${nota} puntos`;
```

La construcción `${expresion}` evalúa la expresión, convierte su resultado a texto y lo inserta. Acepta cualquier expresión, no solo variables:

```
const precio = 1200;
const cantidad = 3;

const detalle = `Total: ${precio * cantidad}`; // "Total: $3600"
const estado = `${nota >= 6 ? "Aprobado" : "Desaprobado"}`;
const conteo = `${productos.length} productos`;
```

Suele ser más legible que concatenar muchas partes:

```
const concatenado = nombre + " obtuvo " + nota + " puntos";
const interpolado = `${nombre} obtuvo ${nota} puntos`;
```

La segunda capacidad es que pueden ocupar **varias líneas**:

```
const correo = `Hola, ${nombre}:

Tu inscripción fue confirmada.
Saludos.`;
```

Los saltos y espacios escritos entre los backticks forman parte de la cadena resultante, incluida la sangría. Es algo a tener en cuenta al usar plantillas dentro de un bloque indentado.

6.2 Las cadenas son inmutables

No podemos cambiar un carácter dentro de una cadena existente:

```
const lenguaje = "javascript";

lenguaje[0] = "J"; // no hace nada: la asignación se ignora

const corregido = "J" + lenguaje.slice(1);
console.log(corregido); // "Javascript"
```

Todos los métodos de cadena crean cadenas nuevas y dejan intacta la original. Esto es coherente con lo que vimos en la sección 2.8: los primitivos son inmutables.

Tiene una consecuencia práctica: construir una cadena larga concatenando dentro de un bucle crea muchas cadenas intermedias. Para textos grandes suele ser mejor acumular las partes en un arreglo y unir las al final con `join()`.

6.3 Operadores sobre cadenas

+ concatena cuando la operación involucra una cadena:

```
const nombreCompleto = "Ana" + " " + "Pérez";
```

+= concatena y reasigna:

```
let mensaje = "Hola";
mensaje += ", Ana";
```

Y las comparaciones funcionan, pero de una manera que conviene mirar de cerca:

```
console.log("Ana" === "ana"); // false
console.log("20" < "3");      // true
```

La primera línea no sorprende: las cadenas distinguen mayúsculas de minúsculas. La segunda sí. `"20" < "3"` es `true` porque la comparación es **lexicográfica**, carácter por carácter: compara `"2"` con `"3"`, encuentra que `"2"` viene antes, y ya tiene el resultado. No compara veinte con tres.

Volveremos sobre este punto en la sección 6.10, donde tiene consecuencias concretas.

6.4 Métodos habituales

```
const materia = "  Programación IV  ";

materia.length;           // 19, incluye espacios
materia.trim();           // "Programación IV"
materia.toUpperCase();     // "  PROGRAMACIÓN IV  "
materia.toLowerCase();    // "  programación iv  "
materia.includes("IV");    // true
materia.startsWith("Programa"); // false: comienza con espacios
materia.endsWith("IV");   // false: termina con espacios
```

Para extraer y buscar partes:

```
const codigo = "TUP-P4-2026";

codigo[0];                // "T"
codigo.at(-1);            // "6"
codigo.indexOf("P4");     // 4
codigo.slice(0, 3);       // "TUP"
codigo.substring(4, 6);   // "P4"
```

`at()` admite índices negativos para contar desde el final; los corchetes no. `slice()` también acepta índices negativos, y `substring()` no: intercambia sus argumentos si el primero es mayor que el segundo. Por esa diferencia de comportamiento, `slice()` suele ser la opción más predecible.

Para reemplazar y dividir:

```
"hola mundo".replace("mundo", "curso"); // "hola curso"
"1-2-3".replaceAll("-", "/");           // "1/2/3"
"uno, dos, tres".split(", ");            // ["uno", "dos", "tres"]
```

`replace()` reemplaza solamente la primera coincidencia textual; `replaceAll()` las reemplaza todas. Ambos aceptan expresiones regulares, que el capítulo 20 desarrolla.

Los métodos pueden encadenarse porque cada uno devuelve un valor nuevo:

```
const entrada = "  ANA@EJEMPLO.COM  ";
const correoNormalizado = entrada.trim().toLowerCase();
```

Hasta aquí todo se comporta como esperaríamos. Ahora empieza la parte interesante.

6.5 `length` no cuenta lo que parece

```
console.log("😊".length); // 2
```

Un emoji, que a simple vista es un solo símbolo, ocupa dos posiciones. Y no es un caso aislado:

```
console.log("A😊B".length); // 4
console.log("a\u0301".length); // 2, y en pantalla se ve como "á"
```

Para entender qué está pasando hay que separar tres conceptos que solemos confundir en uno solo.

6.6 Unidades de código, puntos de código y grafemas

Unicode asigna un número —llamado **punto de código**— a cada carácter que define. Se escriben con el prefijo U+:

```
A    → U+0041
ñ    → U+00F1
€    → U+20AC
😊    → U+1F600
```

Unicode dice qué número corresponde a cada carácter. No dice cómo almacenar ese número; de eso se ocupan las **codificaciones**.

JavaScript representa sus cadenas internamente mediante **UTF-16**, una codificación que usa unidades de 16 bits. Los caracteres con punto de código hasta U+FFFF caben en una unidad. Los que están por encima —muchos emojis, alfabetos históricos, símbolos matemáticos— necesitan dos unidades, llamadas **par sustituto**.

Tomemos el emoji:

```
const s = "😊";
```

Grafema visible:	😊
Punto de código:	U+1F600
Unidades UTF-16:	D83D DE00

Por eso:

```
s.length;           // 2 → cuenta unidades de código UTF-16
[...s].length;       // 1 → cuenta puntos de código
```

La expansión con `...` recorre la cadena por puntos de código, no por unidades.

Pero todavía falta un nivel. Consideremos:

```
const s = "a\u0301";
```

Visualmente se ve una sola letra: á. Sin embargo:

```
[...s]; // ["a", "´"]
```

Hay **dos puntos de código** —la letra a y un acento combinante— aunque una persona vea **una letra**.

Tenemos entonces tres niveles distintos:

```
unidad de código ≠ punto de código ≠ carácter visual
```

El término técnico para el tercero es **grapheme cluster**, o grupo de grafemas: lo que una persona percibe como un carácter.

6.7 Los emojis hacen visible el problema

Consideremos este emoji de familia:



Visualmente es una sola imagen. Internamente está construido combinando varios emojis individuales unidos por un carácter invisible llamado *zero-width joiner*:

```
const familia = "👨👩👦";

familia.length; // bastante mayor que 1
[...familia].length; // también mayor que 1
```

Aunque el usuario ve una sola unidad visual, el programa ve una secuencia larga. Por eso cortar cadenas arbitrariamente por índice puede producir resultados desagradables: un `slice(0, 10)` sobre un texto con emojis puede partir un par sustituto por la mitad y generar un carácter de reemplazo.

6.8 Recorrer correctamente una cadena

La forma aparentemente obvia de recorrer es esta:

```
const texto = "A😊B";

for (let i = 0; i < texto.length; i++) {
  console.log(texto[i]);
}
```

Pero `length` y los índices trabajan con unidades UTF-16, así que el emoji queda partido:


```
A
<primera mitad de 😊>
<segunda mitad de 😊>
B
```

En cambio, `for...of` recorre puntos de código:

```
for (const caracter of texto) {
  console.log(caracter);
}
```

produce:

```
A
😊
B
```

Lo mismo ocurre con la expansión:

```
[... "A😊B"];
// ["A", "😊", "B"]
```

Regla práctica

Para procesar texto que puede contener caracteres fuera del alfabeto básico:

```
for (const c of texto) {
  // ...
}
```

suele ser preferible a:

```
for (let i = 0; i < texto.length; i++) {
  const c = texto[i];
}
```

El bucle con índice sigue siendo correcto cuando trabajamos con posiciones de bytes o unidades, no con caracteres percibidos.

6.9 Intl.Segmenter

JavaScript moderno puede segmentar texto según reglas lingüísticas, y eso resuelve el nivel de los grafemas:

```
const segmentador = new Intl.Segmenter("es", {
  granularity: "grapheme"
});

const texto = "Hola 🧑🏿";
const partes = [...segmentador.segment(texto)];

console.log(partes.length); // cuenta caracteres percibidos
```

`Intl.Segmenter` también admite `granularity: "word"` y `"sentence"`, útiles para contar palabras respetando las reglas de cada idioma.

Los tres niveles, ordenados:

```
"😊".length
  ↓
unidades UTF-16

[..."😊"].length
  ↓
puntos de código

Intl.Segmenter
  ↓
grafemas percibidos
```

`Intl.Segmenter` está disponible en entornos modernos, pero conviene comprobar el soporte si el destino incluye navegadores antiguos.

6.10 Comparación lexicográfica y sus consecuencias

Volvamos a la comparación de la sección 6.3. La idea es comparar posición por posición:

```
casa
cosa
|
c = c      → seguir

casa
cosa
|
a < o      → resultado: "casa" < "cosa"
```

Es parecido a buscar palabras en un diccionario. Pero la comparación básica de JavaScript usa los valores UTF-16, no las reglas lingüísticas de un idioma. Eso tiene dos consecuencias importantes.

Números almacenados como texto

```
"10" < "2"; // true
10 < 2;      // false
```

Estamos comparando **cadenas**, no números. La primera posición decide: '1' es menor que '2'.

El caso se vuelve visible al ordenar:

```
const valores = ["1", "2", "10", "20", "3"];

valores.sort();
// ["1", "10", "2", "20", "3"]
```

El resultado es perfectamente correcto **como texto**, e incorrecto si pretendíamos interpretar esas cadenas como números. La solución es indicar el criterio:

```
valores.sort((a, b) => Number(a) - Number(b));
// ["1", "2", "3", "10", "20"]
```

El capítulo 10 desarrolla el ordenamiento de arreglos y por qué `sort()` sin comparador convierte a texto incluso cuando los elementos son números.

Palabras acentuadas

```
const palabras = ["nube", "Ñandú", "árbol", "zapato", "avión"];

palabras.sort();
```

El resultado no es el que un hablante de español esperaría, porque las mayúsculas tienen puntos de código menores que las minúsculas y las letras acentuadas están fuera del bloque ASCII. Lo que queremos decir no es «ordená por valor Unicode» sino:

ordená esto como lo ordenaría alguien que habla español.

6.11 Comparación lingüística

Para eso existe `localeCompare()`:

```
palabras.sort((a, b) => a.localeCompare(b, "es"));
```

Devuelve un número negativo, cero o positivo, igual que cualquier comparador.

Si vamos a comparar muchas veces, conviene construir el comparador una sola vez:

```
const español = new Intl.Collator("es");

palabras.sort(español.compare);
```

`Intl.Collator` encapsula las reglas de **collation**: el ordenamiento lingüístico de un idioma, que incluye el tratamiento de acentos, mayúsculas, dígrafos y caracteres especiales.

Comparar ignorando mayúsculas y acentos

Para ciertas búsquedas queremos considerar equivalentes:

ÁRBOL árbol Arbol arbol

Podemos construir un comparador con menor sensibilidad:

```
const comparador = new Intl.Collator("es", {
  sensitivity: "base"
});

comparador.compare("árbol", "ARBOL"); // 0
```

El `0` significa: para los criterios de este comparador son equivalentes.

Esto suele ser más correcto que la solución casera:

```
a.toLowerCase() === b.toLowerCase();
```

porque el tratamiento lingüístico de mayúsculas, acentos y alfabetos es más complejo de lo que parece, y varía según el idioma. El caso clásico es el turco, donde la `i` mayúscula no es `I`.

6.12 Convertir a texto

La conversión explícita se hace con `String()`:

```
String(123);      // "123"
String(true);     // "true"
String(null);     // "null"
String(undefined); // "undefined"
```

También existe el método `toString()`:

```
const n = 123;
n.toString(); // "123"
```

Con una diferencia práctica importante:

```
String(null);    // "null"
null.toString(); // TypeError
```

`String(valor)` es la conversión general más segura, porque funciona con `null` y `undefined`, que no tienen métodos.

Números en distintas bases

`toString()` tiene una capacidad particular sobre números: acepta una base.

```
const n = 42;

n.toString(10); // "42"
n.toString(2);  // "101010"
n.toString(16); // "2a"
```

Conceptualmente:

número → representación textual

Vale la pena ser preciso sobre esto: `"101010"` **no es** un número binario almacenado de otra manera. Es simplemente una cadena que representa un número en base 2. El valor numérico original no cambió.

Es el mismo punto de la sección 5.2, visto desde el otro lado.

6.13 Conversión implícita a texto

El operador `+` está sobrecargado: puede significar suma o concatenación.

```
2 + 3;    // 5
"2" + "3"; // "23"
"Edad: " + 25; // "Edad: 25"
```

En el último caso, el número `25` fue convertido implícitamente a `"25"`.

Esto produce resultados que sorprenden hasta que se conoce la regla:

```
1 + 2;    // 3
"1" + 2;   // "12"
1 + "2";   // "12"
```

Y la evaluación de izquierda a derecha hace el resto:

```
1 + 2 + "3";
```

```
1 + 2 → 3
3 + "3" → "33"
```

mientras que:

```
"1" + 2 + 3;
```

```
"1" + 2 → "12"
"12" + 3 → "123"
```

Los paréntesis, o mejor aún la interpolación, evitan el problema:

```
"Total: " + 2 + 3; // "Total: 23"
"Total: " + (2 + 3); // "Total: 5"
`Total: ${2 + 3}`; // "Total: 5"
```

El capítulo 9 desarrolla las reglas completas de coerción, incluido por qué `+` se comporta de manera distinta al resto de los operadores aritméticos.

6.14 Plantillas etiquetadas

Las plantillas literales tienen una capacidad más profunda que la interpolación. Se les puede anteponer una función:

```
html`<h1>${titulo}</h1>`
```

```
sql`
  SELECT *
  FROM usuarios
  WHERE id = ${id}
`
```

Aquí `html` y `sql` son **funciones**, no sintaxis especial del lenguaje. Reciben las partes estáticas y los valores interpolados por separado:

```
function mostrar(partes, ...valores) {
  console.log(partes);
  console.log(valores);
}

const nombre = "Ana";

mostrar`Hola ${nombre}!`;
```

La función recibe:

```
partes: ["Hola ", "!", "  
valores: ["Ana"]
```

Esa separación es lo valioso: permite **procesar los valores antes de construir el resultado**. Es la base conceptual de APIs que escapan caracteres peligrosos en HTML, construyen consultas SQL parametrizadas o generan hojas de estilo. Como la función distingue lo que el programador escribió de lo que vino de una variable, puede tratar cada parte de manera distinta.

String.raw

Normalmente las secuencias de escape se interpretan:

```
console.log("A\\nB");  
// A  
// B
```

Con `String.raw`, no:

```
String.raw`A\\nB`;  
// "A\\nB" – literalmente, con la barra y la ene
```

`String.raw` es una plantilla etiquetada incorporada en el lenguaje. Resulta útil al trabajar con expresiones regulares, rutas de Windows y otros textos donde las barras invertidas tienen significado propio.

6.15 Cadenas y expresiones regulares

Muchas operaciones sobre texto se expresan mejor con expresiones regulares:

```
const texto = "Juan tiene 57 años";  
  
texto.match(/\d+/); // ["57", ...]  
"hola mundo".replace(/\s+/g, " "); // "hola mundo"  
"uno, dos; tres".split(/[;]\s*/); // ["uno", "dos", "tres"]
```

Y Unicode vuelve a aparecer también aquí. Las expresiones regulares modernas tienen una bandera `u` que activa la semántica Unicode:

```
/🤪/u
```

y propiedades Unicode que permiten expresar categorías de caracteres:

```
"ñ".match(/\p{Letter}/u); // ["ñ"]
```

Esto evita la suposición de que «letra» significa [A-Za-z], que en un mundo Unicode es una definición bastante pobre. El capítulo 20 desarrolla el tema completo.

6.16 Un experimento que reúne todo

Este programa merece ejecutarse:

```
const s = "A😊á";

console.log("Texto:", s);

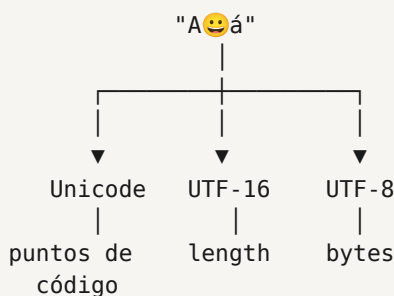
console.log("length:", s.length);

console.log("Puntos de código:", [...s]);

console.log(
  "Unicode:",
  [...s].map(
    (c) => "U+" + c.codePointAt(0).toString(16).toUpperCase()
  )
);

console.log("UTF-8:", [...new TextEncoder().encode(s)]);
```

Estamos observando **el mismo texto desde distintas capas**:



Y esa es probablemente la idea más importante del capítulo:

Una cadena no es «una lista de bytes».

El texto tiene una representación abstracta mediante Unicode. JavaScript expone sus cadenas, por razones históricas, como unidades UTF-16. Y UTF-8 aparece normalmente cuando necesitamos convertir ese texto en bytes para almacenarlo o transmitirlo, que es exactamente lo que hace el capítulo 21 al escribir un archivo.

Entendiendo esa separación dejan de parecer misteriosos `😊.length === 2`, `TextEncoder`, `codePointAt()`, los problemas con los acentos y buena parte de las rarezas del procesamiento de texto.

6.17 Síntesis conceptual

Las cadenas se escriben con comillas simples, dobles o invertidas. Las plantillas literales agregan interpolación y texto multilínea, y suelen ser la opción más legible para construir mensajes.

Las cadenas son inmutables: todos sus métodos devuelven cadenas nuevas. Eso permite encadenar operaciones con confianza, y también explica por qué la concatenación repetida dentro de un bucle es costosa.

El punto conceptual central del capítulo es la distinción de tres niveles: la unidad de código UTF-16 que cuenta `length`, el punto de código Unicode que recorren `for...of` y la expansión, y el grafema percibido que segmenta `Intl.Segmenter`. Para textos ASCII los tres coinciden, y por eso la diferencia pasa desapercibida hasta que aparece un emoji o un acento combinante.

La comparación con `<` y `>` es lexicográfica y usa valores UTF-16. Sirve para ordenamientos internos y falla para el orden que espera una persona: allí corresponden `localeCompare()` o `Intl.Collator`.

La conversión a texto se hace con `String()`, que funciona incluso con `null` y `undefined`. La conversión implícita ocurre sola cuando `+` encuentra una cadena, y produce resultados que dependen del orden de evaluación.

Finalmente, las plantillas etiquetadas convierten una plantilla literal en una llamada a función que recibe por separado el texto y los valores. Es un mecanismo que se usa poco al principio y que sostiene bibliotecas enteras.

Los dos capítulos siguientes son breves y cierran los tipos primitivos: primero las dos formas de ausencia, después el tipo que existe para no colisionar con nada.

Capítulo 7. `undefined` y `null`: dos formas de ausencia

JavaScript tiene dos valores para expresar que no hay un dato utilizable. Otros lenguajes se arreglan con uno solo, así que la duplicación llama la atención y merece una explicación.

Los dos valores comunican intenciones diferentes. Distinguir las hace que el código diga más sobre lo que ocurrió.

7.1 `undefined`: todavía no hay valor

`undefined` suele indicar que un valor **no fue asignado**, que una propiedad no existe o que una función no devolvió un resultado explícito. Aparece por acción del lenguaje, no por decisión nuestra.

```
let telefono;
console.log(telefono); // undefined – declarada sin inicializar

const alumno = { nombre: "Ana" };
console.log(alumno.telefono); // undefined – la propiedad no existe

function registrar() {
  console.log("Registrado");
}

console.log(registrar()); // undefined – la función no retorna nada
```

Hay un cuarto caso: un parámetro que no recibió argumento.

```
function mostrar(a, b) {
  console.log(a, b);
}

mostrar(10); // 10 undefined
```

Los cuatro casos comparten un rasgo: nadie decidió que no hubiera valor. Simplemente no lo hay.

Técnicamente, `undefined` es una propiedad global del lenguaje y no un literal de la gramática, como sí lo es `null`. En el uso cotidiano se escribe directamente y la diferencia no se nota.

7.2 null: la ausencia es la respuesta

`null` representa una ausencia **elegida de manera intencional**:

```
const fechaDeBaja = null;
```

Aquí no olvidamos asignar el dato: estamos afirmando que el alumno no tiene fecha de baja. La ausencia es información.

La diferencia se aprecia bien en un objeto que viene de una base de datos:

```
const alumno = {
  nombre: "Ana",
  fechaDeBaja: null, // sabemos que no tiene: sigue activa
};
```

frente a:

```
const alumno = {
  nombre: "Ana",
  // fechaDeBaja no está: no sabemos si tiene o no
};
```

En el primer caso, leer `alumno.fechaDeBaja` da `null`. En el segundo da `undefined`. Son situaciones distintas y el programa puede distinguirlas.

La particularidad histórica

```
console.log(typeof null); // "object"
```

Esto es un error del diseño original de 1995 que nunca pudo corregirse sin romper código existente. `null` **no** es un objeto. Para comprobarlo se usa igualdad estricta:

```
if (fechaDeBaja === null) {
  console.log("El alumno continúa activo");
}
```

7.3 Cómo se comparan entre sí

```
null === undefined; // false
null == undefined; // true
```

La segunda línea es una regla especial de la igualdad flexible: `null` y `undefined` se consideran iguales entre sí, y **solo** entre sí. Ningún otro valor es `==` a `null`.

Eso hace que exista un idioma reconocible para preguntar «¿es alguna de las dos formas de ausencia?»:

```
if (valor == null) {  
  // true si valor es null o undefined  
}
```

Es prácticamente la única situación donde el uso de `==` está aceptado por convención, precisamente porque su comportamiento en este caso es acotado y bien conocido. Aun así, muchos equipos prefieren la forma explícita:

```
if (valor === null || valor === undefined) {  
  // ...  
}
```

7.4 Tratar ambas ausencias con `??`

El operador de coalescencia nula que vimos en la sección 4.11 está diseñado exactamente para este par: usa el valor alternativo cuando el primero es `null` o `undefined`, y solo entonces.

```
const telefono = alumno.telefono ?? "Sin teléfono";
```

La ventaja frente a `||` es que no descarta valores *falsy* legítimos:

```
const stock = 0;  
  
stock || "sin datos"; // "sin datos" – incorrecto: cero es un stock válido  
stock ?? "sin datos"; // 0 – correcto
```

El nombre del operador viene de ahí: coalesce, «fusionar», sobre valores *nullish*, es decir, `null` o `undefined`.

También existe el operador de asignación lógica correspondiente:

```
let configuracion;  
  
configuracion ??= { tema: "claro" };  
// asigna solo si configuracion era null o undefined
```

7.5 Cuál usar al escribir código

Como regla de diseño:

- **No asignar `undefined` explícitamente.** Es el valor que el lenguaje usa para «no hay nada»; asignarlo a mano borra la distinción.
- **Usar `null` cuando la ausencia sea una decisión** que el programa toma y que quien lee el código debe entender como deliberada.
- **Devolver `null` desde una función de búsqueda** que legítimamente puede no encontrar nada:

```
function buscarAlumno(legajo) {  
  const alumno = alumnos.find((item) => item.legajo === legajo);  
  return alumno ?? null;  
}
```

`find()` devuelve `undefined` cuando no encuentra; convertirlo a `null` comunica que la ausencia es un resultado previsto de la función, no un descuido.

El capítulo 15 retoma esta decisión desde otro ángulo: cuándo una ausencia debe devolverse como valor y cuándo debe lanzarse como excepción.

7.6 Síntesis conceptual

`undefined` y `null` expresan ausencia, pero no la misma ausencia. `undefined` aparece cuando el lenguaje no tiene un valor que dar: una variable sin inicializar, una propiedad inexistente, un parámetro no recibido, una función sin `return`. `null` aparece cuando nosotros decidimos que la ausencia es la respuesta correcta.

`typeof null` devuelve `"object"` por un error histórico; la comprobación correcta es `valor === null`.

Los dos son *falsy*, los dos son `==` entre sí y con nada más, y `??` los trata como un par para proveer valores por defecto sin descartar `0`, `false` ni la cadena vacía.

El próximo capítulo cierra los tipos primitivos con el menos frecuente de todos, y el único que existe para no ser igual a nada.

Capítulo 8. Símbolos y protocolos del lenguaje

`Symbol` es el tipo primitivo que menos aparece en el código de una aplicación común, y sin embargo sostiene mecanismos que usamos todo el tiempo sin verlos. Tiene dos usos relacionados pero distintos: crear claves de propiedad que no choquen con ninguna otra, y permitir que un objeto participe de los protocolos internos del lenguaje.

8.1 Un valor que no es igual a ningún otro

Un símbolo es un valor primitivo único e inmutable. Se crea con la función `Symbol()`:

```
const a = Symbol("id");
const b = Symbol("id");

console.log(a === b); // false
```

La cadena `"id"` es solamente una **descripción** útil para depurar. No define la identidad del símbolo: cada llamada a `Symbol()` produce un valor nuevo y distinto de todos los anteriores.

```
const id = Symbol();
const nombre = Symbol("nombre");

console.log(typeof id);           // "symbol"
console.log(nombre);              // Symbol(nombre)
console.log(nombre.description); // "nombre"
```

No hay sintaxis literal para escribir un símbolo, a diferencia de los demás primitivos. La única forma de obtener uno es llamando a la función.

8.2 Símbolos como claves de propiedad

Su uso más directo es servir de clave en un objeto, usando corchetes:

```
const id = Symbol("id");

const persona = {
  nombre: "Ana",
  [id]: 1234,
};

console.log(persona.nombre); // "Ana"
console.log(persona[id]);    // 1234
```

Los corchetes indican que se usa el **valor** almacenado en `id` como clave, no la palabra `"id"`. Es la sintaxis de claves calculadas que el capítulo 11 desarrolla.

Y aquí está el punto: una propiedad con clave de símbolo **no es** una propiedad con clave de cadena, aunque la descripción coincida.

```
const id = Symbol("id");

const persona = {
  id: 10,
  [id]: 20,
};

console.log(persona.id); // 10
console.log(persona[id]); // 20
```

Conceptualmente:

```
persona
├── "id"      → 10
└── Symbol(id) → 20
```

Son claves completamente diferentes y pueden coexistir sin pisarse.

8.3 El problema que resuelve

Supongamos que una biblioteca recibe objetos creados por otra parte del programa y necesita adjuntarles información interna:

```
function procesar(usuario) {
  usuario.estadoInterno = "procesado";
}
```

Nada garantiza que el objeto no tuviera ya esa propiedad:

```
const usuario = {
  nombre: "Ana",
  estadoInterno: "otra cosa", // se pierde
};
```

La biblioteca acaba de destruir un dato de la aplicación, y la aplicación no tiene forma de saber qué nombres debe evitar.

Con un símbolo, la colisión es imposible:

```
const estadoInterno = Symbol("estadoInterno");

function procesar(usuario) {
  usuario[estadoInterno] = "procesado";
}
```

Ahora ambas propiedades conviven. Nadie más puede generar ese mismo símbolo, ni siquiera escribiendo `Symbol("estadoInterno")`, porque cada llamada crea uno nuevo.

8.4 El registro global: `Symbol.for()`

A veces sí queremos que dos partes del programa obtengan el mismo símbolo. Para eso existe un registro global:

```
const a = Symbol.for("usuario");
const b = Symbol.for("usuario");

console.log(a === b); // true
```

`Symbol.for()` busca primero si ya existe un símbolo registrado con esa clave. Si existe, lo devuelve. Si no existe, lo crea y lo registra.

La diferencia con `Symbol()` es la clave:

```
Symbol("x") === Symbol("x");           // false - dos símbolos distintos
Symbol.for("x") === Symbol.for("x");   // true  - el mismo símbolo
```

`Symbol.keyFor()` recupera la clave de un símbolo del registro:

```
const s = Symbol.for("usuario");
console.log(Symbol.keyFor(s)); // "usuario"

const t = Symbol("usuario");
console.log(Symbol.keyFor(t)); // undefined - no pertenece al registro
```

En la práctica, `Symbol()` sirve para lo privado a un módulo y `Symbol.for()` para protocolos compartidos entre módulos o entre bibliotecas distintas.

8.5 Las propiedades con símbolo no aparecen al enumerar

```
const secreto = Symbol("secreto");

const usuario = {
  nombre: "Ana",
  edad: 25,
  [secreto]: 123,
};

console.log(Object.keys(usuario));
// ["nombre", "edad"]
```

Tampoco aparecen en un `for...in`, ni en `JSON.stringify()`, ni en `Object.entries()`.

Esto es deseable: los metadatos internos de una biblioteca no deberían contaminar el recorrido de propiedades que hace la aplicación.

Pero **no significa que sean privadas**. Se pueden obtener:

```
Object.getOwnPropertySymbols(usuario);
// [Symbol(secreto)]
```

Conviene ser explícito sobre esto, porque es una confusión frecuente:

Symbol no es un mecanismo de seguridad.

Sirve para evitar colisiones, agregar metadatos, implementar protocolos y personalizar el comportamiento de un objeto. No sirve para proteger información secreta. Para privacidad real, JavaScript ofrece los campos privados de clase, que se escriben con `#`, y las clausuras, que el capítulo 17 desarrolla.

8.6 Los símbolos conocidos del lenguaje

Aquí aparece el segundo uso, más profundo. JavaScript define un conjunto de símbolos especiales llamados **well-known symbols**, que funcionan como puntos de enganche: si un objeto tiene una propiedad con uno de esos símbolos, el lenguaje la usa para decidir cómo comportarse.

Los principales:

```
Symbol.iterator  
Symbol.asyncIterator  
Symbol.toPrimitive  
Symbol.toStringTag  
Symbol.hasInstance  
Symbol.match  
Symbol.replace  
Symbol.search  
Symbol.split  
Symbol.species  
Symbol.isConcatSpreadable  
Symbol.unscopables
```

Veamos los más importantes.

8.7 Symbol.iterator: cómo se recorre un objeto

Cuando escribimos:

```
for (const x of objeto) {  
  // ...  
}
```

JavaScript busca `objeto[Symbol.iterator]` y lo invoca. Si no lo encuentra, lanza un `TypeError`.

Ese método debe devolver un **iterador**: un objeto con un método `next()` que en cada llamada devuelve `{ value, done }`.

```

const numeros = {
  desde: 1,
  hasta: 5,

  [Symbol.iterator]() {
    let actual = this.desde;
    const hasta = this.hasta;

    return {
      next() {
        if (actual <= hasta) {
          return { value: actual++, done: false };
        }

        return { done: true };
      },
    };
  },
};

for (const n of numeros) {
  console.log(n);
}
// 1 2 3 4 5

```

Conceptualmente:

```

objeto
|
▼
objeto[Symbol.iterator]()
|
▼
iterador
|
▼
next()
|
├─ { value: ..., done: false }
├─ { value: ..., done: false }
└─ { done: true }

```

Esto explica algo que hasta ahora dábamos por sentado: **los arrays funcionan con `for...of` porque implementan este protocolo.**

```

const numeros = [10, 20, 30];
const iterador = numeros[Symbol.iterator]();

console.log(iterador.next()); // { value: 10, done: false }
console.log(iterador.next()); // { value: 20, done: false }
console.log(iterador.next()); // { value: 30, done: false }
console.log(iterador.next()); // { value: undefined, done: true }

```

Lo mismo vale para las cadenas, los `Map` y los `Set`. Y no solo `for...of` depende de este protocolo: también la expansión con `...`, la desestructuración de arreglos, `Array.from()` y `Promise.all()`.

Por eso `[..."A 😊 B"]` funcionaba en el capítulo 6: `String.prototype` implementa `Symbol.iterator` recorriendo puntos de código.

`Symbol.asyncIterator`

Es el equivalente asincrónico, usado por `for await`:

```
const fuente = {
  async *[Symbol.asyncIterator]() {
    yield 10;
    yield 20;
    yield 30;
  },
};

for await (const x of fuente) {
  console.log(x);
}
```

La sintaxis `async *` combina función generadora y asincronía. El capítulo 18 presenta los generadores; el capítulo 21 usa `for await` para leer un archivo línea por línea.

8.8 `Symbol.toPrimitive`: cómo se convierte a primitivo

Permite controlar qué ocurre cuando un objeto debe convertirse a un valor primitivo:

```
const producto = {
  nombre: "Monitor",
  precio: 500,

  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return this.precio;
    }

    return this.nombre;
  },
};

console.log(+producto); // 500
console.log(`${producto}`); // "Monitor"
```

El parámetro `hint` indica qué clase de primitivo espera el contexto:

```
"number" → el contexto necesita un número: +obj, obj * 2, obj - 1
"string"  → el contexto necesita texto: `${obj}`, String(obj)
"default" → el contexto acepta cualquiera: obj + 1, obj == otro
```

Conceptualmente:

```
objeto
  |
  ▼
Symbol.toPrimitive
  |
  ▼
primitivo
  |
  ▼
operación
```

Este símbolo es el punto exacto donde nuestro código puede intervenir en la coerción implícita que el capítulo 9 describe.

8.9 Symbol.toStringTag: cómo se identifica

Personaliza la etiqueta que devuelve `Object.prototype.toString`:

```
const usuario = {
  [Symbol.toStringTag]: "Usuario",
};

console.log(Object.prototype.toString.call(usuario));
// "[object Usuario]"
```

Sin ese símbolo, cualquier objeto literal produce `"[object Object]"`. Con él, las herramientas de depuración y los mensajes de error pueden identificar el tipo.

8.10 Symbol.hasInstance: cómo responde instanceof

Permite redefinir el comportamiento del operador `instanceof`:

```
class Adulto {
  static [Symbol.hasInstance](objeto) {
    return objeto.edad >= 18;
  }
}

const ana = { edad: 25 };

console.log(ana instanceof Adulto); // true
```

Conceptualmente:

```
ana instanceof Adulto
  |
  ▼
Adulto[Symbol.hasInstance](ana)
```

`ana` no fue construida con `new Adulto()` ni hereda de esa clase, pero el operador responde `true` porque la clase definió su propio criterio. Es un mecanismo potente y, por lo mismo, fácil de usar mal: quien lea `x instanceof Adulto` va a suponer la semántica habitual.

8.11 Los símbolos restantes

De expresiones regulares

```
Symbol.match
Symbol.replace
Symbol.search
Symbol.split
```

Determinan cómo participa un objeto en `texto.match()`, `texto.replace()`, `texto.search()` y `texto.split()`. Es lo que permite que esos métodos acepten un objeto `RegExp` sin que `String.prototype` sepa nada de expresiones regulares:

```
const buscador = {
  [Symbol.match](texto) {
    return texto.includes("JavaScript");
  },
};

console.log("Me gusta JavaScript".match(buscador)); // true
```

`Symbol.isConcatSpreadable`

Controla si un objeto se expande al usar `concat()`:

```
const objeto = {
  0: "A",
  1: "B",
  length: 2,
  [Symbol.isConcatSpreadable]: true,
};

console.log([1, 2].concat(objeto));
// [1, 2, "A", "B"]
```

Symbol.species

Controla qué constructor se usa cuando una operación crea objetos derivados:

```
class MiArray extends Array {  
  static get [Symbol.species]() {  
    return Array;  
  }  
}
```

Con esa definición, `miArray.map(...)` devuelve un `Array` común en lugar de un `MiArray`. Es un mecanismo avanzado y poco frecuente.

Symbol.unscopables

Está relacionado con la sentencia `with`, desaconsejada y no disponible en modo estricto. Tiene principalmente interés histórico.

8.12 Un ejemplo integrador

Una clase que implementa tres protocolos a la vez:

```
class Rango {  
  constructor(desde, hasta) {  
    this.desde = desde;  
    this.hasta = hasta;  
  }  
  
  *[Symbol.iterator]() {  
    for (let n = this.desde; n <= this.hasta; n++) {  
      yield n;  
    }  
  }  
  
  [Symbol.toPrimitive](hint) {  
    if (hint === "number") {  
      return this.hasta - this.desde + 1;  
    }  
  
    return `${this.desde}..${this.hasta}`;  
  }  
  
  get [Symbol.toStringTag]() {  
    return "Rango";  
  }  
}
```

```
const rango = new Range(3, 6);

for (const n of rango) {
  console.log(n);
}
// 3 4 5 6

console.log(+rango);      // 4 – la cantidad de elementos
console.log(`${rango}`);  // "3..6"

console.log(Object.prototype.toString.call(rango));
// "[object Range]"
```

`Range` no hereda de `Array` ni de ninguna clase especial. Simplemente le dijo al lenguaje cómo recorrerlo, cómo convertirlo y cómo nombrarlo, y el lenguaje lo trata en consecuencia.

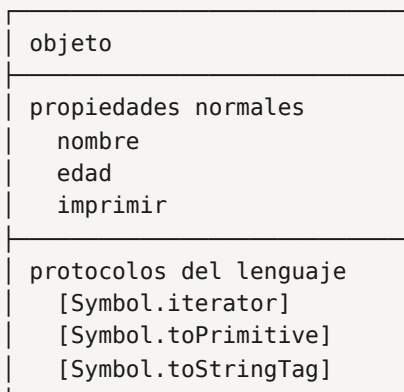
8.13 Síntesis conceptual

`Symbol` tiene dos usos que conviene mantener separados.

El primero es práctico e inmediato: `Symbol("id")` produce una clave de propiedad única, que no puede colisionar con ninguna otra y que no aparece en los recorridos habituales. Sirve para adjuntar metadatos a objetos ajenos. No sirve para ocultar información: `Object.getOwnPropertySymbols()` las revela.

El segundo es estructural. Los símbolos conocidos actúan como **ganchos del lenguaje**. Si JavaScript hubiera usado nombres comunes —`objeto.iterator()`—, cualquier método que un programa llamara `iterator` habría entrado en conflicto con el protocolo. Al usar `objeto[Symbol.iterator]()`, el identificador pertenece al lenguaje y no puede chocar con nada que escribamos.

Podemos imaginar un objeto con dos zonas:



Las propiedades normales describen los datos y comportamientos de la aplicación. Los símbolos conocidos le dicen al lenguaje: cuando quieras hacer determinada operación conmigo, hazla de esta manera.

Con esto termina el recorrido por los tipos primitivos. El capítulo siguiente cierra la parte poniendo en orden el tema que atravesó a todos: cómo pasa un valor de un tipo a otro.

Capítulo 9. Conversiones y coerción de tipos

Los capítulos anteriores mencionaron la coerción varias veces y siempre dejaron la explicación completa para más adelante. Este es ese lugar.

Convertir un tipo significa producir un valor nuevo de otro tipo. Cuando la conversión aparece escrita por quien programa, es **explícita**. Cuando la aplica el lenguaje por su cuenta, porque el contexto lo requiere, se llama **coerción**.

La coerción tiene mala prensa, y en parte se la ganó. Pero conviene ser precisos: no son decisiones arbitrarias del motor. Cada operador y cada contexto tienen reglas definidas en la especificación, deterministas y documentadas. La dificultad aparece cuando esas reglas no coinciden con la intención de quien escribió el código.

El objetivo del capítulo no es memorizar una tabla de rarezas. Es reconocer **dónde** ocurre una conversión automática, para poder decidir si la queremos.

9.1 Conversiones explícitas

A cadena

```
String(42);      // "42"
String(true);    // "true"
String(null);    // "null"
String(undefined); // "undefined"
String([1, 2]);  // "1,2"
String({});      // "[object Object]"
```

`String()` funciona con cualquier valor, incluidos `null` y `undefined`, a diferencia del método `toString()`.

Los números pueden expresarse en otras bases:

```
const valor = 255;

valor.toString(2); // "1111111"
valor.toString(16); // "ff"
```

A número

```
Number("42");      // 42
Number("3.14");     // 3.14
Number("");         // 0
Number(" ");        // 0
Number(true);       // 1
Number(false);      // 0
Number(null);       // 0
Number(undefined); // NaN
Number("42px");     // NaN
```

Dos resultados merecen atención. `Number("")` da `0`, lo cual sorprende: una cadena vacía no representa el cero. Y `Number(null)` también da `0`, mientras que `Number(undefined)` da `NaN`. Estas asimetrías son históricas y conviene no apoyarse en ellas.

`Number()` exige que la cadena completa represente un número, salvo espacios laterales. `Number.parseInt()` y `Number.parseFloat()` leen un prefijo numérico:

```
Number.parseInt("42px", 10); // 42
Number.parseFloat("3.14kg"); // 3.14
```

Esa tolerancia puede ser útil al analizar formatos controlados, pero también puede aceptar una entrada que debería rechazarse. La función se elige según la regla del problema, no por costumbre.

A booleano

```
Boolean(1);      // true
Boolean(0);      // false
Boolean("hola"); // true
Boolean("");     // false
Boolean("false"); // true
Boolean(null);   // false
```

`Boolean()` no interpreta el significado humano de la palabra `"false"`. Solo aplica la regla de *truthy* y *falsy* del capítulo 4: toda cadena no vacía es *truthy*.

A bigint

```
BigInt("9007199254740993"); // 9007199254740993n
BigInt(42);                  // 42n
```

Una cadena con decimales produce un error, porque `bigint` solo representa enteros:

```
// BigInt("3.14"); // SyntaxError
// BigInt(3.14);   // RangeError
```

9.2 Convertir y validar son cosas distintas

Un error frecuente consiste en confundir «la conversión funcionó» con «el dato es correcto».

```
Number("-2"); // -2 – la conversión es perfecta
```

Pero si esperábamos una cantidad, `-2` es inválido. La conversión hizo su trabajo; la validación es otra responsabilidad.

Una conversión robusta de un campo numérico separa los dos pasos:

```
function convertirCantidad(entrada) {
  const texto = entrada.trim();

  if (texto === "") {
    throw new Error("La cantidad es obligatoria");
  }

  const cantidad = Number(texto);

  if (!Number.isInteger(cantidad) || cantidad < 0) {
    throw new Error("La cantidad debe ser un entero no negativo");
  }

  return cantidad;
}
```

Primero comprueba presencia, después convierte, después valida la regla del dominio. Cada paso puede fallar por un motivo distinto y dar un mensaje distinto.

9.3 Dónde ocurre la coerción

Hay cuatro contextos principales donde JavaScript convierte por su cuenta.

Contextos booleanos

Las condiciones de `if` y `while`, el operador ternario y los operadores lógicos aplican la conversión booleana:

```
if ("hola") {
  console.log("Se ejecuta porque la cadena es truthy");
}
```

No cambian el valor original; lo interpretan como condición.

Contextos de texto

Una expresión dentro de una plantilla se convierte a cadena:

```
const cantidad = 3;
const disponible = true;

`${cantidad}`; // "3"
`${disponible}`; // "true"
```

Lo mismo ocurre al usar un valor como clave de propiedad con corchetes, o al concatenar con `+` cuando uno de los operandos es texto.

Contextos numéricos

Los operadores aritméticos, salvo `+`, y los operadores de comparación relacional intentan obtener números.

Contextos de comparación

`==` aplica un algoritmo de conversión antes de comparar. `===` no.

9.4 La regla especial del operador `+`

`+` es el único operador aritmético que también sirve para concatenar. Su regla, simplificada, es esta: después de obtener valores primitivos de ambos operandos, **si alguno es una cadena, concatena como texto**; en caso contrario, suma.

```
5 + 2; // 7
"5" + 2; // "52"
5 + "2"; // "52"
"5" + true; // "5true"
```

La evaluación ocurre de izquierda a derecha, y eso importa:

```
1 + 2 + "3"; // "33": primero 1 + 2 = 3, después 3 + "3"
"1" + 2 + 3; // "123": la concatenación empieza en el primer paso
```

Los paréntesis cambian el resultado:

```
"Total: " + 2 + 3; // "Total: 23"
"Total: " + (2 + 3); // "Total: 5"
```

Y la interpolación suele expresar mejor la intención:

```
`Total: ${2 + 3}`; // "Total: 5"
```

Hay un caso más que conviene conocer:

```
[] + [];    // "" - dos arreglos vacíos convertidos a texto  
[] + {};    // "[object Object]"
```

No son ejemplos de código real, pero explican los resultados extraños que a veces aparecen en una consola: los arreglos y objetos se convierten primero a primitivo y `+` termina concatenando dos cadenas.

9.5 Los demás operadores aritméticos

`-`, `*`, `/`, `%` y `**` no concatenan nunca. Intentan convertir sus operandos a valores numéricos:

```
"10" - 2;    // 8  
"10" * 2;    // 20  
"10" / 2;    // 5  
"10" ** 2;   // 100
```

Si la conversión no produce un número válido, el resultado es `NaN`:

```
"diez" * 2;  // NaN
```

Esta asimetría entre `+` y el resto es la fuente de un error clásico. Que una operación «funcione» no demuestra que la entrada tenía el tipo correcto:

```
const precio = 1200;  
const cantidad = "2";  
  
precio * cantidad; // 2400 - parece correcto  
precio + cantidad; // "12002" - revela el problema
```

La multiplicación convirtió `"2"` y produjo el resultado esperado. La suma concatenó. El dato siempre estuvo mal; solo una de las dos operaciones lo hizo visible.

La solución es convertir una vez, validar y después operar con un tipo conocido.

9.6 Comparaciones relacionales

Si **ambos** operandos son cadenas, `<`, `>`, `<=` y `>=` comparan lexicográficamente. En los demás casos habituales intentan una comparación numérica:

```
"20" < "3"; // true - comparación textual, carácter por carácter  
20 < "3";  // false - "3" se convierte a 3
```

Es el mismo comportamiento que vimos en la sección 6.10. Evitar operandos de tipos diferentes vuelve la comparación mucho más fácil de revisar.

9.7 Igualdad flexible y estricta

`==` y `!=` aplican conversiones cuando los tipos no coinciden:

```
0 == false;      // true
"" == 0;         // true
"5" == 5;        // true
null == undefined; // true
"0" == false;    // true
```

`===` y `!==` no convierten nada:

```
0 === false;     // false
"" === 0;        // false
"5" === 5;       // false
null === undefined; // false
```

Las reglas de `==` son deterministas y no muy largas, pero exigen tenerlas presentes para interpretar una sola línea. Sus casos principales:

Comparación	Resultado	Regla aplicada
<code>null == undefined</code>	<code>true</code>	regla especial: iguales entre sí y con nada más
<code>número == cadena</code>	según valor	la cadena se convierte a número
<code>booleano == cualquier cosa</code>	según valor	el booleano se convierte a número (0 o 1)
<code>objeto == primitivo</code>	según valor	el objeto se convierte a primitivo
<code>NaN == NaN</code>	<code>false</code>	NaN no es igual a nada, ni a sí mismo

Por eso la igualdad estricta debe ser la opción habitual. La excepción aceptada por convención es `valor == null`, que comprueba las dos formas de ausencia a la vez, como vimos en el capítulo 7.

Existe además `Object.is()`, que se comporta como `===` con dos diferencias:

```
Object.is(NaN, NaN); // true
Object.is(0, -0);    // false
```

Es útil en casos muy puntuales, sobre todo al comparar resultados de cálculos numéricos.

9.8 Cómo se convierte un objeto a primitivo

Cuando un objeto aparece en un contexto que necesita un primitivo, JavaScript sigue un procedimiento definido:

1. Si el objeto tiene `Symbol.toPrimitive`, lo invoca con la pista correspondiente y usa su resultado.
2. Si no, y el contexto pide un número, prueba `valueOf()` y después `toString()`.
3. Si el contexto pide texto, prueba `toString()` y después `valueOf()`.

Esto explica varios comportamientos que de otro modo parecerían mágicos:

```
const fecha = new Date("2026-08-30");

fecha + "";    // texto: usa toString()
Number(fecha); // número: usa valueOf(), da la marca de tiempo
```

Y permite definir el comportamiento de nuestros propios objetos:

```
const dinero = {
  monto: 1500,
  valueOf() {
    return this.monto;
  },
};

console.log(dinero * 2); // 3000
```

El capítulo 8 mostró la versión moderna y más explícita de este mecanismo, `Symbol.toPrimitive`, que permite distinguir las tres pistas posibles.

9.9 Coerciones que JavaScript se niega a hacer

No todas las conversiones son automáticas. En dos casos el lenguaje prefiere lanzar un error antes que adivinar:

```
// 1n + 1;           // TypeError: no mezcla bigint con number
// +1n;              // TypeError: el + unario no admite bigint
// "id: " + Symbol("id"); // TypeError: los símbolos no se convierten a texto
```

Las razones son distintas pero comparten el criterio. Mezclar `bigint` y `number` exigiría decidir si el resultado pierde precisión, y ninguna respuesta es segura por defecto. Convertir un símbolo a texto silenciosamente anularía su propósito de no colisionar.

Las conversiones deben hacerse a mano cuando correspondan:

```
1n + BigInt(1); // 2n
`id: ${String(Symbol("id"))}`; // "id: Symbol(id)"
```

Estos errores son una buena noticia: el lenguaje avisa en lugar de producir un resultado inesperado.

9.10 Una estrategia para los datos de entrada

Los formularios, los parámetros de URL, los archivos y los cuerpos HTTP llegan frecuentemente como texto, o como datos cuya forma no controlamos. Una secuencia saludable es esta:

```
entrada externa
  ↓
comprobar presencia y formato
  ↓
convertir explícitamente
  ↓
validar reglas del dominio
  ↓
operar con un valor confiable
```

En código:

```
const precioTexto = "1999.50";
const cantidadTexto = "3";

const precio = Number(precioTexto);
const cantidad = Number(cantidadTexto);

if (!Number.isFinite(precio) || precio < 0) {
  throw new Error("Precio inválido");
}

if (!Number.isInteger(cantidad) || cantidad <= 0) {
  throw new Error("Cantidad inválida");
}

const total = precio * cantidad;
```

Esta versión parece más extensa que confiar en la coerción, y lo es. A cambio, hace visibles cuatro cosas que antes estaban implícitas: qué tipos espera el cálculo, qué entradas son inválidas, en qué punto ocurre la conversión y qué condiciones pertenecen al negocio.

TypeScript puede agregar verificación estática alrededor de este código, pero la entrada sigue necesitando validación durante la ejecución, por la razón que explicamos en la sección 1.9: los tipos desaparecen antes de ejecutar.

9.11 Confusiones frecuentes

Interpretar una cadena no vacía por su significado

```
Boolean("false"); // true
```

La conversión observa si la cadena está vacía, no qué palabra contiene. Un campo que llega como "false" desde un formulario debe compararse explícitamente:

```
const activo = valor === "true";
```

Usar `||` cuando el cero es válido

```
const intentos = 0;

intentos || 3; // 3 - descarta un valor legítimo
intentos ?? 3; // 0 - correcto
```

Formatear un número y creer que sigue siendo numérico

```
const precio = 12.5;
const formateado = precio.toFixed(2);

typeof formateado; // "string"
formateado * 2;    // 25 - funciona por coerción, pero es confuso
formateado + 2;    // "12.502" - el error se hace visible
```

`toFixed()` produce texto listo para mostrar. Conviene formatear al final, nunca en el medio de un cálculo.

Usar `parseInt` sin decidir si se acepta texto adicional

```
Number.parseInt("12px", 10); // 12
Number("12px");              // NaN
```

Ninguno de los dos es incorrecto. La pregunta es si "12px" debe considerarse una entrada válida.

Comparar tipos diferentes con `==`

```
"0" == false; // true
"0" === false; // false
```

Convertir primero y comparar después reduce las sorpresas a cero.

9.12 Síntesis conceptual

Una conversión explícita aparece escrita en el código; una coerción la aplica el lenguaje porque el contexto lo exige. Las dos siguen las mismas reglas: la diferencia está en quién decide.

Hay cuatro contextos donde ocurre coerción: las condiciones, que convierten a booleano; las plantillas y la concatenación, que convierten a texto; los operadores aritméticos distintos de `+`, que convierten a número; y `==`, que convierte antes de comparar.

El operador `+` es el caso especial que produce la mayoría de las sorpresas, porque su significado depende de los tipos de sus operandos. Y las comparaciones con `==` son el segundo, porque obligan a conocer un algoritmo para leer una línea. Preferir `===` y la interpolación elimina casi todos los casos difíciles.

Convertir y validar son pasos distintos. `Number("-2")` convierte correctamente y la regla de negocio puede rechazar el resultado igual. Los datos que entran al sistema desde afuera necesitan los dos pasos, en ese orden, antes de participar de cualquier cálculo.

Con esto cerramos los valores individuales. La parte siguiente pasa a los valores que contienen otros valores, y con ellos aparece un concepto nuevo que cambia varias de las intuiciones construidas hasta acá: la identidad.

Parte II — Estructuras de datos

Capítulo 10. Arrays: secuencias ordenadas

Los tipos primitivos permiten representar valores individuales: un número, una cadena, un booleano o una ausencia. Sin embargo, una aplicación rara vez trabaja con datos aislados. Un curso tiene alumnos, un carrito contiene productos y cada producto reúne un nombre, un precio y una categoría.

Para modelar esas relaciones necesitamos **tipos compuestos**: valores capaces de organizar otros valores. JavaScript ofrece cuatro: arrays y objetos, que son las estructuras fundamentales, y Map y Set, que agregan colecciones especializadas.

El array responde a una pregunta muy concreta: qué valores aparecen en esta secuencia y en qué orden.

10.1 De un dato individual a una estructura

Supongamos que queremos conservar las notas de un alumno. Con variables individuales podríamos escribir:

```
const nota1 = 8;  
const nota2 = 6;  
const nota3 = 9;
```

Esta representación funciona para tres notas conocidas de antemano, pero no expresa que forman una colección. Tampoco permite recorrerlas de manera general, ni agregar una cuarta sin modificar el programa.

Un array reúne los valores bajo una misma estructura:

```
const notas = [8, 6, 9];
```

Ahora el programa puede recorrerlas, contarlas, filtrarlas o promediarlas sin saber cuántas son.

10.2 Índices y posiciones

Un array es una colección ordenada cuyos elementos se asocian con índices enteros que comienzan en cero.

```
const lenguajes = ["JavaScript", "TypeScript", "C#"];
```

La relación entre posiciones e índices es:

posición humana:	primera	segunda	tercera
índice:	0	1	2
valor:	"JavaScript"	"TypeScript"	"C#"

Que el primer índice sea cero no es una decisión estética ni un capricho heredado. El índice expresa el **desplazamiento** desde el comienzo de la colección: el primer elemento se encuentra a cero posiciones del inicio, el segundo a una posición, y así. Con esa convención, el cálculo de la dirección de un elemento es una multiplicación simple, y varias operaciones sobre rangos quedan más limpias.

10.3 Crear un array

La forma habitual es el literal delimitado por corchetes:

```
const vacio = [];
const edades = [19, 22, 21];
const materias = ["Programación IV", "Base de Datos II"];
```

JavaScript permite mezclar tipos:

```
const mezcla = [42, "hola", true, null];
```

Es válido, pero una colección suele ser más fácil de procesar cuando sus elementos cumplen el mismo contrato conceptual. Un array de notas debería contener notas; un array de alumnos debería contener objetos alumno. Si necesitamos mezclar, probablemente lo que corresponda sea un objeto con propiedades con nombre.

Otras formas de creación

`Array.of()` construye un array con los argumentos recibidos:

```
Array.of(3); // [3]
Array.of(1, 2, 3); // [1, 2, 3]
```

`Array.from()` crea un array a partir de un iterable o de un valor similar a un array:

```
Array.from("TUP"); // ["T", "U", "P"]

Array.from({ length: 5 }, (_, indice) => indice);
// [0, 1, 2, 3, 4]
```

La segunda forma es muy útil para generar secuencias: el segundo argumento es una función que se ejecuta una vez por posición.

El constructor `new Array()` existe, pero tiene un caso que confunde:

```
new Array(3); // array de longitud 3 con tres posiciones vacías
Array.of(3);  // [3]
```

Cuando recibe un único número, lo interpreta como longitud en lugar de contenido. Por eso conviene preferir `[]`, `Array.of()` y `Array.from()` según la intención.

length

```
const notas = [8, 6, 9];

console.log(notas.length); // 3
```

En un array normal y denso, los índices válidos van desde `0` hasta `length - 1`.

10.4 Acceso y modificación por índice

Los corchetes permiten leer una posición:

```
const materias = ["Programación IV", "Base de Datos II", "Inglés II"];

console.log(materias[0]); // "Programación IV"
console.log(materias[1]); // "Base de Datos II"
```

También permiten modificarla:

```
const estados = ["pendiente", "pendiente", "pendiente"];

estados[1] = "aprobado";

console.log(estados);
// ["pendiente", "aprobado", "pendiente"]
```

Si el índice no existe, la lectura devuelve `undefined` en lugar de lanzar un error:

```
const notas = [8, 6, 9];

console.log(notas[10]); // undefined
```

Esto es cómodo y peligroso a la vez: un índice mal calculado no falla de inmediato, sino que propaga un `undefined` que puede aparecer varios pasos más adelante.

Posiciones vacías

Asignar muy lejos del final crea posiciones vacías, llamadas *holes*:

```
const valores = [10, 20];
valores[5] = 60;

console.log(valores.length); // 6
// [10, 20, <3 empty items>, 60]
```

Un array con huecos se llama **disperso**, y tiene comportamientos particulares al recorrerlo: algunos métodos saltan las posiciones vacías y otros las tratan como `undefined`. En código de aplicación conviene evitarlos y agregar elementos mediante operaciones explícitas.

Índices calculados

El índice puede ser cualquier expresión que produzca la posición deseada:

```
const notas = [8, 6, 9];
const indice = 2;

console.log(notas[indice]); // 9
console.log(notas[indice - 1]); // 6
```

Acceso desde el final

Los corchetes no interpretan índices negativos como posiciones desde el final:

```
notas[-1]; // undefined
```

El método `at()` sí lo hace:

```
notas.at(0); // 8
notas.at(-1); // 9
notas.at(-2); // 6
```

La forma tradicional, todavía muy usada:

```
notas[notas.length - 1]; // 9
```

10.5 Extraer un rango

JavaScript no tiene una sintaxis especial como `array[inicio:fin]`. El método habitual es `slice()`:


```
const letras = ["a", "b", "c", "d", "e"];

const centro = letras.slice(1, 4);

console.log(centro); // ["b", "c", "d"]
console.log(letras); // el original no cambia
```

El índice inicial se incluye y el final se excluye:

```
slice(1, 4)
   ↑  ↑
   incluye  excluye
```

Esta convención permite calcular la cantidad de elementos como `fin - inicio`, y hace que rangos consecutivos encajen sin superposición: `slice(0, 3)` y `slice(3, 6)` cubren seis elementos sin repetir ninguno.

Variantes frecuentes:

```
const datos = [10, 20, 30, 40, 50];

datos.slice(2);      // [30, 40, 50]
datos.slice(0, 3);   // [10, 20, 30]
datos.slice(-2);     // [40, 50]
datos.slice(1, -1);  // [20, 30, 40]
datos.slice();       // copia superficial completa
```

`slice()` no es `splice()`

Son dos métodos con nombres casi idénticos y comportamientos opuestos:

- `slice(inicio, fin)` extrae una copia y **no** modifica el original;
- `splice(inicio, cantidad, ...elementos)` elimina o inserta **modificando** el array original;
- `toSpliced()` ofrece una versión moderna que devuelve un array nuevo.

```
const original = ["a", "b", "c", "d"];

const nuevo = original.toSpliced(1, 2, "x", "y");

console.log(nuevo); // ["a", "x", "y", "d"]
console.log(original); // ["a", "b", "c", "d"]
```

`toSpliced()`, junto con `toSorted()`, `toReversed()` y `with()`, se incorporó recientemente. Conviene verificar el soporte del entorno de destino antes de usarlos.

10.6 Arrays de arrays y matrices

Un elemento de un array puede ser otro array:

```
const cursos = [
  ["Ana", "Luis", "Marta"],
  ["Pedro", "Julia"],
];
```

El primer índice selecciona el array exterior y el segundo un elemento interior:

```
console.log(cursos[0]); // ["Ana", "Luis", "Marta"]
console.log(cursos[0][1]); // "Luis"
console.log(cursos[1][0]); // "Pedro"
```

Una matriz puede representarse como un array de filas:

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

matriz[0][0]; // 1
matriz[1][2]; // 6
matriz[2][1]; // 8
```

JavaScript no exige que todas las filas tengan la misma longitud. Una matriz rectangular es una convención que el programa debe mantener:

```
const irregular = [
  [1, 2],
  [3, 4, 5],
];
```

Una trampa clásica

Esta forma parece crear tres filas independientes, pero no lo hace:

```
const incorrecta = Array(3).fill(Array(3).fill(0));

incorrecta[0][0] = 9;

console.log(incorrecta);
// [[9, 0, 0], [9, 0, 0], [9, 0, 0]]
```

Modificamos una fila y cambiaron las tres. `fill()` recibió **un** array y colocó una referencia a ese mismo array en cada posición. No hay tres filas: hay una fila referenciada tres veces.

La forma correcta construye una fila nueva por posición:

```
const matriz = Array.from(
  { length: 3 },
  () => Array(3).fill(0)
);

matriz[0][0] = 9;

console.log(matriz);
// [[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

La función se ejecuta tres veces y construye tres arrays diferentes.

Este es el primer encuentro con el concepto de referencia, que el capítulo 12 desarrolla por completo. Por ahora basta la regla: `fill()` con un objeto o un array repite la misma referencia.

Recorrer una matriz

Cuando necesitamos índices de fila y columna, un `for` anidado resulta claro:

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
];

for (let fila = 0; fila < matriz.length; fila += 1) {
  for (let columna = 0; columna < matriz[fila].length; columna += 1) {
    console.log(`matriz[${fila}][${columna}] = ${matriz[fila][columna]}`);
  }
}
```

Si solo importan los valores, `for...of` es más legible:

```
for (const fila of matriz) {
  for (const valor of fila) {
    console.log(valor);
  }
}
```

Y una transformación que no modifica el original puede anidar `map()`:

```
const duplicada = matriz.map((fila) => fila.map((valor) => valor * 2));
// [[2, 4, 6], [8, 10, 12]]
```

10.7 Tres formas de recorrer

El `for` tradicional

Reúne inicialización, condición y actualización:

```
const notas = [8, 6, 9];

for (let indice = 0; indice < notas.length; indice += 1) {
  console.log(indice, notas[indice]);
}
```

Es apropiado cuando necesitamos el índice numérico, un paso distinto de uno, un recorrido hacia atrás, comparar con elementos vecinos o controlar con precisión dónde empieza y termina:

```
for (let indice = notas.length - 1; indice >= 0; indice -= 1) {
  console.log(notas[indice]);
}
```

for...of: recorrer valores

Obtiene los valores producidos por un objeto iterable. Los arrays, las cadenas, los `Map` y los `Set` lo son, porque implementan el protocolo que vimos en la sección 8.7.

```
for (const nota of notas) {
  console.log(nota);
}
```

Cuando no necesitamos el índice, suele ser la opción imperativa más legible.

Si necesitamos índice y valor, podemos recorrer `entries()` y desestructurar cada par:

```
for (const [indice, nota] of notas.entries()) {
  console.log(`La nota ${indice + 1} es ${nota}`);
}
```

for...in: recorrer nombres de propiedades

`for...in` recorre claves de propiedades enumerables, expresadas como cadenas. No recorre valores.

```
const alumno = { nombre: "Ana", legajo: 12345, regular: true };

for (const propiedad in alumno) {
  console.log(propiedad, alumno[propiedad]);
}
```

No se recomienda `for...in` para arrays. Entrega los índices como cadenas y puede incluir propiedades adicionales:

```
const notas = [8, 6, 9];
notas.descripcion = "Primer parcial";

for (const clave in notas) {
  console.log(clave);
}
// "0", "1", "2", "descripcion"
```

Para arrays corresponden `for`, `for...of` o los métodos de la sección siguiente.

Comparación rápida

Construcción	Recorre	Uso típico
<code>for</code>	posiciones controladas por el programa	índices, pasos, recorrido inverso
<code>for...of</code>	valores de un iterable	arrays, Map, Set, cadenas
<code>for...in</code>	nombres de propiedades enumerables	inspección de objetos, con cuidado por la herencia

El capítulo 14 desarrolla las tres construcciones como estructuras de control.

10.8 Agregar y quitar elementos

Estos métodos **modifican** el array original:

```
const nombres = ["Ana", "Luis"];

nombres.push("Marta"); // agrega al final
nombres.pop();         // quita y devuelve el último
nombres.unshift("Pedro"); // agrega al comienzo
nombres.shift();       // quita y devuelve el primero
```

`push()` y `unshift()` devuelven la nueva longitud. `pop()` y `shift()` devuelven el elemento quitado, o `undefined` si el array estaba vacío.

`splice()` permite quitar, insertar o reemplazar en cualquier posición, y también muta:

```
const letras = ["a", "b", "d"];

letras.splice(2, 0, "c");

console.log(letras); // ["a", "b", "c", "d"]
```

Los argumentos son la posición, cuántos elementos eliminar y qué elementos insertar. Con `0` en el segundo lugar, solo inserta.

10.9 Buscar y comprobar

```
const notas = [8, 6, 9, 6];

notas.includes(9);           // true
notas.indexOf(6);           // 1
notas.lastIndexOf(6);       // 3
notas.find((nota) => nota < 7); // 6
notas.findIndex((nota) => nota < 7); // 1
notas.some((nota) => nota === 10); // false
notas.every((nota) => nota >= 6); // true
```

Cada uno expresa una intención distinta:

- `includes(valor)` pregunta si aparece ese valor exacto;
- `indexOf(valor)` devuelve su primera posición, o `-1` si no está;
- `find(predicado)` devuelve el primer elemento que cumple una condición;
- `findIndex(predicado)` devuelve su índice;
- `some(predicado)` pregunta si al menos uno cumple;
- `every(predicado)` pregunta si todos cumplen.

`find()`, `findIndex()`, `some()` y `every()` pueden terminar antes de recorrer toda la colección, en cuanto conocen el resultado. Es el mismo cortocircuito que vimos en la sección 4.7, aplicado a una colección.

Un detalle sobre `every()`: sobre un array vacío devuelve `true`, porque no hay ningún elemento que incumpla. Es la convención matemática y suele ser la respuesta deseada, pero conviene tenerla presente.

10.10 Transformar, seleccionar y acumular

Tres métodos cubren la enorme mayoría del procesamiento de colecciones. Los presentamos aquí en su uso práctico; el capítulo 18 desarrolla la idea que los sostiene, que es la de pasar funciones como argumentos.

`map()` : transformar

Aplica una función a cada elemento y produce un array nuevo con la misma cantidad de posiciones:

```
const precios = [1000, 2500, 800];

const preciosConIva = precios.map((precio) => precio * 1.21);

console.log(preciosConIva); // [1210, 3025, 968]
console.log(precios);      // [1000, 2500, 800] - sin cambios
```

La función recibe el valor, el índice y el array completo:

```
const etiquetas = precios.map(
  (precio, indice) => `Producto ${indice + 1}: ${precio}`
);
```

Se usa `map()` cuando cada elemento de entrada produce un elemento de salida.

filter(): seleccionar

Conserva únicamente los elementos para los cuales el predicado devuelve un valor *truthy*:

```
const notas = [8, 4, 6, 9, 3];

const aprobadas = notas.filter((nota) => nota >= 6);

console.log(aprobadas); // [8, 6, 9]
console.log(notas);     // sin cambios
```

El resultado puede tener desde cero hasta la misma cantidad de elementos que el original.

reduce(): acumular

Combina todos los elementos para obtener un único resultado:

```
const precios = [1000, 2500, 800];

const total = precios.reduce(
  (acumulado, precio) => acumulado + precio,
  0
);

console.log(total); // 4300
```

La estructura es:

```
reduce(función reductora, valor inicial)
```

En cada paso, la función recibe el acumulador y el elemento actual, y devuelve el acumulador siguiente:

```
acumulador inicial = 0
0    + 1000 = 1000
1000 + 2500 = 3500
3500 + 800  = 4300
```

Conviene proporcionar siempre un valor inicial: hace explícito el tipo del resultado y define qué ocurre con un array vacío. Sin valor inicial, `reduce()` sobre un array vacío lanza un error.

`reduce()` no sirve solamente para sumar. Puede construir objetos, agrupaciones o cualquier estructura, siempre que el resultado siga siendo legible.

`forEach()` : ejecutar efectos

Ejecuta una función por cada elemento y devuelve `undefined`:

```
const alumnos = ["Ana", "Luis", "Marta"];

alumnos.forEach((alumno, indice) => {
  console.log(`${indice + 1}. ${alumno}`);
});
```

Es apropiado para efectos: mostrar, registrar, enviar. Si queremos construir un array, `map()` o `filter()` comunican mejor la intención.

Dos limitaciones que conviene conocer. Un `return` dentro del callback no termina el `forEach()` completo, solo esa iteración. Y no admite `break`. Cuando se necesita detener el recorrido, corresponde `for...of`, `find()`, `some()` o `every()`, según la intención.

10.11 Aplanar y combinar

```
const grupos = [["Ana", "Luis"], ["Marta", "Pedro"]];

grupos.flat(); // ["Ana", "Luis", "Marta", "Pedro"]
```

`flat(profundidad)` reduce niveles de anidamiento:

```
[1, [2, [3]]].flat(1); // [1, 2, [3]]
[1, [2, [3]]].flat(2); // [1, 2, 3]
```

`flatMap()` equivale a transformar y luego aplanar un nivel:

```
const frases = ["hola mundo", "programación web"];

const palabras = frases.flatMap((frase) => frase.split(" "));
// ["hola", "mundo", "programación", "web"]
```


`concat()` une arrays sin modificar los originales:

```
const frontend = ["HTML", "CSS"];
const programacion = frontend.concat(["JavaScript"]);
// ["HTML", "CSS", "JavaScript"]
```

La sintaxis de expansión ofrece otra forma, hoy más frecuente:

```
const programacion = [...frontend, "JavaScript"];
```

10.12 Ordenar e invertir

`sort()` modifica el array y, sin comparador, ordena **como cadenas**:

```
const numeros = [10, 2, 1];

numeros.sort();
console.log(numeros); // [1, 10, 2]
```

El resultado no es un error: `sort()` convierte cada elemento a texto y compara lexicográficamente, tal como vimos en la sección 6.10. "10" viene antes que "2" porque "1" viene antes que "2".

Para números hay que proporcionar un comparador:

```
numeros.sort((a, b) => a - b); // ascendente
numeros.sort((a, b) => b - a); // descendente
```

El comparador debe devolver un número negativo si `a` va antes, positivo si va después y cero si son equivalentes para el orden.

Para textos en español conviene `localeCompare()`, por las razones de la sección 7.11:

```
const nombres = ["Ñandú", "árbol", "zapato"];
nombres.sort((a, b) => a.localeCompare(b, "es"));
```

`toSorted()` devuelve una copia ordenada y conserva el original:

```
const originales = [10, 2, 1];
const ordenados = originales.toSorted((a, b) => a - b);

console.log(ordenados); // [1, 2, 10]
console.log(originales); // [10, 2, 1]
```

De manera similar:

- `reverse()` muta y `toReversed()` copia;

- `splice()` muta y `toSpliced()` copia;
- asignar `array[indice]` muta y `with(indice, valor)` copia.

```
const notas = [8, 6, 9];
const corregidas = notas.with(1, 7);

console.log(corregidas); // [8, 7, 9]
console.log(notas);      // [8, 6, 9]
```

Qué muta y qué no

Esta tabla resume una distinción que el nombre del método no siempre permite adivinar:

Intención	Muta	Devuelve una estructura nueva
agregar al final	<code>push()</code>	<code>[...array, elemento]</code> , <code>concat()</code>
agregar al inicio	<code>unshift()</code>	<code>[elemento, ...array]</code>
quitar del final	<code>pop()</code>	<code>slice(0, -1)</code>
quitar del inicio	<code>shift()</code>	<code>slice(1)</code>
insertar o eliminar	<code>splice()</code>	<code>toSpliced()</code>
reemplazar por índice	<code>array[i] = valor</code>	<code>with(i, valor)</code> , <code>map()</code>
ordenar	<code>sort()</code>	<code>toSorted()</code>
invertir	<code>reverse()</code>	<code>toReversed()</code>
rellenar	<code>fill()</code>	crear con <code>map()</code> o <code>Array.from()</code>

El capítulo 12 explica cuándo conviene cada columna.

10.13 Desestructuración de arrays

La desestructuración permite extraer posiciones y vincularlas con nombres:

```
const coordenadas = [-26.83, -65.20];
const [latitud, longitud] = coordenadas;

console.log(latitud); // -26.83
console.log(longitud); // -65.20
```

La correspondencia es **posicional**: el primer nombre recibe el índice `0`, el segundo el índice `1`.

Omitir posiciones

```
const colores = ["rojo", "verde", "azul"];
const [, segundo] = colores;

console.log(segundo); // "verde"
```

La coma sola salta una posición.

Valores predeterminados

Se usan cuando la posición contiene `undefined` o no existe:

```
const [nombre = "Sin nombre", nota = 0] = ["Ana"];

console.log(nombre); // "Ana"
console.log(nota);   // 0
```

No reemplazan `null`, `false`, `0` ni la cadena vacía: solo `undefined`. Es la misma regla que rige `??`.

El resto de los elementos

```
const [primero, ...restantes] = [10, 20, 30, 40];

console.log(primero); // 10
console.log(restantes); // [20, 30, 40]
```

El elemento *rest* debe ser el último y siempre crea un array nuevo.

Intercambiar variables

```
let izquierda = "A";
let derecha = "B";

[izquierda, derecha] = [derecha, izquierda];

console.log(izquierda); // "B"
console.log(derecha);   // "A"
```

Sin variable auxiliar, en una línea.

Desestructuración anidada

```
const matriz = [
  [1, 2],
  [3, 4],
];

const [[primero], [, ultimo]] = matriz;

console.log(primero); // 1
console.log(ultimo); // 4
```

Un punto importante: **la desestructuración no crea copias profundas**. Si extraemos un objeto o un array interior, obtenemos una referencia a ese mismo valor. El capítulo 12 explica las consecuencias.

10.14 Síntesis conceptual

Un array representa una secuencia ordenada y usa índices que empiezan en cero, porque el índice expresa el desplazamiento desde el inicio. `length` indica la cantidad de posiciones, y el último índice de un array denso es `length - 1`.

Leer una posición inexistente devuelve `undefined` en lugar de fallar. Es cómodo y a la vez es la razón por la que un índice mal calculado puede manifestarse lejos de donde se originó.

`slice()` extrae un rango incluyendo el inicio y excluyendo el final, y no modifica el original. `splice()`, pese al nombre parecido, hace lo contrario en ambos aspectos.

Para recorrer hay tres construcciones y cada una tiene su lugar: `for` cuando necesitamos control explícito del índice, `for...of` cuando queremos los valores, y `for...in` para inspeccionar propiedades de objetos, nunca para arrays.

Los métodos que procesan colecciones se agrupan por intención: `map()` transforma uno a uno, `filter()` selecciona, `reduce()` acumula y `forEach()` produce efectos. Elegir el que corresponde a la intención hace que el código se lea como una descripción de lo que hace.

Finalmente, algunos métodos mutan el array y otros devuelven uno nuevo, y el nombre no siempre permite adivinarlo. Esa distinción es la puerta de entrada al capítulo 12, que explica por qué importa tanto.

El próximo capítulo pasa a la otra estructura fundamental: la que responde qué características tiene esta entidad.

Capítulo 11. Objetos: entidades con propiedades

Un array responde bien a la pregunta «¿qué valores hay en esta secuencia y en qué orden?». Pero muchas cosas que necesitamos representar no son secuencias.

Pensemos en un alumno. Tiene un nombre, un legajo y una condición de regularidad. Esos tres datos no son intercambiables ni se identifican naturalmente por una posición: decir «el elemento 1 del alumno» no significa nada. Lo que necesitamos es nombrar cada parte.

```
const alumno = {
  nombre: "Ana Pérez",
  legajo: 12345,
  regular: true,
  notas: [8, 6, 9],
};
```

Un **objeto** reúne asociaciones entre **claves** y **valores**. Cada asociación se llama propiedad.

La elección entre las dos estructuras debería responder al significado del dato:

- un **array** representa una secuencia ordenada de elementos equivalentes;
- un **objeto** representa una entidad mediante propiedades con nombre.

Aunque técnicamente en JavaScript los arrays son objetos especializados, conviene elegir según lo que el dato significa, no según lo que la sintaxis permite hacer.

11.1 Claves y valores

Las claves de un objeto son cadenas o símbolos. Cuando escribimos `id` o `precio` sin comillas dentro de un literal, JavaScript los interpreta como nombres de propiedades:

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 35_000,
  disponible: true,
};
```

Si una clave no es un identificador válido, hay que escribirla entre comillas:

```
const datos = {
  "nombre-completo": "Ana Pérez",
  "código postal": "4000",
};
```

Los valores pueden ser de cualquier tipo, incluidos otros objetos, arrays o funciones:

```
const alumno = {
  nombre: "Ana Pérez",
  contacto: {
    correo: "ana@example.com",
    telefono: null,
  },
  notas: [8, 6, 9],
  saludar() {
    return `Hola, soy ${this.nombre}`;
  },
};
```

Cuando el valor de una propiedad es una función, suele llamarse **método**.

En el ejemplo, una llamada como `alumno.saludar()` hace que `this` se refiera a `alumno`. El valor de `this` depende de **cómo se invoca** la función, no de dónde fue escrita; el capítulo 16 desarrolla ese mecanismo y explica por qué las funciones flecha se comportan distinto.

11.2 Crear, actualizar y eliminar propiedades

```
const alumno = {
  nombre: "Ana",
};

alumno.legajo = 12345; // agrega
alumno.nombre = "Ana Pérez"; // actualiza
delete alumno.legajo; // elimina
```

Que el objeto haya sido declarado con `const` no impide ninguna de estas operaciones. `const` impide asignar **otro objeto** a la variable `alumno`:

```
// alumno = { nombre: "Luis" }; // TypeError
```

Es la distinción que anticipamos en la sección 2.3 y que el capítulo 12 explica por completo.

Una nota sobre `delete`: es la única forma de quitar realmente una propiedad. Asignar `undefined` deja la propiedad presente con ese valor, lo cual es observable:

```
const a = { dato: undefined };
const b = {};

"dato" in a; // true
"dato" in b; // false
```

11.3 Acceso con punto y con corchetes

JavaScript ofrece dos sintaxis para acceder a una propiedad.

Notación de punto

```
const producto = { nombre: "Teclado", precio: 35_000 };  
  
console.log(producto.nombre);  
console.log(producto.precio);
```

Después del punto se escribe directamente el nombre de la propiedad, que debe tener forma de identificador válido. Es la opción preferible cuando la clave es conocida y fija: resulta breve y legible.

Notación de corchetes

```
console.log(producto["nombre"]);  
console.log(producto["precio"]);
```

Dentro de los corchetes se **evalúa una expresión**. Esa es toda la diferencia, y tiene una consecuencia importante:

```
const propiedadElegida = "precio";  
  
console.log(producto[propiedadElegida]); // 35000  
console.log(producto.propiedadElegida); // undefined
```

`producto.propiedadElegida` busca literalmente una propiedad llamada "propiedadElegida", que no existe. `producto[propiedadElegida]` primero evalúa la variable, obtiene "precio" y busca esa.

Esta confusión es frecuente al empezar. La regla es simple:

```
objeto.clave    → busca la propiedad llamada "clave"  
objeto[clave]   → evalúa clave y busca la propiedad con ese valor
```

Los corchetes también son necesarios para claves que no son identificadores válidos:

```
datos["nombre-completo"];  
datos["código postal"];
```

y para claves de tipo símbolo, como vimos en la sección 8.2.

11.4 Propiedades anidadas y encadenamiento opcional

```
const alumno = {
  nombre: "Ana",
  contacto: {
    correo: "ana@example.com",
  },
};

console.log(alumno.contacto.correo);
```

El problema aparece cuando una parte intermedia puede faltar:

```
const alumno = { nombre: "Ana" };

// console.log(alumno.contacto.correo);
// TypeError: no se puede leer una propiedad de undefined
```

`alumno.contacto` da `undefined`, y leer `.correo` sobre `undefined` lanza un error.

El **encadenamiento opcional** `?.` resuelve exactamente ese caso:

```
const correo = alumno.contacto?.correo;
console.log(correo); // undefined, sin error
```

Si el valor a la izquierda de `?.` es `null` o `undefined`, la expresión completa produce `undefined` y no se evalúa el resto. Combinado con `??`, da un patrón muy frecuente:

```
const telefono = alumno.contacto?.telefono ?? "Sin teléfono";
```

`?.` también funciona para índices y llamadas:

```
notas?.[0]; // acceso a índice
alumno.saludar?.(); // llama al método solo si existe
```

Una advertencia importante: `?.` **no reemplaza la validación**. Expresa que la ausencia es una posibilidad admitida en ese acceso concreto. Si un objeto que debería tener contacto no lo tiene, quizás el problema no sea leerlo de forma segura sino que llegó incompleto. Usar `?.` en todas partes puede ocultar errores de datos.

11.5 Comprobar si una propiedad existe

```
"nombre" in alumno; // true, incluye propiedades heredadas
Object.hasOwn(alumno, "nombre"); // true, solo propiedades propias
```


La diferencia importa cuando el objeto hereda propiedades a través de su cadena de prototipos:

```
"toString" in alumno;           // true – heredada de Object.prototype
Object.hasOwn(alumno, "toString"); // false – no es propia
```

Comparar con `undefined` no siempre distingue la ausencia de una propiedad presente cuyo valor es precisamente `undefined`:

```
const objeto = { dato: undefined };

objeto.dato === undefined; // true – pero la propiedad existe
Object.hasOwn(objeto, "dato"); // true
```

`Object.hasOwn()` es la comprobación más precisa y la que conviene usar por defecto.

11.6 Formas abreviadas de escribir objetos

Propiedades abreviadas

Cuando una variable y una propiedad deben tener el mismo nombre, puede evitarse la repetición:

```
const nombre = "Ana";
const legajo = 12345;

const formaCompleta = {
  nombre: nombre,
  legajo: legajo,
};

const formaAbreviada = {
  nombre,
  legajo,
};
```

`{ nombre }` significa «crear una propiedad llamada `nombre` cuyo valor proviene de la variable `nombre`».

Es muy útil al construir respuestas o resultados:

```
function crearAlumno(nombre, legajo) {
  return { nombre, legajo, regular: true };
}
```

Métodos abreviados

```
const contador = {
  valor: 0,

  incrementar() {
    this.valor += 1;
  },
};
```

La forma extensa sería `incrementar: function () { ... }`. La abreviada comunica con claridad que se trata de un método.

Claves calculadas

Dentro de un literal, los corchetes permiten calcular el nombre de una propiedad:

```
const campo = "correo";
const valor = "ana@example.com";

const contacto = {
  [campo]: valor,
};

console.log(contacto.correo); // "ana@example.com"
```

Son útiles para formularios, agrupaciones y transformaciones dinámicas:

```
const indice = 2;

const etiquetas = {
  [`campo${indice}`]: "valor",
};
// { campo2: "valor" }
```

Expansión de propiedades

La sintaxis `...` copia las propiedades enumerables propias dentro de otro objeto:

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 35_000,
};

const actualizado = {
  ...producto,
  precio: 32_000,
};
```

Si una propiedad aparece más de una vez, **gana el último valor**:

```
const a = { precio: 32_000, ...producto }; // precio queda en 35000
const b = { ...producto, precio: 32_000 }; // precio queda en 32000
```

El orden de las expansiones forma parte del significado. La forma habitual es expandir primero y sobrescribir después, como en `b`.

11.7 Funciones estándar para objetos

`Object.keys()` devuelve las claves propias, enumerables y de tipo cadena:

```
const alumno = {
  nombre: "Ana",
  legajo: 12345,
  regular: true,
};

Object.keys(alumno);
// ["nombre", "legajo", "regular"]
```

`Object.values()` devuelve los valores:

```
Object.values(alumno);
// ["Ana", 12345, true]
```

`Object.entries()` devuelve pares `[clave, valor]`:

```
Object.entries(alumno);
// [["nombre", "Ana"], ["legajo", 12345], ["regular", true]]
```

Esta última es especialmente útil, porque convierte un objeto en algo iterable y permite aplicarle todo lo que sabemos sobre arrays:

```
for (const [clave, valor] of Object.entries(alumno)) {
  console.log(`${clave}: ${valor}`);
}
```

`Object.fromEntries()` hace la transformación inversa:

```
const pares = [
  ["tema", "oscuro"],
  ["idioma", "es"],
];

const configuracion = Object.fromEntries(pares);
// { tema: "oscuro", idioma: "es" }
```

Combinadas, permiten transformar las propiedades de un objeto sin mutarlo:

```
const precios = {
  teclado: 35_000,
  mouse: 18_000,
};

const preciosConIva = Object.fromEntries(
  Object.entries(precios).map(([nombre, precio]) => [
    nombre,
    precio * 1.21,
  ])
);
```

El patrón `entries` → `map` → `fromEntries` es el equivalente de `map()` para objetos, y vale la pena reconocerlo.

Otras funciones importantes:

```
Object.assign({}, alumno, { regular: false });
// copia superficial y combina en el objeto destino

Object.freeze(alumno);
// impide cambios en las propiedades del primer nivel
```

`Object.freeze()` es **superficial**: no congela los objetos anidados.

```
const configuracion = Object.freeze({
  tema: "claro",
  avanzado: { depurar: false },
});

configuracion.tema = "oscuro"; // ignorado
configuracion.avanzado.depurar = true; // permitido
```

Un último detalle: los símbolos no aparecen en `Object.keys()`, `Object.values()` ni `Object.entries()`, como vimos en la sección 8.5. Se obtienen con `Object.getOwnPropertySymbols()`.

11.8 Recorrer las propiedades de un objeto

Un objeto común **no es iterable** con `for...of`:

```
const alumno = { nombre: "Ana", legajo: 12345 };

// for (const valor of alumno) {} // TypeError
```

La razón es la de la sección 8.7: no implementa `Symbol.iterator`. Hay dos caminos.

El primero es `for...in`, que recorre nombres de propiedades enumerables, incluidas las heredadas:

```
for (const propiedad in alumno) {
  if (!Object.hasOwn(alumno, propiedad)) {
    continue;
  }

  console.log(propiedad, alumno[propiedad]);
}
```

El segundo, más directo y hoy más frecuente, combina `Object.entries()` con `for...of`:

```
for (const [propiedad, valor] of Object.entries(alumno)) {
  console.log(propiedad, valor);
}
```

La segunda forma no necesita el filtro de propiedades propias, porque `Object.entries()` ya las devuelve solamente a ellas.

11.9 Desestructuración de objetos

La desestructuración de objetos extrae propiedades **por nombre**, no por posición:

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 35_000,
};

const { nombre, precio } = producto;

console.log(nombre); // "Teclado"
console.log(precio); // 35000
```

Como la correspondencia es por nombre, el orden del patrón no importa:

```
const { precio, nombre } = producto; // equivalente
```

Cambiar el nombre local

```
const { nombre: nombreProducto } = producto;

console.log(nombreProducto); // "Teclado"
```

La propiedad se llama `nombre`; la nueva variable se llama `nombreProducto`. Es útil para evitar colisiones o para dar un nombre más específico en el contexto de uso.

Valores predeterminados

```
const { stock = 0 } = producto;

console.log(stock); // 0
```

El predeterminado se usa solamente si la propiedad vale `undefined` o no existe, no si contiene `null`, `false`, `0` o `""`.

Propiedades restantes

```
const { id, ...datosEditables } = producto;

console.log(id); // 1
console.log(datosEditables); // { nombre: "Teclado", precio: 35000 }
```

`datosEditables` es un objeto nuevo con las propiedades enumerables propias restantes. Es un patrón muy usado para separar lo que no debe modificarse de lo que sí.

Desestructuración anidada

```
const alumno = {
  nombre: "Ana",
  contacto: {
    correo: "ana@example.com",
  },
};

const {
  nombre,
  contacto: { correo },
} = alumno;
```

Este patrón crea `nombre` y `correo`, pero **no** crea una variable llamada `contacto`. La parte `contacto:` indica por dónde bajar, no qué extraer.

Parámetros desestructurados

Una función puede expresar directamente qué propiedades usa:

```
function mostrarProducto({ nombre, precio, disponible = true }) {
  const estado = disponible ? "disponible" : "sin stock";
  return `${nombre}: ${precio} - ${estado}`;
}
```

Esto resulta claro para objetos pequeños y contratos conocidos. En patrones muy profundos puede ser preferible recibir el objeto completo y desestructurarlo en líneas separadas, para que la firma de la función siga siendo legible. El capítulo 16 vuelve sobre esta técnica.

11.10 Síntesis conceptual

Un objeto asocia claves con valores y modela una entidad con características nombradas. La elección entre array y objeto no debería depender de qué sintaxis resulta más cómoda, sino de si el dato es una secuencia de elementos equivalentes o una entidad con partes distinguibles.

Las dos notaciones de acceso hacen cosas distintas: el punto usa una clave fija y los corchetes evalúan una expresión. Confundirlas produce el error más común con objetos, y la regla `objeto.clave` frente a `objeto[clave]` lo evita por completo.

El encadenamiento opcional `?.` permite leer propiedades anidadas cuando una parte puede faltar, y se combina naturalmente con `??`. No sustituye a la validación: expresa que la ausencia está contemplada en ese punto.

Las formas abreviadas —propiedades cortas, métodos, claves calculadas y expansión— hacen que construir y actualizar objetos sea conciso. En la expansión, el orden decide qué valor sobrevive.

`Object.keys()`, `Object.values()` y `Object.entries()` tienden un puente entre objetos y arrays, y con `Object.fromEntries()` cierran el círculo. Ese puente es lo que permite aplicar `map()` y `filter()` a las propiedades de un objeto.

Finalmente, la desestructuración de objetos trabaja por nombre, admite renombrar, valores por defecto y propiedades restantes, y no crea copias profundas de lo que extrae.

Esa última observación es la que da pie al capítulo siguiente, que se ocupa del concepto que atraviesa a arrays y objetos por igual y explica varios comportamientos que hasta acá quedaron señalados sin desarrollar.

Capítulo 12. Referencias, copias e inmutabilidad

Los dos capítulos anteriores dejaron varias observaciones pendientes. Que `Array(3).fill([])` comparte la misma fila. Que la desestructuración no crea copias profundas. Que `const` no impide modificar un objeto. Que algunos métodos de array mutan y otros no.

Todas se explican con un solo concepto, y este capítulo se ocupa de él.

Los arrays y los objetos no se manejan como los primitivos. Una variable que contiene un número contiene el número; una variable que contiene un objeto contiene una **referencia** a ese objeto. Esa diferencia, que parece técnica, determina el comportamiento de buena parte del código que escribimos.

12.1 Valor e identidad

Los primitivos se comparan por su valor:

```
console.log(5 === 5);           // true
console.log("hola" === "hola"); // true
```

Dos veces el número cinco son el mismo cinco. No tiene sentido preguntarse si son «el mismo objeto cinco».

Los objetos y los arrays, en cambio, tienen **identidad**. Dos estructuras separadas no son iguales por contener los mismos datos:

```
console.log([1, 2] === [1, 2]); // false
console.log({ id: 1 } === { id: 1 }); // false
```

Cada literal crea un objeto diferente. La comparación con `===` sobre objetos no pregunta «¿contienen lo mismo?» sino «¿son el mismo objeto?».

Esto sorprende al principio, pero es coherente: dos personas con el mismo nombre no son la misma persona. Si queremos comparar contenidos, tenemos que escribir esa comparación nosotros o usar una biblioteca que la haga.

12.2 La asignación copia la referencia

Aquí está el punto central del capítulo:


```
const originales = [8, 6, 9];
const alias = originales;

alias.push(10);

console.log(originales);           // [8, 6, 9, 10]
console.log(alias);               // [8, 6, 9, 10]
console.log(alias === originales); // true
```

Agregamos un elemento a `alias` y cambió `originales`. No es que estén sincronizados: **no hay dos arrays**. Hay un solo array alcanzable mediante dos nombres.

```
originales ┌
           │ → [8, 6, 9, 10]
alias      └
```

Lo mismo ocurre con los objetos:

```
const original = { nombre: "Ana", regular: true };
const alias = original;

alias.regular = false;

console.log(original.regular); // false
console.log(alias === original); // true
```

Y ocurre también al pasar un objeto a una función, porque los argumentos se pasan igual que se asignan. El capítulo 16 desarrolla ese caso.

12.3 Copia superficial

Para obtener una estructura realmente separada hay que pedirla explícitamente.

Para arrays:

```
const original = [1, 2, 3];

const copia1 = [...original];
const copia2 = original.slice();
const copia3 = Array.from(original);

console.log(copia1 === original); // false
```

Ahora sí son estructuras distintas:

```
copia1.push(4);

console.log(original); // [1, 2, 3]
console.log(copia1);   // [1, 2, 3, 4]
```

Para objetos:

```
const original = { nombre: "Ana", regular: true };

const copia1 = { ...original };
const copia2 = Object.assign({}, original);

console.log(copia1 === original); // false
```

Hasta aquí todo funciona como esperábamos. El problema aparece un nivel más abajo.

12.4 Por qué «superficial» es una advertencia

Estas operaciones realizan una **copia superficial**: duplican el contenedor del primer nivel, pero no los objetos que están adentro.

```
const original = [
  { nombre: "Ana", nota: 8 },
  { nombre: "Luis", nota: 6 },
];

const copia = [...original];

copia[0].nota = 10;

console.log(original[0].nota); // 10
```

Los arrays exteriores son diferentes, pero `original[0]` y `copia[0]` apuntan al mismo objeto:

```
original → [ referencia A, referencia B ]
           |
copia    → [ referencia A, referencia B ]
           |
           ↓
        { nombre: "Ana", nota: 10 }
```

Con objetos ocurre exactamente lo mismo:

```
const original = {
  nombre: "Ana",
  contacto: { correo: "ana@example.com" },
};

const copia = { ...original };

console.log(copia === original); // false
console.log(copia.contacto === original.contacto); // true

copia.contacto.correo = "nuevo@example.com";
console.log(original.contacto.correo); // "nuevo@example.com"
```

Copiamos el objeto exterior. La propiedad `contacto` se copió, pero lo que se copió fue la referencia, no el objeto al que apunta.

Esta es también la explicación completa de la trampa de la sección 10.6:

```
Array(3).fill(Array(3).fill(0));
```

`fill()` recibe **un** array y coloca la misma referencia en las tres posiciones.

12.5 Copia profunda

Una copia profunda crea estructuras nuevas también en los niveles interiores.

Cuando los valores son compatibles con el algoritmo de clonación estructurada, puede usarse `structuredClone()`:

```
const original = [{ nombre: "Ana", notas: [8, 9] }];  
  
const copiaProfunda = structuredClone(original);  
  
copiaProfunda[0].notas.push(10);  
  
console.log(original[0].notas);    // [8, 9]  
console.log(copiaProfunda[0].notas); // [8, 9, 10]
```

`structuredClone()` no pertenece al núcleo de ECMAScript: es una API que ofrecen los navegadores modernos y Node.js. Maneja muchos tipos —incluidos `Date`, `Map`, `Set`, `ArrayBuffer` y las referencias circulares— pero **no puede clonar funciones**, elementos del DOM ni objetos con un prototipo particular que deba conservarse.

El atajo que conviene conocer y evitar

A veces se ve esta solución:

```
const copia = JSON.parse(JSON.stringify(original));
```

Funciona para datos simples y falla de maneras silenciosas en el resto de los casos. JSON no tiene representación para varios valores de JavaScript:

```
const original = {
  cuando: new Date("2026-08-30"),
  cuanto: undefined,
  cuantos: 10n,
  etiquetas: new Set(["a", "b"]),
  calcular() { return 1; },
};

JSON.parse(JSON.stringify(original));
// { cuando: "2026-08-30T00:00:00.000Z" }
```

La fecha se convirtió en cadena. `undefined` y la función desaparecieron. El `Set` quedó como objeto vacío. Y `10n` habría lanzado un error, porque `JSON.stringify` no admite `bigint`. Además, las referencias circulares producen una excepción.

No es una solución general de clonación. Sirve como conversión a JSON, que es otra cosa.

La mejor pregunta

Ante un caso concreto, la pregunta útil no siempre es «¿cómo copio todo?». Con frecuencia alcanza con copiar solamente las ramas que la actualización modifica, que es lo que veremos a continuación y suele ser más eficiente y más claro.

12.6 Mutación e inmutabilidad

Una operación **mutable** cambia la estructura existente:

```
const original = [1, 2, 3];
original.push(4); // mutación
```

Una operación **inmutable** conserva la anterior y produce una nueva:

```
const original = [1, 2, 3];
const nuevo = [...original, 4]; // actualización inmutable
```

Ningún enfoque es universalmente correcto. La mutación local puede ser simple y eficiente: acumular en un array dentro de un bucle es perfectamente razonable si ese array es local a la función.

La inmutabilidad resulta valiosa cuando:

- necesitamos conservar estados anteriores, para deshacer o comparar;
- varias partes del programa comparten una estructura;
- una interfaz detecta cambios comparando referencias, como hacen varias bibliotecas de componentes;
- queremos que una función no produzca efectos sobre sus argumentos;
- buscamos facilitar el razonamiento, las pruebas y la depuración.

La regla práctica que funciona bien: **mutar lo local, no mutar lo compartido**. Si una estructura fue creada dentro de la función y no sale de ella, mutarla no tiene consecuencias. Si llegó como argumento o se comparte con otra parte del programa, conviene producir una versión nueva.

12.7 Actualizar sin mutar

El patrón general consiste en copiar el nivel que cambia y reutilizar lo demás.

Un elemento de un array

```
const productos = [
  { id: 1, nombre: "Teclado", stock: 5 },
  { id: 2, nombre: "Mouse", stock: 8 },
];

const actualizados = productos.map((producto) =>
  producto.id === 2
    ? { ...producto, stock: producto.stock - 1 }
    : producto
);
```

El array es nuevo. El producto actualizado también es un objeto nuevo. El producto que no cambió **conserva su referencia**, lo que evita copiar trabajo innecesario y permite que una comparación por identidad detecte exactamente qué cambió.

Una propiedad anidada

```
const original = {
  nombre: "Ana",
  contacto: { correo: "ana@example.com" },
};

const actualizado = {
  ...original,
  contacto: {
    ...original.contacto,
    correo: "nuevo@example.com",
  },
};
```

Se copian el objeto exterior y el objeto `contacto`, porque ambos cambian en el nuevo estado. No hace falta clonar las propiedades que no se modifican.

La forma general del patrón:

```
copiar el contenedor exterior
↓
copiar cada nivel que esté en el camino del cambio
↓
reutilizar todo lo demás por referencia
```

Con estructuras muy profundas esto se vuelve tedioso, y es señal de que quizás la estructura de datos debería ser más plana.

12.8 `const`, `Object.freeze` y qué protege cada uno

Vale la pena poner los tres niveles juntos, porque se confunden:

```
const configuracion = { tema: "claro" };

configuracion.tema = "oscuro";      // permitido: cambia el objeto
// configuracion = { tema: "azul" }; // TypeError: cambia la vinculación
```

`const` protege el **nombre**. Impide que la variable apunte a otro objeto. No dice nada sobre el contenido.

```
const configuracion = Object.freeze({ tema: "claro" });

configuracion.tema = "oscuro";
console.log(configuracion.tema); // "claro" – el cambio se ignoró
```

`Object.freeze()` protege las **propiedades del primer nivel**. En modo estricto, que es el de los módulos, el intento lanza un `TypeError`; fuera de él se ignora en silencio.

```
const configuracion = Object.freeze({
  tema: "claro",
  avanzado: { depurar: false },
});

configuracion.avanzado.depurar = true; // permitido
```

`Object.freeze()` es superficial. Congelar en profundidad requiere recorrer la estructura:

```
function congelarProfundo(objeto) {
  for (const valor of Object.values(objeto)) {
    if (valor !== null && typeof valor === "object") {
      congelarProfundo(valor);
    }
  }

  return Object.freeze(objeto);
}
```

En la práctica, `Object.freeze()` se usa poco en código de aplicación. La disciplina de no mutar suele lograrse por convención y por revisión, no por bloqueo en tiempo de ejecución.

Los tres niveles, resumidos:

Herramienta	Protege	No protege
<code>const</code>	la vinculación del nombre	el contenido del objeto
<code>Object.freeze()</code>	las propiedades del primer nivel	los objetos anidados
<code>congelarProfundo()</code>	toda la estructura	nada, pero tiene costo

12.9 Confusiones frecuentes

Esperar que la asignación copie

```
const copia = original;
```

Crea otra referencia al mismo objeto. Para copiar hacen falta `[...original]`, `{...original}`, `slice()`, `Array.from()` u `Object.assign()`.

Esperar que `...` copie en profundidad

```
const copia = { ...original };
```

Copia un nivel. Los objetos interiores siguen compartidos.

Comparar objetos con `===` esperando comparar contenidos

```
const producto = { id: 1 };
const otro = { id: 1 };

console.log(producto === otro); // false
```

`===` sobre objetos compara identidad. Para comparar contenidos hay que escribirlo:

```
producto.id === otro.id && producto.nombre === otro.nombre;
```

Crear que `Set` elimina objetos con datos repetidos

```
new Set([ { id: 1 }, { id: 1 } ]).size; // 2
```

Son dos objetos distintos, así que el conjunto conserva ambos. El capítulo 13 desarrolla este caso.

Mutar un argumento sin advertirlo

```
function ordenar(notas) {  
  return notas.sort((a, b) => a - b); // muta el array de quien llamó  
}
```

La versión que no sorprende a nadie:

```
function ordenar(notas) {  
  return [...notas].sort((a, b) => a - b);  
}
```

o, en entornos que lo soporten:

```
function ordenar(notas) {  
  return notas.toSorted((a, b) => a - b);  
}
```

12.10 Síntesis conceptual

Los primitivos se comparan por valor; los objetos y arrays tienen identidad y se comparan por referencia. `[1, 2] === [1, 2]` es `false` porque son dos objetos distintos que casualmente contienen lo mismo.

Una asignación no duplica una estructura: copia la referencia. Dos variables pueden alcanzar el mismo objeto, y un cambio a través de una es visible a través de la otra. Lo mismo vale al pasar un objeto como argumento.

La expansión, `slice()`, `Array.from()` y `Object.assign()` producen copias superficiales: duplican el contenedor y comparten lo que hay adentro. Para duplicar en profundidad existe `structuredClone()`, con sus límites, y el atajo con JSON, que pierde información silenciosamente y conviene evitar.

La actualización inmutable copia el contenedor exterior y cada nivel en el camino del cambio, reutilizando todo lo demás. Es más trabajo de escritura y a cambio hace que el estado anterior siga siendo válido y que los cambios sean detectables comparando referencias.

Finalmente, `const` protege el nombre y `Object.freeze()` protege un nivel de propiedades. Ninguno de los dos vuelve inmutable a una estructura anidada.

Con esto, arrays y objetos están completos. El capítulo siguiente agrega las dos colecciones que resuelven problemas que ninguno de los dos resuelve bien.

Capítulo 13. Map y Set

Un objeto puede usarse como diccionario:

```
const preciosPorCodigo = {  
  A10: 1000,  
  B20: 2500,  
};
```

Y funciona. Pero los objetos también modelan entidades con propiedades conocidas, y esa doble función genera fricción: las claves solo pueden ser cadenas o símbolos, contar los elementos requiere un rodeo, no son iterables por defecto y hay que cuidarse de las propiedades heredadas.

`Map` y `Set` son colecciones diseñadas específicamente para dos problemas que los objetos y los arrays resuelven a medias: asociar claves con valores de manera dinámica, y mantener un conjunto de valores sin repetidos.

13.1 ¿Objeto o Map?

Necesidad	Objeto	Map
modelar una entidad con campos conocidos	excelente	poco habitual
claves de tipo cadena o símbolo	sí	sí
claves de cualquier tipo	no	sí
cantidad de elementos	<code>Object.keys(obj).length</code>	<code>map.size</code>
agregar y quitar con frecuencia	posible	diseñado para eso
iteración directa	no es iterable por defecto	sí
orden de inserción garantizado	parcialmente	sí
conversión directa a JSON	natural para datos compatibles	requiere transformación

Como regla orientativa:

- usar **objetos** para registros como `alumno`, `producto` o `configuracion`, donde los nombres de los campos forman parte del diseño;
- usar **Map** para asociaciones dinámicas donde las claves son datos: índices por identificador, cachés, conteos, agrupaciones.

Una pista útil: si los nombres de las propiedades los escribimos nosotros en el código, probablemente sea un objeto. Si vienen de los datos, probablemente sea un `Map`.

13.2 Map: pares clave-valor

Un `Map` conserva pares clave-valor en **orden de inserción**. Cualquier valor, incluso un objeto o una función, puede actuar como clave.

Crear y cargar

```
const stockPorCodigo = new Map();

stockPorCodigo.set("A10", 5);
stockPorCodigo.set("B20", 8);
```

`set()` agrega o reemplaza, y devuelve el mismo mapa, por lo que puede encadenarse:

```
const stock = new Map()
  .set("A10", 5)
  .set("B20", 8);
```

También puede inicializarse con un iterable de pares:

```
const stock = new Map([
  ["A10", 5],
  ["B20", 8],
]);
```

Esta última forma es la más frecuente, sobre todo combinada con `map()`, como veremos en la sección 13.9.

Operaciones frecuentes

```
stock.get("A10");    // 5
stock.has("A10");    // true
stock.size;          // 2
stock.delete("B20"); // true si existía
stock.clear();        // elimina todos los pares
```

`get()` devuelve `undefined` cuando la clave no existe. Si `undefined` puede ser un valor legítimamente almacenado, hay que usar `has()` para distinguir los dos casos:

```
const configuracion = new Map([["tiempoEspera", undefined]]);

configuracion.get("tiempoEspera"); // undefined
configuracion.has("tiempoEspera"); // true – la clave existe
configuracion.has("reintentos");   // false – no existe
```

13.3 Claves de cualquier tipo

Esta es la diferencia estructural con los objetos:

```
const usuario = { id: 1 };
const fecha = new Date();

const datos = new Map();

datos.set("texto", "una cadena como clave");
datos.set(42, "un número como clave");
datos.set(true, "un booleano como clave");
datos.set(usuario, "un objeto como clave");
datos.set(fecha, "una fecha como clave");
```

En un objeto, todas esas claves se habrían convertido a cadena, y `42` y `"42"` habrían colisionado. En un `Map` son claves distintas.

Cuando la clave es un objeto, la comparación es **por identidad**, con la lógica del capítulo 12:

```
const clave = { id: 1 };
const mapa = new Map();

mapa.set(clave, "encontrado");

mapa.get(clave); // "encontrado"
mapa.get({ id: 1 }); // undefined – es otro objeto
```

Para las claves primitivas, `Map` usa una comparación equivalente a `===` con una excepción: `NaN` se considera igual a sí mismo, así que se puede usar como clave y recuperarlo.

13.4 Recorrer un Map

Un `Map` es iterable, y cada elemento producido es un par `[clave, valor]`. La desestructuración resulta natural:

```
const precios = new Map([
  ["teclado", 35_000],
  ["mouse", 18_000],
]);

for (const [producto, precio] of precios) {
  console.log(`${producto}: ${precio}`);
}
```

También pueden recorrerse partes específicas:

```

for (const clave of precios.keys()) {
  console.log(clave);
}

for (const valor of precios.values()) {
  console.log(valor);
}

for (const entrada of precios.entries()) {
  console.log(entrada); // [clave, valor]
}

```

`forEach()` está disponible, aunque conviene notar que **el orden de sus parámetros es valor, clave**, invertido respecto de la iteración:

```

precios.forEach((precio, producto) => {
  console.log(`${producto}: ${precio}`);
});

```

13.5 Convertir entre Map, arrays y objetos

```

const entradas = [...precios];
// [["teclado", 35000], ["mouse", 18000]]

const comoObjeto = Object.fromEntries(precios);
// { teclado: 35000, mouse: 18000 }

const nuevamenteMapa = new Map(Object.entries(comoObjeto));

```

Como un `Map` es iterable de pares, se conecta directamente con `Object.fromEntries()` y con `Object.entries()`.

La conversión a objeto solo conserva fielmente las claves que puedan expresarse como cadenas o símbolos. Un `Map` con claves numéricas u objetos pierde información al convertirse.

Esto importa especialmente al serializar:

```

JSON.stringify(precios); // "{}" – no serializa el contenido
JSON.stringify(Object.fromEntries(precios));
// '{"teclado":35000,"mouse":18000}'

```

`JSON.stringify()` no sabe qué hacer con un `Map`. Hay que convertirlo primero.

13.6 Actualizar un Map sin mutarlo

`set()` muta el mapa. Para conservar el anterior, se crea primero una copia:

```
const original = new Map([["A10", 5]]);

const actualizado = new Map(original);
actualizado.set("A10", 4);

console.log(original.get("A10")); // 5
console.log(actualizado.get("A10")); // 4
```

`new Map(otroMapa)` produce una copia superficial: los valores interiores siguen compartiendo referencias si son objetos, con todas las consecuencias del capítulo 13.

13.7 Set: colecciones sin repetidos

Un `Set` almacena valores únicos y conserva el orden en que fueron agregados por primera vez.

```
const categorias = new Set();

categorias.add("hardware");
categorias.add("software");
categorias.add("hardware");

console.log(categorias.size); // 2
```

El segundo `"hardware"` no crea otra entrada ni produce un error: simplemente no hace nada.

Operaciones frecuentes:

```
categorias.has("hardware"); // true
categorias.delete("software"); // true si existía
categorias.clear(); // elimina todos los valores
```

`add()` devuelve el mismo conjunto y permite encadenar:

```
const permisos = new Set()
  .add("leer")
  .add("editar");
```

Una ventaja de rendimiento que conviene conocer: `set.has(valor)` es una búsqueda directa, mientras que `array.includes(valor)` recorre el array. Para comprobaciones de pertenencia repetidas sobre colecciones grandes, la diferencia es significativa.

13.8 Eliminar duplicados

El uso más frecuente de `Set` es este:

```
const etiquetas = ["web", "js", "web", "backend", "js"];
const unicas = [...new Set(etiquetas)];

console.log(unicas); // ["web", "js", "backend"]
```

El `Set` elimina las repeticiones y la expansión vuelve a producir un array. Es un idioma que conviene reconocer de inmediato.

La identidad vuelve a aparecer

Dos objetos con los mismos datos siguen siendo valores diferentes:

```
const primero = { id: 1 };
const segundo = { id: 1 };

const conjunto = new Set([primero, segundo]);

console.log(conjunto.size); // 2
```

Agregar dos veces **la misma referencia** sí evita la repetición:

```
const conjunto = new Set([primero, primero]);
console.log(conjunto.size); // 1
```

Si buscamos unicidad por `id`, hay que trabajar sobre los identificadores o construir una asociación apropiada:

```
const productos = [
  { id: 1, nombre: "Teclado" },
  { id: 2, nombre: "Mouse" },
  { id: 1, nombre: "Teclado" },
];

const unicos = [...new Map(productos.map((p) => [p.id, p])).values()];

console.log(unicos.length); // 2
```

El `Map` usa el `id` como clave, así que la segunda aparición reemplaza a la primera. Después extraemos los valores.

13.9 Recorrer un Set

```
const tecnologias = new Set(["HTML", "CSS", "JavaScript"]);

for (const tecnologia of tecnologias) {
  console.log(tecnologia);
}
```

También dispone de `values()`, `keys()`, `entries()` y `forEach()`. En un conjunto, `keys()` y `values()` producen los mismos valores; existen por compatibilidad con la interfaz de `Map`.

13.10 Operaciones de conjuntos

Los entornos modernos incluyen operaciones matemáticas que producen conjuntos nuevos sin modificar los originales:

```
const frontend = new Set(["Ana", "Luis", "Marta"]);
const backend = new Set(["Luis", "Pedro"]);

frontend.union(backend);
// Set { "Ana", "Luis", "Marta", "Pedro" }

frontend.intersection(backend);
// Set { "Luis" }

frontend.difference(backend);
// Set { "Ana", "Marta" }

frontend.symmetricDifference(backend);
// Set { "Ana", "Marta", "Pedro" }

frontend.isSubsetOf(backend); // false
frontend.isSupersetOf(backend); // false
frontend.isDisjointFrom(backend); // false
```

Estas operaciones se incorporaron recientemente, así que conviene comprobar el entorno de destino. Una intersección compatible con entornos anteriores puede expresarse mediante arrays:

```
const interseccion = new Set(
  [...frontend].filter((persona) => backend.has(persona))
);
```

Actualizar sin mutar

`add()`, `delete()` y `clear()` mutan. Para conservar el conjunto anterior:


```
const original = new Set(["leer"]);
const actualizado = new Set(original);

actualizado.add("editar");

console.log(original);    // Set { "leer" }
console.log(actualizado); // Set { "leer", "editar" }
```

13.11 WeakMap y WeakSet

Existen dos variantes que conviene al menos reconocer.

`WeakMap` y `WeakSet` solo admiten objetos como claves —o como elementos— y mantienen esas referencias de manera **débil**: si el objeto deja de ser alcanzable desde cualquier otro lugar del programa, el recolector de basura puede recuperarlo aunque siga en la colección.

```
const metadatos = new WeakMap();

let elemento = { id: 1 };
metadatos.set(elemento, { visitas: 0 });

elemento = null;
// el objeto y su entrada en metadatos pueden ser recolectados
```

Con un `Map` normal, la entrada mantendría vivo al objeto para siempre y produciría una fuga de memoria.

A cambio de esa propiedad, no son iterables ni tienen `size`: permitirlo haría observable el momento en que actúa el recolector de basura.

Su uso típico es asociar metadatos a objetos que no controlamos, exactamente el problema que en la sección 8.3 resolvimos con símbolos. La diferencia es que `WeakMap` no modifica el objeto en absoluto.

13.12 Elegir una estructura

Estructura	Pregunta que responde	Acceso principal
Array	«¿Qué valores aparecen en esta secuencia y en qué orden?»	índice numérico
Objeto	«¿Cuáles son las características de esta entidad?»	nombre de propiedad
Map	«¿Qué valor está asociado dinámicamente con esta clave?»	clave de cualquier tipo
Set	«¿Este valor pertenece al conjunto?»	valor único

Ejemplos:

```
// Secuencia ordenada
const notas = [8, 6, 9];

// Entidad con campos conocidos
const alumno = {
  legajo: 12345,
  nombre: "Ana",
};

// Asociación dinámica por identificador
const alumnosPorLegajo = new Map([[12345, alumno]]);

// Pertenencia sin duplicados
const permisos = new Set(["leer", "editar"]);
```

No existe una estructura mejor en abstracto. Existe una estructura que expresa mejor la relación que necesita el dominio.

13.13 Un caso integrador: el carrito de compras

Este ejemplo reúne las cuatro estructuras de la parte III.

Un catálogo se representa como un array de objetos:

```
const productos = [
  { id: 1, nombre: "Teclado", precio: 35_000, categoria: "hardware" },
  { id: 2, nombre: "Mouse", precio: 18_000, categoria: "hardware" },
  { id: 3, nombre: "Curso JS", precio: 25_000, categoria: "software" },
];
```

Un `Map` permite encontrar rápidamente un producto por identificador:

```
const productosPorId = new Map(
  productos.map((producto) => [producto.id, producto])
);
```

Sin el `Map`, cada búsqueda recorrería el array completo. Con él, es directa.

Un `Set` permite obtener las categorías únicas:

```
const categorias = new Set(
  productos.map((producto) => producto.categoria)
);

console.log([...categorias]);
// ["hardware", "software"]
```

El carrito almacena líneas con producto y cantidad:

```
const carrito = [
  { productId: 1, cantidad: 2 },
  { productId: 3, cantidad: 1 },
];
```

Construimos el detalle combinando `map()`, desestructuración y acceso al mapa:

```
const detalle = carrito.map(({ productId, cantidad }) => {
  const producto = productosPorId.get(productId);

  if (!producto) {
    throw new Error(`No existe el producto ${productId}`);
  }

  const { nombre, precio } = producto;

  return {
    productId,
    nombre,
    precioUnitario: precio,
    cantidad,
    subtotal: precio * cantidad,
  };
});
```

Y calculamos el total con `reduce()`:

```
const total = detalle.reduce(
  (acumulado, linea) => acumulado + linea.subtotal,
  0
);
```

La actualización de una cantidad, sin mutar el carrito original:

```
const carritoActualizado = carrito.map((linea) =>
  linea.productId === 1
    ? { ...linea, cantidad: linea.cantidad + 1 }
    : linea
);
```

Cada estructura hace lo que hace bien:

- array para las secuencias de productos y de líneas;
- objeto para describir cada entidad;
- Map para localizar por clave sin recorrer;
- Set para obtener valores únicos;
- desestructuración para extraer los datos que interesan;
- map() para transformar y reduce() para agregar;
- expansión para actualizar sin mutar.

13.14 Síntesis conceptual

Map modela asociaciones dinámicas entre claves y valores. Admite claves de cualquier tipo, conserva el orden de inserción, expone size y es iterable. Es la estructura correcta cuando las claves vienen de los datos y no del diseño del programa.

Cuando la clave es un objeto, la comparación es por identidad. Eso es una ventaja — permite indexar por el objeto mismo— y a la vez la razón por la que un objeto recién construido nunca encuentra su entrada.

Set modela pertenencia y conserva valores únicos, también por identidad. Su uso más frecuente es eliminar duplicados de un array con [...new Set(array)], y su ventaja menos visible es que has() no recorre la colección.

Ninguna de las dos se serializa directamente a JSON: hay que convertirlas antes. Y ninguna de las dos ofrece actualización inmutable propia: se copia el constructor y se modifica la copia.

WeakMap y WeakSet cubren un caso específico: asociar datos a objetos sin impedir que se liberen.

Con esto termina el recorrido por las estructuras de datos. La parte siguiente cambia de tema: pasa de cómo se organizan los datos a cómo se organiza la ejecución.

Parte III — Control del flujo

Capítulo 14. Decisiones y repeticiones

Un programa sencillo ejecuta sus instrucciones en el orden en que fueron escritas. Una aplicación real necesita algo más: elegir caminos, repetir tareas, abandonar una operación o transferir el control cuando algo sale mal.

El **flujo de control** es el orden en el que se ejecutan las instrucciones. Las estructuras de control permiten modificar ese orden de manera explícita:

- `if`, `else` y `switch` seleccionan caminos;
- `while`, `do...while` y `for` repiten instrucciones;
- `break` y `continue` alteran una repetición;
- `return` termina una función;
- `throw` interrumpe el flujo normal y propaga una excepción.

Este capítulo estudia las primeras cuatro. El siguiente se ocupa de las excepciones, que merecen un tratamiento propio.

El objetivo no es solamente aprender la sintaxis, sino poder elegir la forma que haga visible la regla del problema. Casi siempre hay varias maneras correctas de escribir la misma decisión, y no todas se leen igual de bien.

14.1 El flujo normal y sus desvíos

En ausencia de estructuras de control, JavaScript ejecuta una sentencia y continúa con la siguiente:

```
const precio = 1000;
const cantidad = 3;
const total = precio * cantidad;

console.log(total);
```

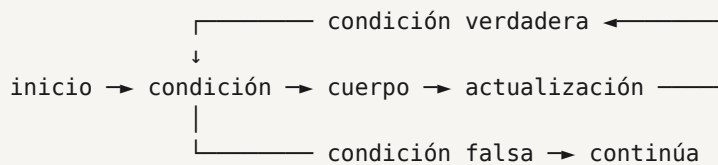
Podemos representar ese flujo como una línea:

```
instrucción 1 → instrucción 2 → instrucción 3 → fin
```

Una condición agrega una bifurcación:

```
inicio → decisión {
  ┌ condición verdadera → camino A ┐
  └ condición falsa → camino B ┘ ──> continúa
```

Un bucle agrega un regreso controlado:



Y una excepción produce una salida abrupta que busca un manejador:

```
función C — throw
  ↑ descarta su ejecución pendiente
función B
  ↑ propaga
función A — catch → maneja o vuelve a lanzar
```

Todas estas construcciones responden a la misma pregunta: **¿cuál es la próxima instrucción que debe ejecutarse?**

14.2 if: ejecutar cuando se cumple una condición

```
if (condicion) {
  // se ejecuta si la condición es truthy
}
```

Por ejemplo:

```
const edad = 20;

if (edad >= 18) {
  console.log("Puede registrarse");
}
```

La condición puede ser cualquier expresión. JavaScript la interpreta en un contexto booleano, con las reglas del capítulo 4: si es *truthy*, ejecuta el bloque; si es *falsy*, lo omite.

```
const nombre = "Ana";

if (nombre) {
  console.log(`Hola, ${nombre}`);
}
```

Las llaves son opcionales para una sola sentencia, pero conviene usarlas siempre. Sin ellas, agregar una segunda línea más adelante produce un error silencioso:

```
if (edad >= 18)
  console.log("Puede registrarse");
  console.log("Bienvenido"); // se ejecuta siempre, sin importar la edad
```

Condiciones expresivas

Una condición debería leerse como una pregunta del dominio:

```
const tieneEdadPermitida = edad >= 18;
const tieneDocumento = documento !== null;
const puedeRegistrarse = tieneEdadPermitida && tieneDocumento;

if (puedeRegistrarse) {
  registrarUsuario();
}
```

Nombrar una condición compleja comunica mejor la regla y permite depurar cada parte por separado.

No hace falta comparar un booleano con `true`:

```
if (estaActivo) {
  // ...
}

// evitar, salvo que exista una razón particular:
if (estaActivo === true) {
  // ...
}
```

La comparación explícita sí tiene sentido cuando queremos distinguir el booleano `true` de otros valores *truthy*, por ejemplo al validar datos que pueden llegar como `"true"` en texto.

14.3 `else`: elegir entre dos caminos

```
if (saldo >= importe) {
  console.log("Compra aprobada");
} else {
  console.log("Saldo insuficiente");
}
```

Solamente uno de los dos bloques se ejecuta:

```
saldo >= importe
├─ true  → compra aprobada
└─ false → saldo insuficiente
```

`else` no lleva condición propia. Representa todos los casos que el `if` anterior no aceptó.

El operador condicional

Cuando la decisión produce uno de dos **valores**, el operador ternario suele ser más directo:

```
const estado = nota >= 6 ? "aprobado" : "desaprobado";
```

Su forma es:

```
condición ? valorSiVerdadera : valorSiFalsa
```

La versión con `if...else` necesita una variable reasignable:

```
let estado;

if (nota >= 6) {
  estado = "aprobado";
} else {
  estado = "desaprobado";
}
```

Aquí aparece la distinción de la sección 3.1. El ternario es una **expresión**: produce un valor y por eso puede inicializar una `const`. `if` es una **sentencia**: controla qué bloque se ejecuta.

Conviene usar el ternario para una elección breve. Los ternarios anidados suelen esconder la jerarquía de condiciones:

```
// difícil de leer
const categoria = nota >= 9
  ? "destacado"
  : nota >= 6
    ? "aprobado"
    : "desaprobado";
```

En ese caso, un encadenamiento o una función con guardas resulta más claro.

14.4 Encadenamiento con `else if`

Cuando hay más de dos alternativas excluyentes, las condiciones se evalúan en secuencia:

```
function clasificarNota(nota) {
  if (nota >= 9) {
    return "destacado";
  } else if (nota >= 6) {
    return "aprobado";
  } else if (nota >= 4) {
    return "recupera";
  } else {
    return "desaprobado";
  }
}
```

JavaScript prueba las condiciones de arriba hacia abajo y ejecuta solamente el primer bloque cuya condición sea *truthy*.

El orden forma parte del significado. Esta versión está mal ordenada:

```
if (nota >= 6) {
  return "aprobado";
} else if (nota >= 9) {
  return "destacado"; // nunca se alcanza
}
```

Toda nota mayor o igual que nueve también es mayor o igual que seis, así que el primer caso la captura antes. La regla general: las condiciones más específicas van primero.

Una cadena `else if` es apropiada cuando las alternativas son mutuamente excluyentes, hay una prioridad clara y solo debe ejecutarse una rama.

Si varias condiciones independientes pueden cumplirse a la vez, corresponden varios `if` separados:

```
if (tieneErrores) {
  mostrarErrores();
}

if (tieneAdvertencias) {
  mostrarAdvertencias();
}
```

14.5 Condiciones anidadas

Un `if` puede aparecer dentro de otro:

```

if (usuario !== null) {
  if (usuario.estaActivo) {
    if (usuario.esAdministrador) {
      mostrarPanelDeAdministracion();
    }
  }
}

```

El anidamiento expresa que una pregunta solamente tiene sentido después de responder otra. No es incorrecto en sí mismo, pero demasiados niveles desplazan la lógica principal hacia la derecha y obligan a recordar todas las condiciones abiertas.

Combinar condiciones

Si las condiciones describen una misma decisión, pueden combinarse:

```

if (
  usuario !== null &&
  usuario.estaActivo &&
  usuario.esAdministrador
) {
  mostrarPanelDeAdministracion();
}

```

El cortocircuito de `&&`, visto en la sección 4.7, evita acceder a `usuario.estaActivo` cuando `usuario` es `null`.

Con encadenamiento opcional queda todavía más breve:

```

if (usuario?.estaActivo && usuario.esAdministrador) {
  mostrarPanelDeAdministracion();
}

```

No todo anidamiento puede reemplazarse mecánicamente por `&&`. Si cada nivel tiene un `else` diferente, las ramas representan decisiones distintas y hay que conservarlas o reorganizarlas con cuidado.

Leyes de De Morgan

Las equivalencias del apéndice A ayudan a simplificar negaciones:

```

!(a && b) equivale a !a || !b
!(a || b) equivale a !a && !b

```

Por ejemplo:

```
if (!(esDocente || esAlumno)) {  
    rechazarAcceso();  
}  
  
// equivale a:  
if (!esDocente && !esAlumno) {  
    rechazarAcceso();  
}
```

La mejor versión es la que expresa con mayor claridad la regla del dominio. «Ninguno de los dos roles» y «no es docente y tampoco alumno» dicen lo mismo, y según el contexto una se lee mejor que la otra.

14.6 Guardas y retorno temprano

Una **guarda** verifica al comienzo una condición que impide continuar. Si se cumple, la función termina temprano.

Comparemos las dos formas. Versión anidada:

```
function procesarCompra(usuario, carrito) {  
    if (usuario !== null) {  
        if (usuario.estaActivo) {  
            if (carrito.length > 0) {  
                return confirmarCompra(usuario, carrito);  
            } else {  
                return { ok: false, motivo: "carrito vacío" };  
            }  
        } else {  
            return { ok: false, motivo: "usuario inactivo" };  
        }  
    } else {  
        return { ok: false, motivo: "falta usuario" };  
    }  
}
```

Versión con guardas:

```
function procesarCompra(usuario, carrito) {
  if (usuario === null) {
    return { ok: false, motivo: "falta usuario" };
  }

  if (!usuario.estaActivo) {
    return { ok: false, motivo: "usuario inactivo" };
  }

  if (carrito.length === 0) {
    return { ok: false, motivo: "carrito vacío" };
  }

  return confirmarCompra(usuario, carrito);
}
```

Las dos hacen exactamente lo mismo. La segunda deja el camino principal al nivel izquierdo, separa cada impedimento con su motivo al lado, y permite agregar una condición nueva sin reindentar nada.

Además, en la primera versión hay que leer hasta el final para descubrir qué pasa cuando todo sale bien. En la segunda, la última línea lo dice.

Retornar o lanzar

Una guarda puede devolver un resultado esperado:

```
if (carrito.length === 0) {
  return { ok: false, motivo: "carrito vacío" };
}
```

O puede lanzar una excepción si continuar violaría el contrato de la función:

```
function calcularPromedio(notas) {
  if (!Array.isArray(notas)) {
    throw new TypeError("notas debe ser un array");
  }

  if (notas.length === 0) {
    throw new RangeError("no se puede promediar un array vacío");
  }

  return notas.reduce((suma, nota) => suma + nota, 0) / notas.length;
}
```

La elección depende del contrato: un resultado de negocio previsible suele devolverse; una imposibilidad para cumplir la operación puede representarse mediante una excepción. El capítulo 15 desarrolla ese criterio.

14.7 while: repetir mientras se cumpla una condición

```
while (condicion) {  
    // cuerpo  
}
```

`while` evalúa la condición **antes** de cada repetición. Si inicialmente es *falsy*, el cuerpo no se ejecuta ninguna vez.

```
let numero = 1;  
  
while (numero <= 5) {  
    console.log(numero);  
    numero += 1;  
}
```

Orden de ejecución:

1. evaluar condición
2. si es falsa, terminar
3. ejecutar cuerpo
4. volver al paso 1

El progreso debe ser visible

Todo bucle necesita una razón para terminar. En el ejemplo, `numero += 1` acerca el estado a la condición de salida.

```
let numero = 1;  
  
while (numero <= 5) {  
    console.log(numero);  
    // falta actualizar numero: bucle infinito  
}
```

No todos los bucles avanzan mediante un contador. También pueden consumir una colección o cambiar un estado:

```
const tareas = ["validar", "guardar", "notificar"];  
  
while (tareas.length > 0) {  
    const tarea = tareas.shift();  
    console.log(`Procesando: ${tarea}`);  
}
```

Esta versión muta la cola: cada `shift()` la acorta, y por eso el bucle termina. Si la colección original debe conservarse, conviene trabajar sobre una copia.

Cuándo elegir `while`

Resulta natural cuando no sabemos de antemano cuántas repeticiones harán falta:

- reintentar hasta alcanzar un límite o tener éxito;
- consumir una cola mientras tenga elementos;
- leer hasta encontrar una marca de finalización;
- repetir un algoritmo hasta que su estado converja;
- mantener un servicio mientras continúe activo.

```
let intentos = 0;
let autenticado = false;

while (intentos < 3 && !autenticado) {
  autenticado = intentarAutenticacion();
  intentos += 1;
}
```

La condición combina las dos razones para detenerse, y por eso el bucle no puede quedarse girando ni terminar antes de tiempo.

14.8 `do...while`: ejecutar al menos una vez

```
do {
  // cuerpo
} while (condicion);
```

Evalúa la condición **después** del cuerpo, así que siempre realiza al menos una iteración:

```
let pagina = 1;
const totalPaginas = 3;

do {
  console.log(`Procesando página ${pagina}`);
  pagina += 1;
} while (pagina <= totalPaginas);
```

Incluso si la condición inicial fuera falsa, el cuerpo se ejecutaría una vez:

```
let valor = 10;

do {
  console.log(valor); // 10
} while (valor < 5);
```

Es útil cuando la primera acción debe ocurrir antes de poder decidir si se repite: mostrar un menú y luego evaluar la opción, solicitar un dato y luego validarlo, ejecutar una operación y decidir si debe reintentarse, procesar la primera página antes de saber si

existe otra.

Bucle	Momento de la condición	Mínimo de ejecuciones
while	antes del cuerpo	0
do...while	después del cuerpo	1

14.9 for: reunir las tres partes

Muchos bucles con contador comparten tres elementos: inicialización, condición y actualización. Con `while` aparecen dispersos:

```
let indice = 0;

while (indice < alumnos.length) {
  console.log(alumnos[indice]);
  indice += 1;
}
```

El `for` los reúne en su cabecera:

```
for (let indice = 0; indice < alumnos.length; indice += 1) {
  console.log(alumnos[indice]);
}
```

Su forma general:

```
for (inicializacion; condicion; actualizacion) {
  // cuerpo
}
```

Y el orden real de ejecución:

1. inicialización una sola vez
2. condición antes de cada vuelta
3. cuerpo si la condición es truthy
4. actualización después del cuerpo
5. volver al paso 2

La ventaja no es la brevedad sino la localidad: las tres partes que gobiernan el bucle están juntas, y la variable de control no sobrevive fuera de él.

`for` es natural cuando la progresión está controlada por un contador:


```

for (let numero = 1; numero <= 10; numero += 1) {
  console.log(numero);
}

for (let par = 0; par <= 20; par += 2) {
  console.log(par);
}

for (let indice = alumnos.length - 1; indice >= 0; indice -= 1) {
  console.log(alumnos[indice]);
}

```

Las tres expresiones son opcionales:

```

for (;;) {
  if (debeTerminar()) {
    break;
  }
}

```

Eso representa un bucle infinito equivalente a `while (true)`, y exige una salida clara dentro del cuerpo.

14.10 break: terminar una repetición

`break` abandona inmediatamente el bucle más cercano:

```

const numeros = [3, 8, -1, 10];

for (const numero of numeros) {
  if (numero < 0) {
    break;
  }

  console.log(numero);
}
// Muestra 3 y 8

```

-1 actúa como señal de finalización. El 10 nunca se procesa.

Un bucle deliberadamente infinito puede terminar mediante una guarda y `break`:

```

let indice = 0;

while (true) {
  if (indice >= datos.length) {
    break;
  }

  procesar(datos[indice]);
  indice += 1;
}

```

Esta forma es válida cuando la condición de salida resulta más clara dentro del cuerpo, por ejemplo si depende de algo que hay que calcular primero. Pero si puede expresarse en la cabecera, es preferible:

```

while (indice < datos.length) {
  procesar(datos[indice]);
  indice += 1;
}

```

La condición en la cabecera se lee de un vistazo; la que está enterrada en el cuerpo hay que buscarla.

14.11 continue: pasar a la siguiente iteración

`continue` omite el resto del cuerpo actual y comienza la próxima iteración:

```

const valores = [10, null, 20, undefined, 30];
let total = 0;

for (const valor of valores) {
  if (valor === null || valor === undefined) {
    continue;
  }

  total += valor;
}

console.log(total); // 60

```

Puede verse como una guarda dentro del bucle: descarta temprano los elementos que no deben procesarse.

Sin `continue`, el mismo código necesita anidamiento:

```

for (const valor of valores) {
  if (valor !== null && valor !== undefined) {
    total += valor;
  }
}

```

Las dos versiones son correctas. `continue` resulta útil cuando hay varias condiciones de descarte y el cuerpo principal es largo.

Una trampa con `while`

En un `for`, después de `continue` se ejecuta la expresión de actualización y luego se vuelve a evaluar la condición. En un `while`, el salto va directamente a la condición, así que hay que asegurarse de no omitir la actualización:

```
let indice = 0;

while (indice < valores.length) {
  const valor = valores[indice];
  indice += 1; // actualizar antes de cualquier continue

  if (valor === null) {
    continue;
  }

  procesar(valor);
}
```

Si `indice += 1` estuviera después del `continue`, el bucle nunca avanzaría al encontrar un `null`.

Bucles anidados y etiquetas

Sin una etiqueta, `break` y `continue` afectan solamente al bucle más cercano. JavaScript admite etiquetas para dirigirlos a un bucle exterior:

```
busqueda:
for (let fila = 0; fila < matriz.length; fila += 1) {
  for (let columna = 0; columna < matriz[fila].length; columna += 1) {
    if (matriz[fila][columna] === objetivo) {
      console.log(fila, columna);
      break busqueda;
    }
  }
}
```

Las etiquetas existen y son válidas, pero su uso frecuente dificulta la lectura. Extraer la búsqueda a una función y usar `return` suele ofrecer una estructura más clara:

```
function buscarPosicion(matriz, objetivo) {
  for (let fila = 0; fila < matriz.length; fila += 1) {
    for (let columna = 0; columna < matriz[fila].length; columna += 1) {
      if (matriz[fila][columna] === objetivo) {
        return { fila, columna };
      }
    }
  }

  return null;
}
```

`return` sale de todos los bucles a la vez, y además la función tiene un nombre que dice qué hace.

14.12 `switch`: seleccionar por un valor

`switch` compara una expresión con una serie de casos:

```
switch (expresion) {
  case valor1:
    // instrucciones
    break;

  case valor2:
    // instrucciones
    break;

  default:
    // si ningún caso coincide
}
```

Por ejemplo:

```
function describirEstado(estado) {
  switch (estado) {
    case "pendiente":
      return "Todavía no fue procesado";

    case "aprobado":
      return "La operación fue aprobada";

    case "rechazado":
      return "La operación fue rechazada";

    default:
      return "Estado desconocido";
  }
}
```

`switch` es apropiado cuando una misma expresión se compara contra valores discretos. La comparación es estricta, equivalente en lo esencial a `===`:

```
const valor = "1";

switch (valor) {
  case 1:
    console.log("número");
    break;

  case "1":
    console.log("cadena"); // este coincide
    break;
}
```

`default` es opcional y puede ubicarse en cualquier posición, aunque escribirlo al final es más claro.

`break` o `return`

`break` termina el `switch` y continúa después de él:

```
let mensaje;

switch (estado) {
  case "ok":
    mensaje = "Correcto";
    break;

  default:
    mensaje = "Desconocido";
}

console.log(mensaje);
```

Si el `switch` está dentro de una función y cada caso devuelve un resultado, `return` ya termina la función y no necesita `break`. Esa forma suele ser más compacta y elimina de raíz el problema de la sección siguiente.

14.13 *Fall-through*: continuar en el caso siguiente

Si un caso no termina con `break`, `return` o `throw`, la ejecución **continúa por las instrucciones del caso siguiente**. Este comportamiento se llama *fall-through*.

```
const nivel = 2;

switch (nivel) {
  case 1:
    console.log("Nivel básico");
    break;

  case 2:
    console.log("Nivel intermedio");
    // falta break

  case 3:
    console.log("Nivel avanzado");
    break;
}
```

La salida es:

```
Nivel intermedio
Nivel avanzado
```

Después de encontrar `case 2`, JavaScript no vuelve a comparar `case 3`: simplemente sigue ejecutando sus instrucciones.

Cuando es un error

La omisión de `break` es una fuente clásica de defectos. Cuando no es intencional, cada caso debe finalizar explícitamente.

Cuando es intencional

Puede usarse para agrupar varios valores bajo el mismo comportamiento:

```
switch (dia) {
  case "sábado":
  case "domingo":
    console.log("Fin de semana");
    break;

  default:
    console.log("Día laborable");
}
```

Los primeros casos no tienen instrucciones; todos conducen al mismo bloque. Esta forma es clara y muy usada.

También puede acumular capacidades:

```

switch (rol) {
  case "administrador":
    permisos.add("eliminar");
    // continúa intencionalmente

  case "editor":
    permisos.add("editar");
    // continúa intencionalmente

  case "lector":
    permisos.add("leer");
    break;
}

```

Un administrador acumula los tres permisos; un editor, dos. Cuando el *fall-through* contiene instrucciones, hay que comentarlo para dejar claro que no falta un `break`.

Alcance dentro de `switch`

Todo el `switch` comparte un único bloque léxico. Declarar el mismo nombre con `const` en varios casos produce un error, aunque solo se ejecute uno:

```

switch (tipo) {
  case "a": {
    const mensaje = "Tipo A";
    console.log(mensaje);
    break;
  }

  case "b": {
    const mensaje = "Tipo B";
    console.log(mensaje);
    break;
  }
}

```

Las llaves adicionales crean un alcance independiente por caso y resuelven el problema.

14.14 Recorrer colecciones

El capítulo 10 desarrolló `for...of` y `for...in` en el contexto de los arrays. Vale la pena recordar aquí su lugar como estructuras de control.

`for...of` recorre los valores producidos por un iterable —arrays, cadenas, `Map`, `Set`—:

```
const stock = new Map([
  ["A10", 5],
  ["B20", 8],
]);

for (const [codigo, cantidad] of stock) {
  console.log(`${codigo}: ${cantidad}`);
}
```

`break` y `continue` funcionan normalmente en `for...of`.

Un objeto común no es iterable de manera directa. Sus propiedades se recorren con `Object.entries()` y `for...of`, o con `for...in` filtrando las heredadas.

14.15 Elegir la construcción apropiada

Pregunta	Construcción sugerida
¿Se repite mientras cambie un estado y no conocemos la cantidad?	<code>while</code>
¿Debe ejecutarse al menos una vez?	<code>do...while</code>
¿Existe un contador con inicio, condición y paso?	<code>for</code>
¿Queremos cada valor de un iterable?	<code>for...of</code>
¿Queremos nombres de propiedades enumerables?	<code>for...in</code>
¿Queremos transformar un array completo?	<code>map()</code>
¿Queremos seleccionar elementos?	<code>filter()</code>
¿Queremos encontrar uno y detenernos?	<code>find()</code> o <code>for...of</code>
¿Queremos saber si alguno cumple?	<code>some()</code>
¿Queremos acumular un resultado?	<code>reduce()</code> o un bucle con acumulador

Los métodos de arrays también controlan la iteración, pero no reemplazan a todos los bucles. `for...of` permite `break`, `continue`, `return` desde la función contenedora y `await` secuencial cuando corresponda. `forEach()` no ofrece ninguna de esas salidas.

14.16 Confusiones frecuentes

Confundir asignación con comparación

```
if (estado = "activo") {  
    // asigna "activo" y la condición resulta truthy: siempre entra  
}
```

La comparación correcta:

```
if (estado === "activo") {  
    // ...  
}
```

Ordenar mal un encadenamiento

Las condiciones más específicas deben aparecer antes que las generales, o nunca se alcanzan.

Anidar todas las condiciones

Las guardas permiten terminar los casos inválidos y mantener visible el camino principal.

Crear un bucle sin progreso

Si la condición nunca cambia, el bucle no termina. Al escribir un bucle conviene preguntarse explícitamente qué estado se acerca a la salida en cada vuelta.

Actualizar después de un `continue`

En un `while`, la actualización debe ocurrir antes de cualquier salida con `continue`.

Olvidar `break` en un `switch`

La ejecución continúa mediante *fall-through*. Si es intencional, debe quedar agrupado o comentado.

Usar `for...in` para valores de un array

`for...in` recorre claves como cadenas y puede incluir propiedades adicionales. Para valores corresponde `for...of`.

14.17 Síntesis conceptual

El flujo de control determina cuál es la próxima instrucción. `if` ejecuta un bloque cuando su condición es *truthy* y `else` cubre el camino contrario. Una cadena `else if` ejecuta solamente la primera rama verdadera, y por eso el orden de las condiciones forma parte

de su significado.

El anidamiento es válido pero cuesta leerlo. Las guardas con retorno temprano resuelven el mismo problema dejando el camino principal a la vista y asociando cada impedimento con su motivo.

`while` comprueba antes del cuerpo y puede ejecutarse cero veces; `do...while` comprueba después y ejecuta al menos una. El `for` reúne inicialización, condición y actualización en un solo lugar, y mantiene la variable de control acotada al bucle. Cualquiera de los tres necesita un estado que progrese hacia la salida.

`break` termina el bucle o el `switch` más cercano; `continue` comienza la siguiente iteración. Para bucles anidados existen las etiquetas, pero extraer una función y usar `return` suele leerse mejor.

`switch` compara con igualdad estricta y, sin `break`, `return` o `throw`, continúa por el caso siguiente. Ese *fall-through* es un error cuando se olvida y una herramienta útil cuando se busca deliberadamente para agrupar casos.

El capítulo siguiente completa el flujo de control con la construcción que no elige un camino sino que abandona el que estaba en curso.

Capítulo 15. Excepciones y manejo de errores

Hay operaciones que pueden fallar mientras el programa se está ejecutando, y queremos decidir qué hacer cuando eso ocurre. Para eso existen las excepciones.

La forma más visible del mecanismo es esta:

```
try {  
    // intento hacer algo  
} catch (error) {  
    // si falla, reacciono al problema  
}
```

Pero para entenderlo bien conviene empezar antes: por qué existe un error, qué significa lanzar una excepción y cómo cambia el flujo del programa. Este capítulo va a llegar a una conclusión que quizás sorprenda:

try...catch no es realmente un mecanismo para «detectar errores». Es un mecanismo para cambiar abruptamente el flujo del programa y transportar información sobre un fallo desde el lugar donde se detecta hasta el lugar donde puede manejarse.

15.1 Un error altera el flujo normal

Un programa normalmente avanza línea por línea:

```
console.log("A");  
console.log("B");  
console.log("C");
```

línea 1 → línea 2 → línea 3

Pero ahora:

```
console.log("A");  
  
algoQueNoExiste();  
  
console.log("C");
```

JavaScript encuentra una llamada a una función que no está definida y produce un error:

```
ReferenceError: algoQueNoExiste is not defined
```

En ese momento ocurre algo fundamental: **JavaScript deja de ejecutar normalmente ese camino del programa**. La última línea no se ejecuta.

```
A  
↓  
B  
↓  
C ✨
```

El error empieza a propagarse buscando código que pueda manejarlo. Ahí aparece `try...catch`.

Una excepción no es simplemente un valor que informa que algo salió mal. Es también un mecanismo de control: interrumpe el recorrido normal y transfiere la ejecución a otro lugar.

15.2 `try` significa «intentá ejecutar esto»

```
try {  
  console.log("A");  
  
  algoQueNoExiste();  
  
  console.log("B");  
}
```

El bloque `try` **no evita** que ocurra el error. No significa «ejecutá esto sin errores». Significa:

Ejecutá esto normalmente, pero si durante esta ejecución aparece una excepción, dame la oportunidad de manejarla.

`try` delimita la región de código cuyas excepciones queremos poder capturar.

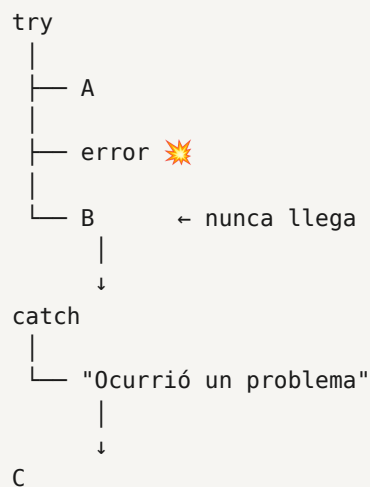
15.3 `catch` captura la excepción

```
try {  
  console.log("A");  
  
  algoQueNoExiste();  
  
  console.log("B");  
} catch (error) {  
  console.log("Ocurrió un problema");  
}  
  
console.log("C");
```

Resultado:

```
A  
Ocurrió un problema  
C
```

Hay algo importante que observar: **B no se ejecutó**. Cuando ocurrió el error, JavaScript abandonó inmediatamente el bloque `try`:



Después de ejecutar el `catch`, el programa continúa con la instrucción posterior a toda la estructura.

15.4 El objeto `Error`

La variable del `catch` recibe aquello que fue lanzado:

```
try {  
  x();  
} catch (error) {  
  console.log(error.name);  
  console.log(error.message);  
}
```

Podría imprimir:

```
ReferenceError  
x is not defined
```

Las propiedades más utilizadas:

- **name**: nombre o categoría del error.
- **message**: explicación legible del problema.
- **stack**: rastro de las llamadas activas cuando ocurrió. Es la propiedad más valiosa para depurar.
- **cause**: causa original opcional, cuando el error fue creado con esa información.

El rastro puede verse así:

```
ReferenceError: x is not defined  
    at programa.js:4  
    at ...
```

No siempre se lanza un objeto `Error`

JavaScript permite técnicamente lanzar cualquier valor:

```
throw "falló";  
throw 404;  
throw { motivo: "sin permiso" };
```

Por eso un `catch` recibe exactamente lo que se lanzó, y no puede dar por sentado que tiene `message` o `stack`. Sin embargo, **es una buena práctica lanzar siempre instancias de `Error` o de sus subclases**:

```
throw new Error("Falló la operación");
```

Así se conservan de forma coherente el mensaje, el tipo y el rastro de la pila. Un `throw "falló"` es legal y deja a quien lo captura sin ninguna información sobre dónde ocurrió.

15.5 `throw`: lanzar una excepción

El concepto central del mecanismo es `throw`:

```
throw new Error("Algo salió mal");
```

En cuanto se ejecuta, se interrumpe el flujo normal del bloque actual:

```
console.log("A");  
  
throw new Error("Problema");  
  
console.log("B"); // nunca se ejecuta
```

`throw` es una instrucción; el valor que aparece después es la información que viaja con la excepción.

También podemos detectar una regla inválida de nuestro propio dominio y lanzar un error:

```
function retirar(saldo, monto) {  
  if (monto > saldo) {  
    throw new Error("Saldo insuficiente");  
  }  
  
  return saldo - monto;  
}
```

Crear no es lanzar

Conviene notar la diferencia:

```
const error = new Error("Algo salió mal");  
  
console.log("El programa continúa");
```

Crear el objeto no interrumpe nada. Para eso hace falta lanzarlo:

```
throw error;
```

```
new Error(...)    → construye información sobre un error  
throw new Error(...) → construye esa información y lanza la excepción
```

15.6 `throw` y `catch` son dos caras del mismo mecanismo

```
try {  
  throw new Error("No hay conexión");  
} catch (error) {  
  console.log(error.message);  
}
```

```
throw
  |
  | lanza una excepción
  ↓
busca un catch
  |
  ↓
catch
```

Por eso suele decirse que una excepción **sube por la pila de llamadas** hasta encontrar un `catch`.

15.7 Un ejemplo con una regla propia

Consideremos esta función:

```
function dividir(a, b) {
  return a / b;
}
```

En JavaScript, `dividir(10, 0)` no produce una excepción: devuelve `Infinity`, por lo que vimos en la sección 5.5. Pero tal vez, para nuestro programa, dividir por cero sea inválido. Podemos imponer esa regla:

```
function dividir(a, b) {
  if (b === 0) {
    throw new RangeError("No se puede dividir por cero");
  }

  return a / b;
}
```

Quien llama a la función decide cómo responder:

```
try {
  const resultado = dividir(10, 0);
  console.log(resultado);
} catch (error) {
  console.log(error.message);
}
```

Las excepciones no sirven únicamente para errores internos del lenguaje. También señalan situaciones que vuelven imposible completar una operación de nuestra aplicación.

¿Por qué no devolver `false`?

Podríamos escribir:


```
function dividir(a, b) {  
  if (b === 0) {  
    return false;  
  }  
  
  return a / b;  
}
```

Pero aparece un problema: ¿qué significa `false`? Podría significar error, resultado válido, dato faltante, permiso denegado o cualquier otra cosa. Quien recibe el valor tiene que adivinar, y peor aún, puede olvidarse de comprobarlo y usar `false` en una operación aritmética.

Una excepción separa claramente dos cosas:

```
resultado normal
```

de:

```
la operación no pudo completarse
```

Esto no significa que siempre haya que usar excepciones. Para un resultado esperable como «no se encontró ningún usuario», una función también puede devolver `null` o un objeto de resultado explícito. La sección 15.16 desarrolla ese criterio.

15.8 La pila de llamadas y la propagación

Cuando una función llama a otra, JavaScript necesita recordar dónde debe regresar. Esas llamadas pendientes forman la **pila de llamadas**.

```
function c() {  
  throw new Error("Problema");  
}  
  
function b() {  
  c();  
}  
  
function a() {  
  b();  
}  
  
a();
```

La pila queda conceptualmente así:

```

a()
└─ b()
    └─ c()

```

Cuando `c` lanza el error, JavaScript busca un `catch`:

1. Primero en `c`. No hay.
2. Sale de `c` y sube a `b`. No hay.
3. Sale de `b` y sube a `a`. No hay.
4. Sale de `a`.
5. Al no encontrar ningún manejador, queda una excepción no capturada.

En un navegador, eso se informa en la consola. En Node.js, una excepción síncrona no capturada normalmente hace terminar el proceso.

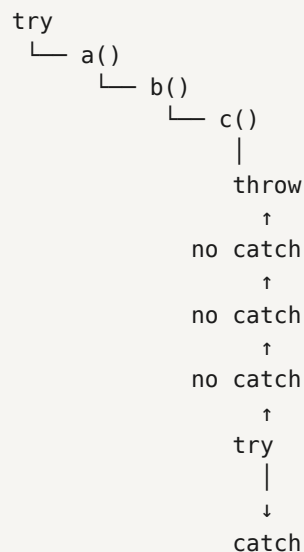
Este proceso se llama **desenrollado de la pila**.

Si lo capturamos en una capa superior:

```

try {
  a();
} catch (error) {
  console.log("Capturado:", error.message);
}

```



Este mecanismo permite que una función profunda diga «no puedo continuar» sin necesitar saber quién finalmente va a resolver, registrar o presentar el problema.

Por qué eso es valioso

Considérese:

```
function leerConfiguracion() {
    return convertirJSON();
}

function convertirJSON() {
    throw new Error("JSON inválido");
}
```

La función `convertirJSON` no tiene que saber si el programa quiere mostrar un mensaje, volver a intentar, escribir un registro, terminar o usar valores por defecto. Simplemente informa que no pudo completar su trabajo. Una capa superior decide:

```
try {
    const config = leerConfiguracion();
    console.log(config);
} catch (error) {
    console.log("No se pudo cargar la configuración");
}
```

Esta separación de responsabilidades es una de las grandes ventajas de las excepciones:

```
capa profunda → detecta e informa
capa superior → decide cómo responder
```

15.9 Tipos de error

JavaScript define varias clases incorporadas:

Tipo	Uso habitual
Error	error general de la aplicación
TypeError	valor de un tipo o forma incompatible
RangeError	valor numérico fuera del rango admitido
ReferenceError	referencia a un nombre inexistente o inaccesible
SyntaxError	código o texto analizado con sintaxis inválida
URIError	uso inválido de funciones relacionadas con URI
AggregateError	agrupación de varios errores

Ejemplos de cada uno:

```
const nombre = null;
nombre.toUpperCase(); // TypeError

new Array(-1); // RangeError

console.log(inexistente); // ReferenceError

JSON.parse("{}"); // SyntaxError
```

El lenguaje genera algunos de estos errores automáticamente. Nuestro código también puede elegir la clase que comunique mejor el contrato incumplido: `TypeError` cuando el argumento tiene la forma equivocada, `RangeError` cuando está fuera de los valores admitidos.

Podemos comprobar el tipo con `instanceof`:

```
try {
  const nombre = null;
  nombre.toUpperCase();
} catch (error) {
  console.log(error instanceof TypeError); // true
}
```

15.10 Actuar según el tipo de error

```
try {
  hacerAlgo();
} catch (error) {
  if (error instanceof TypeError) {
    console.log("Error de tipo");
  } else if (error instanceof RangeError) {
    console.log("Valor fuera de rango");
  } else {
    console.log("Otro error");
  }
}
```

Pero hay una regla importante: **si el `catch` no sabe manejar un tipo de error, debe volver a lanzarlo en vez de ocultarlo.**

```
try {
  hacerAlgo();
} catch (error) {
  if (error instanceof TypeError) {
    corregirDatos();
  } else {
    throw error;
  }
}
```

Esto se llama **relanzamiento**, o *rethrow*. Relanzar el mismo objeto conserva su identidad y su rastro original.

15.11 Errores personalizados

Podemos crear clases que representen problemas específicos de nuestra aplicación:

```
class UsuarioNoEncontradoError extends Error {
  constructor(id) {
    super(`No existe el usuario con ID ${id}`);
    this.name = "UsuarioNoEncontradoError";
    this.id = id;
  }
}
```

Y lanzarlas donde se detecta el problema:

```
function buscarUsuario(id) {
  if (id !== 123) {
    throw new UsuarioNoEncontradoError(id);
  }

  return { id: 123, nombre: "Ana" };
}
```

Para manejarlas:

```
try {
  buscarUsuario(999);
} catch (error) {
  if (error instanceof UsuarioNoEncontradoError) {
    console.log("Usuario inexistente:", error.id);
  } else {
    throw error;
  }
}
```

Un error personalizado permite reconocer el problema con `instanceof`, adjuntar datos estructurados como el `id`, separar el comportamiento del texto del mensaje y evitar decisiones basadas en cadenas frágiles.

Esto último merece énfasis. **No conviene escribir:**

```
if (error.message === "El usuario no existe") {
  // ...
}
```

El mensaje puede cambiar, traducirse o incluir detalles variables. El tipo de error expresa el significado de manera mucho más estable.

Un ejemplo más completo, con varios datos del dominio:

```
class StockInsuficienteError extends Error {
  constructor(productoId, solicitado, disponible) {
    super(`Stock insuficiente para el producto ${productoId}`);
    this.name = "StockInsuficienteError";
    this.productoId = productoId;
    this.solicitado = solicitado;
    this.disponible = disponible;
  }
}

function descontarStock(producto, cantidad) {
  if (cantidad > producto.stock) {
    throw new StockInsuficienteError(producto.id, cantidad, producto.stock);
  }

  return {
    ...producto,
    stock: producto.stock - cantidad,
  };
}
```

Y en el punto donde se sabe qué hacer:

```
try {
  const actualizado = descontarStock(producto, cantidad);
  guardar(actualizado);
} catch (error) {
  if (error instanceof StockInsuficienteError) {
    mostrarMensaje(`Solo quedan ${error.disponible} unidades`);
  } else {
    throw error;
  }
}
```

La excepción esperada se traduce a una respuesta del sistema. Cualquier error desconocido vuelve a propagarse.

Conservar la causa original

Si una capa agrega contexto, puede conservar el error que lo originó:

```
function cargarConfiguracion(texto) {
  try {
    return JSON.parse(texto);
  } catch (error) {
    throw new Error("No se pudo cargar la configuración", {
      cause: error,
    });
  }
}
```

El mensaje superior explica qué operación del dominio falló; `error.cause` conserva el motivo técnico. Sin `cause`, esa información se perdería y la depuración se volvería mucho más difícil.

15.12 finally

Existe un tercer bloque:

```
try {  
    // operación  
} catch (error) {  
    // manejo del error  
} finally {  
    // se ejecuta siempre  
}
```

`finally` significa: ejecutá este bloque haya ocurrido un error o no.

Sin error:

```
try {  
    console.log("Intentando");  
} catch (error) {  
    console.log("Error");  
} finally {  
    console.log("Terminando");  
}
```

```
Intentando  
Terminando
```

Con error:

```
try {  
    throw new Error("Problema");  
} catch (error) {  
    console.log("Error");  
} finally {  
    console.log("Terminando");  
}
```

```
Error  
Terminando
```

Para qué sirve

Principalmente, para garantizar una limpieza o una restauración:

```
const conexion = abrirConexion();

try {
  guardarDatos(conexion);
} catch (error) {
  console.error("No se pudieron guardar los datos", error);
} finally {
  cerrarConexion(conexion);
}
```

```
abrir recurso
  ↓
  try
  |
  trabajar
  ↓
  finally
  |
  cerrar recurso
```

Sus usos típicos: cerrar archivos o conexiones, liberar bloqueos, ocultar un indicador de carga, restaurar una configuración temporal, eliminar un manejador que ya no se necesita.

```
async function guardar() {
  mostrarIndicador();

  try {
    await enviarDatos();
  } finally {
    ocultarIndicador();
  }
}
```

Aunque `enviarDatos()` falle, el indicador se oculta.

finally se ejecuta incluso con `return`

```
function ejemplo() {
  try {
    return 10;
  } finally {
    console.log("finally");
  }
}

console.log(ejemplo());
```

```
finally
10
```


Antes de abandonar la función, JavaScript ejecuta `finally`.

Y también después de `throw`

```
function ejemplo() {  
  try {  
    throw new Error("Problema");  
  } finally {  
    console.log("Limpiando");  
  }  
}
```

Primero se imprime `Limpiando` y después el error continúa propagándose:

```
throw  
↓  
finally  
↓  
el error sigue subiendo
```

Qué no hacer en `finally`

Conviene evitar `return`, `throw`, `break` o `continue` dentro de `finally`, porque pueden reemplazar una salida o una excepción anterior:

```
function peligroso() {  
  try {  
    throw new Error("error original");  
  } finally {  
    return "oculta el error"; // el error desaparece  
  }  
}
```

```
function tambienPeligroso() {  
  try {  
    return 10;  
  } finally {  
    return 20; // el resultado es 20  
  }  
}
```

`finally` debería limpiar, no decidir el resultado principal.

Combinaciones válidas

```
try + catch  
try + finally  
try + catch + finally
```

Un bloque `try` no puede aparecer solo. `try...finally` sin `catch` es perfectamente válido cuando no queremos resolver el error, solo garantizar la limpieza:

```
try {
  hacerAlgo();
} finally {
  limpiar();
}
```

Si `hacerAlgo()` falla, se ejecuta `limpiar()` y el error continúa propagándose.

También podemos omitir el parámetro del `catch` si no lo necesitamos:

```
try {
  hacerAlgo();
} catch {
  console.log("Falló");
}
```

Es válido, aunque si estamos depurando, registrando o decidiendo según el tipo, vamos a necesitar el parámetro.

15.13 Un error frecuente: atrapar demasiado

Esto es tentador:

```
try {
  // 200 líneas de código
} catch {
  console.log("Algo salió mal");
}
```

Un bloque tan grande dificulta saber qué operación falló y mezcla errores que requieren respuestas diferentes.

Peor todavía:

```
try {
  hacerAlgo();
} catch {
}
```

Esto se conoce como «tragarse» la excepción. El programa falló y el código fingió que no pasó nada. Después puede continuar con un estado inválido y producir un problema mucho más difícil de localizar, muy lejos del punto donde se originó.

Un `catch` demasiado amplio también oculta errores de programación:

```
try {
  const configuracion = JSON.parse(texto);
  usarConfiguracion(configuracoin); // nombre mal escrito
} catch {
  return null; // oculta también el ReferenceError
}
```

El `catch` estaba pensado para un JSON inválido y terminó ocultando una errata.

La versión que distingue:

```
function parsearConfiguracion(texto) {
  try {
    return JSON.parse(texto);
  } catch (error) {
    if (error instanceof SyntaxError) {
      throw new Error("La configuración tiene formato inválido", {
        cause: error,
      });
    }
    throw error;
  }
}
```

El patrón general:

```
try {
  operacion();
} catch (error) {
  if (puedoResolverlo(error)) {
    resolver(error);
  } else {
    throw error;
  }
}
```

La idea es: **capturá únicamente aquello que realmente sabés manejar.**

«Manejar» puede significar corregir los datos, reintentar de forma segura, usar un valor alternativo válido, transformar el error en una respuesta adecuada, o informar al usuario dejando la aplicación en un estado coherente. Imprimir un mensaje y continuar no siempre equivale a haber manejado el fallo.

15.14 La asincronía cambia dónde puede capturarse un error

Consideremos:

```
try {
  setTimeout(() => {
    throw new Error("Boom");
  }, 1000);
} catch (error) {
  console.log("Capturado");
}
```

Uno podría esperar que aparezca «Capturado». No ocurre.

Cuando se ejecuta el *callback* de `setTimeout`, el bloque `try` exterior ya terminó hace un segundo:

```
try
├─ programa el callback
└─ termina

...un segundo después...

callback
└─ throw 💣
```

Son ejecuciones separadas. El `catch` solo puede capturar excepciones lanzadas mientras se está ejecutando dinámicamente el código protegido por su `try`.

Para capturar esa excepción, el `try` debe estar dentro:

```
setTimeout(() => {
  try {
    throw new Error("Boom");
  } catch (error) {
    console.log("Capturado:", error.message);
  }
}, 1000);
```

En código moderno, lo habitual es representar el resultado asíncronico con una promesa y manejar su rechazo.

15.15 `async/await` y las promesas

Una función `async` siempre devuelve una promesa. Si dentro devuelve un valor, la promesa se cumple con ese valor. Si lanza una excepción, la promesa se rechaza con ese error.

```
async function obtenerDatos() {
  throw new Error("No se pudieron obtener datos");
}
```

Podemos manejarla con `await`:

```

async function main() {
  try {
    await obtenerDatos();
  } catch (error) {
    console.log(error.message);
  }
}

```

`await` suspende la función `async` hasta que la promesa se resuelva. Si se rechaza, `await` se comporta como si lanzara el motivo del rechazo en ese punto. Por eso el `catch` que rodea al `await` puede capturarlo.

El `await` debe estar **dentro** del `try`:

```

async function cargar() {
  try {
    const datos = await obtenerDatos();
    console.log(datos);
  } catch (error) {
    console.error("Falló la carga:", error);
  }
}

```

Un detalle importante

Crear una promesa dentro de un `try`, pero no esperarla, no permite capturar su rechazo:

```

async function ejemplo() {
  try {
    obtenerDatos(); // falta await
  } catch (error) {
    console.log("Este catch no captura el rechazo");
  }
}

```

La llamada devuelve una promesa inmediatamente y el `try` termina. El rechazo llega después, sin nadie que lo escuche.

Hay que usar:

```

await obtenerDatos();

```

o devolver la promesa a quien vaya a manejarla:

```

return obtenerDatos();

```

El equivalente con métodos de promesa

Con `async/await`:

```

async function main() {
  try {
    const datos = await obtenerDatos();
    console.log(datos);
  } catch (error) {
    console.error(error);
  }
}

```

Con métodos:

```

obtenerDatos()
  .then((datos) => {
    console.log(datos);
  })
  .catch((error) => {
    console.error(error);
  });

```

Ambos modelan lo mismo:

```

promesa cumplida → valor
promesa rechazada → motivo del rechazo

```

Además, si una función pasada a `then` lanza un error, la promesa resultante se rechaza y un `.catch()` posterior puede manejarlo:

```

obtenerDatos()
  .then((datos) => JSON.parse(datos))
  .catch((error) => {
    console.error("La descarga o la conversión falló:", error);
  });

```

Y el método `finally` de promesa sirve para una limpieza que no depende del resultado:

```

obtenerDatos()
  .then(procesar)
  .catch(mostrarError)
  .finally(ocultarIndicador);

```

Conviene no mezclar los dos estilos sin necesidad. Una cadena coherente es más fácil de leer y de razonar.

15.16 `fetch` tiene una sutileza importante

Esto captura, por ejemplo, un rechazo causado por un problema de red o por una cancelación:

```
try {
  const respuesta = await fetch(url);
} catch (error) {
  console.log("No se pudo realizar la solicitud");
}
```

Pero una respuesta HTTP con estado 404, 403 o 500 **no hace que fetch rechace la promesa**. El servidor respondió correctamente a nivel del protocolo, aunque su respuesta represente un resultado fallido para nuestra aplicación.

Hay que verificarlo explícitamente:

```
const respuesta = await fetch(url);

if (!respuesta.ok) {
  throw new Error(`HTTP ${respuesta.status}`);
}
```

Un ejemplo completo:

```
async function cargarDatos(url) {
  try {
    const respuesta = await fetch(url);

    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}`);
    }

    return await respuesta.json();
  } catch (error) {
    console.error("No se pudo obtener la información:", error);
    throw error;
  }
}
```

Ese único `catch` puede recibir errores muy distintos: un rechazo de `fetch` por un fallo de red, el error HTTP que nuestro propio código lanzó, un `SyntaxError` de `respuesta.json()` si el cuerpo no es JSON válido, o cualquier defecto ocurrido dentro del bloque.

Por eso, en una aplicación real, suele convenir distinguirlos con tipos personalizados:

```

class ErrorHttp extends Error {
  constructor(status, url) {
    super(`La solicitud respondió HTTP ${status}`);
    this.name = "ErrorHttp";
    this.status = status;
    this.url = url;
  }
}

async function obtenerUsuario(id) {
  const url = `/api/usuarios/${id}`;
  const respuesta = await fetch(url);

  if (!respuesta.ok) {
    throw new ErrorHttp(respuesta.status, url);
  }

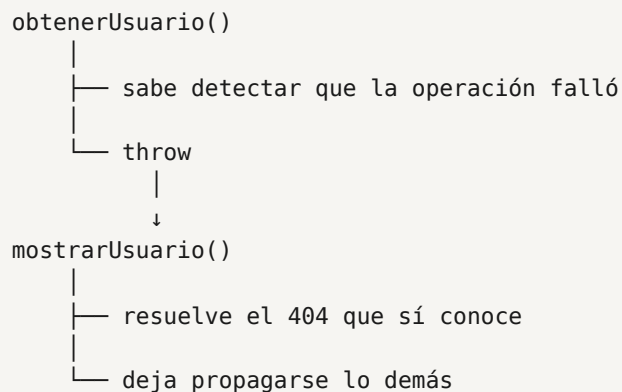
  return respuesta.json();
}

async function mostrarUsuario(id) {
  try {
    const usuario = await obtenerUsuario(id);
    console.log(usuario.nombre);
  } catch (error) {
    if (error instanceof ErrorHttp && error.status === 404) {
      console.log("El usuario no existe");
      return;
    }

    console.error("No se pudo mostrar el usuario:", error);
    throw error;
  }
}

```

La separación de responsabilidades es lo importante:



Esta arquitectura importa más que la sintaxis.

15.17 Condiciones esperables frente a excepciones

No todo resultado negativo es excepcional, y usar excepciones para todo vuelve el código más difícil de leer, no más robusto.

Decisiones esperadas

```
function edadValida(edad) {  
  return edad >= 0 && edad <= 130;  
}  
  
if (!edadValida(edad)) {  
  console.log("Edad inválida");  
}
```

Una edad incorrecta en un formulario es una situación perfectamente esperable. No necesitamos lanzar y capturar un error por cada dato que el usuario puede corregir.

Lo mismo vale para una búsqueda sin resultados:

```
function buscarAlumno(legajo) {  
  const alumno = alumnos.find((item) => item.legajo === legajo);  
  return alumno ?? null;  
}
```

Que una búsqueda no encuentre nada puede ser normal y representarse con `null`.

Operaciones que no pueden cumplir su contrato

```
function obtenerAlumnoObligatorio(legajo) {  
  const alumno = alumnos.find((item) => item.legajo === legajo);  
  
  if (!alumno) {  
    throw new Error(`No existe el alumno ${legajo}`);  
  }  
  
  return alumno;  
}
```

Aquí la función **promete** entregar un alumno. Si no existe, no puede producir un resultado válido, y devolver `null` obligaría a quien llama a comprobarlo cada vez.

Nótese que las dos funciones anteriores son legítimas y pueden coexistir: la diferencia está en lo que cada una promete.

Por qué no reemplazar `if` con excepciones

Esto es posible pero desaconsejable:

```
try {
  if (!usuario.esAdministrador) {
    throw new Error("no es administrador");
  }

  mostrarPanel();
} catch {
  mostrarInicio();
}
```

El flujo normal ya se expresa claramente con:

```
if (usuario.esAdministrador) {
  mostrarPanel();
} else {
  mostrarInicio();
}
```

Al leer un `throw`, debería entenderse que el camino normal de la operación ya no puede continuar. Si las excepciones se convierten en el mecanismo habitual de decisión, esa señal se pierde.

Regla orientativa

```
condición esperable
    ↓
    if

fallo excepcional de una operación
    ↓
    throw / try / catch
```

Ejemplos de condiciones normalmente esperables: un campo vacío antes de enviar un formulario, una búsqueda sin resultados, una opción que el usuario no seleccionó, una contraseña que todavía no cumple las reglas mientras se escribe.

Ejemplos que pueden justificar una excepción: no se pudo analizar una configuración que debía ser válida, se perdió la conexión durante una operación, una dependencia devolvió datos que violan su contrato, una función recibió un argumento imposible, una operación no puede entregar el resultado prometido.

No es una ley absoluta. Lo que importa es que la API tenga un contrato claro y consistente: quien la usa necesita saber qué se devuelve y qué puede lanzar.

Una excepción no es sinónimo de defecto

Un error puede representar un defecto de programación, un fallo externo, datos corruptos o una regla de negocio que impide completar la operación. Son situaciones distintas.

Un `catch` no corrige automáticamente un defecto. Si oculta un `TypeError` inesperado y deja continuar el programa, empeora el problema en lugar de resolverlo.

15.18 Caso integrador

```
class ValidacionError extends Error {
  constructor(mensaje, campo) {
    super(mensaje);
    this.name = "ValidacionError";
    this.campo = campo;
  }
}

function convertirCantidad(valor) {
  const cantidad = Number(valor);

  if (!Number.isInteger(cantidad) || cantidad <= 0) {
    throw new ValidacionError(
      "La cantidad debe ser un entero positivo",
      "cantidad"
    );
  }

  return cantidad;
}

function calcularCompra(productos, solicitud) {
  if (!solicitud) {
    throw new ValidacionError("Falta la solicitud", "solicitud");
  }

  const cantidad = convertirCantidad(solicitud.cantidad);
  let productoEncontrado = null;

  for (const producto of productos) {
    if (producto.id !== solicitud.productoId) {
      continue;
    }

    productoEncontrado = producto;
    break;
  }

  if (!productoEncontrado) {
    return { ok: false, motivo: "producto inexistente" };
  }

  if (productoEncontrado.stock < cantidad) {
    return { ok: false, motivo: "stock insuficiente" };
  }

  let descuento;

  switch (solicitud.tipoCliente) {
    case "premium":
      descuento = 0.15;
      break;

    case "frecuente":
      descuento = 0.05;
      break;
  }
}
```

```

    default:
      descuento = 0;
  }

  const subtotal = productoEncontrado.precio * cantidad;
  const total = subtotal * (1 - descuento);

  return {
    ok: true,
    producto: productoEncontrado.nombre,
    cantidad,
    total,
  };
}

```

Un nivel exterior traduce las excepciones de validación a una respuesta:

```

function atenderCompra(productos, solicitud) {
  try {
    return calcularCompra(productos, solicitud);
  } catch (error) {
    if (error instanceof ValidacionError) {
      return {
        ok: false,
        motivo: error.message,
        campo: error.campo,
      };
    }

    throw error;
  }
}

```

Vale la pena observar las decisiones tomadas. Una cantidad no numérica **lanza**, porque viola el contrato de la función. Un producto inexistente o sin stock **devuelve** un resultado, porque son situaciones previstas del negocio. Y el nivel exterior captura solo lo que sabe traducir; cualquier otra cosa se propaga.

El ejemplo combina guardas para validar precondiciones, `for...of` con `continue` y `break` para recorrer, `if` para resultados normales, `switch` para valores discretos y `try...catch` en el nivel que sabe qué hacer.

15.19 Buenas prácticas

Lanzar objetos `Error`. Conservan tipo, mensaje y pila.

Escribir mensajes con contexto, sin incluir contraseñas, tokens ni datos sensibles:

```

throw new Error(`No se pudo guardar el pedido ${pedido.id}`);

```

Usar tipos personalizados para las decisiones del programa, no el texto del mensaje.

Mantener pequeño el bloque `try`. Proteger la operación que puede fallar, no una función entera.

No ocultar errores desconocidos. Manejar lo que se sabe manejar y relanzar el resto.

Agregar contexto sin perder la causa, con la opción `cause`.

Usar `finally` para limpieza, no para cambiar el resultado.

En asincronía, esperar o devolver la promesa. Una promesa ignorada produce un rechazo ignorado.

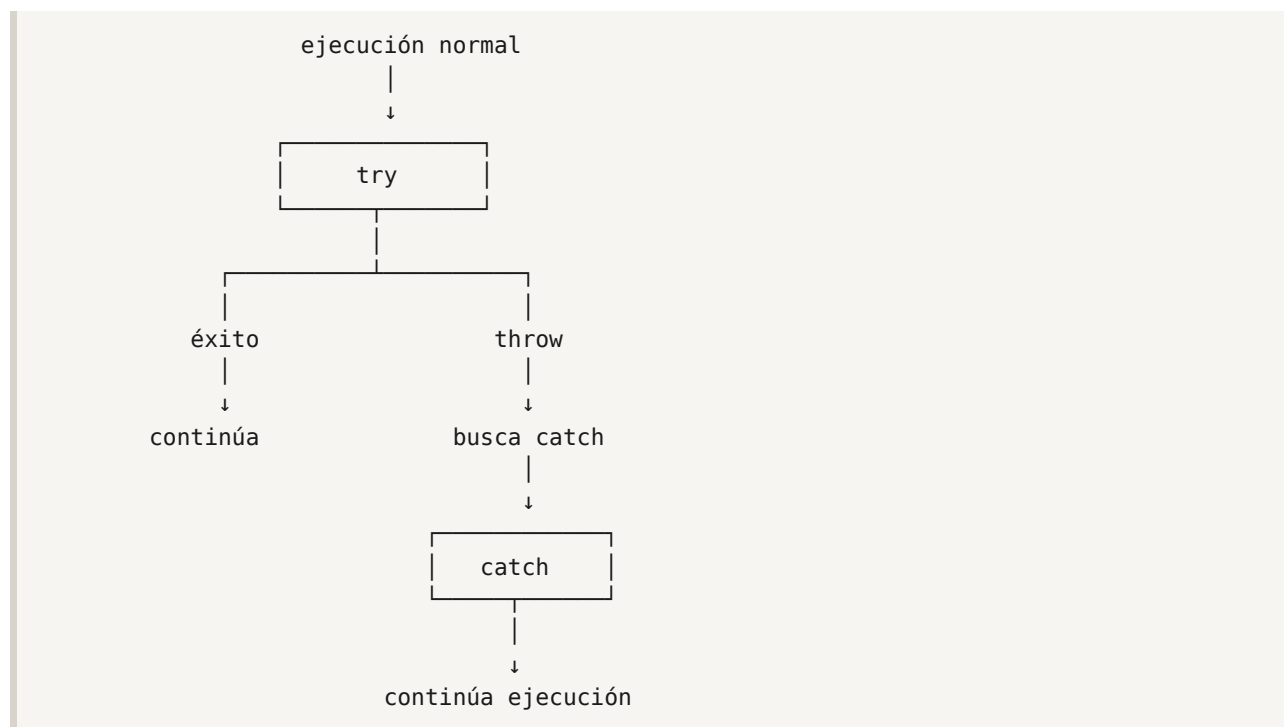
Con `fetch`, verificar `response.ok`. Un estado HTTP fallido no se transforma solo en excepción.

Separar el mensaje para el usuario del diagnóstico técnico. El usuario necesita saber qué hacer; el registro necesita el error, la pila y el contexto.

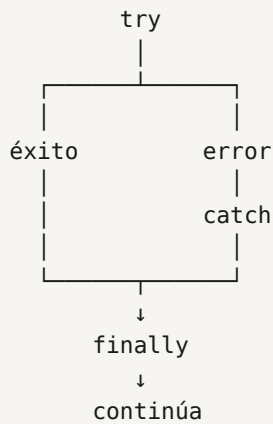
Documentar el contrato. Si una función puede lanzar tipos concretos de error, quien la usa debería poder saberlo.

15.20 Síntesis conceptual

Podemos resumir el mecanismo en un esquema:



Con `finally`:



Y en asincronía:

```
callback posterior de setTimeout
≠
ejecución dinámica del try exterior

promesa rechazada
├─ await dentro de try → catch
└─ cadena de promesa → .catch()
```

Una forma precisa de decirlo en una frase:

Ejecutá normalmente este código; si durante su ejecución se lanza una excepción, interrumpí ese camino, buscá el `catch` correspondiente y transferí allí el control junto con la información del fallo.

Ese concepto de **transferencia del flujo de control** es el corazón de las excepciones. Y su valor principal no es detectar fallos, sino permitir que el lugar donde se detecta un problema y el lugar donde se decide qué hacer sean lugares distintos.

Con esto cerramos el control del flujo. La parte siguiente se ocupa de la construcción que permite darle nombre a un fragmento de programa y reutilizarlo: la función.

Parte IV — Funciones

Capítulo 16. Funciones: definición, parámetros y resultados

Una función es una pieza de código que podemos **definir una vez y ejecutar todas las veces que necesitamos**.

La idea más elemental es esta:

```
datos de entrada
  ↓
función
  ↓
resultado de salida
```

Por ejemplo, una función que recibe dos números y devuelve su suma:

```
function sumar(a, b) {
  return a + b;
}
```

Para usarla, la invocamos:

```
const resultado = sumar(4, 7);

console.log(resultado); // 11
```

En este ejemplo, `sumar` es el nombre de la función, `a` y `b` son sus parámetros, `4` y `7` son los argumentos enviados, `return a + b` produce el resultado, y `resultado` recibe el valor retornado.

16.1 Por qué necesitamos funciones

Sin funciones podríamos escribir:

```
const resultado1 = 4 + 7;
const resultado2 = 10 + 20;
const resultado3 = 100 + 50;
```

Con una operación tan simple no molesta. Pero si fuera más compleja, terminaríamos repitiendo mucho código, y cada corrección habría que aplicarla en todos los lugares.

Una función permite darle nombre a una operación:

```
function calcularPrecioFinal(precio, descuento) {  
  const rebaja = precio * descuento / 100;  
  
  return precio - rebaja;  
}
```

y reutilizarla:

```
console.log(calcularPrecioFinal(1000, 10)); // 900  
console.log(calcularPrecioFinal(500, 20)); // 400  
console.log(calcularPrecioFinal(800, 5)); // 760
```

La función separa dos cosas que antes estaban mezcladas:

```
qué operación queremos realizar  
    ↓  
calcularPrecioFinal  
  
con qué datos queremos realizarla  
    ↓  
1000 y 10
```

Esa separación es lo que permite razonar sobre programas grandes: podemos usar `calcularPrecioFinal` sin recordar cómo está implementada, y podemos cambiar su implementación sin revisar cada uso.

16.2 Definir no es ejecutar

Cuando escribimos:

```
function saludar() {  
  console.log("Hola");  
}
```

estamos **definiendo** la función. El código interno todavía no se ejecutó.

Para ejecutarla hacen falta paréntesis:

```
saludar();
```

La diferencia es fundamental:

```
console.log(saludar);  
// [Function: saludar] – la función como valor  
  
console.log(saludar());  
// ejecuta la función
```

`saludar` representa la función como valor. `saludar()` la invoca.

Esta distinción parece obvia y es la causa de un error muy común al pasar funciones a otras funciones:

```
setTimeout(saludar, 1000); // correcto: pasa la función
setTimeout(saludar(), 1000); // error: ejecuta ahora y pasa su resultado
```

El capítulo 18 desarrolla ese uso.

16.3 Declaración de función

La forma tradicional:

```
function nombre(parametros) {
  instrucciones;

  return resultado;
}
```

Por ejemplo:

```
function multiplicar(a, b) {
  return a * b;
}

console.log(multiplicar(6, 7)); // 42
```

Se la llama habitualmente *function declaration*.

Las declaraciones se elevan

JavaScript permite invocar una función declarada **antes** de su aparición textual:

```
console.log(sumar(3, 4)); // 7

function sumar(a, b) {
  return a + b;
}
```

Esto ocurre porque las declaraciones de función participan del mecanismo de *hoisting* que vimos en la sección 2.6, y a diferencia de `let` y `const`, quedan completamente inicializadas antes de que empiece la ejecución.

Conceptualmente:

1. registrar la función `sumar`
2. ejecutar el programa

No significa que el código se mueva literalmente, pero es un buen primer modelo mental. En la práctica permite escribir un archivo con las funciones auxiliares al final y la lógica principal arriba.

16.4 Expresiones de función

Una función también puede crearse como un valor y almacenarse en una variable:

```
const restar = function (a, b) {  
  return a - b;  
};  
  
console.log(restar(10, 3)); // 7
```

Aquí tenemos una función anónima almacenada en la constante `restar`:

```
restar  
  |  
  ▼  
función
```

La función es un valor, igual que un número, una cadena o un objeto. Esa es una de las ideas centrales de la parte V.

La diferencia con la declaración

Esto funciona:

```
console.log(sumar(2, 3));  
  
function sumar(a, b) {  
  return a + b;  
}
```

Esto no:

```
console.log(restar(5, 2)); // ReferenceError  
  
const restar = function (a, b) {  
  return a - b;  
};
```

La variable `restar` está en la zona muerta temporal hasta que se ejecuta su declaración. La función existe recién a partir de esa línea.

Expresión de función con nombre

Una expresión también puede tener un nombre interno:

```
const factorial = function calcularFactorial(n) {  
  if (n <= 1) {  
    return 1;  
  }  
  
  return n * calcularFactorial(n - 1);  
};  
  
console.log(factorial(5)); // 120
```

El nombre `calcularFactorial` solo es visible dentro de la propia función. Resulta útil para la recursividad —que el capítulo 19 desarrolla—, para los mensajes de error y para la depuración, porque el rastro de pila muestra un nombre en lugar de `<anonymous>`.

16.5 Funciones flecha

Las funciones flecha son una forma más compacta de escribir funciones. Su versión completa:

```
const sumar = (a, b) => {  
  return a + b;  
};
```

Equivale, para este caso sencillo, a:

```
const sumar = function (a, b) {  
  return a + b;  
};
```

La flecha `=>` separa los parámetros del cuerpo:

```
parámetros => cuerpo
```

Las abreviaturas

Conviene aprenderlas de a una.

Forma completa:

```
const doble = (numero) => {  
  return numero * 2;  
};
```

Un solo parámetro: pueden omitirse los paréntesis.

```
const doble = numero => {  
  return numero * 2;  
};
```

Los paréntesis solo pueden omitirse cuando hay exactamente un parámetro simple, sin valor predeterminado ni desestructuración.

Una sola expresión: puede omitirse return.

```
const doble = numero => numero * 2;
```

Se llama **retorno implícito**. El cuerpo sin llaves es una expresión cuyo valor se devuelve automáticamente.

Sin parámetros: los paréntesis son obligatorios.

```
const obtenerSaludo = () => "Hola";
```

Con varios parámetros: también son obligatorios.

```
const sumar = (a, b) => a + b;  
  
// const sumar = a, b => a + b; // SyntaxError
```

Retornar un objeto

Aquí hay una trampa. Queremos escribir esto:

```
const crearPersona = nombre => {  
  return {  
    nombre,  
    activo: true,  
  };  
};
```

Y podríamos intentar abreviarlo así:

```
const crearPersona = nombre => {  
  nombre,  
  activo: true  
};
```

Pero no funciona, porque JavaScript interpreta las llaves como el **cuerpo de la función**, no como un objeto literal.

Para retornar un objeto de manera implícita hay que envolverlo entre paréntesis:

```
const crearPersona = nombre => ({
  nombre,
  activo: true,
});

console.log(crearPersona("Ana"));
// { nombre: "Ana", activo: true }
```

La regla para recordar:

```
x => ({ valor: x })
```

Los paréntesis indican que las llaves pertenecen a un objeto y no al cuerpo.

Cuándo abreviar

Estas cuatro funciones son equivalentes:

```
const cuadrado = function (numero) {
  return numero * numero;
};

const cuadrado = (numero) => {
  return numero * numero;
};

const cuadrado = numero => {
  return numero * numero;
};

const cuadrado = numero => numero * numero;
```

La última es la más compacta, pero no siempre la más clara. La brevedad ayuda cuando la operación es sencilla:

```
const esPar = numero => numero % 2 === 0;
```

Para operaciones más largas conviene conservar el bloque:

```
const calcularPrecio = (precio, descuento, impuesto) => {
  const rebaja = precio * descuento / 100;
  const precioRebajado = precio - rebaja;
  const recargo = precioRebajado * impuesto / 100;

  return precioRebajado + recargo;
};
```

El objetivo no es eliminar caracteres. El objetivo es que el código se entienda.

16.6 Las funciones flecha no son solo una abreviatura

Tienen diferencias de comportamiento que importan.

No tienen su propio `this`

En una función clásica, el valor de `this` depende de **cómo se invoca** la función:

```
const persona = {
  nombre: "Ana",

  saludar() {
    console.log(`Hola, soy ${this.nombre}`);
  },
};

persona.saludar();
// "Hola, soy Ana"
```

Al invocar `persona.saludar()`, `this` es `persona`.

Una función flecha, en cambio, **captura el `this` del entorno donde fue escrita**. Por eso no conviene usarla como método principal de un objeto:

```
const persona = {
  nombre: "Ana",

  saludar: () => {
    console.log(this.nombre); // this no es persona
  },
};
```

Pero esa misma propiedad la vuelve ideal **dentro** de un método:

```
const persona = {
  nombre: "Ana",

  saludarMasTarde() {
    setTimeout(() => {
      console.log(`Hola, soy ${this.nombre}`);
    }, 1000);
  },
};
```

```
saludarMasTarde()
├── this representa persona
│   └── función flecha
│       └── conserva ese this
```


Con una función clásica en el `setTimeout`, `this` habría sido otra cosa y habría hecho falta el viejo truco de guardar `const self = this`.

La regla práctica: **método de objeto o de clase, función clásica; callback dentro de un método, función flecha.**

No tienen `arguments`

Una función clásica tiene un objeto especial llamado `arguments`:

```
function mostrarArgumentos() {
  console.log(arguments);
}

mostrarArgumentos(10, 20, 30);
```

Una función flecha no tiene el suyo. En código moderno se prefieren los parámetros *rest* de la sección 16.10, que además producen un array real:

```
const mostrarArgumentos = (...valores) => {
  console.log(valores);
};
```

No pueden usarse como constructor

```
function Persona(nombre) {
  this.nombre = nombre;
}

const ana = new Persona("Ana"); // funciona
```

```
const Persona = nombre => {
  this.nombre = nombre;
};

// const ana = new Persona("Ana"); // TypeError
```

Para constructores, hoy normalmente usamos clases:

```
class Persona {
  constructor(nombre) {
    this.nombre = nombre;
  }
}
```

16.7 Parámetros y argumentos

Conviene distinguir los dos términos.

En la definición:

```
function sumar(a, b) {  
  return a + b;  
}
```

`a` y `b` son **parámetros**.

En la llamada:

```
sumar(10, 20);
```

`10` y `20` son **argumentos**.

definición:	invocación:
sumar(a, b)	sumar(10, 20)
↑ ↑	↑ ↑
parámetros	argumentos

JavaScript no exige la cantidad exacta

```
function mostrar(a, b) {  
  console.log(a);  
  console.log(b);  
}  
  
mostrar(10);
```

```
10  
undefined
```

Los parámetros faltantes reciben `undefined`. Y también podemos enviar argumentos de más:

```
mostrar(10, 20, 30, 40);
```

La función usa `10` y `20`; los demás no se asignan a ningún parámetro declarado.

JavaScript no controla la cantidad de argumentos como lo haría un lenguaje con firmas estrictas. Esa flexibilidad es cómoda y a la vez significa que un error de llamada no se detecta: se manifiesta más tarde como un `undefined` inesperado.

16.8 Valores predeterminados

Podemos asignar un valor por defecto a un parámetro:

```
function saludar(nombre = "invitado") {  
  return `Hola, ${nombre}`;  
}  
  
console.log(saludar("Ana")); // "Hola, Ana"  
console.log(saludar());      // "Hola, invitado"
```

El valor predeterminado se usa cuando el argumento es `undefined`:

```
console.log(saludar(undefined)); // "Hola, invitado"
```

Pero **no** cuando es `null`:

```
console.log(saludar(null)); // "Hola, null"
```

`null` es un valor enviado explícitamente, y por eso el lenguaje lo respeta. Es la misma regla que rige la desestructuración y el operador `??`.

Los valores predeterminados pueden depender de parámetros anteriores:

```
function crearRango(desde, hasta = desde + 10) {  
  return { desde, hasta };  
}  
  
console.log(crearRango(5));      // { desde: 5, hasta: 15 }  
console.log(crearRango(5, 20)); // { desde: 5, hasta: 20 }
```

Y pueden ser cualquier expresión, incluida una llamada a otra función. Esa expresión se evalúa en cada invocación que necesite el valor por defecto, no una sola vez al definir la función. Eso es lo que hace segura esta forma:

```
function agregarTarea(tarea, lista = []) {  
  lista.push(tarea);  
  return lista;  
}
```

Cada llamada sin `lista` recibe un array nuevo.

16.9 Parámetros *rest*

Un parámetro *rest* permite recibir una cantidad variable de argumentos:

```
function sumarTodos(...numeros) {
  let total = 0;

  for (const numero of numeros) {
    total += numero;
  }

  return total;
}

console.log(sumarTodos(1, 2));           // 3
console.log(sumarTodos(1, 2, 3, 4, 5)); // 15
```

El parámetro `...numeros` reúne los argumentos restantes en un **array** real, con todos sus métodos disponibles:

```
sumarTodos(10, 20, 30)
      |   |   |
      +---+--> [10, 20, 30]
```

Debe aparecer al final:

```
function registrar(nombre, ...telefonos) {
  console.log(nombre);
  console.log(telefonos);
}

registrar("Ana", "3815551111", "3815552222");
// "Ana"
// ["3815551111", "3815552222"]
```

No hay que confundir el parámetro *rest* con la **expansión**, que usa la misma sintaxis en el otro sentido:

```
const numeros = [1, 2, 3];

sumarTodos(...numeros); // expande el array en argumentos separados
```

Uno recoge, el otro reparte. La posición decide cuál es: en una lista de parámetros recoge, en una llamada reparte.

16.10 Parámetros desestructurados

Si una función necesita muchos datos relacionados, puede recibir un objeto:

```
function registrarAlumno(alumno) {
  console.log(alumno.nombre);
  console.log(alumno.legajo);
  console.log(alumno.telefono);
}
```

Podemos desestructurarlo directamente en los parámetros:

```
function registrarAlumno({ nombre, legajo, telefono }) {
  console.log(nombre);
  console.log(legajo);
  console.log(telefono);
}

registrarAlumno({
  nombre: "Ana Pérez",
  legajo: "123456",
  telefono: "3815551234",
});
```

Esto tiene una ventaja importante: **los argumentos tienen nombre en el lugar de la llamada**. Compárese:

```
registrarAlumno("Ana Pérez", "123456", "3815551234", true, 8);
```

con:

```
registrarAlumno({
  nombre: "Ana Pérez",
  legajo: "123456",
  telefono: "3815551234",
  activo: true,
  nota: 8,
});
```

En la primera forma hay que ir a la definición para saber qué significan `true` y `8`. En la segunda se lee directamente. Además, el orden deja de importar y agregar un parámetro nuevo no rompe las llamadas existentes.

Puede combinarse con valores predeterminados:

```
function crearUsuario({ nombre, activo = true, rol = "usuario" }) {
  return { nombre, activo, rol };
}

console.log(crearUsuario({ nombre: "Ana" }));
// { nombre: "Ana", activo: true, rol: "usuario" }
```

Si todos los campos tienen valor por defecto, conviene además dar un predeterminado al objeto entero, para que la función pueda llamarse sin argumentos:

```
function crearUsuario({ nombre = "invitado", rol = "usuario" } = {}) {  
  return { nombre, rol };  
}  
  
crearUsuario(); // funciona
```

También puede desestructurarse un array:

```
function calcularDistancia([x1, y1], [x2, y2]) {  
  const diferenciaX = x2 - x1;  
  const diferenciaY = y2 - y1;  
  
  return Math.sqrt(diferenciaX ** 2 + diferenciaY ** 2);  
}  
  
console.log(calcularDistancia([0, 0], [3, 4])); // 5
```

16.11 ¿Por valor o por referencia?

JavaScript pasa todos los argumentos **por valor**. La sutileza es que, cuando el argumento es un objeto, el valor copiado es una **referencia** a ese objeto. Es exactamente lo que vimos en el capítulo 12, ahora aplicado a las llamadas.

Con valores primitivos

```
function cambiar(numero) {  
  numero = 100;  
}  
  
let valor = 10;  
  
cambiar(valor);  
  
console.log(valor); // 10
```

La función recibió una copia del número:

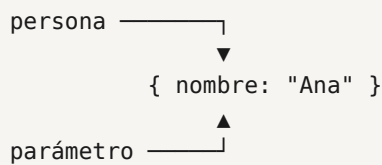
```
valor → 10  
  
cambiar recibe otra variable:  
numero → 10
```

Modificar `numero` no modifica `valor`.

Con objetos

```
function cambiarNombre(persona) {  
  persona.nombre = "María";  
}  
  
const persona = { nombre: "Ana" };  
  
cambiarNombre(persona);  
  
console.log(persona.nombre); // "María"
```

La función recibió una copia de la referencia, pero ambas referencias apuntan al mismo objeto:



Por eso una mutación realizada sobre el objeto es visible fuera de la función.

Reemplazar la referencia no reemplaza la variable exterior

```
function reemplazarPersona(persona) {  
  persona = { nombre: "María" };  
}  
  
const persona = { nombre: "Ana" };  
  
reemplazarPersona(persona);  
  
console.log(persona.nombre); // "Ana"
```

La función solamente reemplazó su copia local de la referencia. La variable exterior sigue apuntando al objeto original.

La descripción precisa:

JavaScript pasa todo por valor. En el caso de los objetos, el valor pasado es una referencia al objeto.

La consecuencia práctica es una regla de diseño: una función que recibe un objeto debería documentar si lo modifica, o directamente no modificarlo y devolver una versión nueva.

16.12 El retorno

La instrucción `return` cumple dos funciones: termina inmediatamente la ejecución de la función y entrega un valor a quien la invocó.

```
function cuadrado(numero) {  
  return numero * numero;  
}  
  
const resultado = cuadrado(5); // 25
```

`return` termina la función

```
function clasificarEdad(edad) {  
  if (edad < 0) {  
    return "Edad inválida";  
  }  
  
  if (edad < 18) {  
    return "Menor";  
  }  
  
  return "Adulto";  
}
```

Cuando JavaScript encuentra un `return`, no continúa ejecutando las instrucciones siguientes. Es lo que hace posibles las guardas de la sección 14.6.

Una función sin `return` devuelve `undefined`

```
function saludar(nombre) {  
  console.log(`Hola, ${nombre}`);  
}  
  
const resultado = saludar("Ana");  
  
console.log(resultado); // undefined
```

La función produce un efecto visible, pero no retorna ningún valor. Conceptualmente es como si existiera un `return undefined` implícito al final.

Esa distinción importa al elegir entre `map()` y `forEach()`, como vimos en la sección 10.10.

16.13 Retornar varios valores

Una función solo puede ejecutar un `return`, pero ese valor puede ser un objeto o un array que contenga varios datos.

Con un objeto

```
function dividir(dividendo, divisor) {  
  return {  
    cociente: Math.trunc(dividendo / divisor),  
    resto: dividendo % divisor,  
  };  
}  
  
const resultado = dividir(17, 5);  
  
console.log(resultado.cociente); // 3  
console.log(resultado.resto);    // 2
```

Y podemos desestructurar en el lugar de uso:

```
const { cociente, resto } = dividir(17, 5);
```

Con un array

```
function minimoYMaximo(numeros) {  
  return [Math.min(...numeros), Math.max(...numeros)];  
}  
  
const [minimo, maximo] = minimoYMaximo([5, 2, 9, 3]);  
  
console.log(minimo); // 2  
console.log(maximo); // 9
```

Los objetos son preferibles cuando cada resultado tiene un significado propio: el nombre lo documenta y el orden deja de importar. Los arrays son apropiados cuando lo que importa es la posición, o cuando quien llama va a renombrar los valores de todos modos.

16.14 Cuidado con el salto de línea después de `return`

Esto no hace lo que parece:

```
function crearResultado() {  
  return  
  {  
    correcto: true  
  };  
}
```

JavaScript inserta un punto y coma automáticamente después de `return`, así que lo interpreta como:

```
function crearResultado() {  
  return;  
  
  {  
    correcto: true  
  }  
}
```

La función devuelve `undefined`.

Debe escribirse:

```
function crearResultado() {  
  return {  
    correcto: true,  
  };  
}
```

La regla práctica es sencilla:

No coloques un salto de línea inmediatamente después de `return`.

Es el caso más conocido de la inserción automática de punto y coma, y vale la pena conocerlo aunque el resto del mecanismo rara vez cause problemas.

16.15 Síntesis conceptual

Una función relaciona entradas con una salida y le da nombre a una operación. Definirla no la ejecuta: los paréntesis son los que invocan, y esa diferencia es la que permite pasar funciones como valores.

Hay tres formas de crear una función. Las declaraciones se elevan por completo y pueden usarse antes de aparecer. Las expresiones de función y las funciones flecha se comportan como cualquier otra vinculación: existen a partir de su línea.

Las funciones flecha no son solo sintaxis más corta. No tienen `this` propio —lo toman del entorno donde fueron escritas—, no tienen `arguments` y no pueden usarse como constructor. Esas diferencias las hacen ideales como *callbacks* dentro de un método y desaconsejables como método de un objeto.

JavaScript no verifica la cantidad de argumentos: los faltantes son `undefined` y los sobrantes se ignoran. Los valores predeterminados cubren solo `undefined`, no `null`. Los parámetros *rest* reúnen lo que sobra en un array, y los parámetros desestructurados permiten que la llamada nombre cada dato, lo cual mejora mucho la legibilidad cuando hay más de dos o tres.

Todos los argumentos se pasan por valor; para los objetos, ese valor es una referencia. Por eso una función puede modificar el contenido de un objeto que recibió, pero no puede reemplazar la variable de quien la llamó.

Finalmente, `return` termina la función y entrega un valor; sin él, la función devuelve `undefined`. Para devolver varios datos se usa un objeto —cuando cada uno tiene un nombre propio— o un array —cuando importa la posición—.

El capítulo siguiente se ocupa de algo que las funciones hacen y que hasta acá solo mencionamos: recordar el entorno en el que fueron creadas.

Capítulo 17. Alcance léxico y clausuras

El capítulo 2 presentó el alcance como la región del código desde la cual una vinculación resulta accesible, y mencionó de pasada que una función puede conservar acceso a variables de su entorno incluso después de que ese entorno terminó. Ese fenómeno se llama **clausura**, y es una de las ideas más importantes de JavaScript.

Este capítulo lo desarrolla desde su origen: primero cómo decide el lenguaje qué vinculación corresponde a cada nombre, y después qué consecuencias tiene esa decisión.

17.1 El alcance, otra vez

Recordemos brevemente los cuatro niveles.

Alcance global. Una variable declarada fuera de todas las funciones y bloques pertenece al ámbito exterior:

```
const nombreAplicacion = "Gestor de alumnos";

function mostrarNombre() {
  console.log(nombreAplicacion);
}

mostrarNombre();
```

La función puede leer una variable de su entorno exterior. Conviene evitar llenar el ámbito global, porque distintas partes del programa pueden interferir entre sí.

Alcance de función. Los parámetros y las variables declaradas dentro de una función solo existen dentro de ella:

```
function calcular() {
  const subtotal = 100;
  const impuesto = 21;

  return subtotal + subtotal * impuesto / 100;
}

// console.log(subtotal); // ReferenceError
```

Podemos imaginar que la función crea una región privada:

```
calcular
subtotal
impuesto
```

Alcance de bloque. `let` y `const` respetan las llaves:

```
if (true) {
  const mensaje = "Hola";
  let contador = 0;

  console.log(mensaje);
}

// console.log(mensaje); // ReferenceError
```

`var`, en cambio, tiene alcance de función y atraviesa los bloques:

```
function ejemplo() {
  if (true) {
    var numero = 10;
  }

  console.log(numero); // 10
}
```

Es una de las razones por las que el código moderno usa `const` y `let`.

Alcance léxico. Es el que nos interesa acá.

17.2 El alcance léxico

JavaScript determina el ámbito de una función **por el lugar donde fue escrita**, no por el lugar desde el cual fue invocada. Eso se llama alcance léxico, o estático.

```

const valor = "global";

function exterior() {
  const valor = "exterior";

  function interior() {
    console.log(valor);
  }

  interior();
}

exterior();
// "exterior"

```

La función `interior` encuentra la variable `valor` en el entorno donde fue definida. La búsqueda conceptual es:

```

¿existe valor dentro de interior?
└─ no
   │
   ▼
¿existe valor en exterior?
└─ sí → "exterior"

```

Si tampoco existiera allí, la búsqueda continuaría hacia afuera hasta el ámbito global, y si no lo encontrara lanzaría un `ReferenceError`.

Esta cadena de entornos es lo que se llama **cadena de alcances**. Es fija: se determina al escribir el código, no al ejecutarlo. Por eso podemos saber a qué variable se refiere un nombre solo leyendo el archivo, sin ejecutar nada.

Vale la pena contrastarlo con `this`, que vimos en la sección 16.6: `this` depende de cómo se invoca la función. Las variables, no. Esa diferencia explica por qué `this` es una fuente de confusión y las clausuras no lo son tanto, una vez entendidas.

17.3 Ocultamiento

Una variable interior puede ocultar temporalmente una exterior con el mismo nombre:

```
const mensaje = "mensaje global";

function mostrar() {
  const mensaje = "mensaje local";

  console.log(mensaje);
}

mostrar();           // "mensaje local"
console.log(mensaje); // "mensaje global"
```

Son dos variables distintas que comparten nombre:

```
ámbito global:
mensaje → "mensaje global"

ámbito de mostrar:
mensaje → "mensaje local"
```

Este fenómeno se conoce como *shadowing*. Es válido y a veces conveniente —por ejemplo, un parámetro que reutiliza el nombre del concepto—, pero si los dos significados no son evidentes, conviene elegir nombres distintos.

17.4 Qué es una clausura

Una **clausura** aparece cuando una función conserva acceso a las variables del entorno donde fue creada, incluso después de que la función exterior haya terminado.

Empecemos con un ejemplo mínimo:

```
function crearSumador(cantidad) {
  return numero => numero + cantidad;
}

const sumar10 = crearSumador(10);
const sumar100 = crearSumador(100);

console.log(sumar10(5)); // 15
console.log(sumar100(5)); // 105
```

¿Qué ocurrió? Al ejecutar:

```
const sumar10 = crearSumador(10);
```

la función exterior recibe `cantidad = 10` y retorna:

```
numero => numero + cantidad
```

Esa función interior conserva acceso a `cantidad`:

```
sumar10
  |
  ▼
numero => numero + cantidad
                |
                └─ cantidad = 10
```

La llamada a `crearSumador` ya terminó. Su marco de ejecución debería haber desaparecido. Pero la variable `cantidad` continúa disponible para la función retornada, porque esa función todavía puede alcanzarla.

Eso es una clausura. Y es la explicación completa de lo que anticipábamos en la sección 2.5: la vida de una variable no depende de dónde fue declarada, sino de si algo todavía puede llegar a ella.

Nótese que `sumar10` y `sumar100` conservan cada una su propia `cantidad`. Cada invocación de `crearSumador` crea un entorno nuevo.

17.5 Las clausuras conservan estado

Podemos construir un contador con estado privado:

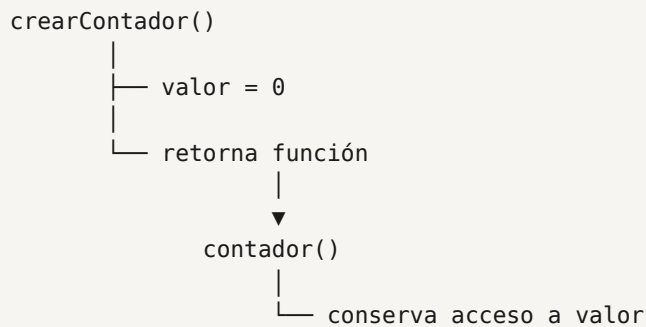
```
function crearContador(inicial = 0) {
  let valor = inicial;

  return function () {
    valor += 1;
    return valor;
  };
}

const contador = crearContador();

console.log(contador()); // 1
console.log(contador()); // 2
console.log(contador()); // 3
```

La variable `valor` no es global. Tampoco es una propiedad de ningún objeto. No se puede leer ni modificar desde afuera. Pero la función retornada conserva acceso a ella y la actualiza en cada llamada:



Esto es privacidad real, a diferencia de los símbolos que vimos en la sección 9.5: no hay ninguna API que permita recuperar `valor` desde fuera.

Un objeto con operaciones privadas

El patrón se extiende naturalmente:

```
function crearContador(inicial = 0) {
  let valor = inicial;

  return {
    incrementar() {
      valor += 1;
    },

    decrementar() {
      valor -= 1;
    },

    obtenerValor() {
      return valor;
    },
  };
}

const contador = crearContador(10);

contador.incrementar();
contador.incrementar();
contador.decrementar();

console.log(contador.obtenerValor()); // 11
```

Las tres funciones comparten la **misma** variable `valor`, porque las tres fueron creadas en el mismo entorno.

Y esto no funciona:

```
contador.valor = 100;
console.log(contador.obtenerValor()); // 11
```

`valor` no es una propiedad del objeto. La asignación crea una propiedad nueva que nadie lee.

```
valor privado
├── incrementar
├── decrementar
└── obtenerValor
```

Esta es una forma de encapsulamiento que no necesita clases ni campos privados. El archivo 046 del material original —hoy el capítulo 19— usa exactamente este patrón para mantener oculta la raíz de un árbol binario.

17.6 La clausura conserva la variable, no una copia

Es un punto que conviene dejar claro, porque la intuición suele fallar:

```
function crearFunciones() {
  let valor = 10;

  const leer = () => valor;
  const cambiar = nuevoValor => {
    valor = nuevoValor;
  };

  return { leer, cambiar };
}

const estado = crearFunciones();

console.log(estado.leer()); // 10

estado.cambiar(50);

console.log(estado.leer()); // 50
```

Las dos funciones comparten la misma variable. `leer` no guardó una fotografía del valor inicial: guardó el acceso a la vinculación.

Dicho de otro modo, una clausura no captura valores. Captura **variables**.

17.7 Clausuras y bucles

Este ejemplo es clásico y explica de golpe por qué `let` fue una mejora importante del lenguaje.

Con `var`:

```
const funciones = [];

for (var i = 0; i < 3; i++) {
  funciones.push(() => i);
}

console.log(funciones[0]()); // 3
console.log(funciones[1]()); // 3
console.log(funciones[2]()); // 3
```

Las tres funciones dan 3. La razón es la de la sección anterior: las tres conservan acceso a **la misma** variable `i`, porque `var` tiene alcance de función y el bucle no crea uno nuevo. Cuando las funciones se ejecutan, el bucle ya terminó e `i` vale 3.

Con `let`:

```
const funciones = [];

for (let i = 0; i < 3; i++) {
  funciones.push(() => i);
}

console.log(funciones[0]()); // 0
console.log(funciones[1]()); // 1
console.log(funciones[2]()); // 2
```

`let` crea una **vinculación diferente para cada iteración** del bucle. Cada función captura la suya.

Antes de que existiera `let`, la solución era crear un alcance a mano con una función:

```
for (var i = 0; i < 3; i++) {
  (function (indice) {
    funciones.push(() => indice);
  })(i);
}
```

Esa construcción —una función que se define y se invoca de inmediato— se llama IIFE, *Immediately Invoked Function Expression*. Aparece mucho en código anterior a 2015 y su propósito casi siempre era exactamente este: crear un alcance donde el lenguaje no lo daba.

17.8 Para qué se usan las clausuras

Más allá de los ejemplos didácticos, las clausuras sostienen varios patrones habituales.

Funciones fábrica

Producir funciones especializadas a partir de una configuración:

```
function crearValidadorDeLongitud(minimo, maximo) {
  return texto => texto.length >= minimo && texto.length <= maximo;
}

const validarClave = crearValidadorDeLongitud(8, 64);
const validarUsuario = crearValidadorDeLongitud(3, 20);

console.log(validarClave("corta")); // false
console.log(validarUsuario("ana")); // true
```

Los límites quedan capturados en cada validador. Quien lo usa no tiene que volver a pasarlos.

Memoización

Recordar resultados ya calculados:

```
function memoizar(funcion) {
  const cache = new Map();

  return function (argumento) {
    if (cache.has(argumento)) {
      return cache.get(argumento);
    }

    const resultado = funcion(argumento);
    cache.set(argumento, resultado);

    return resultado;
  };
}

const cuadradoLento = numero => {
  console.log("calculando...");
  return numero * numero;
};

const cuadrado = memoizar(cuadradoLento);

console.log(cuadrado(4)); // "calculando..." y luego 16
console.log(cuadrado(4)); // 16, sin recalculer
```

El `Map` vive en la clausura. No es global, no contamina nada y persiste entre llamadas.

Estado privado en módulos

Un módulo puede exportar funciones que comparten estado sin exponerlo:

```
let solicitudes = 0;

export function registrarSolicitud() {
  solicitudes += 1;
}

export function obtenerTotal() {
  return solicitudes;
}
```

`solicitudes` es privada al módulo: quien lo importa solo ve las dos funciones.

this capturado

Ya lo vimos en la sección 16.6, y también es una clausura: la función flecha captura `this` del entorno léxico, igual que captura cualquier otra vinculación.

17.9 Una advertencia sobre la memoria

Las clausuras mantienen vivas las variables que capturan. Eso es exactamente lo que queremos, pero conviene tenerlo presente cuando la variable capturada es grande y la función vive mucho tiempo:

```
function crearProcesador(datosEnormes) {
  const total = datosEnormes.length;

  return () => total; // captura todo el entorno, incluidos los datos
}
```

Si la función retornada solo necesita `total`, no hay motivo para que el entorno conserve `datosEnormes`. Algunos motores optimizan este caso y otros no. Cuando importa, conviene extraer explícitamente lo que se necesita antes de crear la clausura, y no capturar estructuras grandes que ya no se usan.

Es la misma consideración que motivó `WeakMap` en la sección 13.11.

17.10 Síntesis conceptual

JavaScript resuelve los nombres por alcance léxico: la vinculación a la que se refiere un identificador queda determinada por dónde está escrito el código, no por dónde se lo invoca. La búsqueda va del alcance más interno hacia afuera, y esa cadena es fija.

Una clausura es la consecuencia natural de esa regla: si una función interior puede alcanzar una variable exterior, esa variable sigue viva mientras la función exista, aunque la función exterior haya terminado hace rato.

Las clausuras capturan variables, no valores. Varias funciones creadas en el mismo entorno comparten las mismas vinculaciones, y por eso pueden coordinarse leyendo y escribiendo el mismo estado sin exponerlo.

El caso del bucle con `var` es la mejor demostración de esa propiedad, y la razón por la que `let` crea una vinculación nueva en cada iteración.

En la práctica, las clausuras sostienen las funciones fábrica, la memoización, el estado privado de un módulo y el `this` capturado por las funciones flecha. Son un mecanismo de encapsulamiento que no requiere clases.

El capítulo siguiente lleva esta idea a su conclusión: si las funciones son valores que recuerdan su entorno, también son valores que podemos pasar, almacenar y combinar.

Capítulo 18. Funciones de orden superior y procesamiento de colecciones

El capítulo 10 mostró `map()`, `filter()` y `reduce()` en su uso práctico. El capítulo 17 mostró funciones que retornan funciones. Los dos apuntaban a la misma idea sin nombrarla del todo:

*En JavaScript, una función no es solamente un bloque de código reutilizable. Es un **valor** que puede almacenarse, pasarse como argumento, retornarse y combinarse.*

Esa propiedad se resume diciendo que las funciones son **ciudadanos de primera clase**. De ella se derivan los *callbacks*, las clausuras, las funciones de orden superior y todo el estilo de procesamiento de colecciones que usamos hoy.

18.1 Las funciones son valores

Podemos almacenarlas en variables, como ya vimos:

```
const sumar = (a, b) => a + b;
```

Guardarlas en un array:

```
const operaciones = [
  (a, b) => a + b,
  (a, b) => a - b,
  (a, b) => a * b,
];

console.log(operaciones[0](10, 5)); // 15
console.log(operaciones[1](10, 5)); // 5
console.log(operaciones[2](10, 5)); // 50
```

`operaciones[0]` obtiene la función; los paréntesis siguientes la invocan.

Colocarlas en un objeto:

```
const operaciones = {
  sumar: (a, b) => a + b,
  restar: (a, b) => a - b,
  multiplicar: (a, b) => a * b,
};

console.log(operaciones.sumar(4, 7)); // 11
```

Este último patrón es una alternativa muy legible a un `switch` largo:

```
function calcular(operacion, a, b) {
  const funcion = operaciones[operacion];

  if (!funcion) {
    throw new Error(`Operación desconocida: ${operacion}`);
  }

  return funcion(a, b);
}
```

Agregar una operación nueva es agregar una entrada al objeto, sin tocar `calcular`.

18.2 Funciones de orden superior

Una **función de orden superior** es una función que cumple al menos una de estas condiciones:

1. recibe una función como argumento;
2. retorna una función.

La primera:

```
function aplicarOperacion(numero, operacion) {
  return operacion(numero);
}

const doble = numero => numero * 2;
const cuadrado = numero => numero ** 2;

console.log(aplicarOperacion(5, doble)); // 10
console.log(aplicarOperacion(5, cuadrado)); // 25
```

También podemos pasar la función directamente, sin nombrarla:

```
console.log(aplicarOperacion(5, numero => numero + 100)); // 105
```

Una función enviada a otra función suele llamarse **callback**:

```
function procesar(valor, callback) {
  return callback(valor);
}

procesar(10, valor => valor * 2); // 20
```

El nombre no implica asincronía. Un *callback* es simplemente una función que entregamos para que otra decida cuándo y cómo ejecutarla. `map()`, `filter()`, `sort()` y `addEventListener()` reciben *callbacks*, y solo el último tiene que ver con eventos futuros.

La segunda condición:


```
function multiplicarPor(factor) {
  return numero => numero * factor;
}

const duplicar = multiplicarPor(2);
const triplicar = multiplicarPor(3);

console.log(duplicar(10)); // 20
console.log(triplicar(10)); // 30
```

`multiplicarPor` es de orden superior porque retorna una función. Y además usa una clausura, porque la función retornada conserva el parámetro `factor`:

```
multiplicarPor(3)
  |
  └─ retorna numero => numero * factor
        |
        └─ factor = 3
```

Las funciones de orden superior y las clausuras suelen trabajar juntas: la primera produce la función, la segunda le da memoria.

18.3 Composición

Podemos escribir una función que combine otras funciones:

```
function componer(f, g) {
  return valor => f(g(valor));
}

const duplicar = numero => numero * 2;
const sumarUno = numero => numero + 1;

const duplicarYSumarUno = componer(sumarUno, duplicar);

console.log(duplicarYSumarUno(5)); // 11
```

El orden de aplicación es de adentro hacia afuera:

```
5
↓
duplicar
↓
10
↓
sumarUno
↓
11
```

`componer(f, g)` aplica primero `g` y después `f`, siguiendo la convención matemática de `f ∘ g`. Si preferimos leer de izquierda a derecha, la variante suele llamarse `encadenar`:

```
function encadenar(...funciones) {  
  return valor => funciones.reduce((actual, funcion) => funcion(actual), valor);  
}  
  
const procesar = encadenar(duplicar, sumarUno);  
  
console.log(procesar(5)); // 11
```

Esta versión usa `reduce()` sobre un array de funciones, lo cual muestra hasta qué punto las funciones son valores como cualquier otro.

La composición es la misma idea del apéndice A —construir lo complejo combinando lo simple— aplicada al nivel del programa.

18.4 Cómo funcionan `map`, `filter` y `reduce` por dentro

Implementarlos aclara qué hacen. No son magia: son bucles con un *callback*.

`map`

```
function mapear(array, transformar) {  
  const resultado = [];  
  
  for (let indice = 0; indice < array.length; indice++) {  
    resultado.push(transformar(array[indice], indice, array));  
  }  
  
  return resultado;  
}  
  
console.log(mapear([1, 2, 3], numero => numero * 2)); // [2, 4, 6]
```

Conceptualmente:

```
para cada elemento:  
  llamar a transformar(elemento)  
  guardar el resultado
```

El array resultante tiene siempre la misma cantidad de elementos que el original:

```

[1, 2, 3, 4]
|  |  |  |
▼  ▼  ▼  ▼
x2 x2 x2 x2
|  |  |  |
▼  ▼  ▼  ▼
[2, 4, 6, 8]

```

filter

```

function filtrar(array, condicion) {
  const resultado = [];

  for (let indice = 0; indice < array.length; indice++) {
    const elemento = array[indice];

    if (condicion(elemento, indice, array)) {
      resultado.push(elemento);
    }
  }

  return resultado;
}

```

El *callback* debe producir un valor que se interprete como booleano:

```

1 → false → descartar
2 → true  → conservar
3 → false → descartar
4 → true  → conservar

```

Los elementos conservados son los mismos objetos, no copias.

reduce

```

function reducir(array, combinar, valorInicial) {
  let acumulador = valorInicial;

  for (let indice = 0; indice < array.length; indice++) {
    acumulador = combinar(acumulador, array[indice], indice, array);
  }

  return acumulador;
}

const total = reducir([10, 20, 30], (acumulador, n) => acumulador + n, 0);
// 60

```

Paso por paso:

```
acumulador inicial = 0
0 + 10 = 10
10 + 20 = 30
30 + 30 = 60
```

Y `reduce()` puede producir **cualquier** tipo de resultado, no solo un número:

```
const personas = [
  { nombre: "Ana", curso: "C1" },
  { nombre: "Juan", curso: "C3" },
  { nombre: "María", curso: "C1" },
];

const porCurso = personas.reduce((grupos, persona) => {
  const curso = persona.curso;

  if (!grupos[curso]) {
    grupos[curso] = [];
  }

  grupos[curso].push(persona);

  return grupos;
}, {});
```

Resultado:

```
{
  C1: [{ nombre: "Ana", curso: "C1" }, { nombre: "María", curso: "C1" }],
  C3: [{ nombre: "Juan", curso: "C3" }],
}
```

Agrupar es un caso muy frecuente de `reduce()`. Los entornos modernos ofrecen además `Object.groupBy()` y `Map.groupBy()`, que hacen exactamente esto con menos código:

```
const porCurso = Object.groupBy(personas, (persona) => persona.curso);
```

18.5 Los tres modelos mentales

Vale la pena fijarlos, porque cubren la mayoría de los casos:

map: muchos elementos → la misma cantidad, transformados.

```
[1, 2, 3].map(x => x * 2); // [2, 4, 6]
```

filter: muchos elementos → algunos de esos mismos elementos.

```
[1, 2, 3, 4].filter(x => x % 2 === 0); // [2, 4]
```

reduce: muchos elementos → un resultado acumulado.

```
[1, 2, 3, 4].reduce((total, x) => total + x, 0); // 10
```

En una línea:

```
map    → transformar
filter → seleccionar
reduce → acumular
```

Elegir el que corresponde a la intención hace que el código se lea como una descripción de lo que hace, y no como una receta de cómo lo hace.

18.6 Encadenamiento

Como `map()` y `filter()` devuelven arrays, pueden encadenarse:

```
const resultado = alumnos
  .filter(alumno => alumno.nota >= 6)
  .map(alumno => ({
    nombre: alumno.nombre,
    nota: alumno.nota,
  }))
  .sort((a, b) => b.nota - a.nota);
```

La lectura es directa:

```
alumnos
  ↓
conservar los aprobados
  ↓
proyectar nombre y nota
  ↓
ordenar por nota descendente
```

Cada paso recibe el resultado del anterior. La cadena describe una transformación, no un procedimiento.

Un detalle sobre `sort()`: como vimos en la sección 10.12, **modifica el array original**. En la cadena anterior no importa, porque `map()` ya produjo un array nuevo. Pero esto sí sería un problema:

```
const ordenados = alumnos.sort((a, b) => b.nota - a.nota); // muta alumnos
```

La versión segura:

```
const ordenados = [...alumnos].sort((a, b) => b.nota - a.nota);
```

18.7 Comparación con LINQ

Para quien viene de C#, la cadena anterior resulta familiar:

```
var resultado = alumnos
    .Where(alumno => alumno.Nota >= 6)
    .Select(alumno => new { alumno.Nombre, alumno.Nota })
    .OrderByDescending(alumno => alumno.Nota)
    .ToList();
```

Las equivalencias aproximadas:

LINQ	JavaScript	Propósito
Where	filter	conservar elementos
Select	map	transformar elementos
SelectMany	flatMap	transformar y aplanar
Aggregate	reduce	acumular
Any	some	comprobar si alguno cumple
All	every	comprobar si todos cumplen
FirstOrDefault	find	encontrar el primero
OrderBy	sort con comparador	ordenar
OrderByDescending	sort inverso	ordenar descendente
Count	length o filter().length	contar
Contains	includes	comprobar existencia
Skip y Take	slice	omitir y tomar
Distinct	Set	eliminar repetidos
Sum	reduce	sumar
GroupBy	reduce u Object.groupBy	agrupar

Algunos casos concretos:

```
alumnos.filter(a => a.nota >= 6);           // Where
alumnos.map(a => a.nombre);                  // Select
alumnos.some(a => a.nota === 10);            // Any
alumnos.every(a => a.asistencia >= 0.75);    // All
alumnos.find(a => a.nombre === "Ana");        // FirstOrDefault
elementos.slice(10, 15);                    // Skip(10).Take(5)
[...new Set(valores)];                      // Distinct
```

`find()` devuelve `undefined` cuando no encuentra, no `null`.

Una suma:

```
const suma = alumnos.reduce((total, alumno) => total + alumno.nota, 0);
```

Y un promedio, contemplando el array vacío:

```
const promedio = alumnos.length === 0
  ? 0
  : alumnos.reduce((total, alumno) => total + alumno.nota, 0) / alumnos.length;
```

Un caso combinado: todos los temas cursados, sin repetición.

```
const temas = [...new Set(alumnos.flatMap(alumno => alumno.temas))];
```

`flatMap()` transforma cada alumno en su array de temas y aplana un nivel; el `Set` elimina las repeticiones; la expansión devuelve un array.

18.8 Una diferencia importante: ejecución inmediata

Aquí las dos tecnologías se separan. Los métodos de arrays de JavaScript se ejecutan **inmediatamente**:

```
const resultado = alumnos
  .filter(alumno => {
    console.log("filtrando", alumno.nombre);
    return alumno.nota >= 6;
  })
  .map(alumno => {
    console.log("transformando", alumno.nombre);
    return alumno.nombre;
  });
```

Primero se ejecuta `filter()` por completo y genera un array. Después `map()` recorre ese array nuevo:

```
array original
  ↓ filter completo
array intermedio
  ↓ map completo
array final
```

LINQ sobre `IEnumerable<T>`, en cambio, usa **ejecución diferida**: la consulta se construye y no se ejecuta hasta que se la enumera. El comportamiento se parece, pero no es idéntico, y esa diferencia se nota con colecciones grandes.

El costo de los intermedios

Esta expresión:

```
const resultado = numeros
  .filter(numero => numero % 2 === 0)
  .map(numero => numero ** 2);
```

crea normalmente dos arrays: uno con los pares y otro con sus cuadrados. Para colecciones de tamaño razonable eso es perfectamente aceptable, y la claridad lo justifica de sobra.

Si la cantidad de datos fuera enorme, podríamos evitar el intermedio con un solo bucle:

```
const resultado = [];

for (const numero of numeros) {
  if (numero % 2 === 0) {
    resultado.push(numero ** 2);
  }
}
```

El bucle puede ser más eficiente en casos extremos, pero mezcla las dos operaciones. La cadena es más declarativa:

```
filtrar pares
↓
elevar al cuadrado
```

El criterio: primero elegir claridad; después, si existe un problema **medido**, optimizar.

18.9 Evaluación perezosa con generadores

JavaScript tiene una manera de acercarse al comportamiento diferido de LINQ: los **generadores**.

Una función generadora se declara con `function*` y produce valores con `yield`. Cada `yield` suspende la ejecución hasta que se pida el valor siguiente.


```

function* where(iterable, condicion) {
  for (const elemento of iterable) {
    if (condicion(elemento)) {
      yield elemento;
    }
  }
}

function* select(iterable, transformar) {
  for (const elemento of iterable) {
    yield transformar(elemento);
  }
}

function* take(iterable, cantidad) {
  let tomados = 0;

  for (const elemento of iterable) {
    if (tomados >= cantidad) {
      return;
    }

    yield elemento;
    tomados++;
  }
}

```

Con esas tres piezas podemos operar sobre una fuente **infinita**:

```

function* naturales() {
  let numero = 1;

  while (true) {
    yield numero;
    numero++;
  }
}

```

Y construir una consulta:

```

const pares = where(naturales(), numero => numero % 2 === 0);
const cuadrados = select(pares, numero => numero ** 2);
const primerosCinco = take(cuadrados, 5);

console.log([...primerosCinco]);
// [4, 16, 36, 64, 100]

```

La fuente no tiene fin, pero solo se generan los valores necesarios:

```
naturales
  ↓
where: solamente pares
  ↓
select: elevar al cuadrado
  ↓
take: detenerse después de 5
```

Nada se calcula hasta que la expansión `[...primerosCinco]` empiece a pedir valores. Con `filter()` y `map()` esto sería imposible: el primer `filter()` nunca terminaría.

Los generadores implementan el protocolo `Symbol.iterator` que vimos en la sección 8.7, y por eso funcionan con `for...of`, con la expansión y con la desestructuración.

El ejemplo pone en juego a la vez funciones de orden superior, *callbacks*, generadores, iteradores, clausuras y composición de operaciones.

18.10 Detalles que conviene conocer

Los *callbacks* reciben más de un argumento

`map()`, `filter()`, `forEach()` y varios más invocan al *callback* con tres argumentos:

```
elemento
índice
array completo
```

```
const nombres = ["Ana", "Juan", "María"];

const numerados = nombres.map(
  (nombre, indice) => `${indice + 1}. ${nombre}`
);
// ["1. Ana", "2. Juan", "3. María"]
```

Esto tiene una consecuencia que sorprende:

```
["1", "2", "3"].map(Number.parseInt);
// [1, NaN, NaN]
```

`map()` pasa el índice como segundo argumento, y `parseInt` lo interpreta como base. La versión correcta es explícita:

```
["1", "2", "3"].map(texto => Number.parseInt(texto, 10)); // [1, 2, 3]
["1", "2", "3"].map(Number);                             // [1, 2, 3]
```

Pasar directamente una función que acepta más argumentos de los que esperamos es una fuente silenciosa de errores.

Olvidar `return` en una flecha con llaves

```
const dobles = [1, 2, 3].map(numero => {
  numero * 2;
});

console.log(dobles); // [undefined, undefined, undefined]
```

Al usar llaves desaparece el retorno implícito. Hay que escribir:

```
const dobles = [1, 2, 3].map(numero => {
  return numero * 2;
});
```

o eliminar las llaves:

```
const dobles = [1, 2, 3].map(numero => numero * 2);
```

Nombrar los *callbacks*

Podemos escribir:

```
const resultado = alumnos
  .filter(a => a.nota >= 6)
  .map(a => a.nombre);
```

O nombrar las operaciones:

```
const estaAprobado = alumno => alumno.nota >= 6;
const obtenerNombre = alumno => alumno.nombre;

const resultado = alumnos
  .filter(estaAprobado)
  .map(obtenerNombre);
```

La segunda versión se lee como una frase. Nombrar un *callback* conviene cuando la condición es compleja, se usa en varios lugares, queremos probarla por separado o su nombre explica mejor la intención que su código.

18.11 Funciones puras

Una función **pura** cumple dos condiciones:

1. para los mismos argumentos produce siempre el mismo resultado;
2. no modifica datos externos.

```
function sumar(a, b) {
  return a + b;
}

function aplicarDescuento(precio, porcentaje) {
  return precio - precio * porcentaje / 100;
}
```

Una función con **efecto secundario**:

```
let total = 0;

function agregar(valor) {
  total += valor;
}
```

`agregar` modifica una variable exterior. Llamarla dos veces con el mismo argumento produce estados distintos.

Los efectos secundarios no son malos: escribir en un archivo, mostrar en pantalla o guardar en una base de datos son efectos, y son el propósito de muchos programas. Lo que conviene es **saber dónde están**.

Los *callbacks* de `map()`, `filter()` y `reduce()` son mucho más fáciles de razonar cuando son puros:

```
const dobles = numeros.map(numero => numero * 2);
```

Podemos leer esa línea sin preguntarnos qué más ocurrió. Si el *callback* mutara algo, tendríamos que revisarlo.

Y para los efectos existe `forEach()`, que anuncia con su nombre que no produce un valor.

18.12 Ejemplo integrador

```
const ventas = [
  { producto: "Teclado", categoria: "Periféricos", precio: 50, cantidad: 2 },
  { producto: "Mouse", categoria: "Periféricos", precio: 20, cantidad: 5 },
  { producto: "Monitor", categoria: "Pantallas", precio: 300, cantidad: 1 },
  { producto: "Webcam", categoria: "Periféricos", precio: 80, cantidad: 0 },
];
```

Queremos conservar los productos vendidos, calcular el importe de cada uno, ordenarlos por importe descendente y obtener el total general.

El detalle:

```
const detalle = ventas
  .filter(venta => venta.cantidad > 0)
  .map(venta => ({
    producto: venta.producto,
    categoria: venta.categoria,
    importe: venta.precio * venta.cantidad,
  }))
  .sort((a, b) => b.importe - a.importe);
```

Resultado:

```
[
  { producto: "Monitor", categoria: "Pantallas", importe: 300 },
  { producto: "Teclado", categoria: "Periféricos", importe: 100 },
  { producto: "Mouse", categoria: "Periféricos", importe: 100 },
]
```

El total:

```
const total = detalle.reduce(
  (acumulador, venta) => acumulador + venta.importe,
  0
);

console.log(total); // 500
```

La consulta completa expresa la intención con claridad:

```
ventas
  ↓
  quedarse con las que tienen cantidad
  ↓
  convertir cada venta en un importe
  ↓
  ordenar por importe
  ↓
  sumar los importes
```

Nótese que `sort()` aquí es seguro: opera sobre el array que `map()` acaba de crear, no sobre `ventas`.

18.13 Síntesis conceptual

En JavaScript las funciones son valores. Pueden guardarse en variables, arrays y objetos, pasarse como argumentos y retornarse desde otras funciones. Esa propiedad es la base de todo lo demás.

Una función de orden superior recibe o retorna funciones. Combinada con las clausuras del capítulo 17, permite fabricar funciones especializadas a partir de una configuración, y componer transformaciones pequeñas para formar otras más grandes.

`map()`, `filter()` y `reduce()` no son construcciones especiales: son bucles con un *callback*, y escribir la propia versión de cada uno deja eso a la vista. Sus tres modelos mentales — transformar, seleccionar, acumular— cubren la mayor parte del procesamiento de colecciones.

El encadenamiento produce código declarativo, que describe qué se quiere en lugar de cómo obtenerlo. A cambio genera arrays intermedios, algo que solo importa con volúmenes grandes y que conviene medir antes de optimizar.

A diferencia de LINQ, los métodos de arrays se ejecutan de inmediato. Para evaluación perezosa —incluidas las fuentes infinitas— JavaScript ofrece los generadores, que producen valores de a uno y solo cuando se los pide.

Dos detalles evitan errores frecuentes: los *callbacks* reciben índice y array además del elemento, y una flecha con llaves necesita `return` explícito.

Finalmente, una función pura es más fácil de leer, probar y reutilizar. Los efectos siguen siendo necesarios; lo que conviene es concentrarlos donde se los espera.

El capítulo siguiente cierra la parte con una técnica que combina todo lo visto: una función que se llama a sí misma.

Capítulo 19. Recursión y estructuras recursivas

Una función puede llamarse a sí misma. Esa posibilidad, que al principio parece un truco, resulta ser la herramienta natural para trabajar con datos cuya estructura se define en términos de sí misma.

19.1 Las dos partes de una función recursiva

Toda función recursiva necesita dos cosas:

- un **caso base**, que resuelve el problema directamente y no vuelve a llamarse;
- un **caso recursivo**, que resuelve una versión más pequeña del problema llamándose a sí misma.

Sin caso base, la función no termina. Sin caso recursivo, no es recursiva.

El ejemplo canónico:

```
function factorial(n) {  
  if (n <= 1) {  
    return 1;           // caso base  
  }  
  
  return n * factorial(n - 1); // caso recursivo  
}  
  
console.log(factorial(5)); // 120
```

La clave está en que `factorial(n - 1)` es **un problema más chico** que `factorial(n)`. Cada llamada se acerca al caso base.

Podemos seguir la ejecución:

```
factorial(5)  
  → 5 * factorial(4)  
    → 4 * factorial(3)  
      → 3 * factorial(2)  
        → 2 * factorial(1)  
          → 1          ← caso base  
        ← 2  
      ← 6  
    ← 24  
  ← 120
```

Las llamadas se apilan hasta llegar al caso base y después los resultados regresan multiplicándose. Es exactamente la pila de llamadas que describimos en la sección 15.8, ahora usada a propósito.

19.2 Confiar en la llamada recursiva

El obstáculo mental más común al aprender recursión es intentar seguir todas las llamadas en la cabeza. Con tres niveles se puede; con veinte, no.

La técnica que funciona es otra. Al escribir el caso recursivo, hay que **suponer que la llamada recursiva ya funciona correctamente** y preguntarse solo qué hacer con su resultado.

Para `factorial(n)`:

1. ¿Qué me devuelve `factorial(n - 1)`? El factorial de `n - 1`.
2. ¿Qué tengo que hacer con eso? Multiplicarlo por `n`.

Listo. No hace falta imaginar la cadena completa.

Ese salto de confianza es lo que vuelve manejable la recursión, y es también lo que la hace parecer mágica al principio. La justificación formal es la inducción matemática: si el caso base es correcto, y el caso recursivo es correcto **suponiendo** que la llamada anterior lo fue, entonces la función es correcta para todo valor.

19.3 Recursión e iteración

Casi toda función recursiva tiene una versión iterativa:

```
function factorial(n) {  
  let resultado = 1;  
  
  for (let i = 2; i <= n; i++) {  
    resultado *= i;  
  }  
  
  return resultado;  
}
```

Esta versión es más eficiente: no apila llamadas y no consume memoria adicional. Para un problema lineal como el factorial, la iteración suele ser la mejor elección.

Entonces, ¿cuándo conviene la recursión?

*Cuando la **estructura de los datos** es recursiva.*

Si un dato se define en términos de sí mismo, escribir un bucle para recorrerlo exige mantener a mano una pila de posiciones pendientes. La recursión ya nos da esa pila gratis.

Ejemplos de estructuras recursivas:

- un árbol, donde cada nodo contiene otros nodos;
- un directorio, que contiene archivos y otros directorios;
- un objeto JSON, cuyos valores pueden ser objetos;
- una expresión aritmética, cuyos operandos pueden ser expresiones.

En el capítulo 21 vamos a recorrer directorios de manera recursiva por exactamente esta razón.

Un aviso sobre la profundidad

Cada llamada pendiente ocupa espacio en la pila, que tiene un límite:

```
function contar(n) {  
  if (n === 0) {  
    return 0;  
  }  
  
  return 1 + contar(n - 1);  
}  
  
// contar(100000); // RangeError: Maximum call stack size exceeded
```

Ese límite depende del motor y ronda las decenas de miles de llamadas. Para recorrer un árbol equilibrado no es un problema: la profundidad crece de manera logarítmica. Para recorrer una lista de un millón de elementos, sí lo es, y corresponde un bucle.

19.4 Un árbol binario de búsqueda

Vamos a construir uno paso a paso: primero la representación de un nodo, después inserción, búsqueda y recorrido, y finalmente todo reunido en una función constructora que mantenga la raíz encapsulada.

Un **árbol binario** es una estructura jerárquica en la que cada nodo puede tener como máximo dos hijos. En un **árbol binario de búsqueda**, los valores menores se ubican a la izquierda y los mayores a la derecha:

```
      8  
     /\   
    3  10  
   /\   
  1  6
```

Esa disposición es lo que le da su utilidad: para encontrar un valor no hace falta recorrer todos los nodos, sino bajar por una sola rama, descartando la mitad del árbol en cada paso.

La forma de representar el árbol es recursiva: cada nodo contiene un valor y puede contener otros nodos como hijos. Un árbol completo está formado por nodos anidados dentro de otros nodos, y **cada subárbol tiene la misma estructura que el árbol original**, aunque sea más pequeño.

Por eso la forma más natural de procesarlo es mediante funciones recursivas. Cada función resuelve una parte del problema llamándose a sí misma con una versión reducida de la estructura: el subárbol izquierdo o el derecho. La recursión termina al llegar a un árbol vacío, que funciona como caso base.

19.5 Representar un nodo

Cada nodo puede ser un objeto. Omitimos las propiedades de los hijos mientras no existan:

```
const nodo = { valor: 8 };
```

Un nodo con hijos:

```
const arbol = {  
  valor: 8,  
  izquierdo: { valor: 3 },  
  derecho: { valor: 10 },  
};
```

Y un árbol vacío se representa con `undefined`. Esa elección es deliberada: al leer `arbol.izquierdo` de un nodo sin hijo izquierdo obtenemos `undefined` naturalmente, sin necesidad de inicializar nada.

19.6 Insertar valores

La inserción compara el valor nuevo con el nodo actual. Si el árbol está vacío, crea un nodo; si el valor es menor, continúa por la izquierda; si es mayor, por la derecha.

```
function insertar(arbol, valor) {
  if (arbol === undefined) {
    return { valor };
  }

  if (valor < arbol.valor) {
    arbol.izquierdo = insertar(arbol.izquierdo, valor);
  } else if (valor > arbol.valor) {
    arbol.derecho = insertar(arbol.derecho, valor);
  }

  return arbol;
}
```

Hay dos decisiones de diseño que vale la pena señalar.

La primera: la función **devuelve el árbol**, incluso cuando no lo cambió. Eso permite que la llamada recursiva resuelva el caso del hijo inexistente sin un tratamiento especial. Cuando `arbol.izquierdo` es `undefined`, la llamada recibe `undefined`, entra en el caso base, crea el nodo y lo devuelve. La asignación `arbol.izquierdo = ...` lo conecta.

La segunda: los valores repetidos no se agregan, porque solo se insertan los casos estrictamente menores o mayores. Si el valor es igual al del nodo actual, no ocurre nada.

En uso:

```
let arbol;

arbol = insertar(arbol, 8);
arbol = insertar(arbol, 3);
arbol = insertar(arbol, 10);
arbol = insertar(arbol, 1);
arbol = insertar(arbol, 6);
```

La primera llamada recibe `undefined` y devuelve la raíz. Por eso hay que reasignar `arbol` en cada llamada.

19.7 Buscar un valor

La búsqueda usa la misma comparación. Si llega a un lugar vacío, el valor no está; si coincide con el nodo actual, lo encontró; de lo contrario continúa por el lado correspondiente.

```
function buscar(arbol, valor) {
  if (arbol === undefined) {
    return false;
  }

  if (valor === arbol.valor) {
    return true;
  }

  if (valor < arbol.valor) {
    return buscar(arbol.izquierdo, valor);
  } else {
    return buscar(arbol.derecho, valor);
  }
}

console.log(buscar(arbol, 6)); // true
console.log(buscar(arbol, 9)); // false
```

Aquí hay dos casos base —árbol vacío y valor encontrado— y dos casos recursivos. Es perfectamente normal: nada obliga a que haya uno de cada.

Al buscar `9` en el árbol del ejemplo, el recorrido es `8 → 10 → undefined`. Se visitaron dos nodos de cinco. Con mil nodos bien distribuidos serían alrededor de diez.

19.8 Recorrer en orden

El recorrido **inorden** visita los nodos así:

```
izquierda → raíz → derecha
```

```
function recorrer(arbol, visita) {
  if (arbol === undefined) {
    return;
  }

  recorrer(arbol.izquierdo, visita);
  visita(arbol.valor);
  recorrer(arbol.derecho, visita);
}

recorrer(arbol, console.log);
// 1, 3, 6, 8, 10
```

En un árbol binario de búsqueda, el recorrido inorden **devuelve los valores ordenados**. No hay ningún paso de ordenamiento: el orden estaba implícito en la estructura y el recorrido simplemente lo hace explícito.

`recorrer` recibe una función `visita` como argumento, así que es una función de orden superior, con la lógica del capítulo 18. Eso la vuelve reutilizable: podemos imprimir, acumular en un array o contar sin modificarla.

```
const valores = [];  
recorrer(arbol, (valor) => valores.push(valor));  
  
console.log(valores); // [1, 3, 6, 8, 10]
```

Cambiando el orden de las tres líneas obtenemos otros recorridos: **preorden** —raíz, izquierda, derecha— y **postorden** —izquierda, derecha, raíz—, útiles para copiar y para liberar estructuras respectivamente.

19.9 Encapsular el árbol

Hasta ahora la raíz estaba expuesta y quien usa el árbol tenía que reasignarla en cada inserción. La siguiente función constructora la guarda internamente y devuelve solamente las operaciones públicas.

Las funciones recursivas internas reciben el nodo actual; los métodos públicos comienzan desde `raiz`.

```

function Arbol(comparar) {
  let raiz;
  comparar ??= (a, b) => String(a).localeCompare(String(b));

  function insertarNodo(arbol, valor) {
    if (arbol === undefined) {
      return { valor };
    }

    const resultado = comparar(valor, arbol.valor);

    if (resultado < 0) {
      arbol.izquierdo = insertarNodo(arbol.izquierdo, valor);
    } else if (resultado > 0) {
      arbol.derecho = insertarNodo(arbol.derecho, valor);
    }

    return arbol;
  }

  function buscarNodo(arbol, valor) {
    if (arbol === undefined) {
      return false;
    }

    const resultado = comparar(valor, arbol.valor);

    if (resultado === 0) {
      return true;
    }

    if (resultado < 0) {
      return buscarNodo(arbol.izquierdo, valor);
    } else {
      return buscarNodo(arbol.derecho, valor);
    }
  }

  function recorrerInorden(arbol, visita) {
    if (arbol === undefined) {
      return;
    }

    recorrerInorden(arbol.izquierdo, visita);
    visita(arbol.valor);
    recorrerInorden(arbol.derecho, visita);
  }

  return {
    insertar(valor) {
      raiz = insertarNodo(raiz, valor);
    },

    buscar(valor) {
      return buscarNodo(raiz, valor);
    },

    recorrer(visita) {

```

```

        recorrerInorden(raiz, visita);
    },
};
}

```

La función recursiva se llama `insertarNodo` para diferenciarla del método público `insertar`. La raíz queda protegida dentro de la clausura y desde afuera solo se accede mediante esos tres métodos.

Es exactamente el patrón de encapsulamiento del capítulo 17: `raiz` y `comparar` no son propiedades del objeto retornado, sino variables del entorno que las tres funciones comparten. Nadie puede corromper la estructura del árbol desde afuera, porque no puede alcanzarla.

19.10 Comparadores

El comparador predeterminado usa `localeCompare`, así que resulta apropiado para textos:

```

const arbol = Arbol();

arbol.insertar("pera");
arbol.insertar("banana");
arbol.insertar("manzana");

console.log(arbol.buscar("banana")); // true

arbol.recorrer(console.log);
// banana, manzana, pera

```

Para números conviene proporcionar un comparador propio, porque compararlos como texto produce un orden incorrecto, por las razones de la sección 6.10:

```

const numeros = Arbol((a, b) => a - b);

numeros.insertar(8);
numeros.insertar(3);
numeros.insertar(10);

numeros.recorrer(console.log); // 3, 8, 10

```

Un comparador devuelve un número menor que `0` cuando `a` va antes que `b`, `0` cuando son equivalentes y un número mayor que `0` cuando `a` va después. Es el mismo contrato que usa `sort()`.

Recibir el comparador como parámetro convierte a `Arbol` en una estructura genérica: sirve para cualquier tipo de dato, siempre que sepamos ordenarlo.

```
const alumnos = Arbol((a, b) => a.legajo - b.legajo);  
  
alumnos.insertar({ legajo: 12345, nombre: "Ana" });  
alumnos.insertar({ legajo: 10001, nombre: "Luis" });
```

19.11 Síntesis conceptual

Una función recursiva necesita un caso base que termine la cadena y un caso recursivo que reduzca el problema. La forma de escribirla no es seguir todas las llamadas mentalmente, sino confiar en que la llamada recursiva resuelve correctamente su parte y decidir solo qué hacer con el resultado.

Casi cualquier recursión puede escribirse como un bucle, y para problemas lineales el bucle suele ser preferible. La recursión gana cuando la estructura de los datos es recursiva: árboles, directorios, JSON anidado, expresiones. En esos casos la alternativa iterativa exige mantener a mano una pila que la recursión provee sola.

El árbol binario de búsqueda ilustra la correspondencia con toda claridad. Un árbol es un nodo con dos árboles adentro, así que insertar, buscar y recorrer se escriben en tres o cuatro líneas cada uno. El caso base es siempre el árbol vacío.

Y el recorrido inorden muestra algo elegante: los valores salen ordenados sin que ninguna función ordene nada. El orden ya estaba en la estructura.

Finalmente, encapsular el árbol con una clausura protege su raíz sin necesidad de clases ni campos privados, y recibir el comparador como parámetro lo vuelve utilizable con cualquier tipo de dato.

Con esto cerramos las funciones. La parte siguiente aplica todo lo construido a dos problemas concretos: describir patrones de texto y trabajar con archivos.

Parte V — Texto y archivos

Capítulo 20. Expresiones regulares

Una **expresión regular** —*regular expression*, o **regex**— es una forma compacta de describir **patrones de texto**.

La idea fundamental es esta:

En lugar de preguntar «¿este texto es exactamente "gato"?», podemos preguntar «¿este texto tiene la forma que estoy buscando?».

Por ejemplo, supongamos que queremos reconocer cualquiera de estos textos:

```
gato  gata  gatos  gatas
```

Podríamos escribir muchas comparaciones:

```
texto === "gato" ||  
texto === "gata" ||  
texto === "gatos" ||  
texto === "gatas"
```

O podemos describir el patrón:

```
/gat[oa]s?/
```

Eso ya es una expresión regular.

Dicho de otro modo, una regex describe un **conjunto de cadenas posibles**. El patrón `\d` no describe un texto concreto: describe cualquiera de los diez dígitos.

20.1 El problema que intentan resolver

JavaScript ya sabe buscar texto:

```
const texto = "El gato duerme";  
  
texto.includes("gato"); // true  
texto.indexOf("gato"); // 3
```

Esto funciona perfectamente cuando conocemos **exactamente** lo que buscamos.

El problema aparece cuando queremos buscar una **forma**, no una cadena concreta:

```
381-555-1234
381-421-8877
351-555-9999
```

Queremos reconocer:

```
tres dígitos
un guion
tres dígitos
un guion
cuatro dígitos
```

Sin regex, tendríamos que recorrer carácter por carácter:

```
function contieneNumero(texto) {
  for (const caracter of texto) {
    if (caracter >= "0" && caracter <= "9") {
      return true;
    }
  }

  return false;
}
```

Funciona, pero expresa un concepto simple de una manera muy mecánica. Con una expresión regular podemos decir «busca un dígito»:

```
/\d/.test("El alumno tiene 27 años"); // true
```

La expresión regular describe **qué buscamos** y no **cómo recorrer el texto para encontrarlo**.

20.2 Escribir y probar una regex

En JavaScript se escribe entre barras:

```
const patron = /gato/;
```

Y se prueba con `test()`, que devuelve un booleano:

```
patron.test("Tengo un gato"); // true
patron.test("Tengo un perro"); // false
```

`test()` responde una sola pregunta:

¿Existe alguna parte del texto que coincida con este patrón?

Conceptualmente:

```
texto
  |
  ▼
¿contiene algo que coincida con el patrón?
  |
  ├── sí → true
  └── no → false
```

Por defecto, las expresiones regulares distinguen mayúsculas de minúsculas:

```
/Ana/.test("Me llamo Ana"); // true
/ana/.test("Me llamo Ana"); // false
```

20.3 Caracteres literales

La regex `/gato/` significa:

```
g seguido de a seguido de t seguido de o
```

Por eso encuentra `"un gato negro"` pero no `"una gata negra"`.

Hasta aquí no ganamos mucho respecto de `texto.includes("gato")`. La potencia comienza cuando algunos caracteres dejan de significar literalmente lo que representan.

20.4 Los metacaracteres

En una expresión regular existen caracteres especiales llamados **metacaracteres**:

```
.    cualquier carácter
[]   conjunto de caracteres
^    comienzo, o negación dentro de []
$    final
*    cero o más
+    uno o más
?    cero o uno
{}   cantidad
()   grupo
|    alternativa
\    escape
```

Vamos a construirlos uno por uno.

El punto: cualquier carácter

La expresión `/g.to/` significa:

```
g
cualquier carácter
t
o
```

Coincide con `gato`, `geto`, `gito`, `glto`, `g-to`. Pero no con `gto`, porque el punto representa **exactamente un** carácter, no cero.

```
/g.to/.test("gato"); // true
/g.to/.test("glto"); // true
/g.to/.test("gto");  // false
```

El punto tampoco coincide con un salto de línea, salvo que usemos la bandera `s` que veremos en la sección 20.10.

Conjuntos con corchetes

Muchas veces no queremos «cualquier carácter», sino uno dentro de ciertas posibilidades:

```
/gat[oa]/
```

significa `gat` seguido de `o` **o** `a`. Reconoce `gato` y `gata`.

```
/gat[oa]/.test("gata"); // true
/[abc]/.test("mesa");   // true, por la "a"
```

Dentro de los corchetes, los metacaracteres pierden su significado especial: `[.]` es un punto literal.

Rangos

Dentro de `[]` podemos especificar rangos:

```
/[0-9]/ // un dígito
/[a-z]/ // una letra minúscula
/[A-Z]/ // una letra mayúscula
```

Y combinarlos:

```
/[a-zA-Z0-9]/
```

que significa una letra minúscula, o una mayúscula, o un número.

Negar un conjunto

Dentro de `[]`, el símbolo `^` **al comienzo** invierte el conjunto:

```
/[^0-9]/
```

significa cualquier carácter que **no** sea un dígito.

```
/[^0-9]/.test("12345"); // false  
/[^0-9]/.test("123A5"); // true, por la "A"
```

Este `^` no tiene nada que ver con el que marca el comienzo del texto. La posición decide cuál es: dentro de corchetes y al principio, niega; fuera, ancla.

Clases abreviadas

Como ciertos conjuntos son extremadamente frecuentes, tienen abreviaturas:

Abreviatura	Equivale aproximadamente a	Significado
<code>\d</code>	<code>[0-9]</code>	un dígito
<code>\D</code>	<code>[^0-9]</code>	cualquier cosa menos un dígito
<code>\w</code>	<code>[A-Za-z0-9_]</code>	carácter de palabra
<code>\W</code>	<code>[^A-Za-z0-9_]</code>	lo contrario
<code>\s</code>	espacio, tabulación, salto de línea...	espacio en blanco
<code>\S</code>	lo contrario	no espacio

Conviene detenerse en `\w`. Su definición tradicional incluye **solo letras inglesas**, dígitos y el guion bajo. No incluye `ñ`, ni vocales acentuadas, ni alfabetos no latinos:

```
/^\w+$/ .test("Jose"); // true  
/^\w+$/ .test("José"); // false
```

Para texto en español eso es una limitación seria. La solución está en la sección 21.16, con las propiedades Unicode.

20.5 Cuantificadores: cuántas veces

Hasta ahora `/\d/` significa «un dígito». Pero un número como `12345` tiene muchos. Necesitamos poder decir «repetí este patrón cierta cantidad de veces».

+: una o más veces

```
/\d+/
```

Encuentra `1`, `42`, `123`, `987654321`.

```
"Tengo 57 años".match(/\d+/);  
// ["57", ...]
```

***: cero o más veces**

```
/a*/
```

Reconoce "", "a", "aa", "aaa". La diferencia con + es importante:

a+	una o más
a*	cero o más

Por eso * puede coincidir incluso sin consumir ningún carácter, lo cual a veces produce resultados inesperados.

?: cero o una vez

```
/a?/
```

La a puede aparecer o no. Permite expresar elementos opcionales:

```
/https?/ // coincide con "http" y con "https"  
/gatos?/ // coincide con "gato" y con "gatos"
```

Hay que leer con cuidado el alcance del cuantificador. En /gatos?/, el ? afecta **solamente a la s**, no a toda la palabra. Por eso no reconoce gata. Para eso combinamos:

```
/gat[oa]s?/ // gato, gata, gatos, gatas
```

Cantidades exactas

```
/\d{3}/ // exactamente tres dígitos  
/\d{2,4}/ // entre dos y cuatro  
/\d{2,}/ // al menos dos
```

Ahora podemos describir el teléfono del comienzo:

```
const telefono = /\d{3}-\d{3}-\d{4}/;  
  
telefono.test("381-555-1234"); // true
```

20.6 Anclajes: buscar frente a validar

Consideremos:

```
/\d{3}/.test("abc123xyz"); // true
/\d{3}/.test("123456");    // true
```

Las dos dan `true`, aunque quizás quisiéramos comprobar que **todo el texto** fuera exactamente tres dígitos. Para eso existen los anclajes.

```
/^Hola/    // "Hola" debe aparecer al comienzo
/mundo$/   // "mundo" debe estar al final
```

```
/^Hola/.test("Hola mundo"); // true
/^Hola/.test("Te digo Hola"); // false

/mundo$/.test("Hola mundo"); // true
/mundo$/.test("mundo cruel"); // false
```

Combinando ambos exigimos que el patrón cubra el texto completo:

```
/^d{3}$/.test("123");    // true
/^d{3}$/.test("1234");   // false
/^d{3}$/.test("abc123"); // false
```

Esta diferencia es fundamental y vale la pena fijarla:

```
/\d{3}/      → busca tres dígitos en alguna parte
/^d{3}$/     → exige que todo el texto sean tres dígitos
```

La primera forma sirve para **buscar**; la segunda, para **validar**.

Un ejemplo de validación:

```
function legajoValido(legajo) {
  return /^d{6}$/.test(legajo);
}

legajoValido("123456"); // true
legajoValido("12345");  // false
legajoValido("A12345"); // false
```

20.7 Agrupar y alternar

Grupos con paréntesis

Supongamos que queremos reconocer `ja`, `jaja`, `jajaja`. Podemos agrupar:

```
/(ja)+/
```

Ahora el cuantificador `+` afecta al grupo completo, no solo a la `a`:


```
 /^(ja)+$/ .test("jajaja"); // true
```

Compárese:

```
 /(ab)+/    // ab, abab, ababab  
 /ab+/      // ab, abb, abbb
```

Alternativas con |

El símbolo | significa «o»:

```
 /gato|perro/ .test("Tengo un perro"); // true
```

Con grupos se vuelve mucho más útil:

```
 /(gato|perro)s?/    // gato, gatos, perro, perros
```

Sin los paréntesis, `/gato|perros?/` significaría «gato» o «perros con s opcional», que es otra cosa. La alternativa tiene la precedencia más baja de todas, así que casi siempre conviene agruparla.

Escapar caracteres especiales

Si queremos buscar un metacarácter literalmente, hay que escaparlos con \:

```
 /\./.test("Hola."); // true
```

Por ejemplo, para buscar 3.14:

```
 /3\.14/    // correcto  
 /3.14/     // también coincidiría con "3X14", "3-14", "3a14"
```

El \ dice: interpreta el siguiente carácter literalmente.

Los caracteres que suelen necesitar escape:

```
. * + ? ^ $ ( ) [ ] { } | \ /
```

20.8 Las regex son objetos

Cuando escribimos:

```
const patron = /\d+/;
```

estamos creando un objeto:

```
typeof patron;           // "object"
patron instanceof RegExp; // true
```

También podemos construirlo explícitamente:

```
const patron = new RegExp("\\d+");
```

Esto equivale aproximadamente a `/\d+/`, con una molestia: dentro de una cadena de JavaScript también hay que escapar la barra invertida. Por eso `\d` se convierte en `"\\d"`.

Cuando el patrón se conoce de antemano, el literal `/.../` es más cómodo y más legible. `new RegExp()` es útil cuando construimos el patrón **dinámicamente**:

```
const palabra = "gato";
const regex = new RegExp(palabra, "i");

regex.test("Un GATO"); // true
```

Una advertencia de seguridad: si el texto viene de un usuario, puede contener metacaracteres que cambien el significado del patrón, o construir una expresión patológicamente lenta. En ese caso hay que escapar la entrada antes de insertarla.

20.9 Banderas

Después de la barra final pueden colocarse opciones llamadas **banderas**:

```
i  ignorar mayúsculas y minúsculas
g  global: todas las coincidencias
m  multilínea: ^ y $ operan por línea
s  dotAll: el punto también coincide con salto de línea
u  semántica Unicode
y  sticky: coincide solo desde la posición actual
d  incluye los índices de cada grupo capturado
```

Las dos más usadas:

```
/gato/i.test("GATO"); // true
```

```
"1 gato, 2 perros y 3 peces".match(/\d/);
// ["1", ...] - solo la primera

"1 gato, 2 perros y 3 peces".match(/\d/g);
// ["1", "2", "3"] - todas
```

20.10 Los métodos principales

Las regex aparecen mediante métodos de `RegExp` y de `String`:

<code>regex.test(texto)</code>	¿hay coincidencia?
<code>texto.match(regex)</code>	encontrar
<code>texto.matchAll(regex)</code>	iterar todas, con grupos
<code>texto.search(regex)</code>	posición de la primera
<code>texto.replace(regex, reemplazo)</code>	reemplazar
<code>texto.replaceAll(regex, reemplazo)</code>	reemplazar todas
<code>texto.split(regex)</code>	separar

`match()`

```
"Tengo 42 años".match(/\d+/);  
// ["42", index: 6, input: "Tengo 42 años", groups: undefined]
```

Sin la bandera `g`, devuelve un objeto con la coincidencia y sus grupos. Con `g`, devuelve un array simple de todas las coincidencias:

```
"Compré 3 panes, 2 leches y 10 huevos".match(/\d+/g);  
// ["3", "2", "10"]
```

Y si no encuentra nada, devuelve `null`, no un array vacío. Eso obliga a comprobarlo:

```
const numeros = texto.match(/\d+/g) ?? [];
```

`matchAll()`

Devuelve un iterador con toda la información de cada coincidencia, incluidos los grupos. Requiere la bandera `g`:

```
const texto = "Ana: 20, Juan: 30, Pedro: 40";  
  
for (const resultado of texto.matchAll(/(\w+):\s*(\d+)/g)) {  
  console.log(resultado[1], resultado[2]);  
}  
// Ana 20  
// Juan 30  
// Pedro 40
```

`replace()` y `replaceAll()`

Las regex son extremadamente útiles para transformar texto:

```
"Juan    Pérez".replace(/\s+/g, " ");  
// "Juan Pérez"  
  
"+54 9 (381) 555-1234".replace(/\D/g, "");  
// "5493815551234"
```

La segunda expresión elimina todo lo que no sea un dígito.

`split()`

```
"Juan , Pedro, Ana".split(/\s*,\s*/);  
// ["Juan", "Pedro", "Ana"]
```

El patrón `\s*,\s*` significa: cero o más espacios, una coma, cero o más espacios. Absorbe la variación de formato.

```
"Ana,Juan;Pedro María".split(/[,\s]+/);  
// ["Ana", "Juan", "Pedro", "María"]
```

20.11 Capturas

Los paréntesis no solamente agrupan. También **capturan** la parte del texto que coincidió.

Supongamos una fecha `24/08/2026`:

```
const resultado = "24/08/2026".match(/(\d{2})\/(\d{2})\/(\d{4})/);  
  
console.log(resultado[0]); // "24/08/2026" – coincidencia completa  
console.log(resultado[1]); // "24" – primer grupo  
console.log(resultado[2]); // "08" – segundo grupo  
console.log(resultado[3]); // "2026" – tercer grupo
```

```
resultado[0] = coincidencia completa  
resultado[1] = primer grupo  
resultado[2] = segundo grupo  
...
```

Grupos con nombre

Trabajar con posiciones numéricas es frágil: agregar un grupo al principio desplaza todos los índices. JavaScript moderno permite nombrarlos:

```
const regex = /(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})/;  
  
const resultado = "24/08/2026".match(regex);  
  
console.log(resultado.groups.dia); // "24"  
console.log(resultado.groups.mes); // "08"  
console.log(resultado.groups.anio); // "2026"
```

Es mucho más legible, y además permite desestructurar:

```
const { dia, mes, anio } = resultado.groups;
```

Grupos sin captura

Si queremos agrupar pero no guardar el resultado, usamos `(?:...)`:

```
/https?:\/\/(?:www\.)?/
```

`www.` queda agrupado para poder hacerlo opcional, pero no ocupa una posición entre las capturas.

Referencias en el reemplazo

Dentro del texto de reemplazo, `$1`, `$2` y demás se refieren a los grupos capturados:

```
const resultado = "15/08/2026".replace(  
  /(\d{2})\/(\d{2})\/(\d{4})/,  
  "$3-$2-$1"  
);  
  
console.log(resultado); // "2026-08-15"
```

Con grupos con nombre se usa `$<nombre>`:

```
"15/08/2026".replace(  
  /(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})/,  
  "$<anio>-$<mes>-$<dia>"  
);
```

Retroreferencias

Dentro del patrón, `\1` significa «lo mismo que capturó el grupo 1». Sirve para detectar repeticiones:

```
const texto = "Esto es muy muy importante";  
const patron = /\b(\w+)\s+\1\b/i;  
  
console.log(patron.test(texto)); // true
```

Un ejemplo completo

```
const texto = "Alumno: Juan Pérez - Legajo: 12345";  
  
const resultado = texto.match(/Legajo:\s*(\d+)/);  
  
console.log(resultado[1]); // "12345"
```

Descompuesto:

Legajo:	texto literal
<code>\s*</code>	cero o más espacios
<code>(\d+)</code>	uno o más dígitos, capturados

Esto muestra el uso conceptualmente más importante de las regex:

convertir texto aparentemente desestructurado en datos estructurados.

20.12 Límites de palabra

Consideremos `/gato/`. También encuentra `gato` dentro de `gatopardo`, porque ahí aparecen esos cuatro caracteres.

Si queremos la palabra completa, usamos `\b`:

```
/\bgato\b/.test("El gato duerme");    // true
/\bgato\b/.test("El gatopardo duerme"); // false
```

`\b` no consume ningún carácter: marca una **posición** donde termina o empieza una palabra, es decir, la frontera entre un carácter de `\w` y uno que no lo es.

```
/\bpan\b/.test("compro pan"); // true
/\bpan\b/.test("pantalla");   // false
```

Como `\b` se define en términos de `\w`, hereda su limitación con los acentos.

20.13 Codicia

Los cuantificadores son **codiciosos** por defecto: intentan consumir todo lo posible.

```
const html = "<b>uno</b><b>dos</b>";

html.match(/<b>.*</b>/)[0];
// "<b>uno</b><b>dos</b>"
```

`.*` consumió hasta el último `</b>` que encontró, no el primero.

Agregando `?` después del cuantificador obtenemos la versión **no codiciosa**, que consume lo mínimo posible:

```
html.match(/<b>.*?</b>/)[0];
// "<b>uno</b>"
```

Conceptualmente:

```
.*    todo lo posible
.*?   lo mínimo posible
```

También existen `+`, `??` y `{n,m}?`.

20.14 Mirar adelante y atrás

A veces queremos exigir que algo esté presente sin incluirlo en la coincidencia.

Lookahead positivo `(?=...)` :

```
/\d+(?=€)/
```

significa: uno o más dígitos **seguidos de** €, sin incluir el €. En "Precio: 100€" encuentra "100".

Lookbehind positivo `(?<=...)` :

```
/(?<=\$)\d+/
```

significa: uno o más dígitos **precedidos por** \$. En "Precio: \$150" encuentra "150".

También existen las versiones negativas: `(?!...)` exige que **no** siga cierto patrón, y `(?<!=...)` que no lo preceda.

```
/\d+(?!%)/ // números que no van seguidos de un porcentaje
```

Estas construcciones no consumen caracteres: son condiciones sobre el contexto.

20.15 Unicode

Una regex como:

```
/^[A-Za-z]+$
```

no contempla adecuadamente nombres como José, María, Ángel o Núñez.

La bandera `u` activa la semántica Unicode y habilita las **propiedades Unicode**:

```
const patron = /\p{L}+$/u;

patron.test("José"); // true
patron.test("Ñandú"); // true
```

`\p{L}` significa cualquier carácter Unicode cuya propiedad sea «letra». Es una definición mucho más apropiada para texto internacional que `[A-Za-z]`.

Otras propiedades útiles:

```
\p{L}    letra
\p{N}    número
\p{Lu}    letra mayúscula
\p{Ll}    letra minúscula
\p{P}    puntuación
```

Combinándolas con `g`:

```
"José, María y Ñandú".match(/\p{L}+/gu);
// ["José", "María", "y", "Ñandú"]
```

La bandera `u` también hace que el punto y los cuantificadores traten correctamente los caracteres representados mediante pares sustitutos, el problema que vimos en la sección 6.6:

```
/^.$/.test("😊"); // false – sin u, ve dos unidades
/^.$/u.test("😊"); // true  – con u, ve un punto de código
```

20.16 Construir una regex en lugar de adivinarla

Este es probablemente el consejo más útil del capítulo. No hay que intentar escribir la expresión completa de memoria. Hay que **partir de la especificación y construirla por partes**.

Supongamos que necesitamos validar un código con esta forma:

```
ABC-1234
```

Tres letras mayúsculas:

```
[A-Z]{3}
```

Después un guion:

```
[A-Z]{3}-
```

Después cuatro números:

```
[A-Z]{3}-\d{4}
```

Y queremos validar el texto completo:

```
 /^[A-Z]{3}-\d{4}$/
```

Probamos:


```
const codigo = /^[A-Z]{3}-\d{4}$/;

codigo.test("ABC-1234"); // true
codigo.test("AB-1234");  // false
codigo.test("abc-1234"); // false
```

La manera correcta de trabajar es:

```
problema → componentes → regex
```

No al revés.

Y a la inversa, para **leer** una regex ajena conviene traducir cada fragmento:

```
/^[A-Z][a-z]+$/
```

^	comienzo
[A-Z]	una mayúscula
[a-z]+	una o más minúsculas
\$	final

Describe textos como Juan, Pedro, Maria, pero no juan, JUAN ni Juan123.

20.17 Algunos casos reales

Una hora válida

Una regex ingenua:

```
/^\d{2}:\d{2}$/
```

aceptaría 99:99. Una mejor versión:

```
/^(?:[01]\d|2[0-3]):[0-5]\d$/
```

Descompuesta:

^	
(?:[01]\d 2[0-3])	hora entre 00 y 23
:	dos puntos
[0-5]\d	minutos entre 00 y 59
\$	

El grupo sin captura permite alternar entre las dos formas de hora sin agregar una captura innecesaria.

Extraer estructura de líneas

Supongamos líneas como:

```
Ana Perez;123456;3815551234
Juan Gomez;654321;3814449876
```

Con grupos con nombre:

```
const patron = /^(?<nombre>.+);(?<legajo>\d{6});(?<telefono>\d+)$/;

const resultado = "Ana Perez;123456;3815551234".match(patron);

console.log(resultado.groups);
// { nombre: "Ana Perez", legajo: "123456", telefono: "3815551234" }
```

La regex no solamente valida: **extrae estructura de un texto plano**. El capítulo 22 usa exactamente esta técnica para procesar registros de eventos.

Correo electrónico

Una validación sencilla:

```
/^[^\s@]+@[^\s@]+\.[^\s@]+$/
```

Sirve para detectar errores evidentes. Pero **no conviene** intentar reproducir toda la especificación formal del correo electrónico mediante una regex: es enormemente compleja y las versiones que circulan suelen rechazar direcciones válidas.

La validación definitiva de un correo consiste en enviarle un mensaje de confirmación.

20.18 Regex no reemplaza a un parser

Las expresiones regulares son excelentes para estructuras relativamente simples:

```
números, fechas, códigos, líneas, identificadores,
fragmentos de texto, separadores, normalización
```

No son la herramienta adecuada para analizar estructuras complejas y anidadas:

```
HTML completo, JavaScript, JSON, SQL, lenguajes de programación
```

La razón es profunda y la veremos en la sección siguiente: una expresión regular, en su forma matemática, no puede contar niveles de anidamiento.

Para JSON:

```
JSON.parse(texto);
```

es muchísimo mejor que intentar construir una regex monstruosa.

La regla práctica: si el problema consiste principalmente en reconocer secuencias de caracteres, una regex puede ser excelente. Si hay una estructura jerárquica, probablemente necesitemos un analizador sintáctico.

Cuando una expresión regular empieza a parecer un conjuro, suele ser el momento de cambiar de herramienta.

20.19 La idea teórica

En términos conceptuales, una regex describe un **lenguaje**: un conjunto de cadenas.

Por ejemplo, `/\d{3}/` describe el conjunto:

```
000, 001, 002, ..., 998, 999
```

No enumera esas mil posibilidades. Describe una **regla generadora** para todas ellas:

```
L = { todas las cadenas formadas por exactamente tres dígitos }
```

La regex es una representación compacta de ese conjunto.

Regex y autómatas

Esta idea conecta directamente con la teoría de la computación y con el álgebra de Boole del apéndice A.

Una expresión regular puede convertirse en una máquina que lee caracteres uno por uno y cambia de estado. Para `/ab+c/`:

```
inicio
|
a
▼
estado 1
|
b
▼
estado 2
|\
| b
|/
|
c
▼
aceptado
```

Reconoce `abc`, `abbc`, `abbbc`, pero no `ac`, `ab` ni `bac`.

Esa máquina es un **autómata finito**. Se llama finito porque tiene una cantidad fija de estados y ninguna memoria adicional. De ahí viene el nombre «expresiones regulares»: describen exactamente los **lenguajes regulares**, que son los que un autómata finito puede reconocer.

Y de ahí viene también la limitación de la sección anterior. Para verificar que los paréntesis de una expresión estén balanceados hay que contar cuántos quedan abiertos, y una cantidad fija de estados no alcanza para contar sin límite. Por eso HTML y JSON quedan fuera del alcance.

Los motores modernos de regex agregaron características que van más allá de las expresiones regulares matemáticas puras —las retrorreferencias, por ejemplo—, pero la intuición sigue siendo excelente.

20.20 Síntesis conceptual

Una regex no conviene aprenderla como una colección de jeroglíficos. Conviene pensarla como un pequeño lenguaje con cinco clases de piezas:

qué carácter quiero	<code>.</code> <code>[abc]</code> <code>\d</code> <code>\s</code>
+	
cuántas veces	<code>*</code> <code>+</code> <code>?</code> <code>{3}</code> <code>{2,5}</code>
+	
en qué orden	<code>abc</code>
+	
entre qué alternativas	<code>a b</code>
+	
en qué posición	<code>^</code> <code>\$</code> <code>\b</code>

Con esas piezas se construye la mayoría de las expresiones útiles. Y con los grupos —capturantes, con nombre o sin captura— se extrae la información que interesa.

La distinción entre buscar y validar es la que más errores evita: `/\d{3}/` busca en cualquier parte, `/^\d{3}$/` exige que el texto entero coincida.

Las banderas modifican el modo de trabajo: `i` ignora mayúsculas, `g` busca todas las coincidencias, `u` habilita la semántica Unicode y las propiedades como `\p{L}`, indispensables para texto en español.

Los cuantificadores son codiciosos por defecto; agregarles `?` los vuelve mínimos.

La forma de trabajar es siempre la misma: partir de la especificación en palabras, traducirla componente por componente y probarla. La habilidad importante no es memorizar una expresión monstruosa, sino poder mirar una necesidad como «tres letras, después un guion, después cuatro números» y construir tranquilamente:

```
/^[A-Z]{3}-\d{4}$/
```

Una regex no es más que eso: una descripción compacta de qué secuencias de caracteres consideramos válidas o interesantes.

El apéndice C reúne la sintaxis en una tabla de consulta rápida.

El capítulo siguiente aplica casi todo lo visto a un problema concreto: leer, escribir y transformar archivos.

Capítulo 21. Gestión de archivos con Node.js

Un archivo parece algo sencillo: tiene un nombre, lo abrimos, guardamos información y lo cerramos. Sin embargo, para programar correctamente con archivos conviene construir primero un modelo mental preciso, porque casi todos los errores frecuentes provienen de confundir capas que en realidad están separadas.

Los ejemplos se ejecutan con Node.js y, salvo indicación en contrario, usan la API asíncrona basada en promesas.

21.1 Un archivo contiene bytes

Una computadora almacena información como secuencias de bits, agrupados de a ocho. Cada grupo forma un **byte**:

```
8 bits = 1 byte
```

Un archivo puede pensarse, en una primera aproximación, como una secuencia ordenada de bytes guardada en un dispositivo:

```
archivo
├─ byte 0
├─ byte 1
├─ byte 2
└─ ...
```

Los bytes, por sí solos, no dicen qué representan. La misma secuencia podría interpretarse como texto, una imagen, audio, un documento PDF, un programa ejecutable o datos comprimidos.

El **formato** establece cómo interpretar esos bytes. Es exactamente la idea de codificación de la sección A.2: los bits no tienen significado intrínseco; se lo damos con una convención.

La extensión no transforma el contenido

El nombre `foto.jpg` sugiere que el archivo contiene una imagen JPEG, pero la extensión es solamente parte del nombre. Si renombramos:

```
foto.jpg → foto.txt
```

los bytes internos no cambian. El archivo continúa conteniendo una imagen, aunque ahora su nombre resulte engañoso.

nombre y extensión → orientan sobre el contenido
bytes del archivo → son el contenido real

21.2 Archivo, directorio, ruta y metadatos

Un sistema de archivos organiza la información mediante varios conceptos relacionados pero distintos.

Archivo. Contiene una secuencia de bytes.

Directorio. Contiene entradas que relacionan nombres con archivos u otros directorios. Coloquialmente lo llamamos carpeta.

```
curso/  
├─ alumnos.txt  
├─ notas.json  
└─ entregas/  
    └─ trabajo-01.txt
```

Ruta. Describe la ubicación de una entrada dentro del sistema de archivos:

```
curso/entregas/trabajo-01.txt
```

Metadatos. Son datos acerca del archivo: tamaño, fechas de modificación y acceso, permisos, tipo de entrada, propietario.

Un detalle que explica varias cosas: **el nombre no forma parte de los bytes almacenados dentro del archivo.** El directorio relaciona ese nombre con la entrada correspondiente. Por eso renombrar un archivo normalmente no implica reescribir todo su contenido, y por eso mover un archivo dentro del mismo disco es instantáneo aunque pese gigabytes.

21.3 Node.js como intermediario

JavaScript, por sí solo, no define una operación para abrir un archivo del disco. Es la separación entre lenguaje y entorno que vimos en la sección 1.5. Node.js ofrece módulos que hablan con el sistema operativo:

node:fs	operaciones con archivos y directorios
node:fs/promises	las mismas ideas usando promesas
node:path	construcción y análisis de rutas
node:readline	lectura de texto línea por línea
node:buffer	manipulación explícita de bytes

Usaremos especialmente:

```
import * as fs from "node:fs/promises";
import path from "node:path";
```

El prefijo `node:` deja claro que estamos importando un módulo incorporado y no un paquete descargado.

Tres estilos de API

Muchas operaciones del módulo `fs` tienen tres variantes.

Asíncrona con promesas, que será nuestro estilo principal:

```
import { readFile } from "node:fs/promises";

const texto = await readFile("mensaje.txt", "utf8");
```

No bloquea el hilo de JavaScript mientras Node.js espera la operación, y se combina bien con `async` y `await`.

Asíncrona con callback, común en código antiguo:

```
import { readFile } from "node:fs";

readFile("mensaje.txt", "utf8", (error, texto) => {
  if (error) {
    console.error(error);
    return;
  }

  console.log(texto);
});
```

Síncrona, que detiene la ejecución hasta terminar:

```
import { readFileSync } from "node:fs";

const texto = readFileSync("mensaje.txt", "utf8");
```

Puede ser razonable en un script muy pequeño, durante el arranque de un programa o en una herramienta que hace una sola tarea. En un servidor puede impedir que se atiendan otras solicitudes mientras espera al disco.

21.4 Preparar un proyecto mínimo

Creamos un directorio con un archivo `package.json`:


```
{
  "type": "module"
}
```

La propiedad `"type": "module"` permite usar `import` y `export` en archivos con extensión `.js`.

Ahora podemos crear `inicio.js`:

```
import { writeFile } from "node:fs/promises";

await writeFile("saludo.txt", "Hola desde Node.js\n", "utf8");

console.log("Archivo creado");
```

y ejecutarlo:

```
node inicio.js
```

También podríamos usar la extensión `.mjs` sin crear el `package.json`.

21.5 Rutas relativas y absolutas

Una operación necesita saber **dónde** está el archivo.

Ruta relativa

```
datos/alumnos.txt
```

Se interpreta a partir del directorio de trabajo actual del proceso, que podemos consultar:

```
console.log(process.cwd());
```

`cwd` significa *current working directory*. Un punto importante: si ejecutamos el mismo programa desde dos directorios diferentes, una misma ruta relativa puede señalar lugares diferentes. **La ruta relativa no parte del archivo JavaScript, sino del lugar donde se lanzó el proceso.**

Ruta absoluta

Describe una ubicación desde la raíz del sistema:

```
/home/ana/proyecto/datos/alumnos.txt
```

En Windows:

```
C:\Users\Ana\proyecto\datos\alumnos.txt
```

Construir rutas con `node:path`

No conviene concatenar separadores manualmente:

```
// frágil
const ruta = directorio + "/" + nombre;
```

Es preferible:

```
import path from "node:path";

const ruta = path.join("datos", "alumnos", "ana.txt");
```

`path.join()` usa las convenciones del sistema operativo y normaliza separadores, así que el mismo código funciona en Linux, macOS y Windows.

Para obtener una ruta absoluta:

```
const rutaAbsoluta = path.resolve("datos", "alumnos.txt");
```

`path.resolve()` parte del directorio de trabajo, salvo que encuentre antes una ruta absoluta.

Y para analizar una ruta:

```
const ruta = "/curso/datos/alumnos.csv";

path.basename(ruta); // "alumnos.csv"
path.extname(ruta);  // ".csv"
path.dirname(ruta);  // "/curso/datos"
path.parse(ruta);
```

`path.parse()` devuelve un objeto:

```
{
  root: "/",
  dir: "/curso/datos",
  base: "alumnos.csv",
  ext: ".csv",
  name: "alumnos"
}
```

21.6 Crear directorios

```
import { mkdir } from "node:fs/promises";

await mkdir("datos");
```

Esto falla si `datos` ya existe o si falta algún directorio intermedio. Para crear toda la cadena necesaria sin fallar si ya existe:

```
await mkdir("datos/2026/agosto", { recursive: true });
```

```
mkdir("a/b/c", { recursive: true })
  |
  ├── crea a si falta
  ├── crea b si falta
  └── crea c si falta
```

La opción `recursive` es la que se usa casi siempre, porque expresa la intención habitual: «asegurate de que este directorio exista».

21.7 Escribir un archivo de texto

```
import { writeFile } from "node:fs/promises";

await writeFile(
  "mensaje.txt",
  "Primera línea\nSegunda línea\n",
  "utf8"
);
```

Si `mensaje.txt` no existe, se crea. **Si ya existe, su contenido se reemplaza por completo.** No hay advertencia ni confirmación.

Evitar sobrescribir

La opción `flag: "wx"` significa: abrir para escritura, pero fallar si el archivo ya existe.

```
await writeFile(
  "configuracion.txt",
  "modo=desarrollo\n",
  { encoding: "utf8", flag: "wx" }
);
```

Las banderas más habituales al escribir:

Bandera	Significado práctico
"w"	Escribe desde cero; crea o reemplaza
"wx"	Escribe desde cero; falla si ya existe
"a"	Agrega al final; crea si no existe
"ax"	Agrega; falla si ya existe

`writeFile` usa "w" por defecto.

Agregar sin borrar lo anterior

```
import { appendFile } from "node:fs/promises";

await appendFile("eventos.log", "Inicio\n", "utf8");
await appendFile("eventos.log", "Alumno creado\n", "utf8");
await appendFile("eventos.log", "Fin\n", "utf8");
```

```
Inicio
Alumno creado
Fin
```

La diferencia esencial:

```
writeFile → reemplaza el contenido
appendFile → conserva lo anterior y agrega al final
```

21.8 Leer un archivo completo

```
import { readFile } from "node:fs/promises";

const contenido = await readFile("mensaje.txt", "utf8");

console.log(contenido);
```

Al indicar "utf8", pedimos que los bytes se **decodifiquen** y el resultado sea un `string`.

Si omitimos la codificación:

```
const contenido = await readFile("mensaje.txt");

console.log(contenido);
// <Buffer 48 6f 6c 61 ...>
```

obtenemos un `Buffer`, es decir, una representación de los bytes crudos. Podemos decodificarlo después:

```
const bytes = await readFile("mensaje.txt");
const texto = bytes.toString("utf8");
```

Esta distinción es fundamental:

```
readFile(ruta)           → Buffer, bytes
readFile(ruta, "utf8") → string, texto decodificado
```

21.9 Texto, Unicode y codificación

El capítulo 6 desarrolló Unicode desde el punto de vista de las cadenas en memoria. Acá lo retomamos desde el otro extremo: los bytes que van al disco.

Un archivo no contiene «letras»: contiene bytes. Para guardar texto necesitamos una regla que relacione caracteres y bytes. Esa regla es una **codificación de caracteres**.

Unicode asigna un **punto de código** a cada carácter:

```
A    → U+0041
ñ    → U+00F1
€    → U+20AC
😊    → U+1F600
```

Unicode define qué número representa cada carácter. Una codificación como UTF-8 define cómo expresar esos números mediante bytes.

UTF-8

UTF-8 usa una cantidad variable de bytes:

```
carácter ASCII común    → 1 byte
muchos caracteres latinos → 2 bytes
otros caracteres        → 3 bytes
muchos emojis           → 4 bytes
```

Podemos observarlo:

```
console.log(Buffer.byteLength("A", "utf8")); // 1
console.log(Buffer.byteLength("ñ", "utf8")); // 2
console.log(Buffer.byteLength("😊", "utf8")); // 4
```

Por eso no debemos suponer que un carácter ocupa un byte. Y tampoco hay que confundir las tres mediciones posibles:

```
const ejemplo = "😊";

Buffer.byteLength(ejemplo, "utf8"); // 4 – bytes UTF-8
ejemplo.length; // 2 – unidades UTF-16
Array.from(ejemplo).length; // 1 – puntos de código
```

Como vimos en la sección 6.6, ni siquiera un punto de código equivale siempre a un símbolo percibido por una persona. Toda medición debe aclarar qué unidad cuenta.

El ciclo completo

```
string de JavaScript
  |
  | codificar como UTF-8
  ▼
bytes
  |
  | decodificar como UTF-8
  ▼
string de JavaScript
```

```
const textoOriginal = "Tucumán 😊";

const bytes = Buffer.from(textoOriginal, "utf8");
const textoRecuperado = bytes.toString("utf8");
```

Cuando la codificación no coincide

Si los bytes fueron producidos con una codificación y los interpretamos con otra, aparecen caracteres incorrectos:

```
información correcta → bytes → interpretación incorrecta → texto dañado
```

Los textos como `JosÃ©` son el síntoma clásico: bytes UTF-8 leídos como si fueran Latin-1.

Node.js reconoce, entre otras, `utf8`, `utf16le`, `latin1`, `ascii`, y también `base64`, `base64url` y `hex` para representar bytes mediante texto. **Para archivos de texto nuevos, UTF-8 es normalmente la elección apropiada.**

La marca de orden de bytes

Algunos archivos comienzan con una marca llamada **BOM**. UTF-8 no la necesita, pero ciertas herramientas la agregan. Si aparece como primer carácter al leer JSON o CSV, podemos retirarla:

```
const sinBOM = texto.replace(/^\uFEFF/, "");
```

Salto de línea

Los sistemas han usado diferentes secuencias para separar líneas:

```
Linux y macOS actuales → LF    → \n
Windows                → CRLF → \r\n
macOS clásico          → CR    → \r
```

Para dividir texto que puede venir de distintas plataformas:

```
const lineas = texto.split(/\r\n|\n|\r/);
```

Para escribir con el separador del sistema actual:

```
import { EOL } from "node:os";

const texto = ["uno", "dos", "tres"].join(EOL);
```

En formatos intercambiados entre sistemas suele ser más simple elegir `\n` de manera consistente.

21.10 Transformar y volver a guardar

La secuencia general es:

```
leer → decodificar → transformar → codificar → escribir
```

```
import { readFile, writeFile } from "node:fs/promises";

const texto = await readFile("entrada.txt", "utf8");
const transformado = texto.toLocaleUpperCase("es");

await writeFile("salida.txt", transformado, "utf8");
```

Leer y escribir el mismo archivo es válido:

```
const texto = await readFile("entrada.txt", "utf8");
const transformado = texto.trim() + "\n";

await writeFile("entrada.txt", transformado, "utf8");
```

Pero si el proceso falla durante la escritura podríamos dejar el archivo incompleto, y el contenido original ya se perdió. Cuando el dato es importante, es más seguro escribir primero un archivo temporal y luego renombrarlo:

```
import { rename, writeFile } from "node:fs/promises";

await writeFile("entrada.txt.tmp", transformado, "utf8");
await rename("entrada.txt.tmp", "entrada.txt");
```

El renombrado es la operación que se completa o no se completa, y por eso protege el archivo original. Conviene que el temporal esté en el mismo directorio que el destino, porque renombrar entre sistemas de archivos distintos no es una operación atómica.

21.11 Listar el contenido de un directorio

```
import { readdir } from "node:fs/promises";

const nombres = await readdir("datos");

for (const nombre of nombres) {
  console.log(nombre);
}
```

Para distinguir archivos de directorios usamos `withFileTypes`:

```
const entradas = await readdir("datos", { withFileTypes: true });

for (const entrada of entradas) {
  if (entrada.isDirectory()) {
    console.log(`[DIR] ${entrada.name}`);
  } else if (entrada.isFile()) {
    console.log(`[FILE] ${entrada.name}`);
  } else {
    console.log(`[OTRO] ${entrada.name}`);
  }
}
```

Cada elemento es un `Dirent` y ofrece, entre otros:

```
isFile()      ¿es un archivo común?
isDirectory() ¿es un directorio?
isSymbolicLink() ¿es un enlace simbólico?
```

Filtrar por extensión combina lo anterior con los métodos de array del capítulo 10:

```
const archivosTxt = entradas
  .filter((entrada) => entrada.isFile())
  .filter((entrada) => path.extname(entrada.name).toLowerCase() === ".txt")
  .map((entrada) => entrada.name);
```


21.12 Recorrer subdirectorios

Un directorio contiene archivos y otros directorios, que a su vez contienen archivos y directorios. Es una estructura recursiva, y por las razones del capítulo 19 la recorreremos con una función recursiva:

```
import { readdir } from "node:fs/promises";
import path from "node:path";

async function listarArchivos(directorio) {
  const resultados = [];

  const entradas = await readdir(directorio, { withFileTypes: true });

  for (const entrada of entradas) {
    const rutaCompleta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      const archivosInternos = await listarArchivos(rutaCompleta);
      resultados.push(...archivosInternos);
    } else if (entrada.isFile()) {
      resultados.push(rutaCompleta);
    }
  }

  return resultados;
}

const archivos = await listarArchivos("datos");
```

```
visitar directorio
├── si encuentra archivo → guardarlo
└── si encuentra directorio → visitar ese directorio
```

El caso base no aparece como un `if` explícito: es el directorio sin subdirectorios, donde el bucle no vuelve a llamarse.

Este ejemplo ignora los enlaces simbólicos deliberadamente: seguirlos sin cuidado podría producir ciclos infinitos o salir del árbol esperado.

21.13 Consultar información de un archivo

```
import { stat } from "node:fs/promises";

const informacion = await stat("mensaje.txt");

console.log("Tamaño:", informacion.size, "bytes");
console.log("Modificado:", informacion.mtime);
console.log("Es archivo:", informacion.isFile());
console.log("Es directorio:", informacion.isDirectory());
```

Algunas propiedades útiles:

Propiedad	Significado
size	Tamaño en bytes
mtime	Última modificación del contenido
atime	Último acceso, si el sistema lo registra
birthtime	Fecha de creación, cuando el sistema la provee

No todos los sistemas de archivos mantienen todas las fechas con la misma precisión ni la misma semántica. No conviene tratarlas como verdades universales.

Si la ruta es un enlace simbólico, `stat()` informa sobre el destino y `lstat()` sobre el enlace mismo:

```
import { lstat } from "node:fs/promises";

const informacion = await lstat("acceso-directo");
console.log(informacion.isSymbolicLink());
```

21.14 Comprobar si una ruta existe

Podemos intentar acceder:

```
import { access } from "node:fs/promises";

async function existe(ruta) {
  try {
    await access(ruta);
    return true;
  } catch {
    return false;
  }
}
```

Sin embargo, **no siempre conviene preguntar primero y operar después**:

1. comprobar que existe
2. otro proceso lo elimina
3. intentar leerlo 🌟

El estado puede cambiar entre los pasos. Es un caso clásico de condición de carrera. Cuando lo que realmente queremos es leer, suele ser mejor intentar leer y manejar el posible error:

```
import { readFile } from "node:fs/promises";

try {
  const texto = await readFile("mensaje.txt", "utf8");
  console.log(texto);
} catch (error) {
  if (error.code === "ENOENT") {
    console.log("El archivo no existe");
  } else {
    throw error;
  }
}
```

Esto aplica exactamente el patrón del capítulo 15: capturar lo que sabemos manejar y relanzar lo demás.

21.15 Buscar archivos

Por nombre

```
async function buscarPorNombre(directorio, fragmento) {
  const coincidencias = [];
  const buscado = fragmento.toLocaleLowerCase("es");

  const entradas = await readdir(directorio, { withFileTypes: true });

  for (const entrada of entradas) {
    const rutaCompleta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      const internas = await buscarPorNombre(rutaCompleta, fragmento);
      coincidencias.push(...internas);
    } else if (
      entrada.isFile() &&
      entrada.name.toLocaleLowerCase("es").includes(buscado)
    ) {
      coincidencias.push(rutaCompleta);
    }
  }

  return coincidencias;
}
```

Por contenido

Buscar por contenido requiere leer los archivos:

encontrar archivos → leerlos como texto → comparar el contenido

```
import { readFile, readdir } from "node:fs/promises";
import path from "node:path";

async function buscarEnTextos(directorio, textoBuscado) {
  const coincidencias = [];
  const buscado = textoBuscado.toLocaleLowerCase("es");

  const entradas = await readdir(directorio, { withFileTypes: true });

  for (const entrada of entradas) {
    const rutaCompleta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      const internas = await buscarEnTextos(rutaCompleta, textoBuscado);
      coincidencias.push(...internas);
      continue;
    }

    const esTexto =
      entrada.isFile() &&
      path.extname(entrada.name).toLowerCase() === ".txt";

    if (!esTexto) {
      continue;
    }

    const contenido = await readFile(rutaCompleta, "utf8");

    if (contenido.toLocaleLowerCase("es").includes(buscado)) {
      coincidencias.push(rutaCompleta);
    }
  }

  return coincidencias;
}
```

Filtramos por extensión porque intentar decodificar indiscriminadamente imágenes, ejecutables o archivos comprimidos no constituye una búsqueda de texto confiable: produciría coincidencias falsas y consumiría memoria innecesariamente.

Para informar el número y el contenido de cada línea coincidente:

```

async function buscarLineas(ruta, textoBuscado) {
  const contenido = await readFile(ruta, "utf8");
  const lineas = contenido.split(/\r\n|\n|\r/);
  const buscado = textoBuscado.toLocaleLowerCase("es");

  return lineas
    .map((texto, indice) => ({ numero: indice + 1, texto }))
    .filter((linea) =>
      linea.texto.toLocaleLowerCase("es").includes(buscado)
    );
}

```

`indice + 1` convierte el índice de un array, que empieza en cero, en un número de línea para personas, que empieza en uno.

21.16 Copiar, mover y eliminar

Copiar un archivo

```

import { copyFile } from "node:fs/promises";

await copyFile("original.txt", "copia.txt");

```

Por defecto, si el destino existe puede ser reemplazado. Para impedirlo:

```

import { constants } from "node:fs";
import { copyFile } from "node:fs/promises";

await copyFile("original.txt", "copia.txt", constants.COPYFILE_EXCL);

```

Copiar un directorio completo

```

import { cp } from "node:fs/promises";

await cp("datos", "respaldo/datos", { recursive: true });

```

Con opciones más específicas:

```

await cp("datos", "respaldo/datos", {
  recursive: true,
  force: false,
  errorOnExist: true,
  preserveTimestamps: true,
});

```

La combinación exacta depende de la política deseada: reemplazar, omitir o fallar cuando el destino ya existe.

Renombrar y mover

```
import { rename } from "node:fs/promises";

await rename("borrador.txt", "informe.txt");
```

Si cambiamos también el directorio, estamos moviendo la entrada:

```
await rename("pendientes/informe.txt", "terminados/informe.txt");
```

Renombrar y mover dentro de un mismo sistema de archivos son, conceptualmente, variaciones de la misma operación: cambiar la ruta con la que se encuentra la entrada. Por eso son instantáneas.

El directorio de destino debe existir:

```
import { mkdir, rename } from "node:fs/promises";

await mkdir("terminados", { recursive: true });
await rename("pendientes/informe.txt", "terminados/informe.txt");
```

Mover entre dispositivos o sistemas de archivos diferentes puede fallar con `EXDEV`. En ese caso hace falta una estrategia de copiar, verificar y luego eliminar el origen.

Eliminar

Eliminar es una operación destructiva. Hay que validar cuidadosamente la ruta.

```
import { unlink, rmdir, rm } from "node:fs/promises";

await unlink("temporal.txt");           // un archivo
await rmdir("directorio-vacio");        // un directorio vacío

await rm("temporales", {                 // un árbol completo
  recursive: true,
  force: true,
});
```

`recursive: true` amplía enormemente el alcance de la operación. Antes de usarlo conviene resolver e inspeccionar la ruta, y nunca aplicarlo a una ruta vacía o proveniente de una entrada no confiable.

21.17 Extraer información de un texto

Una vez leído, el contenido es un `string` y podemos usar todo lo del capítulo 6 y del capítulo 20.

Contar líneas

```
function contarLineas(texto) {
  if (texto.length === 0) {
    return 0;
  }

  const partes = texto.split(/\r\n|\n\r/);
  const terminaConSalto = /(?:\r\n|\n\r)$/.test(texto);

  return partes.length - (terminaConSalto ? 1 : 0);
}
```

Aquí adoptamos una convención explícita: el archivo vacío tiene cero líneas, y el salto final termina la última línea pero no crea una adicional. Otras herramientas definen el conteo de otra manera; lo importante es conocer y documentar la regla elegida.

Contar palabras Unicode

```
function extraerPalabras(texto) {
  return texto.match(/\p{L}\p{N}+/gu) ?? [];
}

const palabras = extraerPalabras(texto);
console.log("Palabras:", palabras.length);
```

`\p{L}` representa letras Unicode y `\p{N}` números Unicode; la bandera `u` activa ese tratamiento. Es la solución de la sección 20.15, indispensable para texto en español.

El `?? []` cubre el caso en que `match()` devuelve `null` por no encontrar nada.

La definición de «palabra» depende del problema. Este patrón es una buena aproximación, pero no reemplaza a un segmentador lingüístico como el `Intl.Segmenter` de la sección 6.9 en todos los idiomas.

Contar apariciones y extraer números

```
function contarApariciones(texto, palabra) {
  const palabras = texto.match(/\p{L}\p{N}+/gu) ?? [];
  const buscada = palabra.toLocaleLowerCase("es");

  return palabras.filter(
    (actual) => actual.toLocaleLowerCase("es") === buscada
  ).length;
}
```

```
const texto = "Ana obtuvo 8, Juan 10 y Sol 7";
const numeros = (texto.match(/\d+/g) ?? []).map(Number);

console.log(numeros); // [8, 10, 7]
```

21.18 Trabajar con JSON

JSON es texto con una gramática específica.

Archivo `alumnos.json`:

```
[
  { "nombre": "Ana", "nota": 8 },
  { "nombre": "Luis", "nota": 6 },
  { "nombre": "Sol", "nota": 10 }
]
```

Lectura:

```
import { readFile } from "node:fs/promises";

const texto = await readFile("alumnos.json", "utf8");
const alumnos = JSON.parse(texto.replace(/\uFEFF/, ""));

const aprobados = alumnos.filter((alumno) => alumno.nota >= 6);
const promedio =
  alumnos.reduce((acumulado, alumno) => acumulado + alumno.nota, 0) /
  alumnos.length;
```

Escritura:

```
import { writeFile } from "node:fs/promises";

const texto = JSON.stringify(alumnos, null, 4) + "\n";

await writeFile("alumnos.json", texto, "utf8");
```

Los argumentos `null, 4` de `JSON.stringify()` agregan sangría para facilitar la lectura humana y las comparaciones en un sistema de control de versiones.

Conviene recordar los límites que vimos en la sección 12.5: JSON no representa `undefined`, funciones, `Date` —las convierte a cadena—, `Map`, `Set` ni `bigint`.

Validar la estructura, no solo la sintaxis

Que el JSON sea válido no significa que contenga los datos esperados:


```

if (!Array.isArray(alumnos)) {
  throw new TypeError("Se esperaba un arreglo de alumnos");
}

for (const alumno of alumnos) {
  if (
    typeof alumno.nombre !== "string" ||
    typeof alumno.nota !== "number"
  ) {
    throw new TypeError("Alumno inválido");
  }
}

```

Son dos preguntas diferentes:

```

¿el texto respeta la gramática JSON?    → JSON.parse
¿los datos tienen la forma que necesito? → validación

```

Es exactamente la distinción de la sección 1.10: un archivo es una frontera del sistema, y todo lo que entra por una frontera se valida.

21.19 Trabajar con CSV

Archivo `alumnos.csv`:

```

nombre,nota
Ana,8
Luis,6
Sol,10

```

Si controlamos el formato y sabemos que ningún campo contiene comas, comillas ni saltos de línea, un análisis elemental alcanza:

```

const texto = await readFile("alumnos.csv", "utf8");
const lineas = texto
  .replace(/^    /, "")
  .trim()
  .split(/\r\n|\n|\r/);

const [encabezado, ...filas] = lineas;

if (encabezado !== "nombre,nota") {
  throw new Error("Encabezado CSV inesperado");
}

const alumnos = filas.map((fila) => {
  const [nombre, notaTexto] = fila.split(",");

  return { nombre, nota: Number(notaTexto) };
});

```

Pero un CSV real puede contener:

```
"Pérez, Ana",8
"Texto con ""comillas""",9
```

En esos casos `split(",")` es directamente incorrecto: partiría el primer campo en dos. Para archivos CSV generales conviene usar una biblioteca que implemente correctamente la variante del formato que recibimos.

Es un buen ejemplo del límite que señalamos en la sección 20.18: el CSV con comillas escapadas tiene un contexto que un análisis línea a línea no captura.

21.20 Procesar registros de eventos

Supongamos este archivo `servidor.log`:

```
2026-08-24 INFO servidor iniciado
2026-08-24 ERROR conexión rechazada
2026-08-24 INFO reintento
```

Podemos extraer fecha, nivel y mensaje con una expresión regular:

```
const texto = await readFile("servidor.log", "utf8");
const lineas = texto.split(/\r\n|\n|\r/);

const patron = /^(\d{4}-\d{2}-\d{2}) (INFO|WARN|ERROR) (.+)$/;

const eventos = [];

for (const linea of lineas) {
  const coincidencia = linea.match(patron);

  if (!coincidencia) {
    continue;
  }

  const [, fecha, nivel, mensaje] = coincidencia;

  eventos.push({ fecha, nivel, mensaje });
}

const errores = eventos.filter((evento) => evento.nivel === "ERROR");
```

Nótese la desestructuración `const [, fecha, nivel, mensaje]`: la coma inicial salta la coincidencia completa, que ocupa la posición cero.

La expresión regular no «comprende» el archivo: describe la estructura que esperamos encontrar. Las líneas que no coinciden se descartan con `continue`, lo cual vuelve el programa tolerante a líneas mal formadas.

21.21 Archivos grandes: *streams*

`readFile` carga el archivo completo en memoria:

```
disco → archivo completo en RAM → procesamiento
```

Es cómodo para archivos pequeños y medianos. Para un archivo de varios gigabytes puede ser imposible, y casi siempre es innecesario.

Un **stream** entrega fragmentos a medida que están disponibles:

```
disco → fragmento → procesar
      → fragmento → procesar
      → fragmento → procesar
```

Leer línea por línea

```
import { createReadStream } from "node:fs";
import { createInterface } from "node:readline";

const flujo = createReadStream("servidor.log", { encoding: "utf8" });

const lineas = createInterface({
  input: flujo,
  crlfDelay: Infinity,
});

let cantidadErrores = 0;

for await (const linea of lineas) {
  if (linea.includes(" ERROR ")) {
    cantidadErrores++;
  }
}

console.log({ cantidadErrores });
```

`crlfDelay: Infinity` hace que `\r\n` se trate como un solo salto de línea.

El `for await` es el recorrido de un iterador asincrónico, el protocolo `Symbol.asyncIterator` que vimos en la sección 8.7. La memoria utilizada ya no crece con el tamaño total del archivo.

Conectar lectura y escritura

Para una copia simple es preferible `copyFile`. Los *streams* son útiles cuando además queremos transformar o inspeccionar el contenido durante el recorrido:

```
import { createReadStream, createWriteStream } from "node:fs";
import { pipeline } from "node:stream/promises";

await pipeline(
  createReadStream("origen.bin"),
  createWriteStream("destino.bin")
);
```

`pipeline` conecta los flujos, propaga los errores correctamente y espera a que el proceso termine. Conectar flujos a mano con `.pipe()` no propaga los errores, y por eso `pipeline` es la forma recomendada.

21.22 Abrir un archivo explícitamente

Las funciones de alto nivel abren, operan y cierran el archivo por nosotros. También podemos abrirlo explícitamente:

```
import { open } from "node:fs/promises";

const archivo = await open("mensaje.txt", "r");

try {
  const contenido = await archivo.readFile("utf8");
  console.log(contenido);
} finally {
  await archivo.close();
}
```

El sistema operativo mantiene un recurso abierto y Node.js lo representa mediante un `FileHandle`. **Hay que cerrarlo incluso si ocurre un error**, y por eso se usa `finally`. Es el caso de uso canónico que presentamos en la sección 15.12.

Abrir explícitamente resulta útil cuando necesitamos varias operaciones sobre el mismo archivo, o control de bajo nivel sobre posiciones y fragmentos.

21.23 Errores habituales

Las operaciones pueden fallar por muchas razones: la ruta no existe, el proceso no tiene permiso, una parte de la ruta no es un directorio, el destino ya existe, el disco está lleno, el archivo está bloqueado, la operación cruza sistemas de archivos.

Node.js entrega objetos de error que suelen incluir una propiedad `code`:

Código	Significado habitual
ENOENT	No existe la ruta
EACCES	Acceso denegado
EEXIST	El destino ya existe
EISDIR	Se esperaba un archivo, pero era un directorio
ENOTDIR	Una parte que debía ser directorio no lo es
ENOSPC	No queda espacio disponible
EXDEV	La operación cruza sistemas de archivos

```
try {
  const contenido = await readFile("config.json", "utf8");
  console.log(contenido);
} catch (error) {
  switch (error.code) {
    case "ENOENT":
      console.error("No se encontró config.json");
      break;

    case "EACCES":
      console.error("No hay permiso para leer config.json");
      break;

    default:
      throw error;
  }
}
```

El `default` que relanza es la parte importante: si no sabemos resolver un error, hay que dejarlo propagar. Un `catch` vacío convierte fallos reales en resultados silenciosamente incorrectos.

21.24 Operaciones secuenciales y concurrentes

Con `await`, estas escrituras ocurren una después de otra:

```
await writeFile("a.txt", "A", "utf8");
await writeFile("b.txt", "B", "utf8");
await writeFile("c.txt", "C", "utf8");
```

Si son independientes, podemos iniciarlas juntas:

```
await Promise.all([
  writeFile("a.txt", "A", "utf8"),
  writeFile("b.txt", "B", "utf8"),
  writeFile("c.txt", "C", "utf8"),
]);
```

Pero **no** debemos lanzar varias modificaciones concurrentes sobre el mismo archivo sin diseñar cómo se coordinarán:

```
// resultado no confiable: compiten por el mismo destino
await Promise.all([
  writeFile("resultado.txt", "uno", "utf8"),
  writeFile("resultado.txt", "dos", "utf8"),
]);
```

El sistema de archivos no convierte automáticamente una secuencia lógica de varias operaciones en una transacción. Si necesitamos atomicidad, hay que construirla, por ejemplo con el patrón de archivo temporal y renombrado de la sección 21.10.

21.25 Seguridad de las rutas

Si un usuario controla una parte de la ruta, podría intentar salir del directorio permitido:

```
../../archivo-secreto.txt
```

Esto se conoce como **path traversal**, o recorrido de rutas, y es una de las vulnerabilidades más frecuentes en aplicaciones que sirven archivos.

Una estrategia básica consiste en resolver la ruta y verificar que permanezca dentro del directorio autorizado:

```
import path from "node:path";

function resolverDentroDe(base, entrada) {
  const baseAbsoluta = path.resolve(base);
  const candidata = path.resolve(baseAbsoluta, entrada);
  const relativa = path.relative(baseAbsoluta, candidata);

  const saleDeLaBase =
    relativa === ".." ||
    relativa.startsWith(`${path.sep}`) ||
    path.isAbsolute(relativa);

  if (saleDeLaBase) {
    throw new Error("La ruta sale del directorio permitido");
  }

  return candidata;
}

console.log(resolverDentroDe("datos", "alumnos/ana.txt"));
```

La clave es resolver **antes** de comprobar: una comprobación textual sobre la entrada sin resolver puede burlarse con codificaciones o segmentos intermedios.

En aplicaciones expuestas a usuarios también hay que definir políticas sobre enlaces simbólicos, permisos, nombres permitidos, tipos de archivo y tamaños máximos.

21.26 Un programa integrador

El siguiente programa reúne casi todo el capítulo. Recorre un directorio, encuentra archivos `.txt`, obtiene sus metadatos y calcula líneas, palabras y caracteres.

```

import { readFile, readdir, stat } from "node:fs/promises";
import path from "node:path";

function contarLineas(texto) {
  if (texto.length === 0) {
    return 0;
  }

  const partes = texto.split(/\r\n|\n|\r/);
  const terminaConSalto = /(?:\r\n|\n|\r)$/.test(texto);

  return partes.length - (terminaConSalto ? 1 : 0);
}

function contarPalabras(texto) {
  return (texto.match(/\p{L}\p{N}+/gu) ?? []).length;
}

async function encontrarTextos(directorio) {
  const rutas = [];
  const entradas = await readdir(directorio, { withFileTypes: true });

  for (const entrada of entradas) {
    const ruta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      rutas.push(...(await encontrarTextos(ruta)));
    } else if (
      entrada.isFile() &&
      path.extname(entrada.name).toLowerCase() === ".txt"
    ) {
      rutas.push(ruta);
    }
  }

  return rutas;
}

async function analizarArchivo(ruta) {
  const [texto, metadatos] = await Promise.all([
    readFile(ruta, "utf8"),
    stat(ruta),
  ]);

  return {
    ruta,
    bytes: metadatos.size,
    puntosDeCodigo: Array.from(texto).length,
    lineas: contarLineas(texto),
    palabras: contarPalabras(texto),
    modificado: metadatos.mtime.toISOString(),
  };
}

const directorio = process.argv[2] ?? ".";

try {
  const rutas = await encontrarTextos(directorio);

```



```

const informes = [];

for (const ruta of rutas) {
  informes.push(await analizarArchivo(ruta));
}

console.table(informes);
} catch (error) {
  console.error(`No se pudo analizar: ${error.message}`);
  process.exitCode = 1;
}

```

Se ejecuta así:

```
node analizar-textos.js datos
```

`process.argv[2]` es el primer argumento escrito después del nombre del programa. Si no se proporciona, se usa `"."`, el directorio actual.

El recorrido del programa:

```

argumento de la terminal
  |
  ▼
recorrido recursivo del directorio
  |
  ▼
selección de archivos .txt
  |
  ▼
lectura de contenido + metadatos
  |
  ▼
extracción de información
  |
  ▼
tabla de resultados

```

Vale la pena notar el uso de `Promise.all()` en `analizarArchivo`: leer el contenido y consultar los metadatos son operaciones independientes, así que se lanzan juntas. En cambio, el bucle exterior procesa los archivos de a uno, lo cual evita abrir cientos de archivos simultáneamente en un directorio grande.

21.27 Criterios para elegir una técnica

Necesidad	Herramienta apropiada
Leer un archivo pequeño como texto	<code>readFile(ruta, "utf8")</code>
Obtener los bytes sin interpretarlos	<code>readFile(ruta)</code>
Reemplazar o crear contenido	<code>writeFile</code>
Agregar al final	<code>appendFile</code>
Crear directorios intermedios	<code>mkdir(..., { recursive: true })</code>
Listar una carpeta	<code>readdir</code>
Obtener tamaño y fechas	<code>stat</code>
Copiar un archivo	<code>copyFile</code>
Copiar un árbol	<code>cp(..., { recursive: true })</code>
Mover o cambiar el nombre	<code>rename</code>
Procesar un archivo grande por líneas	<code>createReadStream + readline</code>
Conectar lectura y escritura por fragmentos	<code>pipeline</code>
Varias operaciones sobre una misma apertura	<code>open</code> y <code>FileHandle</code>

21.28 Confusiones frecuentes

Confundir extensión con formato. Cambiar `.jpg` por `.txt` no convierte una imagen en texto.

Omitir la codificación al esperar texto. `readFile("archivo.txt")` devuelve bytes; hay que agregar `"utf8"`.

Suponer que un carácter ocupa un byte. En UTF-8, `A`, `ñ` y `😊` ocupan cantidades diferentes.

Suponer que una ruta relativa parte del archivo JavaScript. Parte de `process.cwd()`, el directorio de trabajo del proceso.

Sobrescribir sin querer. `writeFile` reemplaza el contenido existente. Si eso no está permitido, corresponde la bandera `wx`.

Cargar archivos enormes con `readFile`. Para procesamiento secuencial de archivos grandes hay que preferir *streams*.

Dividir cualquier CSV con una coma. Los campos pueden contener comas, comillas y saltos de línea.

Ignorar errores. Un `catch` vacío transforma fallos reales en resultados silenciosamente incorrectos.

Mezclar operaciones concurrentes sobre el mismo archivo. El orden final puede no ser el que imaginamos y pueden perderse datos.

21.29 Síntesis conceptual

La gestión de archivos se vuelve mucho más clara si mantenemos separadas seis capas:

1. El sistema de archivos organiza nombres, rutas y metadatos.
2. Un archivo contiene bytes.
3. Un formato indica cómo interpretar esos bytes.
4. Una codificación relaciona texto y bytes.
5. Node.js solicita las operaciones al sistema operativo.
6. Nuestro programa transforma los datos y decide cómo manejar errores.

El recorrido típico de un programa que trabaja con archivos:

```
construir ruta
  ↓
abrir o solicitar operación
  ↓
leer bytes
  ↓
decodificar si son texto
  ↓
buscar, validar o transformar
  ↓
codificar
  ↓
escribir, copiar o mover
  ↓
manejar el resultado o el error
```

Tres ideas conviene retener.

La primera: **un archivo no es, en esencia, «un documento de texto» ni «una imagen»**. Es una secuencia de bytes a la que damos significado mediante convenciones. Desde ese modelo, leer, escribir, buscar o extraer información dejan de ser acciones misteriosas y se convierten en transformaciones bien definidas.

La segunda: **un archivo es una frontera del sistema**. Todo lo que entra por ahí —la codificación, la estructura, el formato— es una suposición que hay que verificar, no un hecho. Es la misma regla que enunciamos en el capítulo 1 sobre las solicitudes HTTP.

La tercera: **las operaciones sobre archivos fallan**, y fallan por razones variadas y a menudo transitorias. Un programa serio distingue los errores que sabe manejar de los que debe propagar, y protege los datos importantes con patrones como el archivo

temporal más renombrado.

Apéndices

Apéndice A. Fundamentos de la computación digital

Una computadora hace cosas notablemente complicadas. Procesa texto e imágenes, ejecuta videojuegos, resuelve cálculos científicos, reproduce música, se comunica por Internet y entrena sistemas de inteligencia artificial. Frente a esa variedad, resulta razonable suponer que adentro debe haber algo igualmente complicado.

No lo hay. Toda esa complejidad se construye a partir de elementos extraordinariamente simples, combinados en cantidades enormes:

Un comportamiento complejo puede construirse combinando una gran cantidad de operaciones extremadamente simples.

Para ver cómo es posible hacen falta tres piezas: una forma de representar información con valores discretos, un conjunto de operaciones lógicas sobre esos valores, y un método para componer operaciones simples y obtener otras más complejas. Las construimos en ese orden, hasta llegar desde un único interruptor a un circuito que suma.

De acá salen las respuestas a varias preguntas que aparecen más adelante: por qué `0.1 + 0.2` no da exactamente `0.3`, por qué existe un tipo `bigint` además de `number`, qué hacen realmente los operadores `&`, `|` y `^`, y por qué una expresión regular es, en el fondo, una máquina que lee caracteres.

A.1 Representar información con dos estados

La forma más sencilla de representar información consiste en distinguir únicamente entre dos estados. Podemos llamarlos:

0 1

Pero esos dos símbolos son una notación, no una sustancia. Los mismos dos estados podrían escribirse:

falso	verdadero
apagado	encendido
no	sí
abierto	cerrado
0 V	5 V

Lo importante no es qué representan físicamente. Lo importante es que podamos distinguirlos de manera confiable. Un sistema que debe diferenciar diez niveles de tensión es mucho más frágil frente al ruido eléctrico que uno que solo debe decidir si hay

corriente o no la hay. Esa robustez es la razón práctica por la que la computación terminó siendo binaria.

A cada unidad de información que puede tomar uno de dos valores la llamamos **bit**, contracción de *binary digit*.

A.2 Los bits no significan nada por sí solos

Un bit aislado no dice qué representa. Somos nosotros quienes decidimos su significado mediante una **codificación**.

Imaginemos un dado. Tiene seis resultados posibles:

```
1 2 3 4 5 6
```

Con un solo bit podemos representar dos posibilidades. Con dos bits, cuatro:

```
00
01
10
11
```

Con tres bits llegamos a ocho combinaciones:

```
000 001 010 011
100 101 110 111
```

Ocho alcanza para las seis caras. Podríamos codificarlas así:

Cara del dado	Binario
1	001
2	010
3	011
4	100
5	101
6	110

Las combinaciones 000 y 111 quedan sin usar. No es un problema: sobra capacidad.

La regla general es que con n bits se pueden distinguir 2^n valores diferentes. Para representar k posibilidades necesitamos, como mínimo, tantos bits como para que $2^n \geq k$.

Esto revela algo importante:

Una secuencia de bits puede representar información que en su origen no tiene nada de binaria.

Puede representar números, letras, colores, sonidos, la posición de las piezas en un tablero o cualquier otra cosa que sepamos codificar. Y, a la inversa, una misma secuencia de bits puede significar cosas distintas según qué convención usemos para leerla. Esta idea reaparecerá con toda su fuerza en el capítulo 21, cuando veamos que un archivo es una secuencia de bytes a la que un formato le da sentido.

A.3 De representar información a procesarla

Representar no alcanza. Queremos calcular.

Supongamos que tiramos dos dados. Cada uno se representa con tres bits:

$$\begin{aligned} A &= a_2 \ a_1 \ a_0 \\ B &= b_2 \ b_1 \ b_0 \end{aligned}$$

Queremos construir un sistema que responda una pregunta muy concreta: **¿los dos dados suman siete?**

La respuesta también cabe en un bit:

$$\begin{aligned} 0 &\rightarrow \text{no} \\ 1 &\rightarrow \text{sí} \end{aligned}$$

Tenemos entonces una función que va de seis bits de entrada a un bit de salida:

$$f(A, B) \rightarrow \{0, 1\}$$

Algunos casos:

$$\begin{array}{ll} 1 + 6 \rightarrow 1 & 1 + 1 \rightarrow 0 \\ 2 + 5 \rightarrow 1 & 2 + 2 \rightarrow 0 \\ 3 + 4 \rightarrow 1 & 3 + 2 \rightarrow 0 \\ 4 + 3 \rightarrow 1 & 6 + 6 \rightarrow 0 \end{array}$$

Aquí aparece la idea que convierte a la electrónica en computación:

Un cálculo puede entenderse como una función que transforma una combinación de bits de entrada en bits de salida.

Sumar, comparar, decidir, ordenar, dibujar un píxel: todo eso, en el nivel más bajo, es una función de bits a bits.

A.4 Las tres operaciones fundamentales

Cuando las variables solo pueden valer 0 o 1, disponemos de un aparato matemático llamado **álgebra de Boole**, desarrollado por George Boole a mediados del siglo XIX, casi un siglo antes de que existieran las computadoras electrónicas.

Sus operaciones fundamentales son tres.

NOT

Invierte el valor.

A	NOT A
0	1
1	0

Se escribe también $\neg A$.

AND

Produce 1 solamente cuando ambas entradas valen 1.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Se escribe también $A \wedge B$.

OR

Produce 1 cuando al menos una entrada vale 1.

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Se escribe también $A \vee B$.

Estas tres tablas son todo el vocabulario que necesitamos. Vale la pena detenerse en lo pequeño que es ese vocabulario antes de ver hasta dónde llega.

A.5 Componer operaciones

La potencia aparece al combinarlas. Por ejemplo:

```
(A AND B) OR (NOT C)
```

o, en notación simbólica:

```
(A  $\wedge$  B)  $\vee$   $\neg$ C
```

Ya no estamos realizando una operación elemental. Estamos construyendo una función nueva usando funciones más sencillas como piezas. Y nada nos impide repetir el procedimiento sobre el resultado:

```
operaciones simples
  ↓
combinaciones
  ↓
funciones más complejas
  ↓
combinaciones de funciones
  ↓
sistemas todavía más complejos
```

Esta composición es una de las ideas esenciales de la computación, y no aparece solamente en el nivel del hardware. En el capítulo 18 veremos funciones de JavaScript que se combinan para formar transformaciones más elaboradas, con exactamente el mismo espíritu.

A.6 La tabla de verdad como definición

Una función booleana queda completamente descrita por su **tabla de verdad**.

Con dos entradas hay solo cuatro combinaciones posibles:

```
00  01  10  11
```

Así que podemos definir cualquier función de dos variables simplemente diciendo qué devuelve en cada una:

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

La tabla no explica *cómo* se calcula *F*. Solo enumera qué resultado debe producir para cada entrada posible. Por eso una tabla de verdad es una **definición por enumeración** de una función.

Esto es cómodo porque describir un comportamiento suele ser más fácil que diseñarlo. Podemos decir qué queremos que ocurra en cada caso sin saber todavía cómo lograrlo. Lo que sigue resuelve exactamente ese problema.

A.7 De una tabla de verdad a una expresión lógica

Existe un procedimiento mecánico para convertir cualquier tabla de verdad en una expresión construida únicamente con NOT, AND y OR. Veámoslo sobre la tabla anterior.

Nos interesan las filas donde *F* = 1. Hay dos.

Primera fila con salida 1:

```
A = 0
B = 1
```

Queremos una expresión que sea verdadera exactamente para esa combinación y falsa para todas las demás. Como *A* debe valer cero, lo negamos. Como *B* debe valer uno, lo dejamos tal cual. Y exigimos ambas condiciones a la vez con AND:

```
(NOT A) AND B      →      ¬A ∧ B
```

Conviene comprobarlo: esa expresión vale 1 solo si *A* es 0 y *B* es 1. Para cualquier otra combinación, alguno de los dos operandos del AND es 0.

Segunda fila con salida 1:

```
A = 1
B = 0
```

Por el mismo razonamiento:

```
A AND (NOT B)      →      A ∧ ¬B
```

Finalmente, queremos aceptar una combinación o la otra, así que las unimos con OR:

```
((NOT A) AND B) OR (A AND (NOT B))
```

o, en símbolos:

```
(¬A ∧ B) ∨ (A ∧ ¬B)
```

Obtuvimos una expresión lógica directamente a partir de la tabla, sin ingenio ni inspiración. Solo siguiendo una receta.

Dicho sea de paso, esta función tiene nombre: es el **OR exclusivo**, o XOR, que vale 1 cuando las entradas difieren. Volveremos a encontrarla enseguida.

A.8 El procedimiento general: forma normal disyuntiva

El método que acabamos de usar se llama **forma normal disyuntiva**, o **suma de productos**. Su algoritmo es este:

Paso 1. Buscar todas las filas donde la salida sea 1.

Paso 2. Para cada una de esas filas, construir una expresión AND que incluya todas las variables. Para cada variable:

```
si en esa fila vale 1 → usar la variable
si en esa fila vale 0 → usar NOT variable
```

Por ejemplo, la fila:

```
A = 1    B = 0    C = 1
```

produce:

```
A AND (NOT B) AND C
```

Paso 3. Unir todas esas expresiones con OR:

```
fila1 OR fila2 OR fila3 OR ...
```

El resultado reproduce exactamente la tabla de verdad original, porque cada término AND se activa en una sola fila y el OR acepta cualquiera de ellas.

Un caso límite merece mención: si la tabla no tiene ninguna fila con salida 1, la función es constantemente 0 y no hay términos que unir.

A.9 Una consecuencia fundamental

Detengámonos en lo que acabamos de demostrar, porque es más profundo de lo que parece.

El procedimiento funciona para **cualquier** tabla de verdad, de cualquier cantidad de variables. Y toda función booleana puede escribirse como una tabla de verdad, porque hay una cantidad finita de combinaciones de entrada. Por lo tanto:

NOT, AND y OR son suficientes para expresar cualquier función booleana.

Esta propiedad se llama **completitud funcional**.

No dice que la expresión obtenida sea la más corta ni la más eficiente: la forma normal disyuntiva suele producir expresiones más grandes de lo necesario, y existen técnicas de simplificación para reducirlas. Dice algo más básico y más poderoso: **siempre podemos construirla**.

Si sabemos describir un comportamiento, sabemos construirlo con tres operaciones.

A.10 El álgebra de Boole permite transformar expresiones

Las expresiones booleanas forman un álgebra, así que admiten manipulación simbólica, igual que las expresiones numéricas.

Distributividad

$$A \text{ AND } (B \text{ OR } C) = (A \text{ AND } B) \text{ OR } (A \text{ AND } C)$$

También en sentido inverso, factorizando:

$$(A \text{ AND } B) \text{ OR } (A \text{ AND } C) = A \text{ AND } (B \text{ OR } C)$$

La versión factorizada usa menos operaciones. Traducida a circuitos, significa menos compuertas, menos espacio y menos consumo.

Leyes de De Morgan

Dos transformaciones especialmente importantes:

$$\begin{aligned} \text{NOT } (A \text{ AND } B) &= (\text{NOT } A) \text{ OR } (\text{NOT } B) \\ \text{NOT } (A \text{ OR } B) &= (\text{NOT } A) \text{ AND } (\text{NOT } B) \end{aligned}$$

En símbolos:

$$\neg(A \wedge B) = \neg A \vee \neg B$$

$$\neg(A \vee B) = \neg A \wedge \neg B$$

Enunciadas en palabras resultan naturales: «no es cierto que ambos» equivale a «alguno de los dos falla», y «no es cierto que alguno» equivale a «fallan los dos».

Las leyes de De Morgan permiten convertir un AND en un OR y viceversa, siempre que estemos dispuestos a negar. Las vamos a necesitar dos veces más: en la sección A.15, para construir compuertas universales, y en el capítulo 14, para simplificar condiciones de un `if`.

A.11 De la matemática a un mecanismo físico

Hasta aquí todo podría parecer un ejercicio de matemática abstracta. El paso siguiente es notar que estas operaciones pueden construirse con objetos.

Necesitamos un elemento que permita que una señal controle a otra. Históricamente, el primero disponible fue el **relé**: un interruptor accionado eléctricamente. Una corriente en su bobina decide si otro circuito queda abierto o cerrado.

Con interruptores alcanza para las dos operaciones binarias.

AND es una conexión en serie

+ ---- [A] ---- [B] ----  ---- -

La corriente circula solamente si A está cerrado y B está cerrado.

A	B	Lámpara
0	0	0
0	1	0
1	0	0
1	1	1

Es exactamente la tabla de verdad de AND.

OR es una conexión en paralelo

+ --- [A] ---+
 + ----+ +----  ---- -
 + --- [B] ---+

Hay corriente si está cerrado A o está cerrado B.

A	B	Lámpara
0	0	0
0	1	1
1	0	1
1	1	1

Otra vez, la tabla que habíamos definido matemáticamente.

NOT requiere un relé de contacto normalmente cerrado: la señal presente abre el circuito en lugar de cerrarlo.

Con esto, el puente está tendido:

```
álgebra de Boole
  ↓
expresiones lógicas
  ↓
compuertas
  ↓
circuitos
  ↓
sistemas digitales
```

Los primeros computadores electromecánicos usaron relés. Después vinieron las válvulas de vacío y más tarde los transistores, que son varios órdenes de magnitud más rápidos, más pequeños y más baratos. La tecnología física cambió por completo tres veces. La lógica que implementa no cambió nunca.

A.12 Representar números en binario

Hasta ahora usamos 0 y 1 como valores lógicos. También sirven para representar cantidades.

Nuestro sistema decimal se apoya en potencias de diez. El número 583 significa:

$$5 \times 10^2 + 8 \times 10^1 + 3 \times 10^0$$

El sistema binario usa la misma idea con potencias de dos. El número 1011_2 significa:

$$\begin{aligned} &1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 8 + 0 + 2 + 1 \\ &= 11 \end{aligned}$$

Por lo tanto:

$$1011_2 = 11_{10}$$

No estamos usando una matemática distinta. Estamos cambiando la base de representación. El número once es el mismo; lo que cambia es cómo lo escribimos.

Esta distinción entre un valor y su representación va a volver varias veces. En el capítulo 5 veremos que `255`, `0b11111111`, `0o377` y `0xff` producen el mismo valor `number` en JavaScript, y que una vez creado, ese valor no recuerda en qué base fue escrito.

A.13 Sumar en binario

La suma binaria funciona igual que la decimal. En decimal sabemos que `9 + 1` da `10`: escribimos `0` y llevamos `1` a la columna siguiente, porque nos quedamos sin dígitos.

En binario nos quedamos sin dígitos mucho antes:

```
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10    ← escribo 0, llevo 1
```

Por ejemplo:

```
  101
+ 011
-----
 1000
```

Verifiquemos en decimal: `1012` es `5`, `0112` es `3`, y `10002` es `8`. El procedimiento es idéntico al que aprendimos en la escuela; solo cambia cuándo hay que llevarse una.

A.14 El medio sumador: aritmética construida con lógica

Ahora podemos conectar las dos mitades del recorrido.

La suma de dos bits produce dos resultados: el dígito de la suma y el acarreo que se lleva a la columna siguiente. Escribamos su tabla de verdad:

A	B	Suma	Acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Tenemos dos entradas y dos salidas. Cada salida es, por sí sola, una función booleana. Así que podemos aplicarle a cada una el procedimiento de la sección 1.8.

La columna **Suma** vale 1 en las filas A=0, B=1 y A=1, B=0:

```
S = ((NOT A) AND B) OR (A AND (NOT B))
```

Es la expresión que ya habíamos deducido en la sección A.7. La suma de dos bits es un XOR.

La columna **Acarreo** vale 1 en una sola fila, A=1, B=1:

```
C = A AND B
```

Ese circuito se llama **medio sumador**, o *half adder*. Se lo llama «medio» porque no recibe un acarreo de entrada: para sumar números de varios bits hace falta el **sumador completo**, que toma tres entradas —los dos bits y el acarreo que llega de la columna anterior— y encadena tantas copias como bits tenga el número.

Aquí aparece la conclusión que veníamos persiguiendo:

La aritmética puede construirse con lógica booleana.

No hay un módulo aparte que «sepa sumar». Sumar es aplicar una función booleana, construida con las mismas tres operaciones de la sección A.4, que a su vez son interruptores en serie y en paralelo.

La multiplicación sigue el mismo camino y resulta incluso más simple, porque solo hay dos dígitos:

```
0 × 0 = 0      1 × 0 = 0
0 × 1 = 0      1 × 1 = 1
```

Multiplicar un número por un bit significa elegir entre 0 y el número mismo. Con esa operación elemental, desplazamientos y sumas se construye un multiplicador. Otra vez el mismo principio:

```
operaciones elementales → combinación → operaciones aritméticas complejas
```

A.15 ¿Hacen falta las tres compuertas?

Dijimos que NOT, AND y OR bastan para cualquier función booleana. Se puede ir todavía más lejos.

Consideremos la operación **NAND**, que es simplemente la negación de AND:

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Resulta que **NAND sola alcanza para construir cualquier función booleana**. Para demostrarlo basta con construir las tres operaciones básicas usando solo NAND, porque ya sabemos que con esas tres se puede construir todo lo demás.

NOT a partir de NAND

Conectamos las dos entradas al mismo valor:

A NAND A

Por definición de NAND:

A NAND A = NOT (A AND A)

Y como A AND A es simplemente A —una proposición no agrega nada al conjugarse consigo misma—, queda:

A NAND A = NOT A

Comprobémoslo con la tabla:

A	A AND A	NOT (A AND A)
0	0	1
1	1	0

Es exactamente NOT.

AND a partir de NAND

NAND es un AND negado, así que para recuperar AND alcanza con negar otra vez. Y ya sabemos negar:

$$A \text{ AND } B = \text{NOT } (A \text{ NAND } B) = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$$

Necesitamos dos compuertas: una calcula $A \text{ NAND } B$, la otra invierte el resultado.

OR a partir de NAND

Aquí entra en juego De Morgan. Partimos de:

$$A \text{ OR } B$$

Aplicamos doble negación, que no cambia el valor:

$$A \text{ OR } B = \text{NOT } (\text{NOT } (A \text{ OR } B))$$

Y transformamos la negación interior con la segunda ley de De Morgan:

$$\text{NOT } (A \text{ OR } B) = (\text{NOT } A) \text{ AND } (\text{NOT } B)$$

Reemplazando:

$$A \text{ OR } B = \text{NOT } ((\text{NOT } A) \text{ AND } (\text{NOT } B))$$

El lado derecho es un NAND aplicado a las entradas negadas:

$$A \text{ OR } B = (\text{NOT } A) \text{ NAND } (\text{NOT } B)$$

Y como ya sabemos construir NOT con NAND:

$$A \text{ OR } B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$$

Tres compuertas NAND: dos para negar cada entrada, una para combinarlas.

Verifiquemos el caso $A = 0, B = 0$:

$$\begin{aligned} A \text{ NAND } A &= 1 \\ B \text{ NAND } B &= 1 \\ 1 \text{ NAND } 1 &= 0 \quad \leftarrow \text{correcto: } 0 \text{ OR } 0 = 0 \end{aligned}$$

Y el caso $A = 1, B = 0$:

$$\begin{aligned} A \text{ NAND } A &= 0 \\ B \text{ NAND } B &= 1 \\ 0 \text{ NAND } 1 &= 1 \quad \leftarrow \text{correcto: } 1 \text{ OR } 0 = 1 \end{aligned}$$

La conclusión

Como NAND puede construir NOT, AND y OR, y esas tres pueden construir cualquier función booleana, se sigue que:

NAND por sí sola es funcionalmente completa.

Una compuerta se dice **universal** cuando tiene esta propiedad. NOR, la negación de OR, también lo es, por un argumento simétrico.

Esto tiene consecuencias prácticas reales. Un fabricante de circuitos integrados puede optimizar la producción de un único tipo de compuerta y construir con ella sistemas arbitrariamente complejos. La uniformidad simplifica el diseño, la verificación y la fabricación.

A.16 Síntesis conceptual

Recorrimos un camino que empieza en un interruptor y termina en un circuito que suma. Vale la pena mirarlo completo:

```
dos estados distinguibles
  ↓
un bit
  ↓
codificación: los bits representan lo que decidamos
  ↓
funciones booleanas: bits de entrada → bits de salida
  ↓
NOT, AND, OR
  ↓
forma normal disyuntiva: toda tabla de verdad es construible
  ↓
relés en serie y en paralelo: la lógica se vuelve física
  ↓
representación binaria de números
  ↓
medio sumador: la aritmética es un caso de lógica
  ↓
NAND universal: una sola operación alcanza
```

Tres ideas conviene retener por encima de las demás.

La primera es que **la representación es una convención**. Los bits no tienen significado intrínseco; se lo damos nosotros al elegir una codificación. Un mismo patrón de bits puede ser un número, una letra o un color según cómo decidamos leerlo.

La segunda es que **describir un comportamiento equivale a poder construirlo**. La forma normal disyuntiva convierte una tabla de verdad —que es una descripción— en una expresión —que es una construcción— de manera mecánica.

La tercera es que **la complejidad se construye por composición**. No hay ningún componente misterioso que explique la capacidad de una computadora. Hay una operación simple repetida y combinada miles de millones de veces.

Esa misma idea gobierna la programación. Un programa complejo no se escribe de una sola vez: se construye componiendo funciones simples que hacen una cosa bien, la técnica que el capítulo 18 desarrolla de manera explícita. Ya estaba presente en el nivel de los interruptores.

Apéndice B. Un tokenizador Byte Pair Encoding

Un tokenizador convierte texto en una secuencia de números y puede reconstruir el texto original a partir de ellos. El algoritmo que vamos a construir se llama **Byte Pair Encoding**, o BPE, y es la base de los tokenizadores que usan los modelos de lenguaje.

B.1 El problema: convertir texto en números

Una computadora no trabaja directamente con palabras. Trabaja con números. Antes de enviar texto a un modelo, necesitamos una función que lo transforme en una secuencia numérica:

```
"hola" → [104, 111, 108, 97]
```

A esos números los llamaremos **tokens**.

Pero aparece inmediatamente una pregunta: ¿qué debería representar cada token? Podríamos usar un token por carácter, uno por palabra, uno por byte, o tokens que representen fragmentos frecuentes de texto.

BPE parte de una idea muy simple:

Empezar con unidades pequeñas y unir repetidamente los pares que aparecen con mayor frecuencia.

En nuestro caso, las unidades iniciales serán **bytes**.

B.2 Del texto a los bytes

JavaScript maneja cadenas Unicode:

```
const texto = "hola";
```

Para trabajar con bytes necesitamos convertir esa cadena a su representación UTF-8. Node.js lo permite directamente:

```
const bytes = Buffer.from("hola", "utf8");

console.log([...bytes]);
// [104, 111, 108, 97]
```

Cada número está entre 0 y 255, porque un byte tiene ocho bits:

```
00000000 → 0
11111111 → 255
```

En ASCII muchos caracteres ocupan un solo byte:

```
h → 104    o → 111    l → 108    a → 97
```

Pero Unicode contiene muchos más caracteres:

```
[...Buffer.from("ñ", "utf8")];
// [195, 177]

[...Buffer.from("😊", "utf8")];
// cuatro bytes
```

Es exactamente lo que vimos en la sección 21.9: UTF-8 usa una cantidad variable de bytes por carácter.

Esto es importante porque nuestro tokenizador **no necesita conocer letras, palabras ni idiomas**. Solo necesita conocer bytes.

B.3 Por qué empezar por bytes

Supongamos que hiciéramos un tokenizador basado únicamente en palabras. Durante el entrenamiento encontramos:

```
casa    perro    gato
```

Pero después aparece:

```
ornitorrinco
```

Si esa palabra nunca fue aprendida, no sabríamos cómo representarla. Es el problema del vocabulario cerrado.

Con bytes eso nunca ocurre. Todo texto UTF-8 puede convertirse en bytes. Por lo tanto, comenzamos con un vocabulario universal de 256 tokens:

```
0, 1, 2, ..., 255
```

Cada token representa exactamente un byte, así que el tokenizador siempre puede representar cualquier texto, incluidos caracteres y palabras que nunca vio.

B.4 El problema de usar solamente bytes

Funciona, pero produce demasiados tokens. Por ejemplo, `banana` en UTF-8 es:

```
[98, 97, 110, 97, 110, 97]
```

Seis tokens para una palabra de seis letras. Pero observemos la estructura:

```
b a n a n a
```

El par `a n` aparece dos veces. Podríamos inventar un token nuevo que lo represente. Entonces:

```
b [an] [an] a
```

Ahora tenemos cuatro tokens. BPE hace exactamente eso, de manera sistemática.

B.5 La idea del algoritmo

El aprendizaje puede resumirse así:

1. Convertir el texto de entrenamiento en bytes.
2. Buscar todos los pares de tokens consecutivos.
3. Encontrar el par más frecuente.
4. Crear un token nuevo para ese par.
5. Reemplazar todas sus apariciones.
6. Repetir.

Sigamos el ejemplo. Inicialmente:

```
98 97 110 97 110 97
```

Los pares consecutivos son:

```
98 97
97 110
110 97
97 110
110 97
```

Contamos:

```
98 97   → 1 vez
97 110  → 2 veces
110 97  → 2 veces
```


Hay un empate; nuestro programa elegirá simplemente el primero que encuentre. Supongamos que selecciona 97 110. Creamos el token 256 con el significado:

```
256 = [97, 110]
```

Entonces:

```
98 97 110 97 110 97
```

se transforma en:

```
98 256 256 97
```

Y volvemos a contar pares. BPE continúa aprendiendo estructuras cada vez mayores.

Los tokens nuevos son composiciones

El vocabulario comienza con los bytes originales y va creciendo:

```
0..255 = bytes originales  
256 = [97, 110]  
257 = [98, 256]  
258 = [256, 97]
```

Un token nuevo puede contener tokens que a su vez contienen otros tokens. BPE construye así un vocabulario jerárquico:

```
256 = "an"  
257 = "ban"  
258 = "ana"  
259 = "banana"
```

No necesariamente aprenderá exactamente esos tokens, pero conceptualmente puede hacerlo.

Es la misma idea de composición del apéndice A: piezas simples que se combinan para formar piezas mayores.

B.6 Las piezas del programa

Vamos a construir tres operaciones:

```
train(texto, fusiones)  
encode(texto)  
decode(tokens)
```

Con este uso:

```
const bpe = new BPE();

bpe.train(textoBase, 50);

const tokens = bpe.encode("banana");
console.log(tokens);
console.log(bpe.decode(tokens));
```

Y una propiedad que debe cumplirse siempre:

```
bpe.decode(bpe.encode(texto)) === texto
```

Convertir entre texto y bytes

```
function aBytes(texto) {
  return [...Buffer.from(texto, "utf8")];
}

function desdeBytes(bytes) {
  return Buffer.from(bytes).toString("utf8");
}
```

```
const bytes = aBytes("hola"); // [104, 111, 108, 97]
console.log(desdeBytes(bytes)); // "hola"
```

Esta ida y vuelta es la base de todo el tokenizador.

Contar pares

Supongamos:

```
const tokens = [98, 97, 110, 97, 110, 97];
```

Queremos contar cuántas veces aparece cada par consecutivo. Usamos un `Map`:

```
function contarPares(tokens) {
  const conteos = new Map();

  for (let i = 0; i < tokens.length - 1; i++) {
    const clave = `${tokens[i]},${tokens[i + 1]}`;

    conteos.set(clave, (conteos.get(clave) ?? 0) + 1);
  }

  return conteos;
}
```

La clave `"97,110"` representa el par `[97, 110]`. No es la representación más eficiente posible —un `Map` con arrays como clave no serviría, porque compararía por identidad, como vimos en la sección 13.3—, pero es muy fácil de entender.

El bucle llega hasta `length - 1` porque el último token no tiene un siguiente con el cual formar par.

Encontrar el par más frecuente

```
function parMasFrecuente(tokens) {
  const conteos = contarPares(tokens);

  let mejorPar = null;
  let mejorConteo = 0;

  for (const [par, conteo] of conteos) {
    if (conteo > mejorConteo) {
      mejorPar = par;
      mejorConteo = conteo;
    }
  }

  if (mejorPar === null) {
    return null;
  }

  return mejorPar.split(",").map(Number);
}
```

La comparación estricta `conteo > mejorConteo` hace que, ante un empate, gane el primero encontrado. Como un `Map` conserva el orden de inserción, ese primero es el que aparece antes en el texto.

Fusionar un par

Ahora necesitamos transformar `98 97 110 97 110 97` usando la regla `97 110 → 256`, para obtener `98 256 256 97`:

```
function fusionar(tokens, par, tokenNuevo) {
  const resultado = [];

  for (let i = 0; i < tokens.length; i++) {
    if (
      i < tokens.length - 1 &&
      tokens[i] === par[0] &&
      tokens[i + 1] === par[1]
    ) {
      resultado.push(tokenNuevo);
      i++;
    } else {
      resultado.push(tokens[i]);
    }
  }

  return resultado;
}
```

El detalle importante es el `i++` dentro del `if`. Cuando encontramos el par consumimos **dos** tokens, así que hay que saltar el segundo. Sin esa línea, el segundo elemento del par se agregaría de nuevo al resultado.

B.7 La clase completa

```
class BPE {
  constructor() {
    this.fusiones = [];
  }

  train(texto, cantidadDeFusiones) {
    let tokens = [...Buffer.from(texto, "utf8")];

    this.fusiones = [];

    for (let n = 0; n < cantidadDeFusiones; n++) {
      const par = this.buscarMejorPar(tokens);

      if (!par) {
        break;
      }

      const tokenNuevo = 256 + this.fusiones.length;

      this.fusiones.push({ par, token: tokenNuevo });

      tokens = this.fusionar(tokens, par, tokenNuevo);
    }
  }

  encode(texto) {
    let tokens = [...Buffer.from(texto, "utf8")];

    for (const { par, token } of this.fusiones) {
      tokens = this.fusionar(tokens, par, token);
    }

    return tokens;
  }

  decode(tokens) {
    const bytes = [];

    for (const token of tokens) {
      this.expandir(token, bytes);
    }

    return Buffer.from(bytes).toString("utf8");
  }

  buscarMejorPar(tokens) {
    const conteos = new Map();

    for (let i = 0; i < tokens.length - 1; i++) {
      const clave = `${tokens[i]},${tokens[i + 1]}`;
      conteos.set(clave, (conteos.get(clave) ?? 0) + 1);
    }

    let mejorPar = null;
    let mejorConteo = 0;
  }
}
```

```

    for (const [clave, conteo] of conteos) {
        if (conteo > mejorConteo) {
            mejorPar = clave;
            mejorConteo = conteo;
        }
    }

    if (mejorPar === null) {
        return null;
    }

    return mejorPar.split(",").map(Number);
}

fusionar(tokens, par, tokenNuevo) {
    const resultado = [];

    for (let i = 0; i < tokens.length; i++) {
        if (
            i < tokens.length - 1 &&
            tokens[i] === par[0] &&
            tokens[i + 1] === par[1]
        ) {
            resultado.push(tokenNuevo);
            i++;
        } else {
            resultado.push(tokens[i]);
        }
    }

    return resultado;
}

expandir(token, bytes) {
    if (token < 256) {
        bytes.push(token);
        return;
    }

    const { par } = this.fusiones[token - 256];

    this.expandir(par[0], bytes);
    this.expandir(par[1], bytes);
}
}

```

Esta es prácticamente toda la implementación. El resto del apéndice explica por qué funciona.

B.8 Cómo aprende `train`

```
const bpe = new BPE();  
bpe.train("banana banana banana", 5);
```

Primero convierte el texto a bytes. Después repite:

```
const par = this.buscarMejorPar(tokens);
```

Supongamos que devuelve `[97, 110]`. Creamos:

```
const tokenNuevo = 256;
```

y guardamos:

```
{ par: [97, 110], token: 256 }
```

Conceptualmente:

```
256 = 97 + 110
```

o, interpretado como texto:

```
256 = "an"
```

Después reemplazamos las apariciones y volvemos a buscar. El segundo token nuevo será `257`, el tercero `258`, y así sucesivamente.

El número `256` como punto de partida no es arbitrario: los valores `0` a `255` ya están reservados para los bytes originales.

La tabla de fusiones es el modelo

Después de entrenar:

```
console.log(bpe.fusiones);
```

podríamos obtener algo parecido a:

```
[  
  { par: [97, 110], token: 256 },  
  { par: [98, 256], token: 257 },  
  { par: [256, 97], token: 258 },  
]
```

Esto significa:

```
256 = [97, 110]
257 = [98, 256]
258 = [256, 97]
```

Vale la pena subrayarlo:

*BPE no aprende una red neuronal. Aprende una **lista ordenada de fusiones**.*

Esa lista es todo su modelo.

B.9 Cómo funciona `encode`

Supongamos que ya aprendimos:

```
97 110 → 256
98 256 → 257
```

Codificamos `"banana"`. Primero se convierte en:

```
98 97 110 97 110 97
```

Aplicamos la primera regla, `97 110 → 256`:

```
98 256 256 97
```

Después la segunda, `98 256 → 257`:

```
257 256 97
```

Y esos son los tokens finales.

La parte crucial es esta:

```
for (const { par, token } of this.fusiones) {
  tokens = this.fusionar(tokens, par, token);
}
```

Las reglas se aplican **en el mismo orden en que fueron aprendidas**.

Por qué importa el orden

Supongamos:

```
256 = "an"
257 = "ban"
```


El token `257` está definido como `[98, 256]`: depende del `256`. No podríamos crear `257` antes de haber creado `256`, ni aplicarlo antes.

Por eso las reglas de BPE tienen un orden estricto, y por eso `fusiones` es un array y no un `Map`.

B.10 Cómo funciona `decode`

Decodificar parece más complicado, pero es un caso claro de recursión, con la estructura del capítulo 19.

Recordemos:

```
257 = [98, 256]
256 = [97, 110]
```

Entonces `257` se expande como `98 256`, y `256` se expande como `97 110`. Resultado: `98 97 110`, que en UTF-8 representa `"ban"`.

La función `expandir` hace esto recursivamente.

El caso base

```
if (token < 256) {
  bytes.push(token);
  return;
}
```

Si el token es menor que `256`, ya es un byte original. No hay nada más que expandir.

El caso recursivo

Si `token >= 256`, es un token aprendido. Buscamos su regla:

```
const { par } = this.fusiones[token - 256];
```

Por ejemplo, para `token = 257`:

```
257 - 256 = 1 → this.fusiones[1]
```

Y expandimos las dos partes:

```
this.expandir(par[0], bytes);
this.expandir(par[1], bytes);
```

Cada componente puede ser a su vez un token aprendido, así que la recursión continúa. Eventualmente todo termina convertido en bytes originales, porque cada fusión solo puede referirse a tokens **anteriores**, y la cadena desciende hasta llegar por debajo de 256.

Es el recorrido de un árbol binario, exactamente como en el capítulo 19: cada token aprendido es un nodo con dos hijos, y cada byte original es una hoja.

B.11 El programa completo

```
// bpe.js

class BPE {
  constructor() {
    this.fusiones = [];
  }

  train(texto, cantidadDeFusiones) {
    let tokens = [...Buffer.from(texto, "utf8")];

    this.fusiones = [];

    for (let n = 0; n < cantidadDeFusiones; n++) {
      const par = this.buscarMejorPar(tokens);

      if (!par) break;

      const tokenNuevo = 256 + this.fusiones.length;

      this.fusiones.push({ par, token: tokenNuevo });

      tokens = this.fusionar(tokens, par, tokenNuevo);
    }
  }

  encode(texto) {
    let tokens = [...Buffer.from(texto, "utf8")];

    for (const { par, token } of this.fusiones) {
      tokens = this.fusionar(tokens, par, token);
    }

    return tokens;
  }

  decode(tokens) {
    const bytes = [];

    for (const token of tokens) {
      this.expandir(token, bytes);
    }

    return Buffer.from(bytes).toString("utf8");
  }

  buscarMejorPar(tokens) {
    const conteos = new Map();

    for (let i = 0; i < tokens.length - 1; i++) {
      const clave = `${tokens[i]},${tokens[i + 1]}`;
      conteos.set(clave, (conteos.get(clave) ?? 0) + 1);
    }

    let mejorPar = null;
    let mejorConteo = 0;
  }
}
```

```

    for (const [clave, conteo] of conteos) {
        if (conteo > mejorConteo) {
            mejorPar = clave;
            mejorConteo = conteo;
        }
    }

    if (mejorPar === null) return null;

    return mejorPar.split(",").map(Number);
}

fusionar(tokens, par, tokenNuevo) {
    const resultado = [];

    for (let i = 0; i < tokens.length; i++) {
        if (
            i < tokens.length - 1 &&
            tokens[i] === par[0] &&
            tokens[i + 1] === par[1]
        ) {
            resultado.push(tokenNuevo);
            i++;
        } else {
            resultado.push(tokens[i]);
        }
    }

    return resultado;
}

expandir(token, bytes) {
    if (token < 256) {
        bytes.push(token);
        return;
    }

    const { par } = this.fusiones[token - 256];

    this.expandir(par[0], bytes);
    this.expandir(par[1], bytes);
}

// -----
// Ejemplo de uso
// -----

const bpe = new BPE();

const textoDeEntrenamiento = `
banana banana banana
bandana banana
`;

bpe.train(textoDeEntrenamiento, 10);

console.log("Fusiones:");
console.log(bpe.fusiones);

```

```
const texto = "banana";
const tokens = bpe.encode(texto);

console.log("Texto:", texto);
console.log("Tokens:", tokens);
console.log("Decodificado:", bpe.decode(tokens));

console.log("Reversible:", bpe.decode(tokens) === texto);
```

Se ejecuta con:

```
node bpe.js
```

La última línea debe imprimir `true`. Esa es la propiedad que define un tokenizador correcto: codificar y decodificar debe devolver exactamente el texto original.

B.12 Unicode

Como el algoritmo trabaja sobre bytes UTF-8, también funciona con texto que contenga caracteres fuera del alfabeto básico:

```
const texto = "mañana 😊";
const tokens = bpe.encode(texto);

console.log(tokens);
console.log(bpe.decode(tokens)); // "mañana 😊"
```

Aunque ñ ocupe dos bytes y 😊 ocupe cuatro, la reconstrucción es exacta.

Esta es precisamente la ventaja de construir el tokenizador sobre bytes en lugar de sobre caracteres: no hay ningún texto que el algoritmo no pueda representar.

B.13 Qué es realmente un token

Al principio:

```
token 97 = byte 97 = "a"
```

Pero después del entrenamiento:

```
token 256 podría representar "an"
token 257 podría representar "ban"
otro token podría representar "banana"
```

Por lo tanto, un token no es necesariamente una letra, ni una sílaba, ni una palabra. Es simplemente:

una secuencia de bytes que el algoritmo decidió que era útil agrupar.

Esa observación explica varios comportamientos que se ven en la práctica con modelos de lenguaje: por qué una palabra frecuente cuesta un token y una rara cuesta cinco, o por qué un texto en un idioma poco representado en el entrenamiento consume muchos más tokens que su equivalente en inglés.

B.14 El compromiso fundamental

Si hacemos pocas fusiones:

```
vocabulario pequeño  
muchos tokens por texto
```

Si hacemos muchas:

```
vocabulario grande  
menos tokens por texto
```

Ese es uno de los compromisos centrales del diseño de un tokenizador. En nuestro programa:

```
bpe.train(texto, 100);
```

aprende cien fusiones, así que el vocabulario total tendrá como máximo:

```
256 + 100 = 356 tokens
```

B.15 Una mejora sencilla

La versión actual aprende el par más frecuente aunque aparezca una sola vez. Para fines didácticos eso simplifica el código, pero no tiene sentido crear un token para algo que no se repite.

Podemos hacer que `buscarMejorPar` devuelva también la frecuencia:

```
return {  
  par: mejorPar.split(", ").map(Number),  
  conteo: mejorConteo,  
};
```

y detener el entrenamiento cuando ya no haya patrones repetidos:

```
if (!mejor || mejor.conteo < 2) {  
  break;  
}
```

Así solo aprendemos patrones que realmente aparecen más de una vez.

B.16 Qué hemos simplificado

Esta implementación está diseñada para mostrar el algoritmo, no para competir con tokenizadores de producción. Deliberadamente omitimos:

- optimizaciones de rendimiento;
- estructuras especiales para contar pares de manera incremental;
- pretokenización, que separa el texto antes de aplicar BPE;
- reglas especiales para los espacios;
- tokens reservados, como los marcadores de inicio y fin;
- almacenamiento del vocabulario en archivos;
- entrenamiento sobre corpus grandes;
- paralelización;
- codificación incremental.

En particular, el rendimiento de `encode` es pobre: recorre la lista completa de fusiones aplicando cada una al texto entero. Un tokenizador real usa estructuras que localizan directamente los pares candidatos.

La esencia de BPE, sin embargo, permanece intacta:

```
bytes  
↓  
contar pares  
↓  
fusionar el más frecuente  
↓  
guardar la regla  
↓  
repetir
```

B.17 El algoritmo en pseudocódigo

Entrenamiento:

```
tokens = UTF8(texto)

repetir N veces:
    contar pares consecutivos
    elegir el más frecuente
    crear un token nuevo
    guardar la regla: par → token nuevo
    reemplazar el par
```

Codificación:

```
tokens = UTF8(texto)

para cada regla aprendida, en orden:
    aplicar la fusión

devolver tokens
```

Decodificación:

```
para cada token:
    si es byte:
        devolver el byte
    si es token aprendido:
        expandir sus dos componentes

convertir los bytes UTF-8 a texto
```

Eso es BPE en su forma esencial.

B.18 Qué conviene recordar

No hace falta memorizar el código. Conviene recordar el proceso:

```
texto
↓
UTF-8
↓
bytes
↓
pares frecuentes
↓
fusiones
↓
tokens cada vez mayores
```

El punto central es este:

BPE construye un vocabulario descubriendo secuencias frecuentes a partir de unidades elementales.

Y al trabajar sobre bytes obtenemos además una propiedad muy útil:

Cualquier texto Unicode puede representarse, incluso si contiene caracteres o palabras que nunca aparecieron durante el entrenamiento.

Las tres operaciones del tokenizador son:

```
bpe.train(textoBase, cantidadDeFusiones);  
  
const tokens = bpe.encode(texto);  
const textoOriginal = bpe.decode(tokens);
```

Ese pequeño ciclo contiene la esencia de un tokenizador BPE.

Apéndice C. Referencia rápida de expresiones regulares

Sintaxis de consulta rápida. La explicación está en el capítulo 20.

C.1 Átomos: qué carácter

Expresión	Significado
abc	la secuencia literal abc
.	cualquier carácter, salvo el salto de línea
\d	un dígito, equivale a [0-9]
\D	cualquier cosa que no sea un dígito
\w	carácter de palabra: [A-Za-z0-9_]
\W	lo contrario de \w
\s	espacio en blanco: espacio, tabulación, salto de línea
\S	lo contrario de \s
[abc]	uno de estos caracteres
[a-z]	un carácter del rango
[^abc]	cualquier carácter salvo estos
\p{L}	carácter Unicode con propiedad «letra»; requiere la bandera u
\p{N}	carácter Unicode con propiedad «número»
\uXXXX	un carácter por su código de cuatro dígitos hexadecimales
\u{1F600}	un punto de código Unicode completo; requiere u

C.2 Cuantificadores: cuántas veces

Expresión	Significado
*	cero o más
+	una o más
?	cero o una
{3}	exactamente tres
{2,5}	entre dos y cinco
{2,}	al menos dos
*?, +?, ??, {n,m}?	versión no codiciosa: lo mínimo posible

Los cuantificadores afectan al átomo o al grupo inmediatamente anterior.

C.3 Posición

Expresión	Significado
^	comienzo del texto, o de la línea con la bandera m
\$	final del texto, o de la línea con la bandera m
\b	límite de palabra
\B	no límite de palabra

Ninguno consume caracteres: marcan posiciones.

C.4 Estructura

Expresión	Significado
(abc)	grupo con captura
(?:abc)	grupo sin captura
(?<nombre>abc)	grupo con captura, con nombre
a\b	alternativa: a o b
\1	retorreferencia al primer grupo capturado
\k<nombre>	retorreferencia a un grupo con nombre

C.5 Contexto

Expresión	Significado
(?=...)	lookahead positivo: debe seguir esto
(?!...)	lookahead negativo: no debe seguir esto
(?<=...)	lookbehind positivo: debe preceder esto
(?<!...)	lookbehind negativo: no debe preceder esto

Tampoco consumen caracteres: son condiciones sobre el entorno de la coincidencia.

C.6 Banderas

Bandera	Nombre	Efecto
i	ignoreCase	ignora mayúsculas y minúsculas
g	global	busca todas las coincidencias, no solo la primera
m	multiline	^ y \$ operan por línea
s	dotAll	el punto también coincide con el salto de línea
u	unicode	semántica Unicode; habilita <code>\p{...}</code> y <code>\u{...}</code>
v	unicodeSets	versión ampliada de u, con operaciones sobre conjuntos
y	sticky	coincide solo desde la posición actual
d	hasIndices	agrega los índices de cada grupo capturado

Se escriben después de la barra final: `/patron/gi`.

C.7 Escapes

Estos caracteres tienen significado especial y deben escaparse con `\` para buscarlos literalmente:

```
. * + ? ^ $ ( ) [ ] { } | \ /
```

Dentro de una clase de caracteres `[]`, la mayoría pierde su significado especial. Los que siguen necesitando escape allí son `]`, `\`, `^` (solo si va primero) y `-` (salvo al principio o al final).

Al construir una regex con `new RegExp()`, la barra invertida debe duplicarse, porque la cadena de JavaScript también la interpreta:

```
/\d+/           // literal  
new RegExp("\\d+") // equivalente
```

C.8 Métodos de JavaScript

Operación	Sintaxis	Devuelve
Comprobar	<code>regex.test(texto)</code>	<code>true</code> o <code>false</code>
Encontrar la primera	<code>texto.match(regex)</code>	objeto con la coincidencia y sus grupos, o <code>null</code>
Encontrar todas	<code>texto.match(regex)</code> con <code>g</code>	array de cadenas, o <code>null</code>
Iterar todas con grupos	<code>texto.matchAll(regex)</code> con <code>g</code>	iterador de coincidencias
Posición de la primera	<code>texto.search(regex)</code>	índice, o <code>-1</code>
Reemplazar la primera	<code>texto.replace(regex, reemplazo)</code>	cadena nueva
Reemplazar todas	<code>texto.replaceAll(regex, reemplazo)</code> con <code>g</code>	cadena nueva
Separar	<code>texto.split(regex)</code>	array de partes
Avanzar sobre el texto	<code>regex.exec(texto)</code> con <code>g</code>	coincidencia siguiente, o <code>null</code>

En el texto de reemplazo:

Referencia	Significado
\$1, \$2, ...	grupo capturado por número
\$<nombre>	grupo capturado por nombre
\$&	la coincidencia completa
\$`	el texto anterior a la coincidencia
\$'	el texto posterior a la coincidencia
\$\$	un signo de dólar literal

El reemplazo también puede ser una función, que recibe la coincidencia y los grupos como argumentos:

```
"1-2-3".replace(/\d/g, (digito) => Number(digito) * 2);
// "2-4-6"
```

C.9 Patrones frecuentes

Necesidad	Expresión
Solo dígitos, texto completo	/^\d+\$/
Exactamente seis dígitos	/^\d{6}\$/
Tres letras, guion, cuatro dígitos	/^[A-Z]{3}-\d{4}\$/
Hora entre 00:00 y 23:59	/^(?:[01]\d 2[0-3]):[0-5]\d\$/
Fecha dd/mm/aaaa con grupos	/^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})\$/
Correo electrónico, validación básica	/^[^\s@]+@[^\s@]+\.[^\s@]+\$/
Normalizar espacios múltiples	texto.replace(/\s+/g, " ")
Quitar todo lo que no sea dígito	texto.replace(/\D/g, "")
Separar por coma con espacios variables	texto.split(/\s*,\s*/)
Palabras respetando acentos	texto.match(/\p{L}+/gu)
Palabra completa, no fragmento	/\bpalabra\b/
Contenido mínimo entre etiquetas	/ (.*?)/
Quitar la marca BOM inicial	texto.replace(/^\uFEFF/, "")
Dividir por saltos de línea de cualquier sistema	texto.split(/\r\n \n \r/)

C.10 Recordatorios

Buscar no es validar. `/\d{3}/` encuentra tres dígitos en cualquier parte; `/^\d{3}$/` exige que el texto entero sean tres dígitos.

`match()` devuelve `null` si no encuentra, no un array vacío. Conviene escribir `texto.match(regex) ?? []`.

Los cuantificadores son codiciosos. Agregar `?` los vuelve mínimos.

`\w` y `\b` no contemplan acentos ni `ñ`. Para texto en español conviene `\p{L}` con la bandera `u`.

Una regex con `g` y `exec()` conserva estado en su propiedad `lastIndex`. Reutilizar el mismo objeto en dos recorridos distintos puede dar resultados inesperados; `matchAll()` evita el problema.

Regex no reemplaza a un analizador sintáctico. Para HTML, JSON o cualquier estructura anidada corresponde una herramienta específica.

Apéndice D. Glosario

Términos técnicos, con el capítulo donde se desarrollan.

Acarreo. En una suma, el dígito que se traslada a la columna siguiente cuando el resultado excede la base. En el medio sumador es una de las dos salidas. *Apéndice A.*

Alcance (*scope*). Región del código desde la cual una vinculación resulta accesible. JavaScript tiene alcance global, de función y de bloque. *Capítulo 2.*

Alcance léxico. Regla por la cual el ámbito de una función queda determinado por el lugar donde fue escrita, no por el lugar desde donde se la invoca. Es la base de las clausuras. *Capítulo 17.*

Argumento. Valor concreto que se pasa a una función al invocarla. Se distingue del parámetro, que es el nombre declarado. *Capítulo 16.*

Array. Colección ordenada cuyos elementos se asocian con índices enteros que comienzan en cero. *Capítulo 10.*

Asociatividad. Regla que decide cómo se agrupan operadores de la misma precedencia. La mayoría asocia de izquierda a derecha; la exponenciación y la asignación, de derecha a izquierda. *Capítulo 3.*

Autómata finito. Máquina abstracta con una cantidad fija de estados que lee símbolos uno por uno. Las expresiones regulares describen exactamente los lenguajes que un autómata finito puede reconocer. *Apéndice A y capítulo 20.*

Bandera (*flag*). En expresiones regulares, opción que modifica el modo de trabajo del patrón: `i`, `g`, `m`, `s`, `u`, `y`, `d`. *Capítulo 20.*

Bandera de bits. Técnica que usa cada bit de un entero como una opción independiente, activada con `|`, consultada con `&`, apagada con `& ~` y alternada con `^`. *Capítulo 5.*

BigInt. Tipo primitivo para enteros de precisión arbitraria. Su literal termina en `n`. No admite decimales ni se mezcla automáticamente con `number`. *Capítulo 5.*

Bit. Unidad mínima de información, que puede tomar uno de dos valores. *Apéndice A.*

BOM (*Byte Order Mark*). Marca opcional al comienzo de un archivo de texto. UTF-8 no la necesita, pero algunas herramientas la agregan. *Capítulo 21.*

BPE (*Byte Pair Encoding*). Algoritmo de tokenización que parte de bytes y une repetidamente los pares más frecuentes. *Apéndice B.*

Buffer. En Node.js, representación de una secuencia de bytes crudos. *Capítulo 21.*

Bytecode. Representación intermedia que un motor genera a partir del código fuente y que puede interpretar rápidamente antes de compilar las partes más usadas. *Apéndice A*.

Byte. Grupo de ocho bits. *Apéndice A y capítulo 21*.

Cadena de alcances. Secuencia de entornos que el motor recorre, del más interno hacia afuera, para resolver un identificador. *Capítulo 17*.

Callback. Función que se entrega a otra función para que decida cuándo y cómo ejecutarla. No implica asincronía. *Capítulo 18*.

Clausura (closure). Función que conserva acceso a las variables del entorno donde fue creada, incluso después de que ese entorno terminó. *Capítulo 17*.

Codificación de caracteres. Regla que relaciona caracteres con bytes. UTF-8 es la más usada para archivos de texto. *Capítulo 6 y capítulo 21*.

Coerción. Conversión de tipo aplicada automáticamente por el lenguaje porque el contexto lo requiere. Se distingue de la conversión explícita, escrita por el programador. *Capítulo 9*.

Collation. Reglas de ordenamiento lingüístico de un idioma. En JavaScript se accede mediante `localeCompare()` e `Intl.Collator`. *Capítulo 6*.

Compilador. Programa que traduce código a otra representación antes de ejecutarlo. Se contrapone al intérprete, aunque los motores modernos combinan ambas estrategias. *Apéndice A*.

Complejidad funcional. Propiedad de un conjunto de operaciones lógicas que permite expresar cualquier función booleana. `{NOT, AND, OR}` la tiene; NAND sola también. *Apéndice A*.

Copia profunda. Duplicación que crea estructuras nuevas también en los niveles anidados. En JavaScript se obtiene con `structuredClone()`, con limitaciones. *Capítulo 12*.

Copia superficial. Duplicación que crea un contenedor nuevo pero comparte las referencias interiores. Es lo que producen `...`, `slice()` y `Object.assign()`. *Capítulo 12*.

Cortocircuito. Comportamiento de `&&` y `||` por el cual el operando derecho no se evalúa si el izquierdo ya determina el resultado. *Capítulo 4*.

Cuantificador. En expresiones regulares, símbolo que indica cuántas veces puede repetirse el elemento anterior: `*`, `+`, `?`, `{n,m}`. *Capítulo 20*.

Desestructuración. Sintaxis que extrae valores de un array por posición o de un objeto por nombre, y los vincula a variables. *Capítulo 10 y capítulo 11*.

Desenrollado de la pila. Proceso por el cual una excepción abandona sucesivamente las funciones pendientes hasta encontrar un manejador. *Capítulo 15*.

ECMAScript. Especificación estandarizada del lenguaje, definida en el estándar ECMA-262. JavaScript es el nombre habitual de sus implementaciones. *Apéndice A*.

Efecto secundario. Modificación de estado externo producida por una función, más allá de su valor de retorno. *Capítulo 18.*

Encadenamiento opcional (`?.`). Operador que devuelve `undefined` en lugar de lanzar un error cuando la parte izquierda es `null` o `undefined`. *Capítulo 11.*

Entorno de ejecución (*host*). Capa que agrega al lenguaje capacidades para interactuar con el mundo: DOM en el navegador, archivos y red en Node.js. *Apéndice A.*

Excepción. Mecanismo que interrumpe el flujo normal de ejecución y transfiere el control, junto con información sobre el fallo, al primer manejador que la capture. *Capítulo 15.*

Expresión. Porción de código que produce un valor. Se distingue de la sentencia, que ejecuta una acción. *Capítulo 3.*

Expresión regular. Descripción compacta de un patrón de texto, es decir, de un conjunto de cadenas posibles. *Capítulo 20.*

Fall-through. En un `switch`, continuación de la ejecución en el caso siguiente cuando el actual no termina con `break`, `return` o `throw`. *Capítulo 14.*

Falsy. Valor que se convierte en `false` en un contexto booleano. La lista completa es `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined` y `NaN`. *Capítulo 4.*

FileHandle. En Node.js, objeto que representa un archivo abierto y debe cerrarse explícitamente. *Capítulo 21.*

Forma normal disyuntiva. Procedimiento que convierte cualquier tabla de verdad en una expresión construida con NOT, AND y OR. *Apéndice A.*

Función de orden superior. Función que recibe una función como argumento, retorna una función, o ambas cosas. *Capítulo 18.*

Función pura. Función que para los mismos argumentos produce siempre el mismo resultado y no modifica datos externos. *Capítulo 18.*

Generador. Función declarada con `function*` que produce valores de a uno mediante `yield`, suspendiendo su ejecución entre uno y otro. *Capítulo 18.*

Grafema (*grapheme cluster*). Lo que una persona percibe como un solo carácter, que puede estar compuesto por varios puntos de código. *Capítulo 6.*

Guarda. Condición verificada al comienzo de una función que, al cumplirse, termina la ejecución temprano. Alternativa al anidamiento profundo. *Capítulo 14.*

Hoisting. Registro de las declaraciones de un alcance antes de ejecutar sus instrucciones. Con `var` la variable queda inicializada en `undefined`; con `let` y `const` queda en la zona muerta temporal. *Capítulo 2.*

Identidad. Propiedad por la cual dos objetos son iguales solo si son el mismo objeto, sin importar que contengan los mismos datos. *Capítulo 12.*

Identificador. Nombre válido para una variable, función o propiedad. *Capítulo 2.*

Índice. Posición de un elemento dentro de un array, contada como desplazamiento desde el comienzo. Por eso empieza en cero. *Capítulo 10.*

Inmutabilidad. Propiedad de una estructura que no se modifica: las actualizaciones producen valores nuevos. Los primitivos son inmutables por definición. *Capítulo 2 y capítulo 12.*

Interpolación. Inserción del valor de una expresión dentro de una plantilla literal mediante `${...}`. *Capítulo 6.*

Intérprete. Programa que analiza instrucciones y las ejecuta directamente, sin producir antes un programa traducido. *Apéndice A.*

Iterable. Objeto que implementa `Symbol.iterator` y por lo tanto puede recorrerse con `for...of`, expandirse con `...` y desestructurarse. *Capítulo 8.*

JIT (Just-In-Time). Compilación realizada durante la ejecución, usando información que solo puede conocerse mientras el programa corre. *Apéndice A.*

Lenguaje regular. Conjunto de cadenas que un autómata finito puede reconocer. Es exactamente lo que describe una expresión regular en su forma matemática. *Capítulo 20.*

Lookahead y lookbehind. Construcciones de una expresión regular que exigen la presencia o ausencia de un patrón antes o después de la coincidencia, sin incluirlo en ella. *Capítulo 20.*

Map. Colección de pares clave-valor que admite claves de cualquier tipo, conserva el orden de inserción y es iterable. *Capítulo 13.*

Medio sumador. Circuito que suma dos bits y produce la suma y el acarreo, sin recibir acarreo de entrada. *Apéndice A.*

Memoización. Técnica que guarda los resultados ya calculados de una función para no volver a computarlos. Se implementa naturalmente con una clausura. *Capítulo 17.*

Metacarácter. En una expresión regular, carácter con significado especial: `.`, `*`, `+`, `?`, `^`, `$`, `[`, `]`, `(`, `)`, `{`, `}`, `|`, `\`. *Capítulo 20.*

Método. Función almacenada como propiedad de un objeto. *Capítulo 11.*

Motor. Programa que implementa la especificación del lenguaje y ejecuta el código: V8, SpiderMonkey, JavaScriptCore. *Apéndice A.*

Mutación. Modificación de una estructura existente, en oposición a producir una estructura nueva. *Capítulo 12.*

NaN (Not a Number). Valor de tipo `number` que representa el resultado de una operación aritmética que no puede producir un número válido. No es igual a sí mismo. *Capítulo 5.*

Objeto. Colección de asociaciones entre claves y valores. Todo lo que no es un primitivo es un objeto. *Capítulo 11.*

Parámetro. Nombre declarado en la definición de una función. Recibe el valor del argumento correspondiente. *Capítulo 16.*

Parámetro rest. Parámetro precedido por `...` que reúne los argumentos restantes en un array. Debe ser el último. *Capítulo 16.*

Path traversal. Vulnerabilidad que permite salir de un directorio autorizado mediante segmentos `..` en una ruta controlada por el usuario. *Capítulo 21.*

Pila de llamadas (*call stack*). Registro de las llamadas a función pendientes de retornar. Determina el recorrido de una excepción y limita la profundidad de la recursión. *Capítulo 15 y capítulo 19.*

Plantilla etiquetada. Plantilla literal precedida por una función, que recibe por separado las partes estáticas y los valores interpolados. *Capítulo 6.*

Plantilla literal. Cadena delimitada por comillas invertidas, que admite interpolación y varias líneas. *Capítulo 6.*

Polyfill. Código que implementa una funcionalidad ausente en un entorno, para que el resto del programa pueda usarla. *Apéndice A.*

Precedencia. Regla que decide cómo se agrupan operadores distintos en una expresión sin paréntesis. *Capítulo 3.*

Primitivo. Valor de uno de los siete tipos básicos: `boolean`, `number`, `bigint`, `string`, `undefined`, `null`, `symbol`. Todos son inmutables. *Capítulo 2.*

Promesa. Objeto que representa el resultado futuro de una operación asíncrona. Puede cumplirse con un valor o rechazarse con un motivo. *Capítulo 15.*

Punto de código. Número que Unicode asigna a un carácter, escrito como `U+XXXX`. *Capítulo 6.*

Recursión. Técnica en la que una función se llama a sí misma con una versión reducida del problema. Requiere un caso base y un caso recursivo. *Capítulo 19.*

Referencia. Valor que identifica un objeto. Las variables de objeto contienen referencias, no los objetos mismos. *Capítulo 12.*

Relanzamiento (*rethrow*). Volver a lanzar una excepción capturada que este nivel no sabe resolver, para que la maneje una capa superior. *Capítulo 15.*

Set. Colección de valores únicos, comparados por identidad, que conserva el orden de inserción. *Capítulo 13.*

Sombreado (*shadowing*). Situación en la que una variable de un alcance interno oculta otra del mismo nombre en un alcance externo. *Capítulo 2 y capítulo 17.*

Stream. Flujo que entrega datos por fragmentos, a medida que están disponibles, en lugar de cargarlos completos en memoria. *Capítulo 21.*

Symbol. Tipo primitivo que produce valores únicos, usados como claves de propiedad sin riesgo de colisión y como ganchos de los protocolos del lenguaje. *Capítulo 8.*

Tabla de verdad. Definición por enumeración de una función booleana: qué salida corresponde a cada combinación de entradas. *Apéndice A.*

Tipado dinámico. Sistema en el que los tipos se asocian a los valores durante la ejecución y una variable puede referirse a valores de tipos distintos. *Apéndice A.*

Tipado estático. Sistema en el que una herramienta verifica los tipos antes de ejecutar el programa. *Apéndice A.*

Token. En un tokenizador, número que representa una secuencia de bytes que el algoritmo decidió agrupar. *Apéndice B.*

Transpilación. Traducción de un lenguaje fuente a otro de nivel semejante, por ejemplo de TypeScript a JavaScript. *Apéndice A.*

Truthy. Valor que se convierte en `true` en un contexto booleano. Es todo lo que no está en la lista de valores *falsy*, incluidos los arrays y objetos vacíos. *Capítulo 4.*

TypeScript. Superconjunto de JavaScript que agrega verificación estática de tipos. Sus anotaciones desaparecen antes de la ejecución. *Apéndice A.*

Unicode. Estándar que asigna un número a cada carácter de prácticamente todos los sistemas de escritura. *Capítulo 6.*

UTF-8. Codificación de longitud variable que representa los puntos de código Unicode mediante uno a cuatro bytes. Es la habitual en archivos y en la web. *Capítulo 6 y capítulo 21.*

UTF-16. Codificación que usa unidades de 16 bits. Es la que JavaScript emplea internamente para sus cadenas, y la razón por la que `"😊".length` da 2. *Capítulo 6.*

Variable. Nombre vinculado a un valor dentro de un alcance. *Capítulo 2.*

Vinculación (*binding*). Asociación entre un identificador y un valor dentro de un entorno. Es lo que `const` protege. *Capítulo 2.*

WeakMap y WeakSet. Colecciones que solo admiten objetos y mantienen referencias débiles, permitiendo que el recolector de basura libere sus elementos. *Capítulo 13.*

Zona muerta temporal (*Temporal Dead Zone*). Región entre el comienzo de un alcance y la inicialización de una variable declarada con `let` o `const`, donde acceder a ella produce un `ReferenceError`. *Capítulo 2.*

Fuentes y lecturas recomendadas

Referencias consultadas y ampliaciones útiles, agrupadas por tema. Salvo indicación en contrario, la documentación de MDN está disponible también en español, aunque las versiones en inglés suelen estar más actualizadas.

La especificación y su historia

- [ECMAScript Language Specification — TC39](#)
- [Archivo de ediciones de ECMAScript — Ecma International](#)
- [ECMA-262, primera edición, 1997](#)
- [JavaScript: historia y definición — MDN](#)

Motores y ejecución

- [Diez años de V8 — blog de V8](#)
- [El intérprete Ignition y la compilación JIT — blog de V8](#)

Fundamentos del lenguaje

- [Gramática, declaraciones, tipos y literales — MDN](#)
- [Gramática léxica e identificadores — MDN](#)
- [Tipos y estructuras de datos — MDN](#)
- [Expresiones y operadores — MDN](#)
- [Precedencia de operadores — MDN](#)
- [Coerción de tipos — MDN](#)

Tipos primitivos

- [Referencia de `Number` — MDN](#)
- [Referencia de `BigInt` — MDN](#)
- [Referencia de `String` — MDN](#)
- [Plantillas literales — MDN](#)
- [Referencia de `Symbol` — MDN](#)
- [Símbolos conocidos del lenguaje — MDN](#)

Unicode e internacionalización

- [El estándar Unicode — Unicode Consortium](#)
- [Intl.Collator — MDN](#)
- [Intl.Segmenter — MDN](#)
- [TextEncoder — MDN](#)

Estructuras de datos

- [Colecciones indexadas y arrays — MDN](#)
- [Referencia de Array — MDN](#)
- [Trabajo con objetos — MDN](#)
- [Inicializadores de objetos — MDN](#)
- [Desestructuración — MDN](#)
- [Sintaxis de expansión — MDN](#)
- [Referencia de Map — MDN](#)
- [Referencia de Set — MDN](#)
- [Clonación estructurada — MDN](#)

Control del flujo

- [Flujo de control y manejo de errores — MDN](#)
- [Bucles e iteración — MDN](#)
- [switch — MDN](#)
- [for...of — MDN](#)
- [for...in — MDN](#)
- [try...catch...finally — MDN](#)
- [Objeto Error — MDN](#)
- [Causa de un error — MDN](#)
- [Uso de promesas — MDN](#)

Funciones

- [Funciones — MDN](#)
- [Funciones flecha — MDN](#)
- [Clausuras — MDN](#)
- [Iteradores y generadores — MDN](#)

Expresiones regulares

- [Guía de expresiones regulares — MDN](#)
- [Referencia de RegExp — MDN](#)
- [Escapes de propiedades Unicode — MDN](#)

Node.js

- [Introducción a Node.js — documentación oficial](#)
- [Sistema de archivos, `node:fs`](#)
- [Rutas, `node:path`](#)
- [Buffers y codificaciones](#)
- [Lectura línea por línea, `node:readline`](#)
- [Streams](#)

TypeScript

- [TypeScript para quien comienza a programar — documentación oficial](#)
- [Manual de TypeScript — documentación oficial](#)